Multi-Cloud Engineering Program

Recorrido de Amazon Web Services

27 clases · 116 horas

La plataforma esencial de AWS y su arquitectura, automatización y operación en producción, con las clases de otras partes que tratan el proveedor.

Extracto del manual integral, generado desde las mismas fuentes versionadas.

Alcance de este cuaderno

Componente	Cobertura
Partes propias	02, 17
Clases de esas partes	24
Clases de otras partes	3
Clases totales	27
Horas estimadas	116

Índice

Alcance de este cuaderno 2

Índice 3

Parte 02 — AWS: plataforma esencial 4

025 — Organizations, cuentas, OU, SCP y landing zone . . . 4
026 — IAM, roles, políticas, STS y federación . . . 12
027 — VPC, subredes, rutas, NAT, endpoints y seguridad . . . 20
028 — EC2, AMI, EBS y selección de capacidad . . . 28
029 — Elastic Load Balancing y Auto Scaling . . . 36
030 — S3: objetos, versionado, lifecycle y replicación . . . 44
031 — RDS, DynamoDB y ElastiCache: decisión de datos . . . 52
032 — Lambda, API Gateway y Step Functions . . . 60
033 — SQS, SNS y EventBridge . . . 68
034 — CloudWatch, CloudTrail, Config y Systems Manager . . . 76
035 — KMS, Secrets Manager, WAF y controles de seguridad . . . 84
036 — Proyecto: aplicación de tres capas en AWS . . . 93

Parte 06 — Kubernetes y plataformas administradas 101

083 — EKS, AKS y GKE: similitudes y diferencias . . . 101

Parte 07 — Infraestructura como código y configuración 110

093 — CloudFormation, Bicep, Pulumi y Terraform . . . 110

Parte 17 — AWS: arquitectura, automatización y operación en producción 119

205 — Hosting progresivo con Amplify, S3 y CloudFront . . . 119
206 — OIDC de GitHub y GitLab hacia AWS sin secretos . . . 129
207 — SAM, Lambda, API Gateway y despliegue serverless . . . 140
208 — DynamoDB por patrones de acceso y single-table design . . . 151
209 — Cognito, JWT authorizers, WAF y defensa en profundidad . . . 162
210 — EventBridge, SQS, DLQ, replay e idempotencia . . . 173
211 — CloudWatch, X-Ray y observabilidad como código . . . 184
212 — ECR, ECS Fargate, ALB y autoscaling . . . 194
213 — EKS, IRSA, GitOps y operación de clúster . . . 204
214 — Budgets, Cost Explorer, etiquetado y FinOps automatizado . . . 215
215 — Multi-región, Route 53, failover y game day . . . 226
216 — Proyecto: CloudShop productivo en AWS . . . 237

Parte 22 — Especializaciones, certificaciones y práctica profesional 248

273 — Mapeo AWS, Azure, Google Cloud, Kubernetes y FinOps . . . 248

Parte 02 — AWS: plataforma esencial

025 — Organizations, cuentas, OU, SCP y landing zone

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: governance · Estado: EXECUTABLECORE

Propósito

Construir la estructura de cuentas que sostendrá todo lo que se despliegue en AWS durante el resto del programa. Es la decisión más irreversible de la parte: mover cuentas entre unidades organizativas cambia las políticas que se les aplican, y separar más tarde una cuenta compartida exige migrar recursos. Aquí se aplica a AWS lo que la clase 017 estableció de forma neutral.

Resultados de aprendizaje

Al finalizar podrás:

1. Diseñar una jerarquía de unidades organizativas por régimen de gobierno y no por organigrama.
2. Escribir una SCP que restrinja sin inutilizar la cuenta, exceptuando correctamente los servicios globales.
3. Explicar por qué una SCP no concede permisos y qué implica al depurar un acceso denegado.
4. Desplegar una landing zone con cuentas de auditoría, red y seguridad separadas desde el inicio.
5. Verificar que los guardarraíles funcionan mediante prueba negativa, no por inspección de la configuración.

Conceptos centrales

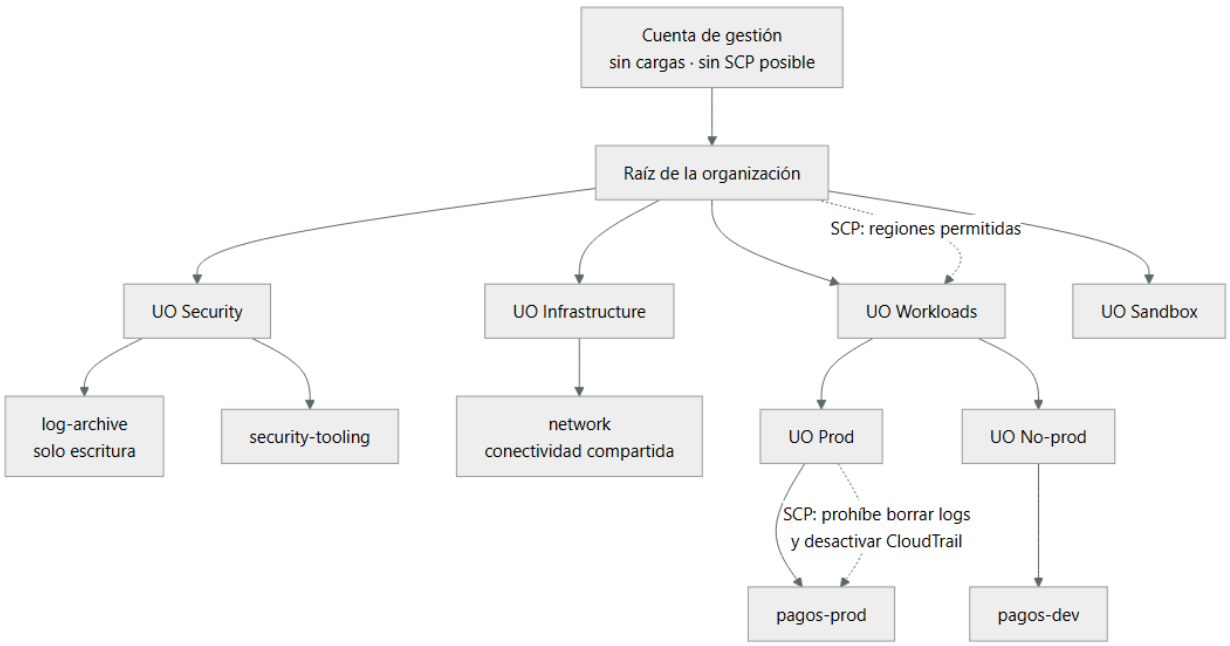
Concepto	Comprensión verificable
SCP	Service Control Policy: política heredada que define el techo máximo de permisos de las cuentas bajo una unidad organizativa. Nunca concede: solo limita lo que otras políticas pueden otorgar.
landing zone	Estructura base de cuentas, red, identidad y registro sobre la que se despliega todo lo demás. Su valor está en existir antes de la primera carga, porque retrofitarla es caro.
cuenta de gestión	La que crea la organización. No admite SCP sobre sí misma, así que no debe alojar cargas: cualquier compromiso ahí no tiene barrera superior que lo contenga.
guardarraíl preventivo	Control que impide la acción antes de que ocurra, normalmente una SCP. Se distingue del detectivo, que avisa después, y exige prueba negativa para demostrar que efectivamente deniega.
cuenta de auditoría	Destino centralizado de registros de todas las demás, con escritura permitida y borrado prohibido. Es lo que evita que quien comprometa una cuenta pueda borrar su propio rastro.

Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. La cuenta de gestión no aloja cargas

AWS Organizations tiene una asimetría deliberada: la cuenta de gestión no puede tener SCP aplicadas sobre sí misma. Es la única de toda la organización sin barrera superior.

La consecuencia es directa: cualquier recurso que viva ahí opera sin los guardarraíles que protegen a las demás, y cualquier credencial comprometida en esa cuenta tiene el poder de modificar la organización entera —crear cuentas, mover unidades, desactivar políticas—.

Lo que sí pertenece a la cuenta de gestión:

```
- la propia organización y sus SCP
- la agregación de facturación
- nada más
```

Y lo que hay que hacer con ella el primer día:

```
# Activar MFA en el usuario raíz y no volver a usarlo
$ aws iam get-account-summary --query 'SummaryMap.AccountMFAEnabled'
1
# Cero claves de acceso del usuario raíz
$ aws iam get-account-summary --query 'SummaryMap.AccountAccessKeysPresent'
0
# Alerta ante cualquier uso del usuario raíz
```

La tercera línea es el control más rentable de toda la landing zone: el uso del usuario raíz es un evento que debería ocurrir una o dos veces al año —para tareas que solo él puede hacer— y cualquier otra aparición merece una llamada telefónica.

AWS Control Tower automatiza buena parte de esta estructura, y conviene saber qué hace por debajo antes de usarlo: crea las unidades organizativas Security y Sandbox, las cuentas de archivo de registros y de auditoría, y aplica un conjunto de guardarraíles. Usarlo sin entender la estructura produce dependencia de la herramienta para operaciones que después hay que hacer a mano.

2. SCP: el orden de evaluación decide el diagnóstico

Una SCP no concede nada. Es un filtro sobre lo que las políticas de identidad pueden otorgar dentro de esa rama de la organización:

```
permiso efectivo = SCP ∩ política de identidad ∩ política de recurso ∩ límite de permisos
```

Dos consecuencias operativas:

1. Adjuntar una SCP con Allow: no da acceso a nadie. Si ninguna política de identidad lo concede, no hay permiso.
2. Un Deny en la SCP gana sobre cualquier concesión, incluida la del administrador de la cuenta. No hay escalada posible desde dentro.

Las SCP admiten dos estrategias, y mezclarlas causa confusión:

```
lista de permitidos (allowlist):  quitar FullAWSAccess y enumerar lo permitido
  → muy restrictivo, alto mantenimiento: cada servicio nuevo exige tocar la SCP

lista de denegados (denylist):    mantener FullAWSAccess y denegar lo prohibido
  → lo habitual: menos mantenimiento, protege contra lo que se sabe peligroso
```

El error de diagnóstico más caro es buscar el problema en la política de identidad cuando lo deniega una SCP. El mensaje de error de AWS lo distingue, y conviene leerlo con atención:

```
AccessDenied ... with an explicit deny in a service control policy
AccessDenied ... with an explicit deny in an identity-based policy
AccessDenied ... no identity-based policy allows the action
```

La tercera es denegación implícita —falta conceder—; las dos primeras dicen exactamente qué documento hay que revisar. Añadir permisos ante la primera no cambia nada.

3. Restringir regiones sin romper la cuenta

La SCP más común y la que más cuentas ha inutilizado. Restringir por región parece trivial hasta que se descubre que varios servicios de AWS son globales y sus llamadas se emiten contra us-east-1.

La versión que rompe:

```
{
  "Effect": "Deny",
  "Action": "*",
  "Resource": "*",
  "Condition": {"StringNotEquals": {"aws:RequestedRegion": ["us-east-1", "sa-east-1"]}}
}
```

En cuanto se aplica, deja de funcionar IAM, la facturación, el soporte y Route 53 desde cualquier contexto que no resuelva a esas regiones. Y con Action: "" se bloquea incluso la posibilidad de quitar la política desde la propia cuenta.

La versión correcta excluye los servicios sin región:

```
{
  "Effect": "Deny",
  "NotAction": [
    "iam:*", "organizations:*", "support:*", "sts:*",
    "route53:*", "cloudfront:*", "budgets:*", "ce:*",
    "health:*", "waf:*", "shield:*"
  ],
  "Resource": "*",
  "Condition": {
    "StringNotEquals": {"aws:RequestedRegion": ["us-east-1", "sa-east-1"]}
  }
}
```

Y un matiz sobre sts:: excluirlo es necesario porque el punto de acceso global de STS resuelve a us-east-1; sin la excepción, asumir roles falla desde otras regiones.

Toda SCP debe probarse en una cuenta de sandbox antes de aplicarla a una unidad organizativa productiva. No hay simulador que capture todos los casos, y el coste de equivocarse es una cuenta que no se puede administrar.

4. Cuentas de la base: auditoría, red y seguridad

Tres cuentas que conviene separar desde el primer día, porque retrofitarlas exige mover recursos con estado:

Archivo de registros (log-archive). Recibe CloudTrail, Config y logs de acceso de todas las cuentas. Su política de bucket permite escritura desde la organización y prohíbe el borrado a todo el mundo, incluido su propio administrador:

```
{
  "Effect": "Deny",
  "Principal": "*",
  "Action": ["s3:DeleteObject", "s3:DeleteObjectVersion", "s3:PutBucketPolicy"],
  "Resource": "arn:aws:s3:::org-log-archive/*"
```

```
}
```

Con bloqueo de objetos en modo cumplimiento, ni siquiera la cuenta raíz puede borrarlos antes de que expire la retención. Ese es el punto: un atacante que compromete una cuenta no puede borrar la evidencia de lo que hizo.

Red (network). Aloja el Transit Gateway, las zonas privadas de DNS y los endpoints compartidos. Separarla evita que el ciclo de vida de la conectividad dependa de un producto concreto, y permite que el equipo de red tenga permisos ahí sin tenerlos en las cuentas de carga.

Herramientas de seguridad (security-tooling). Cuenta delegada como administradora de GuardDuty, Security Hub y Config. La delegación es importante: permite operar esos servicios en toda la organización sin usar la cuenta de gestión, que es donde no hay barreras.

Las tres tienen algo en común: su valor aparece durante un incidente, y para entonces ya no se pueden crear.

5. Un guardarraíl sin prueba negativa es una intención

Configurar una SCP no demuestra que deniegue. La configuración puede estar adjuntada a la unidad equivocada, tener una condición que nunca se cumple, o quedar anulada por un NotAction demasiado amplio.

La verificación exige intentar la acción y comprobar que falla:

```
# Prueba negativa 1: región no permitida
$ aws ec2 describe-instances --region eu-west-1
An error occurred (UnauthorizedOperation) ... with an explicit deny in a
service control policy                                        ✓ deniega

# Prueba negativa 2: desactivar el rastro de auditoría
$ aws cloudtrail stop-logging --name org-trail
An error occurred (AccessDeniedException) ... explicit deny in a service
control policy                                                ✓ deniega

# Prueba positiva: lo que debe seguir funcionando
$ aws iam list-roles --max-items 1 >/dev/null && echo "IAM operativo  ✓"
$ aws sts get-caller-identity >/dev/null && echo "STS operativo  ✓"
```

Las dos últimas líneas son tan importantes como las dos primeras: una SCP que deniega lo que debe y también lo que no, es un fallo igual de grave, solo que se descubre más tarde y en peor momento.

El simulador de políticas de IAM ayuda pero no basta: no evalúa SCP en todos los escenarios ni captura las llamadas que los servicios hacen entre sí. La única evidencia válida es el intento real desde una cuenta bajo la unidad organizativa correspondiente.

Estas pruebas deben ser automáticas y periódicas, no de una sola vez. Una SCP correcta hoy puede quedar anulada mañana por otra adjuntada a un nivel distinto de la jerarquía, sin que nadie modifique la primera.

Ejemplo trabajado

CloudShop opera en una sola cuenta de AWS y migra a una landing zone. Se ejecuta y se verifica paso a paso.

Estado inicial y sus tres riesgos concretos:

```
$ aws organizations describe-organization 2>&1 | head -1
AWSOrganizationsNotInUseException
$ aws cloudtrail describe-trails --query 'trailList[0].[Name,S3BucketName,IsMultiRegionTrail]' --
output text
cloudshop-trail   cloudshop-logs   False
```

```
1. Sin organización: no hay SCP posible, ningún guardarraíl preventivo.
2. CloudTrail en la MISMA cuenta: quien la comprometa borra su rastro.
3. Rastro de una sola región: la actividad en otras regiones es invisible.
```

Paso 1 — organización y unidades por régimen de gobierno, no por equipo:

```
$ aws organizations create-organization --feature-set ALL
$ for uo in Security Infrastructure Workloads Sandbox; do
    aws organizations create-organizational-unit --parent-id r-abcd --name $uo
  done
$ aws organizations create-organizational-unit --parent-id ou-work --name Prod
$ aws organizations create-organizational-unit --parent-id ou-work --name NonProd
```

Paso 2 — cuentas de la base:

```
log-archive        Security         destino de logs, borrado prohibido
security-tooling   Security         administrador delegado de GuardDuty y Config
network            Infrastructure   Transit Gateway y DNS privado
cloudshop-prod     Workloads/Prod   la carga actual, migrada aquí
cloudshop-dev      Workloads/NonProd
```

Paso 3 — SCP de regiones, probada antes en Sandbox:

```
$ aws organizations attach-policy --policy-id p-regiones --target-id ou-sandbox
# desde una cuenta de sandbox:
$ aws ec2 describe-instances --region ap-south-1
AccessDenied ... explicit deny in a service control policy      ✓
$ aws iam list-roles --max-items 1 >/dev/null && echo ok
ok                                                              ✓ IAM intacto
$ aws sts get-caller-identity >/dev/null && echo ok
ok                                                              ✓ STS intacto
```

La primera versión de la SCP sí rompió STS: sin sts: en el NotAction, asumir roles desde sa-east-1 fallaba porque el punto de acceso global resuelve a us-east-1. Se detectó en sandbox, que es exactamente para lo que sirve.

Solo entonces se aplica a Workloads.

Paso 4 — SCP de protección de la evidencia sobre Prod:

```
{
  "Effect": "Deny",
  "Action": [
    "cloudtrail:StopLogging", "cloudtrail:DeleteTrail",
    "config:DeleteConfigurationRecorder", "config:StopConfigurationRecorder",
    "guardduty:DeleteDetector", "guardduty:DisassociateFromMasterAccount"
  ],
  "Resource": "*"
}
```

```
$ aws cloudtrail stop-logging --name org-trail   # desde cloudshop-prod
AccessDeniedException ... explicit deny in a service control policy   ✓
```

Paso 5 — verificación del aislamiento de cuotas, que era el fallo de la clase 017:

```
$ for c in cloudshop-prod cloudshop-dev; do
    printf "%-18s " $c
    aws service-quotas get-service-quota --service-code ec2 \
      --quota-code L-1216C47A --profile $c --query 'Quota.Value' --output text
  done
cloudshop-prod     256
cloudshop-dev      128
```

Cuotas independientes: un experimento en desarrollo ya no puede consumir la capacidad de producción.

Resultado, con las pruebas que lo demuestran:

```
cuentas                                          1          5      —
guardarraíles preventivos                        0          2      prueba negativa ✓
logs borrables por quien compromete la cuenta    sí         no     stop-logging denegado ✓
cuota compartida entre entornos                  sí         no     256 y 128 separadas ✓
visibilidad multi-región                         no         sí     rastro de organización ✓
```

Lo que hizo útil el ejercicio no fue la estructura sino las pruebas negativas. La primera SCP parecía correcta por inspección y rompía STS; solo intentar la acción lo reveló.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/025-organizations-cuentas-ou-scp-y-landing-zone/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa landing-zone-aws en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye landing-zone-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

■■ Errores frecuentes

Síntoma	Causa probable	Corrección
Tras aplicar una SCP de regiones deja de funcionar IAM y no se puede quitar la política	Los servicios globales emiten contra us-east-1 y no se exceptuaron	Usa NotAction con iam, organizations, sts, route53, cloudfront y facturación; prueba siempre en sandbox primero.
Se añaden permisos a un rol y el acceso sigue denegado	Lo deniega una SCP, y ninguna concesión posterior la revierte	Lee el mensaje: «explicit deny in a service control policy» indica qué documento revisar.
Un atacante borra los registros de la cuenta que comprometió	CloudTrail escribía en la misma cuenta y nada impedía detenerlo	Cuenta de archivo separada, bloqueo de objetos y SCP que deniegue StopLogging y DeleteTrail.
La cuenta de gestión aloja cargas de trabajo	Es la única sin SCP posible: no tiene barrera superior	Deja en ella solo la organización y la facturación; mueve todo lo demás a cuentas bajo unidades organizativas.
Un guardarraíl configurado no impide la acción que debía impedir	Se verificó por inspección de la configuración y no por intento real	Prueba negativa automatizada y periódica, más prueba positiva de lo que debe seguir funcionando.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Por qué la cuenta de gestión no debe alojar cargas de trabajo?
2. Adjuntas una SCP con Allow: a una unidad organizativa. ¿Quién obtiene acceso y por qué?
3. ¿Qué servicios hay que exceptuar al restringir regiones, y qué ocurre concretamente si olvidas sts:?
4. ¿Qué dos propiedades debe tener la cuenta de archivo de registros para que la evidencia sobreviva a un compromiso?

5. ¿Por qué una prueba negativa no basta sin su prueba positiva correspondiente?

Referencias

- AWS (2024). Organizations: service control policies — evaluación, estrategias y límites. <https://docs.aws.amazon.com/organizations/latest/userguide/orgsmanagepoliciesscps.html>
- AWS (2024). Organizing your AWS environment using multiple accounts — cuentas de la base y unidades recomendadas. <https://docs.aws.amazon.com/whitepapers/latest/organizing-your-aws-environment/organizing-your-aws-environment.html>
- AWS (2024). Control Tower: how controls work — guardarraíles preventivos y detectivos. <https://docs.aws.amazon.com/controltower/latest/controlreference/controls.html>
- AWS (2024). CloudTrail: security best practices — rastro de organización, validación de integridad y bloqueo de objetos. <https://docs.aws.amazon.com/awscloudtrail/latest/userguide/best-practices-security.html>
- AWS (2024). AWS services that work with IAM — qué servicios son globales y cómo se comportan con condiciones de región. <https://docs.aws.amazon.com/IAM/latest/UserGuide/referenceaws-services-that-work-with-iam.html>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 02 en PDF · Recorrido de AWS en PDF · Manual integral

Anterior	Índice	Siguiente
← 024 · Proyecto: decisión de migración sustentada con ADR	Parte 02 · Programa	026 · IAM, roles, políticas, STS y federación →

Evaluación — 025 Organizations, cuentas, OU, SCP y landing zone

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega landing-zone-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

026 — IAM, roles, políticas, STS y federación

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: iam · Estado: EXECUTABLECORE

Propósito

Dominar el modelo de identidad de AWS hasta poder predecir el resultado de una petición sin ejecutarla, y sustituir toda credencial de larga vida por identidad temporal. Es la clase que hace posible que en la parte 17 un pipeline despliegue sin secretos, y la que explica por qué la mayoría de brechas en AWS no son fallos de AWS.

Resultados de aprendizaje

Al finalizar podrás:

1. Reproducir el orden de evaluación de una petición y predecir el resultado ante políticas en conflicto.
2. Escribir una política con condiciones que acoten origen, etiqueta y presencia de MFA.
3. Configurar federación OIDC desde un pipeline con una condición de sujeto que impida la suplantación.
4. Usar límites de permisos para delegar creación de roles sin permitir escalada de privilegios.
5. Diagnosticar un acceso denegado leyendo el mensaje para saber qué documento revisar.

Conceptos centrales

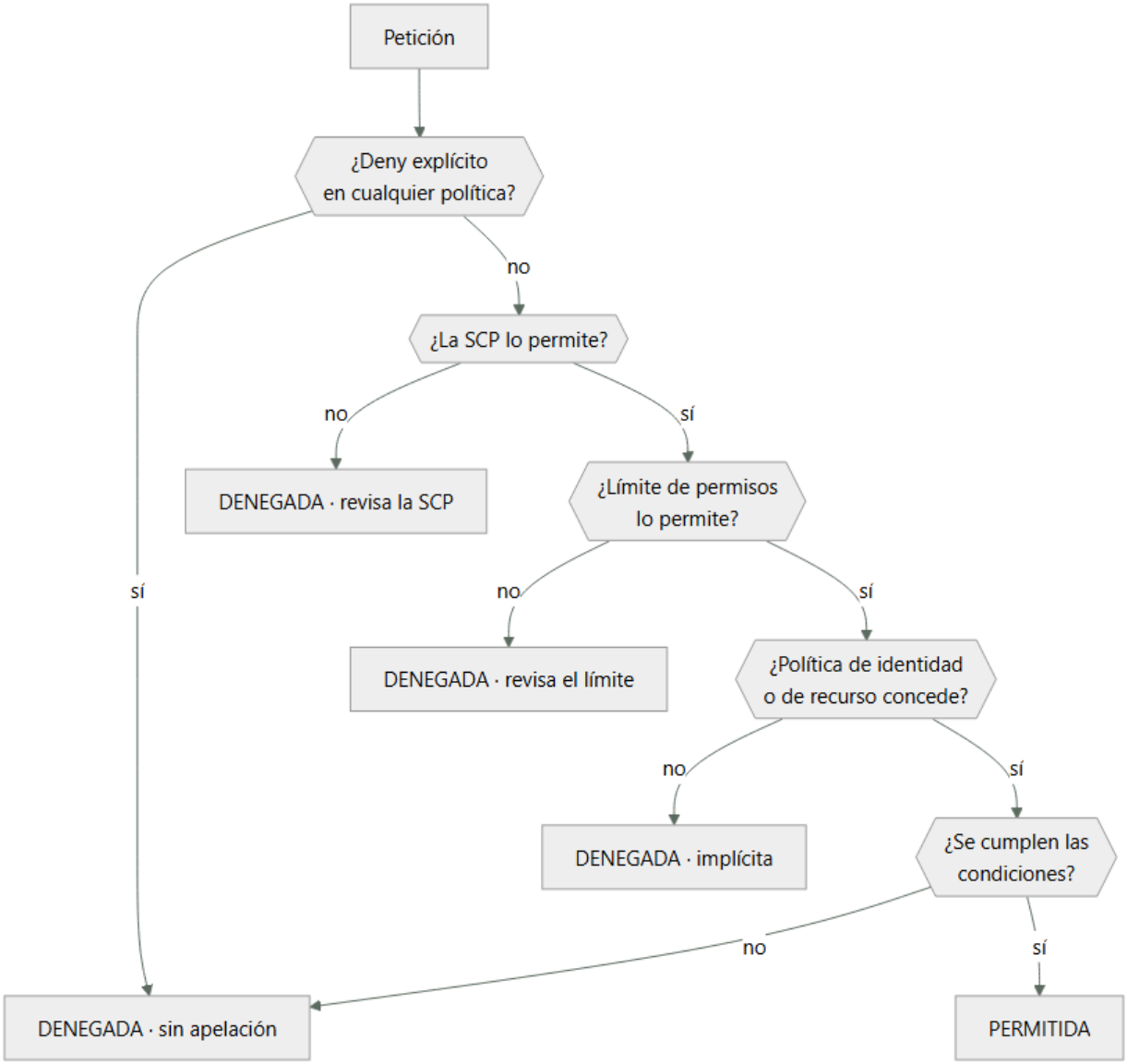
Concepto	Comprensión verificable
rol	Identidad sin credenciales permanentes que se asume temporalmente. Tiene dos documentos: la política de confianza —quién puede asumirlo— y las de permisos —qué puede hacer una vez dentro—.
política de confianza	Documento que declara qué principales pueden asumir el rol y bajo qué condiciones. Es la puerta de entrada, y un error aquí es más grave que uno en los permisos.
límite de permisos	Política adjunta a una identidad que define el techo de lo que puede hacer, aunque otras políticas concedan más. Permite delegar la creación de roles sin permitir escalada.
confused deputy	Ataque en el que un tercero legítimo es inducido a actuar contra un recurso ajeno. Se mitiga con condiciones sobre la cuenta de origen y el identificador externo.
clave de condición	Atributo evaluable de la petición: origen, hora, MFA, etiquetas, VPC. Convierte una política de «puede» en «puede, si además se cumple esto».

Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. El orden de evaluación, y cómo leer el error

AWS evalúa cada petición en una secuencia fija. Conocerla convierte «no tengo permiso» en un diagnóstico de un minuto:

1. Deny explícito en cualquier política → denegada, sin excepción.
2. SCP de la organización → si no lo permite, denegada.
3. Límite de permisos, si existe → si no lo permite, denegada.
4. Política de identidad o de recurso → alguna debe conceder.
5. Condiciones → deben cumplirse todas.

El mensaje de error dice exactamente dónde mirar, y casi nadie lo lee:

```
"...with an explicit deny in a service control policy"
  → revisa las SCP. Añadir permisos a la identidad no hará nada.

"...with an explicit deny in a permissions boundary"
  → revisa el límite de permisos del rol.

"...with an explicit deny in an identity-based policy"
```

```
  → hay un Deny en la política del rol o del usuario.

"...because no identity-based policy allows the action"
  → denegación implícita: falta conceder. Este es el único caso en que
    añadir un permiso resuelve el problema.
```

Solo el cuarto caso se arregla concediendo. Los tres primeros exigen tocar otro documento, y perseguirlos con más permisos es la causa habitual de que un rol acabe con AdministratorAccess.

Hay un matiz sobre acceso entre cuentas: cuando el recurso está en otra cuenta hacen falta las dos políticas —la de identidad en la cuenta que llama y la de recurso en la que responde—. Una sola nunca basta, y el mensaje no siempre distingue cuál falta.

2. Condiciones: donde una política deja de ser un cheque en blanco

Un permiso sin condiciones es válido desde cualquier lugar, a cualquier hora y en cualquier contexto. Las claves de condición acotan eso:

```
{
  "Effect": "Allow",
  "Action": ["s3:GetObject", "s3:PutObject"],
  "Resource": "arn:aws:s3:::cloudshop-facturas/*",
  "Condition": {
    "Bool": {"aws:MultiFactorAuthPresent": "true"},
    "StringEquals": {"aws:PrincipalTag/equipo": "finanzas"},
    "IpAddress": {"aws:SourceIp": ["203.0.113.0/24"]},
    "DateLessThan": {"aws:CurrentTime": "2026-12-31T23:59:59Z"}
  }
}
```

Las cuatro condiciones se combinan con Y lógico: todas deben cumplirse. Dentro de una misma clave con varios valores, la relación es O.

Dos condiciones merecen atención especial:

aws:PrincipalTag habilita control de acceso por atributos: en vez de una política por equipo, una sola política que compara la etiqueta del principal con la del recurso. Escala mucho mejor que enumerar.

aws:SourceIp no funciona como se espera con endpoints de VPC. Cuando la llamada viaja por un endpoint, la IP de origen es privada y la condición falla. Para ese caso hay que usar aws:SourceVpce o aws:SourceVpc, y es un fallo que se descubre justo después de endurecer la red.

Y una condición que evita el confused deputy al conceder acceso a un tercero:

```
"Condition": {"StringEquals": {"sts:ExternalId": "identificador-unico-por-cliente"}}
```

Sin ella, un proveedor que gestiona varias cuentas podría ser inducido a actuar contra la tuya usando su propio rol legítimo.

3. Federación OIDC: desplegar sin ningún secreto

El mecanismo que elimina las claves de acceso de los pipelines, aplicado a AWS. El intercambio, sin secreto compartido en ningún punto:

```
1. El pipeline pide a su plataforma un token OIDC que declara quién es.
2. Llama a sts:AssumeRoleWithWebIdentity presentando ese token.
3. AWS verifica la firma contra las claves públicas del emisor y comprueba
   que las declaraciones cumplen la política de confianza.
4. Devuelve credenciales temporales de 1 hora.
```

La política de confianza es donde se juega todo:

```
{
  "Effect": "Allow",
  "Principal": {"Federated": "arn:aws:iam::123456789012:oidc-provider/token.actions.githubusercontent.
com"},
  "Action": "sts:AssumeRoleWithWebIdentity",
  "Condition": {
    "StringEquals": {
      "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
      "token.actions.githubusercontent.com:sub": "repo:miorg/cloudshop:environment:produccion"
    }
```

```
  }
}
```

Sin la condición sobre sub, cualquier repositorio de GitHub del mundo puede asumir el rol. La firma será válida porque procede del mismo emisor; lo único que distingue a tu repositorio de otro es esa declaración.

Los errores por grado de peligro:

```
sin condición sobre sub                    → cualquiera, en cualquier parte
"sub": "repo:miorg/*"                      → cualquier repositorio de tu organización
StringLike "repo:miorg/cloudshop:*"        → cualquier rama, incluida una de un fork
StringEquals "...:environment:produccion"  → correcto
```

Usar environment en lugar de ref añade una capa: los entornos de GitHub admiten revisores obligatorios, así que el rol solo es asumible tras una aprobación humana.

4. Límites de permisos: delegar sin permitir escalada

El problema: quieres que los equipos creen sus propios roles sin abrir la puerta a que se concedan AdministratorAccess. Conceder iam:CreateRole y iam:AttachRolePolicy es, de hecho, conceder administrador.

Un límite de permisos define el techo de lo que una identidad puede hacer, con independencia de lo que sus políticas concedan:

```
// Límite: nada fuera de estos servicios, y prohibido tocar IAM salvo lo mínimo
{
  "Version": "2012-10-17",
  "Statement": [
    {"Effect": "Allow", "Action": ["s3:*", "dynamodb:*", "logs:*", "sqs:*"], "Resource": "*"},
    {"Effect": "Deny", "Action": ["iam:*", "organizations:*", "account:*"], "Resource": "*"}
  ]
}
```

Y la política que permite delegar obligando a aplicar ese límite:

```
{
  "Effect": "Allow",
  "Action": ["iam:CreateRole", "iam:AttachRolePolicy"],
  "Resource": "arn:aws:iam::*:role/equipos/*",
  "Condition": {
    "StringEquals": {"iam:PermissionsBoundary": "arn:aws:iam::123456789012:policy/limite-equipos"}
  }
}
```

La condición es la clave: solo se pueden crear roles que lleven el límite adjunto. Sin ella, el equipo crearía roles sin límite y la delegación sería una escalada.

Falta una pieza para cerrarlo del todo:

```
{"Effect": "Deny", "Action": ["iam:DeleteRolePermissionsBoundary", "iam:PutRolePermissionsBoundary"],
"Resource": "*"}
```

Sin esta denegación, el equipo podría crear el rol con límite y quitárselo después. Es el hueco que convierte un diseño correcto en uno inútil, y no es evidente hasta que alguien lo prueba.

5. Privilegio mínimo con las herramientas de AWS

Adivinar la lista de permisos produce o bien exceso o bien bloqueo. AWS ofrece tres herramientas para partir de lo observado:

```
# 1. Qué servicios ha usado realmente un rol
$ aws iam generate-service-last-accessed-details --arn arn:aws:iam::...:role/ci-deploy
$ aws iam get-service-last-accessed-details --job-id ... \
    --query 'ServicesLastAccessed[?TotalAuthenticatedEntities>`0`].[ServiceName,LastAuthenticated]'

# 2. Generar una política a partir de la actividad de CloudTrail
$ aws accessanalyzer start-policy-generation --policy-generation-details \
    '{"principalArn":"arn:aws:iam::...:role/ci-deploy"}' --cloud-trail-details ...

# 3. Validar la política antes de aplicarla
$ aws accessanalyzer validate-policy --policy-document file://politica.json \
    --policy-type IDENTITY_POLICY --query 'findings[?findingType==`SECURITY_WARNING`]'
```

La tercera detecta patrones peligrosos que pasan desapercibidos: comodines en el principal, iam:PassRole sin restricción de recurso, o acciones que permiten escalada.

iam:PassRole merece atención propia. Permite entregar un rol a un servicio, y sin restricción de recurso equivale a conceder cualquier permiso que exista en la cuenta: basta con pasar un rol de administrador a una función o a una instancia que tú controlas.

```
// Peligroso
{"Effect": "Allow", "Action": "iam:PassRole", "Resource": "*"}

// Correcto: solo roles concretos y solo al servicio esperado
{
  "Effect": "Allow", "Action": "iam:PassRole",
  "Resource": "arn:aws:iam::*:role/cloudshop-tarea-*",
  "Condition": {"StringEquals": {"iam:PassedToService": "ecs-tasks.amazonaws.com"}}
}
```

Y el límite del método basado en registros, ya señalado en la clase 018: los caminos de error usan permisos distintos. Un rol que escribe en una cola pero no en su cola de mensajes fallidos funciona hasta el primer fallo.

Ejemplo trabajado

El pipeline de CloudShop despliega en producción con una clave de acceso de 2 años y medio y PowerUserAccess. Se migra a OIDC con privilegio mínimo, verificando cada paso.

Estado inicial:

```
$ aws iam list-access-keys --user-name ci-deploy \
    --query 'AccessKeyMetadata[0].[CreateDate,Status]' --output text
2023-03-14T10:22:00Z   Active
$ aws iam list-attached-user-policies --user-name ci-deploy --query 'AttachedPolicies[].PolicyName'
["PowerUserAccess"]
```

```
antigüedad          2 años 5 meses, 0 rotaciones
copias conocidas    secreto del repositorio, 2 portátiles, 1 runbook
permisos            ~8.000 acciones
ventana si se filtra ilimitada
```

Paso 1 — proveedor OIDC y prueba de la política de confianza insegura, para verla fallar:

```
$ aws iam create-open-id-connect-provider \
    --url https://token.actions.githubusercontent.com --client-id-list sts.amazonaws.com
```

Con la condición solo sobre aud, desde un repositorio de laboratorio ajeno a la organización:

```
$ aws sts assume-role-with-web-identity --role-arn ...:role/ci-deploy-oidc ...
{"Credentials": {...}}          ← ÉXITO desde un repositorio ajeno
```

Se corrige acotando el sujeto al entorno:

```
$ aws iam update-assume-role-policy --role-name ci-deploy-oidc \
    --policy-document file://confianza.json    # sub: repo:miorg/cloudshop:environment:produccion
```

Pruebas negativas y positiva:

```
desde otro repositorio                → AccessDenied  ✓
desde miorg/cloudshop, rama de trabajo → AccessDenied  ✓
desde miorg/cloudshop, entorno prod    → credenciales de 1 h  ✓
```

Paso 2 — recortar permisos con lo observado en 30 días:

```
$ aws accessanalyzer start-policy-generation --policy-generation-details \
    '{"principalArn":"arn:aws:iam::123456789012:user/ci-deploy"}'
$ aws accessanalyzer get-generated-policy --job-id ... \
    --query 'generatedPolicyResult.generatedPolicies[0].policy' | jq -r '.Statement[].Action' | wc -l
34
```

```
permisos concedidos antes    ~8.000 acciones
acciones realmente usadas        34
servicios usados                  5   (s3, cloudfront, ecs, ecr, logs)
```

Se ejercitan también los caminos de error antes de aplicar. Aparecen dos acciones más que ningún despliegue correcto usaba:

```
ecs:StopTask                al abortar un despliegue fallido
logs:PutLogEvents sobre el grupo de errores
```

Paso 3 — validar la política antes de aplicarla:

```
$ aws accessanalyzer validate-policy --policy-document file://ci-deploy.json \
    --policy-type IDENTITY_POLICY \
    --query 'findings[?findingType==`SECURITY_WARNING`].[issueCode,learnMoreLink]' --output text
PASS_ROLE_WITH_STAR_IN_RESOURCE   https://docs.aws.amazon.com/...
```

El validador detecta el fallo que la generación automática no evita: iam:PassRole con Resource: "", que permite pasar cualquier rol —incluido uno de administrador— a un servicio controlado por el pipeline. Se acota:

```
{
  "Effect": "Allow", "Action": "iam:PassRole",
  "Resource": "arn:aws:iam::*:role/cloudshop-tarea-*",
  "Condition": {"StringEquals": {"iam:PassedToService": "ecs-tasks.amazonaws.com"}}
}
```

```
$ aws accessanalyzer validate-policy ... --query 'findings[?findingType==`SECURITY_WARNING`]'
[]                                                              ✓
```

Paso 4 — retirar la credencial, con observación previa:

```
$ aws iam update-access-key --user-name ci-deploy --access-key-id AKIA... --status Inactive
# 7 días sin fallos ni llamadas registradas con esa clave
$ aws iam delete-access-key --user-name ci-deploy --access-key-id AKIA...
$ aws iam delete-user --user-name ci-deploy
```

Resultado:

```
credencial               permanente, 2,5 años   token de 1 h
secretos almacenados     4 copias               0
acciones permitidas      ~8.000                 36
iam:PassRole             sin restricción        1 patrón, 1 servicio
superficie si se filtra  ilimitada              1 h, solo ese repo y entorno
```

El paso que más aportó fue el validador, no la generación automática: la política generada a partir del uso real contenía un PassRole sin acotar que habría permitido escalar a administrador desde el propio pipeline.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/026-iam-roles-politicas-sts-y-federacion/lab.py
```

El laboratorio selecciona el motor de práctica iam y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa politica-iam-aws en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una matriz de acceso mínimo con prueba de denegación. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye politica-iam-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.

- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

■■ Errores frecuentes

Síntoma	Causa probable	Corrección
Cualquier repositorio de GitHub puede asumir el rol de despliegue	La política de confianza no condiciona la declaración sub	Condiciona con StringEquals sobre repositorio y entorno; repo:org/ o StringLike con comodín no bastan.
Se conceden más permisos y el acceso sigue denegado	El deny está en una SCP o en un límite de permisos	Lee el mensaje de error: nombra el documento exacto que deniega.
Una política endurecida con aws:SourceIp rompe el acceso desde la VPC	A través de un endpoint de VPC la IP de origen es privada	Usa aws:SourceVpce o aws:SourceVpc para tráfico que viaja por endpoints.
Un equipo con permiso para crear roles se concede administrador	Se delegó iam:CreateRole sin exigir límite de permisos ni impedir retirarlo	Condiciona con iam:PermissionsBoundary y deniega Put/DeleteRolePermissionsBoundary.
Un pipeline con permisos mínimos consigue privilegios de administrador	iam:PassRole con Resource permite pasar cualquier rol a un servicio propio	Acota PassRole por patrón de recurso y por iam:PassedToService.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. Un error dice «explicit deny in a service control policy». ¿Sirve de algo añadir permisos al rol? ¿Qué documento revisas?
2. ¿Por qué aws:SourceIp deja de funcionar cuando el tráfico pasa por un endpoint de VPC, y qué clave se usa entonces?
3. ¿Qué dos piezas hacen falta para delegar la creación de roles sin permitir escalada de privilegios?
4. ¿Por qué iam:PassRole con Resource: "" equivale a conceder administrador?
5. ¿Qué distingue exactamente tu repositorio del de un atacante en un intercambio OIDC?

Referencias

- AWS (2024). Policy evaluation logic — orden de evaluación completo con SCP y límites de permisos. <https://docs.aws.amazon.com/IAM/latest/UserGuide/referencepoliciesevaluation-logic.html>
- AWS (2024). Global condition context keys — catálogo de claves de condición y su comportamiento. <https://docs.aws.amazon.com/IAM/latest/UserGuide/referencepoliciescondition-keys.html>
- AWS (2024). Permissions boundaries for IAM entities — delegación segura de creación de roles. <https://docs.aws.amazon.com/IAM/latest/UserGuide/accesspoliciesboundaries.html>
- AWS (2024). IAM Access Analyzer policy validation — comprobaciones de seguridad antes de aplicar una política. <https://docs.aws.amazon.com/IAM/latest/UserGuide/access-analyzer-policy-validation.html>
- Hardt, D., ed. (2012). RFC 6749: The OAuth 2.0 Authorization Framework — base del intercambio de token por credenciales. <https://www.rfc-editor.org/rfc/rfc6749>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Anterior	Índice	Siguiente
← 025 · Organizations, cuentas, OU, SCP y landing zone	Parte 02 · Programa	027 · VPC, subredes, rutas, NAT, endpoints y seguridad →

Evaluación — 026 IAM, roles, políticas, STS y federación

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega politica-iam-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

027 — VPC, subredes, rutas, NAT, endpoints y seguridad

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

Propósito

Diseñar una VPC entendiendo qué componente decide cada cosa: la tabla de rutas determina por dónde sale el tráfico, el grupo de seguridad quién puede hablar, y la elección entre NAT y endpoint decide una partida de la factura que suele sorprender. Es la base de las clases 028 a 036 y el origen de la mayoría de incidentes de conectividad del programa.

Resultados de aprendizaje

Al finalizar podrás:

1. Planificar un rango CIDR que admita crecimiento y no colisione con la red corporativa ni con futuros pares.
2. Determinar si una subred es pública o privada por su tabla de rutas, no por su nombre.
3. Elegir entre grupo de seguridad y ACL de red sabiendo cuál tiene estado y cuál no.
4. Calcular el ahorro de sustituir tráfico por NAT gateway por endpoints de VPC.
5. Diagnosticar un fallo de conectividad recorriendo las capas en orden y con los registros de flujo.

Conceptos centrales

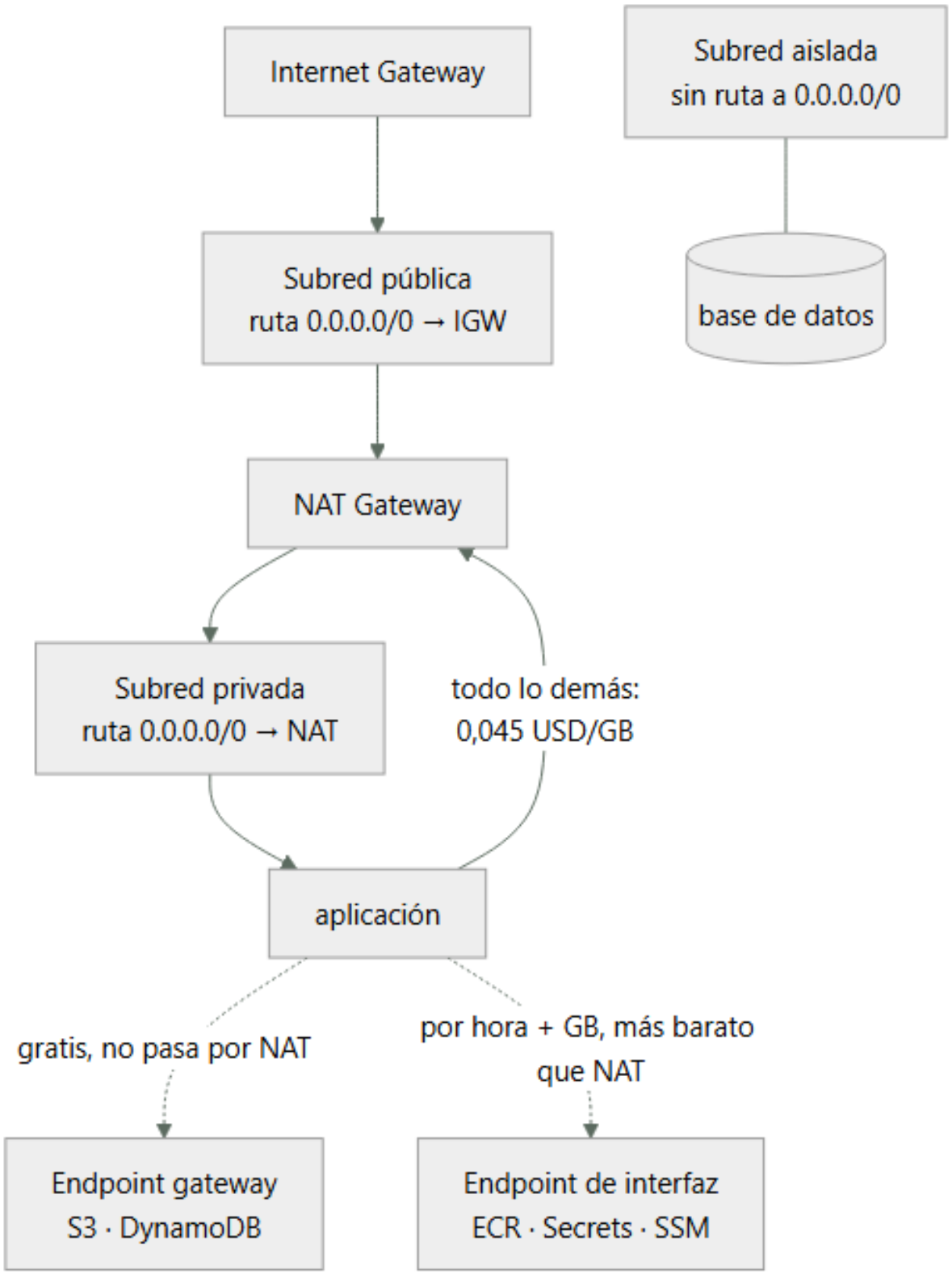
Concepto	Comprensión verificable
tabla de rutas	Conjunto de reglas que decide por dónde sale el tráfico de una subred. Es lo único que hace pública a una subred: una ruta hacia una puerta de enlace de internet.
grupo de seguridad	Cortafuegos con estado a nivel de interfaz de red. Solo tiene reglas de permiso y la respuesta al tráfico permitido vuelve automáticamente, sin regla de salida que la autorice.
ACL de red	Cortafuegos sin estado a nivel de subred. Admite denegaciones explícitas, y al no tener estado exige regla de vuelta para los puertos efímeros.
NAT gateway	Servicio que permite salida a internet desde subredes privadas. Cobra por hora y por GB procesado, lo que lo convierte en una de las partidas más caras y menos vigiladas.
endpoint de VPC	Ruta privada hacia un servicio de AWS sin pasar por internet. Los de tipo gateway —S3 y DynamoDB— son gratuitos; los de interfaz cobran por hora y por GB, pero menos que el NAT.

Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Planificar el CIDR antes de crear nada

El rango de una VPC no se puede cambiar; solo se pueden añadir bloques secundarios, con restricciones. Un error aquí se paga durante años.

Tres reglas que evitan los problemas más caros:

1. No solapar con nada con lo que puedas necesitar hablar. Si la red corporativa usa 10.0.0.0/8 y creas la VPC en 10.0.0.0/16, la VPN o el emparejamiento serán imposibles: el enrutamiento no admite solapamiento. Un registro central de rangos asignados es más barato que una migración.

2. Dimensionar con holgura. AWS admite de /16 a /28. Un /16 da 65.536 direcciones y no cuesta nada reservarlo; un /24 se agota en cuanto se añaden zonas o servicios que consumen direcciones.

3. Contar las direcciones que AWS reserva. En cada subred se reservan 5, no 2:

```
Subred 10.20.1.0/24 → 256 direcciones teóricas
  .0    dirección de red
  .1    puerta de enlace de la VPC
  .2    servidor DNS
  .3    reservada para uso futuro
  .255  difusión
  ----------------------------
  251 direcciones utilizables
```

Un esquema que crece sin colisiones:

```
VPC produccion     10.20.0.0/16
  pública    zona a  10.20.0.0/20    4.091 utilizables
  pública    zona b  10.20.16.0/20
  privada    zona a  10.20.32.0/20
  privada    zona b  10.20.48.0/20
  aislada    zona a  10.20.64.0/20
  aislada    zona b  10.20.80.0/20
  libre              10.20.96.0/19   reservado para crecer
```

El bloque libre al final no es desperdicio: es lo que permite añadir una tercera zona sin rehacer el esquema.

2. Pública o privada lo decide la tabla de rutas

Los nombres «pública» y «privada» son convención humana. Lo único que hace pública a una subred es una ruta hacia una puerta de enlace de internet:

```
Subred pública        0.0.0.0/0 → igw-xxxx
Subred privada        0.0.0.0/0 → nat-xxxx
Subred aislada        sin ruta hacia 0.0.0.0/0
```

De ahí un fallo que aparece con regularidad: una subred llamada «privada» con ruta al IGW es pública, y cualquier recurso con IP pública en ella es accesible desde internet. El nombre no protege nada.

La comprobación es directa:

```
$ aws ec2 describe-route-tables \
    --filters Name=association.subnet-id,Values=subnet-0a1b2c \
    --query 'RouteTables[0].Routes[?DestinationCidrBlock==`0.0.0.0/0`].GatewayId' --output text
igw-0f9e8d          # es PÚBLICA, se llame como se llame
```

Y hay una segunda condición para que una instancia sea alcanzable desde internet: necesita IP pública además de la ruta. Una instancia sin IP pública en una subred pública no es accesible desde fuera, aunque sí puede salir si hay NAT.

La tercera categoría —aislada, sin ruta a internet en absoluto— es donde deben vivir las bases de datos. No es lo mismo que privada: una subred privada puede iniciar conexiones salientes a internet a través del NAT, y eso es exactamente lo que usa un atacante para exfiltrar datos o descargar herramientas.

3. Grupo de seguridad y ACL: con estado y sin estado

La diferencia decisiva:

	Grupo de seguridad	ACL de red
Ámbito	Interfaz de red	Subred entera
Estado	Con estado	Sin estado
Reglas	Solo permitir	Permitir y denegar
Evaluación	Todas las reglas	Por orden de número
Referencia a otro grupo	Sí	No, solo CIDR

Con estado significa que la respuesta al tráfico permitido vuelve automáticamente. Si un grupo permite entrada al 443, la respuesta sale sin necesidad de regla de salida.

Sin estado significa que hay que autorizar ambos sentidos. Este es el error clásico de las ACL:

```
ACL con entrada 443 permitida y salida solo 443:
  petición entrante al 443          ✓ permitida
  respuesta desde el 443 hacia el
  puerto efímero del cliente (52341) ✗ BLOQUEADA
```

La conexión se establece y se cuelga. Hay que permitir la salida hacia el rango efímero 1024-65535, y es la causa habitual de que «el firewall parece bien configurado y no funciona».

La capacidad de referenciar otro grupo de seguridad en vez de un CIDR es lo que hace mantenible el diseño:

```
$ aws ec2 authorize-security-group-ingress --group-id sg-db \
    --protocol tcp --port 5432 --source-group sg-app
```

Esto dice «lo que esté en sg-app puede hablar con la base de datos», y sigue siendo cierto cuando las instancias cambian de IP, se escalan o se sustituyen. Un CIDR fijo exige mantenimiento cada vez que la topología cambia.

La recomendación práctica: grupos de seguridad para casi todo, ACL solo para denegaciones amplias —bloquear un rango de IP concreto—, porque su falta de estado las hace fáciles de configurar mal.

4. NAT gateway: la partida cara que nadie mira

Un NAT gateway cobra dos veces: por hora de existencia y por GB procesado. La segunda es la que sorprende.

```
precio por hora            ~0,045 USD  →  32,85 USD/mes
precio por GB procesado    ~0,045 USD
```

Con 8 TB mensuales de tráfico saliente hacia servicios de AWS:

```
8.000 GB × 0,045 = 360 USD/mes solo de procesamiento
más 32,85 de la hora, por cada NAT
con un NAT por zona (3 zonas): 98,55 + 360 = 458,55 USD/mes
```

Y lo importante: buena parte de ese tráfico no necesita salir a internet. Descargar imágenes de ECR, leer secretos, escribir logs o hablar con S3 son llamadas a servicios de AWS que un endpoint resuelve por la red interna.

```
tráfico por NAT gateway            0,045 USD
tráfico por endpoint de interfaz   0,010 USD
tráfico por endpoint gateway       0,000 USD   ← S3 y DynamoDB
```

El endpoint gateway es gratuito y se instala como una ruta en la tabla; no tener uno para S3 es dinero regalado sin ninguna contrapartida. El de interfaz cuesta 0,01 USD por hora por zona más 0,01 por GB, así que compensa a partir de un volumen modesto:

```
umbral del endpoint de interfaz (1 zona):
  7,30 USD/mes de coste fijo / (0,045 − 0,010) USD/GB ≈ 209 GB/mes
```

Por encima de 209 GB mensuales hacia ese servicio, el endpoint sale más barato. Y además del ahorro, aporta seguridad: el tráfico no sale de la red de AWS, lo que permite subredes aisladas que hablan con los servicios que necesitan sin ninguna ruta a internet.

5. Diagnóstico de conectividad, de abajo hacia arriba

El orden ahorra horas porque cada paso descarta una capa completa:

```
# 1. ¿Existe ruta hacia el destino?
$ aws ec2 describe-route-tables --filters Name=association.subnet-id,Values=subnet-xxx \
    --query 'RouteTables[0].Routes[].[DestinationCidrBlock,GatewayId,NatGatewayId]' --output text

# 2. ¿Lo permite la ACL de la subred, en AMBOS sentidos?
$ aws ec2 describe-network-acls --filters Name=association.subnet-id,Values=subnet-xxx \
    --query 'NetworkAcls[0].Entries[?RuleNumber<`32767`]'

# 3. ¿Lo permite el grupo de seguridad de origen y el de destino?
$ aws ec2 describe-security-groups --group-ids sg-app sg-db \
    --query 'SecurityGroups[].[GroupId,IpPermissions[].FromPort]'

# 4. Analizador de accesibilidad: evalúa la ruta completa
$ aws ec2 create-network-insights-path --source i-origen --destination i-destino \
    --protocol tcp --destination-port 5432
```

```
$ aws ec2 start-network-insights-analysis --network-insights-path-id nip-xxx
```

El paso 4 es el más rentable y el menos usado: evalúa la configuración completa y dice qué componente bloquea, sin necesidad de tráfico real.

Y cuando la configuración parece correcta pero algo falla, los registros de flujo dan el veredicto:

```
$ aws logs filter-log-events --log-group-name /aws/vpc/flowlogs \
    --filter-pattern '[version, account, eni, source, destination, srcport,
                       destport=5432, protocol, packets, bytes, start, end,
                       action=REJECT, status]' --max-items 5
```

Un REJECT indica que una regla lo bloqueó y hay que revisar grupos y ACL. La ausencia total de registros es un diagnóstico distinto y más útil: significa que el paquete nunca llegó a la interfaz, así que el problema está en enrutamiento o resolución de nombres, no en el firewall.

Ejemplo trabajado

La factura de red de CloudShop es de 1.180 USD/mes y nadie sabe explicarla. Además, la base de datos está en una subred llamada «privada». Se auditan ambas cosas.

Parte 1 — de dónde viene el gasto:

```
$ aws ce get-cost-and-usage --time-period Start=2026-07-01,End=2026-08-01 \
    --granularity MONTHLY --metrics UnblendedCost \
    --filter '{"Dimensions":{"Key":"USAGE_TYPE_GROUP","Values":["EC2: NAT Gateway"]}}' \
    --query 'ResultsByTime[0].Total.UnblendedCost.Amount' --output text
842.31
```

842 USD de 1.180 son NAT gateway. Se desglosa qué tráfico lo atraviesa con los registros de flujo:

```
destino                         GB/mes    % del tráfico por NAT
S3 (misma región)                9.140          49 %
ECR (descarga de imágenes)       4.620          25 %
Secrets Manager y SSM              310           2 %
CloudWatch Logs                  1.850          10 %
internet real (APIs de terceros) 2.680          14 %
                                 ------
total                           18.600 GB
```

El 86 % del tráfico que paga NAT no va a internet: va a servicios de AWS de la misma región.

Cálculo del cambio a endpoints:

```
actual
  3 NAT × 32,85 USD/mes de hora            =    98,55
  18.600 GB × 0,045 USD                    =   837,00
                                              --------
                                                935,55

con endpoints
  gateway S3 (gratis)          9.140 GB   →      0,00
  interfaz ECR   3 zonas × 7,30 + 4.620 × 0,010 = 68,10
  interfaz Secrets+SSM  2 svc × 3 × 7,30 + 310 × 0,010 = 46,90
  interfaz Logs  3 × 7,30 + 1.850 × 0,010  =    40,40
  NAT solo para internet real
    3 × 32,85 + 2.680 × 0,045              =   219,15
                                              --------
                                                374,55

ahorro: 935,55 − 374,55 = 561 USD/mes  (−60 %)
```

Se verifica el umbral antes de crear cada endpoint de interfaz, para no añadir coste fijo sin retorno:

```
Secrets Manager: 310 GB/mes
  coste fijo 3 zonas: 21,90 USD
  ahorro por GB: 310 × (0,045 − 0,010) = 10,85 USD
  → NO compensa por volumen, pero SÍ por seguridad: permite que la subred
    aislada lea secretos sin ninguna ruta a internet. Se crea igualmente,
    con el sobrecoste de 11 USD/mes declarado como decisión de seguridad.
```

Parte 2 — la subred «privada» de la base de datos:

```
$ aws ec2 describe-route-tables \
    --filters Name=association.subnet-id,Values=subnet-0db1 \
```

```
    --query 'RouteTables[0].Routes[].[DestinationCidrBlock,GatewayId,NatGatewayId]' --output text
10.20.0.0/16    local     None
0.0.0.0/0       None      nat-0a7f
```

No es pública —sale por NAT, no por IGW— pero sí puede iniciar conexiones salientes a internet. Para una base de datos eso es un camino de exfiltración que no aporta nada:

```
$ aws ec2 create-route-table --vpc-id vpc-0c1d --tag-specifications \
    'ResourceType=route-table,Tags=[{Key=Name,Value=aislada}]'
# sin ruta 0.0.0.0/0; solo local y los endpoints necesarios
$ aws ec2 associate-route-table --route-table-id rtb-nueva --subnet-id subnet-0db1
```

Prueba negativa desde la instancia de base de datos:

```
$ curl -m 5 https://example.com
curl: (28) Connection timed out                    ✓ sin salida a internet
$ aws secretsmanager get-secret-value --secret-id cloudshop/db --query Name
"cloudshop/db"                                     ✓ sigue accediendo por endpoint
```

Resultado:

```
coste de red                        1.180 USD     619 USD    (−48 %)
tráfico por NAT                    18.600 GB     2.680 GB
salida a internet desde la BD          sí            no
endpoints                               0             5
```

El diagnóstico no fue de red sino de rutas. El 86 % del gasto venía de que todo el tráfico interno salía por un componente pensado para internet, y la base de datos tenía un camino de salida que nadie había decidido darle: lo heredó de la tabla de rutas por defecto.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/027-vpc-subredes-rutas-nat-endpoints-y-seguridad/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa vpc-aws en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye vpc-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.

- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

■■ Errores frecuentes

Síntoma	Causa probable	Corrección
Una subred llamada «privada» resulta accesible desde internet	Su tabla de rutas apunta a la puerta de enlace de internet: el nombre no decide nada	Verifica la ruta 0.0.0.0/0; pública, privada y aislada se distinguen solo por ahí.
Una conexión se establece y se cuelga pese a que el firewall permite el puerto	La ACL de red no tiene estado y falta permitir la vuelta por el rango efímero	Autoriza 1024-65535 en el sentido de retorno, o usa grupos de seguridad, que sí tienen estado.
La factura de NAT gateway es la mayor partida de red	El tráfico hacia servicios de AWS de la misma región atraviesa el NAT y paga 0,045 USD/GB	Endpoint gateway para S3 y DynamoDB (gratis) y de interfaz por encima de 209 GB/mes.
No se puede ampliar el rango de la VPC ni emparejar con la red corporativa	El CIDR se solapa y no se puede cambiar tras la creación	Planifica el rango contra un registro central antes de crear nada; deja bloques libres para crecer.
Una base de datos puede iniciar conexiones salientes a internet	Está en subred privada con ruta al NAT, heredada de la tabla por defecto	Usa subred aislada sin ruta a 0.0.0.0/0 y endpoints para lo que necesite; verifica con prueba negativa.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Cuántas direcciones utilizables tiene una subred /24 en AWS y por qué no son 254?
2. ¿Qué convierte a una subred en pública, y basta con eso para que una instancia sea alcanzable desde internet?
3. Una ACL permite entrada al 443 y salida solo por el 443. ¿Qué ocurre y por qué?
4. A partir de cuántos GB mensuales compensa un endpoint de interfaz frente al NAT, y de dónde sale ese número?
5. Los registros de flujo no muestran ninguna entrada para una conexión fallida. ¿Qué te dice eso frente a un REJECT?

Referencias

- AWS (2024). VPC user guide: subnets and routing — direcciones reservadas y tablas de rutas. <https://docs.aws.amazon.com/vpc/latest/userguide/configure-subnets.html>
- AWS (2024). Compare security groups and network ACLs — con estado frente a sin estado. <https://docs.aws.amazon.com/vpc/latest/userguide/infrastructure-security.html>
- AWS (2024). AWS PrivateLink and VPC endpoints — tipos de endpoint, precios y restricciones. <https://docs.aws.amazon.com/vpc/latest/privatelink/what-is-privatelink.html>
- AWS (2024). VPC Flow Logs — formato de registro y campos para diagnóstico. <https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs.html>
- AWS (2024). Reachability Analyzer — análisis estático de la ruta entre dos recursos. <https://docs.aws.amazon.com/vpc/latest/reachability/what-is-reachability-analyzer.html>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 02 en PDF · Recorrido de AWS en PDF · Manual integral

Evaluación — 027 VPC, subredes, rutas, NAT, endpoints y seguridad

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega vpc-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

028 — EC2, AMI, EBS y selección de capacidad

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: compute · Estado: EXECUTABLECORE

Propósito

Elegir capacidad de cómputo con criterio medible en vez de por costumbre. La familia de instancia, el tipo de volumen y el modelo de compra son tres decisiones independientes, y equivocarse en cualquiera produce o bien un cuello de botella invisible o bien una factura que triplica lo necesario.

Resultados de aprendizaje

Al finalizar podrás:

1. Descifrar el nombre de una instancia y deducir familia, generación, procesador y capacidades adicionales.
2. Elegir tipo de volumen a partir de IOPS, throughput y patrón de acceso, no por tamaño.
3. Detectar el agotamiento de créditos de una instancia ráfaga y decidir entre modo ilimitado o cambiar de familia.
4. Calcular el umbral de utilización a partir del cual compensa un plan de ahorro frente a bajo demanda.
5. Construir una AMI reproducible y explicar por qué el estado local es el enemigo de la elasticidad.

Conceptos centrales

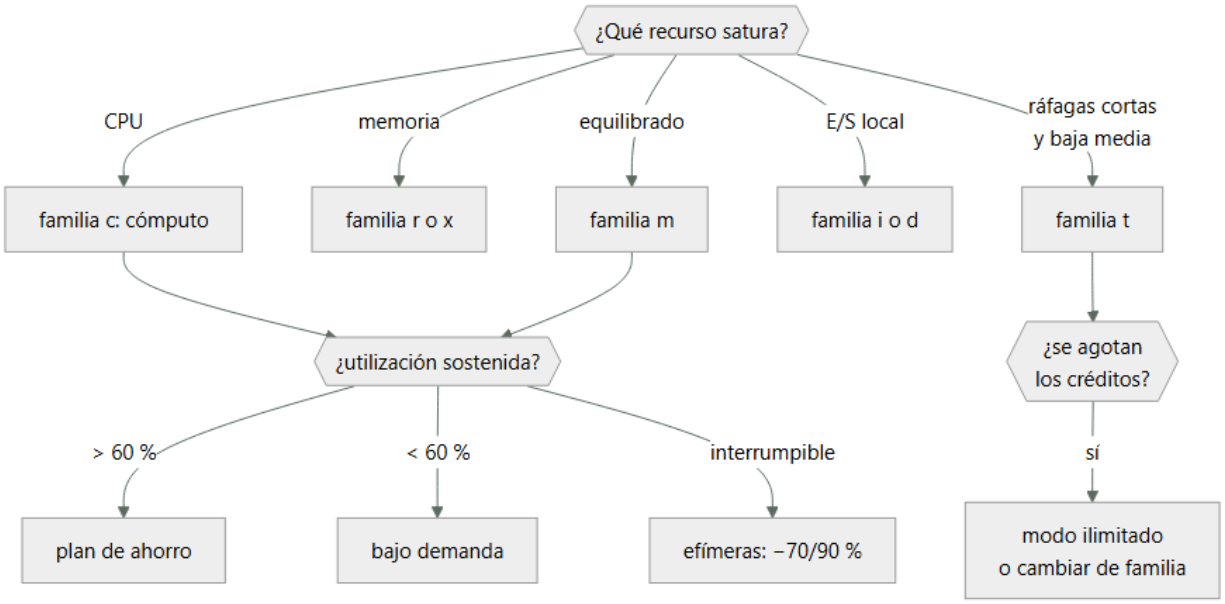
Concepto	Comprensión verificable
familia de instancia	Perfil de recursos optimizado para un uso: propósito general, cómputo, memoria, almacenamiento o aceleración. Elegir mal la familia produce que se pague de más en la dimensión que no se usa.
crédito de CPU	Saldo que acumulan las instancias ráfaga mientras están ociosas y gastan al trabajar. Al agotarse, el rendimiento cae a la línea base sin que ninguna métrica de la aplicación lo explique.
IOPS	Operaciones de entrada/salida por segundo. Junto con el throughput y el tamaño de bloque determina el rendimiento real de un volumen: 16.000 IOPS de 4 KB no equivalen a 16.000 de 256 KB.
AMI	Imagen de máquina que define el estado inicial de una instancia. Construirla de forma reproducible es lo que permite sustituir instancias en lugar de repararlas.
instancia efímera	Instancia que puede reclamarse con dos minutos de preaviso a cambio de un descuento de hasta el 90 %. Correcta para trabajo interrumpible; nunca para el camino crítico de una petición.

Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. El nombre de la instancia es una especificación

m7g.2xlarge no es un código arbitrario: cada carácter dice algo, y leerlo evita elegir a ciegas.

```
m      familia: propósito general (c=cómputo, r=memoria, i=E/S, t=ráfaga)
7      generación: cuanto mayor, mejor relación precio-rendimiento
g      procesador: g=Graviton (ARM), i=Intel, a=AMD; sin letra = Intel
.2xlarge  tamaño: 8 vCPU y 32 GB
```

Sufijos adicionales que cambian el precio y el comportamiento:

Sufijo	Significa
d	Almacenamiento NVMe local, efímero
n	Ancho de banda de red mejorado
e	Memoria o almacenamiento ampliados
flex	Rendimiento sostenido reducido, más barato

La generación es la palanca más rentable y la más ignorada: pasar de la 5 a la 7 suele dar mejor rendimiento por menos dinero, y el cambio no requiere tocar la aplicación salvo por la arquitectura del procesador.

Sobre Graviton: es ARM, así que exige binarios para arm64. Para lenguajes interpretados o con máquina virtual —Python, Java, Node— el cambio suele ser transparente; para binarios compilados o imágenes de contenedor hay que reconstruir. La relación precio-rendimiento típicamente mejora entre un 20 % y un 40 %, así que la reconstrucción se amortiza rápido en flotas grandes.

El tamaño escala linealmente dentro de la familia: 2xlarge es exactamente el doble de xlarge en vCPU, memoria, red y precio. Eso hace que la aritmética de dimensionado sea directa.

2. Créditos de CPU: el desplome que no aparece en las métricas

Las instancias de la familia t no ofrecen su vCPU completa de forma continua. Tienen una línea base —un porcentaje del rendimiento nominal— y acumulan créditos mientras consumen por debajo de ella.

```
t3.medium: línea base 20 % por vCPU
  ociosa al 5 %   → acumula créditos
  al 80 % de CPU  → gasta créditos
  saldo a cero    → el rendimiento se limita al 20 %
```

El síntoma es característico y desconcierta: la aplicación se vuelve cuatro o cinco veces más lenta sin ningún cambio en el código, la carga ni la configuración. Ninguna métrica del sistema operativo lo explica, porque desde dentro la CPU parece ocupada y disponible.

El diagnóstico exige una métrica del proveedor, no del sistema:

```
$ aws cloudwatch get-metric-statistics --namespace AWS/EC2 \
    --metric-name CPUCreditBalance --dimensions Name=InstanceId,Value=i-0a1b2c \
    --start-time 2026-08-01T00:00:00Z --end-time 2026-08-01T12:00:00Z \
    --period 300 --statistics Minimum --query 'Datapoints[?Minimum<`10`]|length(@)'
17
```

Diecisiete intervalos con el saldo por debajo de 10 créditos: la instancia lleva horas limitada.

Dos salidas, con consecuencias distintas:

```
modo ilimitado   permite superar la línea base pagando el exceso.
                 Correcto para picos ocasionales; ruinoso si la carga es
                 sostenida, porque el sobrecargo puede superar el precio
                 de una instancia normal.
cambiar familia  si la utilización media supera la línea base de forma
                 sostenida, la familia t es la elección equivocada.
```

La regla: la familia t es para cargas con media baja y picos cortos. Si la media supera la línea base, el descuento desaparece y el riesgo permanece.

3. Volúmenes: IOPS, throughput y tamaño de bloque

Elegir volumen por tamaño es el error más común. Lo que decide el rendimiento son tres números y su interacción:

Tipo	IOPS máx	Throughput máx	Precio relativo	Para
gp3	16.000	1.000 MB/s	1,0	Propósito general; IOPS y throughput independientes del tamaño
gp2	3 por GB, tope 16.000	250 MB/s	1,25	Generación anterior: el rendimiento depende del tamaño
io2	256.000	4.000 MB/s	3-5	Bases de datos exigentes, con durabilidad mayor
st1	500	500 MB/s	0,3	Lectura secuencial grande: logs, big data

La diferencia entre gp2 y gp3 es económicamente relevante: en gp2 hay que sobredimensionar el tamaño para obtener IOPS. Para 6.000 IOPS hacen falta 2.000 GB aunque solo se usen 200. En gp3 se configuran por separado:

```
gp2 para 6.000 IOPS:  2.000 GB × 0,10 USD/GB          = 200,00 USD/mes
gp3 equivalente:       200 GB × 0,08 + 3.000 IOPS extra × 0,005
                    =  16,00 + 15,00                  =  31,00 USD/mes
```

Un factor de 6,5 por la misma capacidad efectiva. Migrar de gp2 a gp3 es de las optimizaciones de mayor retorno y menor riesgo que existen.

Y el detalle que descoloca al medir: una operación de E/S se cuenta hasta 256 KB. Una lectura de 1 MB consume 4 IOPS, no 1. Por eso una carga secuencial agota antes el throughput que las IOPS, y una aleatoria de bloques pequeños agota antes las IOPS:

```
16.000 IOPS × 4 KB   =   64 MB/s  ← limita el IOPS
16.000 IOPS × 256 KB = 4.096 MB/s ← limita el throughput (tope 1.000)
```

Hay un segundo límite que se olvida: la instancia también tiene un tope de ancho de banda hacia EBS, independiente del volumen. Un gp3 de 1.000 MB/s conectado a una instancia con 600 MB/s de ancho de banda entrega 600.

4. Modelos de compra y el umbral que los separa

Cuatro formas de pagar el mismo cómputo:

Modelo	Descuento	Compromiso	Riesgo
Bajo demanda	0 %	Ninguno	Ninguno
Savings Plan 1 año	28 %	Gasto por hora, no instancia concreta	Sobredimensionar
Savings Plan 3 años	45 %	Ídem	Cambio tecnológico

Modelo	Descuento	Compromiso	Riesgo
Efímeras	70-90 %	Ninguno	Reclamación con 2 min de aviso

El umbral de utilización a partir del cual compensa comprometerse sale de igualar costes, como en la clase 015:

```
u × 730 h × P = 730 h × 0,72 P   →   u = 0,72
```

Con más del 72 % de uso sostenido, el plan de un año sale a cuenta. Y hay un matiz que lo hace más atractivo de lo que parece: los Savings Plans de cómputo comprometen gasto por hora, no un tipo de instancia. Se puede cambiar de familia, de tamaño y de región sin perder el descuento, lo que elimina buena parte del riesgo de sobredimensionar.

Las instancias efímeras merecen su propio criterio. El descuento es enorme y el riesgo concreto: dos minutos de preaviso. Son correctas para trabajo por lotes reanudable, transcodificación, entrenamiento con puntos de control y entornos de prueba. No lo son para nada que sostenga una petición de usuario, salvo que la arquitectura absorba la pérdida sin degradación visible.

La estrategia habitual en una flota:

```
base estable (60 % de la capacidad)  → Savings Plan
variación previsible (25 %)          → bajo demanda
picos y lotes (15 %)                 → efímeras
```

5. El estado local es el enemigo de la elasticidad

Una instancia que guarda estado en su disco no se puede sustituir: hay que repararla. Y reparar es lo contrario de operar en cloud.

La distinción práctica, con lo que hay que hacer con cada cosa:

```
logs        → fuera del host, a un recolector (clase 007)
sesiones    → almacén compartido, no memoria del proceso
ficheros    → almacenamiento de objetos o sistema de ficheros compartido
caché       → externa; si es local, debe poder perderse sin consecuencias
datos       → base de datos gestionada
configuración → parámetros o secretos, inyectados al arrancar
```

Si todo eso está fuera, una instancia es desechable: se puede terminar y crear otra sin ceremonia. Esa propiedad es la que hacen posibles el autoescalado, los despliegues sin interrupción y el parcheo por sustitución.

Y exige una AMI reproducible. Construirla a mano —arrancar, instalar, guardar imagen— produce una imagen que nadie sabe recrear:

```
# Fragmento de plantilla declarativa
source "amazon-ebs" "cloudshop" {
  source_ami_filter {
    filters = { name = "al2023-ami-*-arm64" }
    owners  = ["amazon"]
    most_recent = true
  }
  instance_type = "t4g.small"
  ami_name      = "cloudshop-{{timestamp}}"
}
```

Dos propiedades que hay que exigir a la imagen:

1. Reproducible: la misma plantilla produce una imagen funcionalmente equivalente.
2. Fechada e inmutable: nunca se sobrescribe una AMI; se crea otra y se cambia la referencia. Es la misma lógica de la clase 003 sobre etiquetas mutables.

Y una advertencia de seguridad: una AMI compartida por error es pública para siempre a efectos prácticos. Antes de compartir hay que comprobar que no contiene claves, historial de shell ni datos residuales.

Ejemplo trabajado

El servicio de catálogo de CloudShop se degrada cada día entre las 11:00 y las 14:00. No hay despliegues ni aumento de tráfico en esa franja.

Las métricas de la aplicación no explican nada:

```
peticiones/s        estables en 340
tasa de error       0,02 %, sin cambio
CPU del sistema     94 % durante la degradación
latencia p95        88 ms → 410 ms
```

La CPU al 94 % sugiere saturación, pero el tráfico no subió. Se mira la métrica del proveedor:

```
$ aws cloudwatch get-metric-statistics --namespace AWS/EC2 \
    --metric-name CPUCreditBalance --dimensions Name=InstanceId,Value=i-0a1b2c \
    --start-time 2026-08-01T08:00:00Z --end-time 2026-08-01T15:00:00Z \
    --period 3600 --statistics Average --query 'Datapoints[].[Timestamp,Average]' --output text | sort
2026-08-01T08:00:00Z    142.0
2026-08-01T10:00:00Z     38.0
2026-08-01T11:00:00Z      0.0        ← agotados
2026-08-01T14:00:00Z      0.0
```

Créditos agotados a las 11:00. Las instancias son t3.large con línea base del 30 %; al agotar el saldo, el rendimiento se limita a esa fracción:

```
rendimiento nominal      2 vCPU
línea base 30 %          0,6 vCPU efectivas
factor de degradación    3,3×
latencia observada       88 × 3,3 = 290 ms teórico; medido 410 con encolamiento
```

Se comprueba la utilización media real para elegir la salida:

```
utilización media diaria    47 %
línea base de t3.large      30 %
```

La media supera la línea base, así que la familia t es la elección equivocada: el modo ilimitado cobraría el exceso todo el día. Se dimensiona una alternativa por la ley de Little:

```
λ = 340 peticiones/s, W = 0,088 s → L = 30 concurrentes
a ρ = 0,70 → 43 concurrentes → con 16 por instancia, 3 instancias
```

Comparación de tres opciones para 3 instancias:

```
3 × t3.large (actual)              6    182,50 USD   no: se limita
3 × t3.large modo ilimitado        6    182,50 + ~95 sobrecargo = 277,50
3 × m7g.large (Graviton)           6    139,00 USD   sí, sostenido
```

m7g.large cuesta menos que la opción actual degradada y sostiene el rendimiento. Requiere reconstruir la imagen para arm64; la aplicación es Python y la reconstrucción resultó transparente salvo por una dependencia con extensión nativa.

Se aprovecha para revisar los volúmenes:

```
$ aws ec2 describe-volumes --filters Name=attachment.instance-id,Values=i-0a1b2c \
    --query 'Volumes[].[VolumeType,Size,Iops]' --output text
gp2     500     1500
```

```
gp2 de 500 GB: 1.500 IOPS, uso real 92 GB → se pagaba tamaño para obtener IOPS
gp3 de 120 GB con 3.000 IOPS configurados:
  gp2: 500 × 0,10                       = 50,00 USD/mes
  gp3: 120 × 0,08                       =  9,60 USD/mes
  (3.000 IOPS entran en la base gratuita)
  ahorro por instancia                   = 40,40 USD/mes
```

Resultado combinado:

```
instancias              3 × t3.large   3 × m7g.large
cómputo                   182,50 USD     139,00 USD
almacenamiento            150,00 USD      28,80 USD
total                     332,50 USD     167,80 USD   (−50 %)
latencia p95 11:00-14:00      410 ms          91 ms
degradación diaria          3 horas        ninguna
```

El diagnóstico no estaba en la aplicación ni en el sistema operativo: estaba en una métrica que solo el proveedor publica. Y la corrección salió más barata que el problema.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/028-ec2-ami-ebs-y-seleccion-de-capacidad/lab.py
```

El laboratorio selecciona el motor de práctica compute y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa instancia-aws en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una selección de capacidad justificada y observable. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye instancia-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

■■ Errores frecuentes

Síntoma	Causa probable	Corrección
El rendimiento se desploma a diario sin cambios de código ni de carga	Agotamiento de créditos de CPU en una instancia ráfaga	Vigila CPUCreditBalance; si la media supera la línea base, cambia de familia en vez de activar modo ilimitado.
Se paga un volumen enorme del que se usa una fracción	En gp2 las IOPS dependen del tamaño, así que hay que sobredimensionar	Migra a gp3 y configura IOPS y throughput por separado; el ahorro suele superar el 60 %.
Un volumen con 1.000 MB/s entrega bastante menos	La instancia tiene su propio tope de ancho de banda hacia EBS	Comprueba el límite de la instancia además del volumen; el menor de los dos manda.
Una instancia no se puede sustituir sin perder datos	Guarda estado local: sesiones, ficheros o logs	Externaliza todo el estado; una instancia que no es desechable impide el autoescalado y el parcheo por sustitución.
Un trabajo en instancias efímeras pierde horas de progreso	Se usó capacidad interrumpible sin puntos de control	Reserva las efímeras para trabajo reanudable con checkpoints; el preaviso es de dos minutos.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. Descompón c7gn.4xlarge: ¿qué dice cada parte del nombre?
2. Una instancia t3 tiene utilización media del 47 % y línea base del 30 %. ¿Conviene modo ilimitado? Justifica.
3. ¿Cuántas IOPS consume una lectura secuencial de 1 MB, y por qué eso cambia qué límite alcanzas antes?
4. ¿Por qué gp3 con 200 GB puede rendir más que gp2 con 2.000 GB y costar seis veces menos?
5. ¿A partir de qué utilización sostenida compensa un Savings Plan de un año, y qué riesgo elimina que comprometa gasto y no tipo de instancia?

Referencias

• AWS (2024). Amazon EC2 instance types — nomenclatura, familias y capacidades adicionales. <https://docs.aws.amazon.com/ec2/latest/instancetypes/instance-types.html>

• AWS (2024). Burstable performance instances — línea base, créditos y modo ilimitado. <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/burstable-performance-instances.html>

• AWS (2024). Amazon EBS volume types — IOPS, throughput y tamaño de operación de E/S. <https://docs.aws.amazon.com/ebs/latest/userguide/ebs-volume-types.html>

• AWS (2024). Savings Plans — compromiso por gasto horario frente a instancias reservadas. <https://docs.aws.amazon.com/savingsplans/latest/userguide/what-is-savings-plans.html>

• AWS (2024). Spot Instances: interruption notices — preaviso de dos minutos y patrones de uso seguro. <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/spot-interruptions.html>

• Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 02 en PDF · Recorrido de AWS en PDF · Manual integral

Anterior	Índice	Siguiente
← 027 · VPC, subredes, rutas, NAT, endpoints y seguridad	Parte 02 · Programa	029 · Elastic Load Balancing y Auto Scaling →

Evaluación — 028 EC2, AMI, EBS y selección de capacidad

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega instancia-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

• [ ] Resultado reproducible por una segunda persona.

• [ ] Evidencia vinculada a cada afirmación técnica.

• [ ] Prueba negativa o de fallo incluida.

- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

029 — Elastic Load Balancing y Auto Scaling

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: reliability · Estado: EXECUTABLECORE

Propósito

Construir una capa elástica que reparta tráfico y ajuste capacidad sin oscilar ni tumbar el servicio al desplegar. Aquí se aplican a AWS los cálculos de la clase 016 —histéresis, margen de arranque, rechazo de carga— y se añade lo que ninguna fórmula anticipa: que el drenado de conexiones y el tipo de comprobación de estado deciden si un despliegue pierde peticiones.

Resultados de aprendizaje

Al finalizar podrás:

1. Elegir entre balanceador de aplicación y de red según protocolo, latencia y necesidad de terminar TLS.
2. Configurar comprobaciones de estado que distingan una instancia arrancando de una averiada.
3. Dimensionar un grupo de autoescalado con umbrales, enfriamiento y periodo de gracia coherentes.
4. Evitar la pérdida de peticiones al reducir capacidad, mediante drenado de conexiones.
5. Diagnosticar un 502 y un 504 del balanceador sabiendo qué componente señala cada uno.

Conceptos centrales

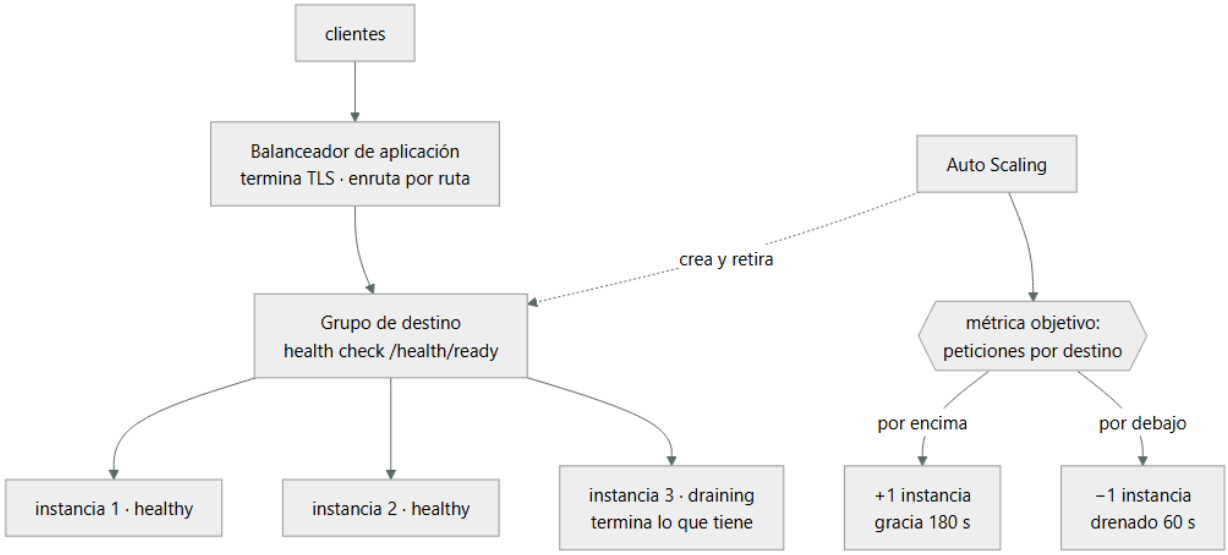
Concepto	Comprensión verificable
grupo de destino	Conjunto de destinos que reciben tráfico, con su propia comprobación de estado y política de enrutamiento. Es la unidad sobre la que actúan el balanceador y el autoescalado.
comprobación de estado	Sondeo periódico que decide si un destino recibe tráfico. Debe reflejar readiness —¿puede servir?— y no liveness, por lo visto en la clase 012.
drenado de conexiones	Periodo durante el cual un destino que se retira deja de recibir peticiones nuevas pero termina las que tiene en curso. Sin él, reducir capacidad corta transacciones a medias.
periodo de gracia	Tiempo que el autoescalado espera antes de evaluar la salud de una instancia recién creada. Corto de más, mata instancias que aún arrancan y entra en bucle.
seguimiento de objetivo	Política de escalado que ajusta capacidad para mantener una métrica en un valor deseado. Gestiona la histéresis automáticamente, a diferencia de las políticas por umbral.

Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Cuatro balanceadores, tres decisiones

	Aplicación (ALB)	Red (NLB)	Gateway (GWLB)
Capa	7 (HTTP)	4 (TCP/UDP)	3 (GENEVE)
Enruta por	Ruta, host, cabecera, método	Puerto	Todo el paquete
Termina TLS	Sí	Sí, opcionalmente	No
IP fija	No	Sí, por zona	No
Latencia añadida	5-10 ms	1 ms	Depende
Preserva IP de origen	En cabecera X-Forwarded-For	En el paquete	Sí

Tres criterios deciden:

Protocolo. Si es HTTP y hace falta enrutar por ruta o host, ALB. Si es TCP, UDP o un protocolo propio, NLB.

IP fija. El NLB tiene una IP estática por zona; el ALB resuelve por DNS y sus IP cambian. Esto importa cuando un cliente externo debe incluir tu servicio en una lista de permitidos: con ALB no hay IP estable que dar.

IP de origen. El NLB preserva la IP del cliente en el propio paquete; el ALB la pone en X-Forwarded-For. Una aplicación que aplica límites de tasa por IP y lee la IP de la conexión verá la del balanceador —y limitará a todos como si fueran uno— salvo que lea la cabecera. Es un fallo que solo aparece bajo ataque.

Y un detalle de coste: ambos cobran por hora y por unidad de capacidad consumida, una métrica compuesta de conexiones nuevas, conexiones activas, ancho de banda y reglas evaluadas. La dimensión que domine determina la factura, y suele ser conexiones nuevas por segundo en servicios que no reutilizan conexión —lo de la clase 005—.

2. La comprobación de estado debe reflejar readiness

Un balanceador retira del reparto lo que falla la comprobación. Por tanto la comprobación debe responder «¿puede servir ahora?» y no «¿está el proceso vivo?».

```
camino          /health/ready       ← no /  ni /health/live
intervalo       10 s
plazo           5 s                 ← menor que el intervalo
umbral sano     2 comprobaciones    ← vuelve rápido
umbral insano   3 comprobaciones    ← no se va por un fallo aislado
códigos válidos 200
```

La asimetría entre los dos umbrales es deliberada: entrar rápido y salir despacio. Un fallo aislado no debe retirar un destino sano, pero un destino que se recupera debe volver a recibir tráfico cuanto antes.

El tiempo de detección se calcula:

```
detección de fallo = intervalo × umbral insano = 10 × 3 = 30 s
```

Treinta segundos durante los cuales una fracción del tráfico va a un destino averiado. Bajarlo tiene coste: intervalos muy cortos generan carga de sondeo y falsos positivos bajo saturación. Con nueve destinos y 10 segundos son 54 sondeos por minuto, despreciable; con 200 destinos y 5 segundos son 2.400, que ya se nota.

El error que convierte una degradación en caída total, ya visto en la clase 012: si /health/ready comprueba la base de datos y esta cae, los nueve destinos fallan a la vez y el balanceador se queda sin ninguno sano. AWS mitiga esto con el modo fail open —si ningún destino está sano, envía tráfico a todos—, pero apoyarse en eso es apoyarse en un comportamiento de emergencia. Lo correcto es que la comprobación distinga dependencias esenciales de opcionales.

3. Autoescalado: los cuatro parámetros y su interacción

El seguimiento de objetivo es preferible a los umbrales manuales porque gestiona la histéresis solo: se declara el valor deseado de una métrica y el servicio calcula los ajustes.

```
$ aws autoscaling put-scaling-policy --auto-scaling-group-name cloudshop-asg \
    --policy-name seguimiento-peticiones --policy-type TargetTrackingScaling \
    --target-tracking-configuration '{
      "PredefinedMetricSpecification": {
        "PredefinedMetricType": "ALBRequestCountPerTarget",
        "ResourceLabel": "app/cloudshop-alb/50dc.../targetgroup/cloudshop-tg/6d0e..."
      },
      "TargetValue": 420.0,
      "ScaleInCooldown": 300,
      "ScaleOutCooldown": 60
    }'
```

Cuatro decisiones en ese documento:

La métrica. ALBRequestCountPerTarget es mejor que la CPU para servicios web: mide demanda directamente y no depende de que el trabajo sea de cómputo. Una carga limitada por E/S nunca dispara un escalado basado en CPU aunque esté saturada.

El valor objetivo. Se deriva de la capacidad medida por destino y del objetivo de utilización de la clase 016:

```
capacidad por destino a saturación   600 peticiones/s
objetivo de utilización              0,70
valor objetivo                       420 peticiones/s
```

Los enfriamientos asimétricos. Subir rápido (60 s) y bajar despacio (300 s). Retirar capacidad demasiado pronto es lo que produce oscilación, y el coste de sobrar una instancia unos minutos es mucho menor que el de faltar.

El periodo de gracia, que se configura en el grupo:

```
periodo de gracia > tiempo de arranque hasta servir tráfico
medido: 118 s  →  se fija en 180 s
```

Si la gracia es menor que el arranque, el grupo marca insanas a las instancias nuevas, las termina y crea otras: un bucle que consume dinero y nunca converge.

4. Drenado: por qué reducir capacidad pierde peticiones

Cuando el autoescalado retira una instancia, el balanceador deja de enviarle peticiones nuevas pero puede tener transacciones en curso. Sin drenado, esas se cortan.

```
drenado 0 s    la instancia se termina de inmediato
               → peticiones en vuelo cortadas → errores para el usuario
drenado 60 s   deja de recibir nuevas, termina las que tiene
               → cero errores si ninguna dura más de 60 s
```

El valor debe ser mayor que la petición más larga:

```
$ aws elbv2 modify-target-group-attributes --target-group-arn arn:...:targetgroup/... \
    --attributes Key=deregistration_delay.timeout_seconds,Value=60
```

Y hay una segunda pieza que casi siempre falta: la aplicación debe cooperar. El balanceador deja de enviar tráfico, pero el orquestador envía SIGTERM a la instancia. Si el proceso no lo atrapa y cierra de inmediato, el drenado del balanceador no sirve de nada —lo de la clase 002—:

```
def apagar(signum, frame):
```

```
    servidor.stop_accepting()      # no aceptar conexiones nuevas
    servidor.wait_for_inflight(55) # terminar las en curso, con margen
    sys.exit(0)

signal.signal(signal.SIGTERM, apagar)
```

Hay un orden que importa y que se equivoca a menudo: el proceso debe primero fallar la comprobación de readiness y solo después dejar de aceptar conexiones. Si deja de aceptar antes de que el balanceador se entere, hay una ventana de segundos en la que el balanceador sigue enviando tráfico a un puerto cerrado, y el usuario recibe un 502.

La secuencia correcta, con sus tiempos:

```
t+0   llega SIGTERM
t+0   /health/ready empieza a devolver 503
t+30  el balanceador ha detectado el fallo (intervalo × umbral) y deja de enviar
t+30  el proceso deja de aceptar conexiones nuevas
t+85  terminan las peticiones en curso
t+85  el proceso sale con 0
```

5. 502 y 504 del balanceador señalan cosas distintas

Los errores del balanceador se distinguen de los de la aplicación en las métricas y en los logs, y confundirlos alarga los incidentes:

Código	Métrica de CloudWatch	Qué ocurrió	Dónde mirar
502	HTTPCodeELB502Count	El destino cerró la conexión o devolvió algo inválido	Logs del destino
503	HTTPCodeELB503Count	No hay destinos sanos	Comprobación de estado
504	HTTPCodeELB504Count	El destino no respondió dentro del plazo	Latencia y saturación del destino

La causa más frecuente de 502 sin ningún error en la aplicación es una discordancia de tiempos de espera:

```
tiempo de espera del balanceador   60 s
tiempo de espera keep-alive del destino  5 s     ← MENOR
```

El destino cierra la conexión inactiva a los 5 segundos; el balanceador, que la creía viva, envía una petición por ella y recibe un cierre. El resultado es un 502 intermitente, con más frecuencia cuanto menor sea el tráfico —porque las conexiones se quedan inactivas—.

La regla: el keep-alive del destino debe ser mayor que el tiempo de espera de inactividad del balanceador. Con 60 s en el balanceador, 75 s en el destino.

Y los logs del balanceador dan el detalle que las métricas no:

```
request_processing_time  target_processing_time  response_processing_time
     0.001                       -1                        -1
```

Un -1 en targetprocessingtime significa que el destino nunca respondió: el problema no está en la aplicación sino en la conexión hacia ella. Distinguir eso de un 59.998 —el destino tardó y agotó el plazo— cambia por completo dónde hay que buscar.

Ejemplo trabajado

Cada despliegue de CloudShop produce entre 200 y 400 errores 502, y el autoescalado oscila creando y destruyendo instancias. Se atacan los dos problemas.

Problema 1 — errores en cada despliegue.

```
$ aws elbv2 describe-target-group-attributes --target-group-arn arn:...:targetgroup/cloudshop-tg \
    --query 'Attributes[?Key==`deregistration_delay.timeout_seconds`].Value' --output text
0
```

Drenado a cero: las instancias se terminan con peticiones en vuelo. Se mide la duración de las peticiones para elegir el valor:

```
p50   38 ms
```

```
p95   91 ms
p99  185 ms
máx  4,2 s   (exportación de informes)
```

Se fija en 60 s, con margen amplio sobre los 4,2 s del peor caso.

Pero al repetir el despliegue siguen apareciendo 502, solo que menos:

```
antes del cambio   340 errores
después            96 errores
```

La aplicación no atrapaba SIGTERM. Se añade el manejador y se comprueba el orden:

```
versión inicial:  SIGTERM → cierra el socket inmediatamente
                  el balanceador tarda 30 s en enterarse → 502 durante 30 s
versión correcta: SIGTERM → /health/ready devuelve 503
                            espera 35 s (intervalo 10 × umbral 3 + margen)
                            deja de aceptar y drena lo en curso

tras el arreglo completo:  0 errores en 5 despliegues consecutivos
```

Problema 2 — oscilación del autoescalado.

```
$ aws autoscaling describe-scaling-activities --auto-scaling-group-name cloudshop-asg \
    --max-items 8 --query 'Activities[].[StartTime,Description]' --output text
2026-08-01T11:42  Terminating EC2 instance: i-0f3a
2026-08-01T11:39  Launching a new EC2 instance: i-0f3a
2026-08-01T11:36  Terminating EC2 instance: i-0c8b
2026-08-01T11:33  Launching a new EC2 instance: i-0c8b
```

Crea y destruye cada 3 minutos. La configuración:

```
política       umbral simple: sube al 70 % de CPU, baja al 65 %
enfriamiento   120 s
gracia         60 s
tiempo de arranque medido  118 s
```

Dos fallos independientes:

```
1. Histéresis insuficiente: con 8 instancias al 71 %, añadir una baja
   la utilización a 63 % → dispara la reducción → vuelve a 71 %.
2. Gracia (60 s) < arranque (118 s): las instancias nuevas se marcan
   insanas antes de poder servir y se terminan.
```

Se sustituye por seguimiento de objetivo sobre peticiones por destino:

```
capacidad medida por destino a saturación   600 peticiones/s
objetivo 0,70                               420 peticiones/s
enfriamiento de subida                       60 s
enfriamiento de bajada                      300 s
periodo de gracia                           180 s   (> 118 medidos)
```

Se elige peticiones por destino y no CPU porque el servicio pasa el 60 % del tiempo esperando a la base de datos: la CPU nunca refleja la saturación real.

Resultado en 7 días:

```
errores 502 por despliegue             340          0
actividades de escalado/día             48          6
instancias en régimen              6-11 osc.      7-9
p95 en hora punta                   410 ms       94 ms
coste mensual de cómputo            412 USD     338 USD
```

El coste bajó un 18 % al dejar de oscilar: cada instancia creada y destruida a los 3 minutos se facturaba igual y no llegaba a servir tráfico útil.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/029-elastic-load-balancing-y-auto-scaling/lab.py
```

El laboratorio selecciona el motor de práctica reliability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.

2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa servicio-elastico-aws en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un escenario de fallo con objetivo y recuperación medida. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye servicio-elastico-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

■■ Errores frecuentes

Síntoma	Causa probable	Corrección
Cada despliegue produce cientos de 502	Drenado a cero: las instancias se terminan con peticiones en vuelo	Fija el drenado por encima de la petición más larga y atrapa SIGTERM en la aplicación.
Se configura el drenado y siguen apareciendo 502, aunque menos	La aplicación cierra el socket antes de que el balanceador detecte el cambio de estado	Al recibir SIGTERM, falla readiness primero y espera intervalo × umbral antes de dejar de aceptar.
El autoescalado crea y destruye instancias cada pocos minutos	Umbrales de subida y bajada demasiado próximos, o gracia menor que el arranque	Usa seguimiento de objetivo y fija la gracia por encima del arranque medido.
El servicio se satura y el autoescalado no reacciona	La métrica es la CPU y la carga está limitada por E/S	Escala por peticiones por destino o por una métrica que refleje la demanda real.
Aparecen 502 intermitentes, más frecuentes con poco tráfico	El keep-alive del destino es menor que el tiempo de espera del balanceador	Configura el keep-alive del destino por encima del plazo de inactividad del balanceador.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿En qué dos casos concretos necesitas un balanceador de red en vez de uno de aplicación?
2. Con intervalo de 10 s y umbral insano de 3, ¿cuánto tarda en detectarse un fallo y qué ocurre entretanto?
3. ¿Por qué los umbrales sano e insano deben ser asimétricos, y en qué dirección?
4. Describe la secuencia correcta desde que llega SIGTERM hasta que el proceso sale sin perder peticiones.

5. Un log del balanceador muestra targetprocessingtime = -1. ¿Qué significa y dónde buscas?

Referencias

- AWS (2024). Application Load Balancer: target groups and health checks — parámetros y semántica. <https://docs.aws.amazon.com/elasticloadbalancing/latest/application/target-group-health-checks.html>
- AWS (2024). Deregistration delay — drenado de conexiones y su interacción con el ciclo de vida. <https://docs.aws.amazon.com/elasticloadbalancing/latest/application/edit-target-group-attributes.html>
- AWS (2024). Target tracking scaling policies — métricas predefinidas y enfriamientos. <https://docs.aws.amazon.com/autoscaling/ec2/userguide/as-scaling-target-tracking.html>
- AWS (2024). Troubleshoot your Application Load Balancer — causas de 502, 503 y 504 y campos del log de acceso. <https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-troubleshooting.html>
- Beyer, B. et al., eds. (2016). Site Reliability Engineering, cap. 20 — reparto de carga y comprobaciones de salud. <https://sre.google/sre-book/load-balancing-datacenter/>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 02 en PDF · Recorrido de AWS en PDF · Manual integral

Anterior	Índice	Siguiente
← 028 · EC2, AMI, EBS y selección de capacidad	Parte 02 · Programa	030 · S3: objetos, versionado, lifecycle y replicación →

Evaluación — 029 Elastic Load Balancing y Auto Scaling

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega servicio-elastico-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

030 — S3: objetos, versionado, lifecycle y replicación

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: storage · Estado: EXECUTABLECORE

Propósito

Operar almacenamiento de objetos entendiendo qué garantiza y qué no. Los once nueves de durabilidad no protegen de un borrado autorizado —lo estableció la clase 010— y el modelo de acceso de S3 es el origen de una parte desproporcionada de las brechas públicas atribuidas a la nube. Aquí se convierte todo eso en configuración verificable.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar por qué el versionado y el bloqueo de objetos son controles distintos y qué protege cada uno.
2. Diseñar una política de ciclo de vida que reduzca coste sin romper la recuperación ni el cumplimiento.
3. Calcular si una transición a una clase más fría ahorra, contando el mínimo de días y el coste de recuperación.
4. Bloquear el acceso público en todos los niveles y verificarlo con prueba negativa.
5. Distinguir replicación de copia de seguridad, y por qué la primera propaga los borrados.

Conceptos centrales

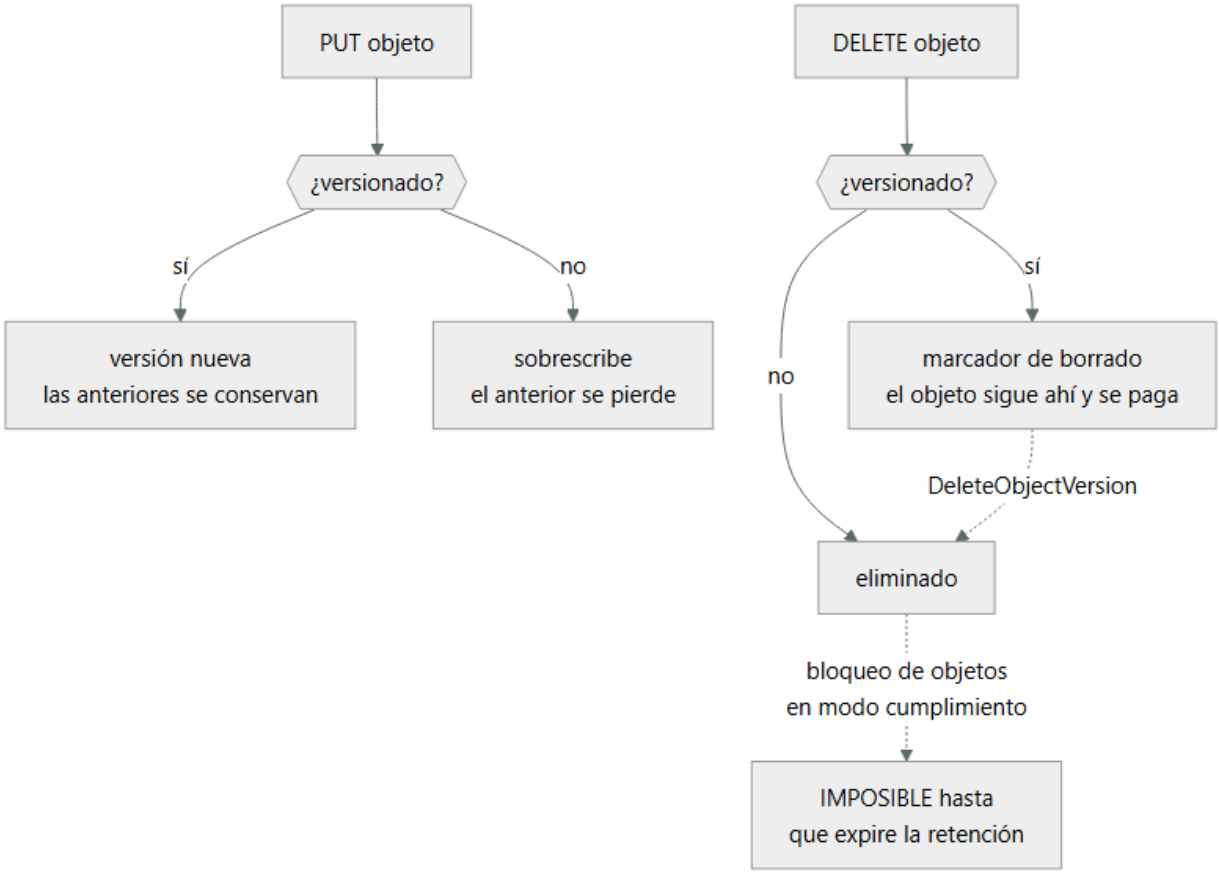
Concepto	Comprensión verificable
versionado	Conservar todas las versiones de un objeto, incluidas las borradas mediante un marcador. Protege del error humano, pero no de quien tenga permiso para borrar versiones.
bloqueo de objetos	Retención inmutable que impide borrar o sobrescribir durante un periodo. En modo cumplimiento ni siquiera la cuenta raíz puede saltárselo, que es lo que lo hace útil frente a ransomware.
clase de almacenamiento	Perfil de coste y acceso: estándar, infrecuente, archivo. Las frías cobran menos por GB y añaden coste de recuperación, mínimos de duración y a veces latencia de horas.
marcador de borrado	Versión especial que oculta un objeto sin eliminarlo. Explica por qué un bucket versionado sigue creciendo y costando después de «vaciarlo».
punto de acceso	Endpoint con su propia política, asociado a un bucket. Permite dar permisos acotados por aplicación sin que la política del bucket crezca sin control.

Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Versionado y bloqueo protegen de cosas distintas

Se confunden y cubren riesgos que no se solapan:

Riesgo	Versionado	Bloqueo de objetos
Sobrescritura accidental	✓	✓
Borrado accidental	✓	✓
Borrado deliberado con permisos	✗	✓
Ransomware con credenciales válidas	✗	✓
Corrupción por fallo de disco	✓ (11 nueves)	✓

Las dos filas centrales son la diferencia. Con versionado, quien tenga s3:DeleteObjectVersion borra todas las versiones y el objeto desaparece de verdad. El bloqueo en modo cumplimiento lo impide hasta que expire la retención, y no admite excepción para nadie —incluida la cuenta raíz—.

```
$ aws s3api put-object-lock-configuration --bucket cloudshop-facturas \
    --object-lock-configuration '{"ObjectLockEnabled":"Enabled",
      "Rule":{"DefaultRetention":{"Mode":"COMPLIANCE","Days":2555}}}'
```

Los dos modos no son intercambiables:

```
GOVERNANCE  quien tenga s3:BypassGovernanceRetention puede saltárselo.
            Útil para protección operativa con escape controlado.
COMPLIANCE  nadie puede. Ni el administrador, ni la raíz, ni AWS.
            Es el único que protege frente a un atacante con permisos.
```

Y una advertencia que hay que entender antes de activarlo: el modo cumplimiento no se puede reducir ni desactivar. Un objeto con 7 años de retención ocupará y costará 7 años. Es exactamente lo que se quiere para evidencia legal y una trampa cara si se aplica por defecto a todo un bucket de datos operativos.

El bloqueo exige versionado activado y solo puede habilitarse al crear el bucket, o mediante una solicitud a soporte. Otra razón para decidirlo al principio.

2. El marcador de borrado explica la factura que no baja

En un bucket versionado, DELETE no borra: crea un marcador que oculta el objeto. Las versiones anteriores siguen ahí y siguen facturándose.

```
$ aws s3 rm s3://cloudshop-datos/informe.csv
delete: s3://cloudshop-datos/informe.csv
$ aws s3 ls s3://cloudshop-datos/informe.csv
# vacío: parece borrado
$ aws s3api list-object-versions --bucket cloudshop-datos --prefix informe.csv \
    --query '[Versions[].[Size,VersionId],DeleteMarkers[].VersionId]'
[[[4823901, "3HL4kqt..."]], ["Ktr5nQ..."]]
```

El objeto de 4,8 MB sigue existiendo y pagándose. En un bucket con años de operación, las versiones no vigentes pueden ser la mayor parte del almacenamiento:

```
$ aws s3api list-object-versions --bucket cloudshop-datos --output json \
  | jq '{vigentes: ([.Versions[]|select(.IsLatest)|.Size]|add),
         antiguas: ([.Versions[]|select(.IsLatest|not)|.Size]|add)}'
{"vigentes": 412000000000, "antiguas": 1840000000000}
```

1,84 TB de versiones antiguas frente a 412 GB vigentes: el 82 % del coste es historial.

La solución no es desactivar el versionado —se perdería la protección— sino caducar lo antiguo con ciclo de vida:

```
{
  "Rules": [{
    "ID": "caducar-versiones-antiguas",
    "Status": "Enabled",
    "Filter": {"Prefix": ""},
    "NoncurrentVersionExpiration": {"NoncurrentDays": 90, "NewerNoncurrentVersions": 3},
    "Expiration": {"ExpiredObjectDeleteMarker": true},
    "AbortIncompleteMultipartUpload": {"DaysAfterInitiation": 7}
  }]
}
```

La última regla merece mención aparte: las subidas multiparte abandonadas no aparecen en ningún listado normal y se facturan indefinidamente. Es una partida invisible que en buckets con mucha escritura fallida puede alcanzar cientos de gigabytes.

3. Clases de almacenamiento: el ahorro que a veces no lo es

Clase	USD/GB/mes	Recuperación	Mínimo	Latencia
Standard	0,023	0	—	ms
Intelligent-Tiering	0,023 → 0,0125	0	—	ms
Standard-IA	0,0125	0,01 USD/GB	30 días	ms
Glacier Instant	0,004	0,03 USD/GB	90 días	ms
Glacier Flexible	0,0036	0,01-0,03	90 días	min-horas
Deep Archive	0,00099	0,02 USD/GB	180 días	12 h

Las dos columnas de la derecha son las que invalidan la mitad de las transiciones que se configuran:

El mínimo de duración se factura completo. Un objeto movido a Standard-IA y borrado a los 10 días paga 30. Para datos con vida corta, la transición cuesta más de lo que ahorra.

La recuperación se cobra. Si los datos se leen con frecuencia, el coste de acceso supera el ahorro de almacenamiento:

```
1 TB en Standard:      1.024 × 0,023 = 23,55 USD/mes
1 TB en Standard-IA:   1.024 × 0,0125 = 12,80 USD/mes
ahorro bruto                         = 10,75 USD/mes

umbral de lecturas: 10,75 / 0,01 USD/GB = 1.075 GB/mes
→ si se lee más del 105 % del volumen al mes, Standard-IA sale MÁS CARO
```

Y hay un coste que se olvida: la propia transición cuesta unos 0,01 USD por cada 1.000 objetos. Con 40 millones de objetos pequeños, mover a otra clase cuesta 400 USD de una vez, y si los objetos son de 20 KB el ahorro mensual puede no compensarlo nunca.

De ahí que Intelligent-Tiering sea razonable cuando el patrón de acceso es desconocido: mueve automáticamente según el uso real, sin coste de recuperación, a cambio de una pequeña tarifa de monitorización por objeto que solo compensa con objetos mayores de 128 KB.

4. Bloquear el acceso público en todos los niveles

Las brechas de almacenamiento expuesto siguen ocurriendo porque hay cuatro mecanismos independientes que pueden conceder acceso público, y bloquear uno no bloquea los demás:

```
1. ACL del bucket
2. ACL del objeto
3. Política del bucket
4. Política del punto de acceso
```

El bloqueo de acceso público los cubre todos, y debe aplicarse a nivel de cuenta, no solo de bucket:

```
$ aws s3control put-public-access-block --account-id 123456789012 \
    --public-access-block-configuration \
    'BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true'
```

Los cuatro parámetros hacen cosas distintas y hay que entender por qué son cuatro:

```
BlockPublicAcls        impide CREAR ACL públicas nuevas
IgnorePublicAcls       IGNORA las ACL públicas que ya existan
BlockPublicPolicy      impide crear políticas públicas nuevas
RestrictPublicBuckets  bloquea el acceso a través de políticas ya existentes
```

Las de «crear» no arreglan lo que ya está mal; las de «ignorar» y «restringir» sí. Aplicar solo las dos primeras deja abierto lo que ya estaba abierto, que es exactamente el caso de la brecha de la clase 010.

La verificación no puede ser por inspección:

```
$ curl -s -o /dev/null -w '%{http_code}\n' https://cloudshop-facturas.s3.amazonaws.com/f-1042.pdf
403                                                  ✓ sin credenciales
$ aws s3api put-bucket-acl --bucket cloudshop-facturas --acl public-read
AccessDenied ... blocked by the public access block    ✓ no se puede abrir
$ aws s3api get-object --bucket cloudshop-facturas --key f-1042.pdf /dev/null
{"ContentLength": 48219}                             ✓ con credenciales sí
```

Las tres pruebas juntas: niega sin credenciales, impide abrirlo, y permite el acceso legítimo. Solo la primera no demuestra nada — el objeto podría estar simplemente mal nombrado.

5. Replicación no es copia de seguridad

La confusión es cara. La replicación copia cambios a otro bucket, y un borrado es un cambio:

```
replicación   protege de: pérdida de una región, latencia de lectura lejana
              NO protege de: borrado, cifrado por ransomware, corrupción lógica
              porque propaga esas operaciones al destino

copia de seguridad  protege de: todo lo anterior
              porque es un punto en el tiempo aislado del origen
```

Hay un matiz importante: por defecto la replicación no replica los borrados —los marcadores de borrado no se replican salvo que se active DeleteMarkerReplication—. Eso da una protección parcial, pero no cubre el caso peligroso: si un atacante borra versiones con DeleteObjectVersion, esa operación sí se propaga.

La configuración que sí resiste:

```
origen    versionado + bloqueo de objetos + política que deniega DeleteObjectVersion
destino   cuenta DISTINTA, con sus propias credenciales y su propio bloqueo
```

La cuenta distinta es la pieza decisiva, por lo visto en la clase 017: si el destino está en la misma cuenta, unas credenciales comprometidas alcanzan ambos. Con cuentas separadas, el atacante necesita comprometer dos.

Y como estableció la clase 019, nada de esto vale sin la prueba:

```
$ aws s3api list-object-versions --bucket cloudshop-backup --prefix f-1042.pdf \
    --query 'Versions[0].[VersionId,LastModified]' --output text
3sL9mQt...   2026-08-01T07:12:44Z
```

```
$ aws s3api get-object --bucket cloudshop-backup --key f-1042.pdf \
    --version-id 3sL9mQt... /tmp/r.pdf && sha256sum /tmp/r.pdf
```

Restaurar y comparar la suma de comprobación con el original. Que exista la copia no demuestra que sea recuperable ni que sea correcta.

Ejemplo trabajado

Auditoría del almacenamiento de CloudShop: 2,25 TB facturados, un bucket con facturas de clientes y ninguna prueba de recuperación.

Hallazgo 1 — el 82 % del coste es historial invisible:

```
$ aws s3api list-object-versions --bucket cloudshop-datos --output json \
  | jq '{vigentes:([.Versions[]|select(.IsLatest)|.Size]|add),
         antiguas:([.Versions[]|select(.IsLatest|not)|.Size]|add),
         marcadores:(.DeleteMarkers|length)}'
{"vigentes": 412000000000, "antiguas": 1840000000000, "marcadores": 28417}
```

```
vigentes     412 GB ×  0,023 =  9,48 USD/mes
antiguas   1.840 GB ×  0,023 = 42,32 USD/mes   ← el 82 %
```

Se añaden subidas multiparte abandonadas, que no salían en ningún listado:

```
$ aws s3api list-multipart-uploads --bucket cloudshop-datos --query 'length(Uploads)'
1247
```

1.247 subidas incompletas, algunas de 2024, facturándose desde entonces.

Ciclo de vida aplicado:

```
{"Rules":[{"ID":"limpieza","Status":"Enabled","Filter":{"Prefix":""},
  "NoncurrentVersionExpiration":{"NoncurrentDays":90,"NewerNoncurrentVersions":3},
  "Expiration":{"ExpiredObjectDeleteMarker":true},
  "AbortIncompleteMultipartUpload":{"DaysAfterInitiation":7}}]}
```

```
almacenamiento tras 90 días:  412 GB vigentes + ~180 GB de historial reciente
coste            51,80 → 13,62 USD/mes   (−74 %)
```

Hallazgo 2 — una transición configurada que costaba dinero. El equipo había movido las miniaturas a Standard-IA:

```
objetos           38.400.000 miniaturas de ~18 KB
volumen           691 GB
lecturas/mes      1.240 GB   (las miniaturas se leen constantemente)

Standard:     691 × 0,023                    = 15,89 USD/mes
Standard-IA:  691 × 0,0125 + 1.240 × 0,01    =  8,64 + 12,40 = 21,04 USD/mes
```

La transición encarecía un 32 %. Además, los objetos de 18 KB están por debajo del umbral de 128 KB, así que ni Intelligent-Tiering compensaría. Se devuelven a Standard.

Hallazgo 3 — el bucket de facturas.

```
$ aws s3api get-public-access-block --bucket cloudshop-facturas \
    --query 'PublicAccessBlockConfiguration' --output json
{"BlockPublicAcls": true, "IgnorePublicAcls": false,
 "BlockPublicPolicy": true, "RestrictPublicBuckets": false}
```

Dos de cuatro. Impide crear accesos públicos nuevos y no restringe los que ya existen. La política del bucket tenía un Principal: "" de 2024, exactamente el caso de la clase 010.

```
$ curl -s -o /dev/null -w '%{http_code}\n' https://cloudshop-facturas.s3.amazonaws.com/f-1042.pdf
200                                              ← accesible sin credenciales
```

Corrección en los cuatro parámetros y a nivel de cuenta:

```
$ aws s3control put-public-access-block --account-id 123456789012 \
    --public-access-block-configuration \
    'BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true'
$ curl -s -o /dev/null -w '%{http_code}\n' https://cloudshop-facturas.s3.amazonaws.com/f-1042.pdf
403                                              ✓
$ aws s3api put-bucket-acl --bucket cloudshop-facturas --acl public-read
AccessDenied                                     ✓ no se puede reabrir
$ aws s3api get-object --bucket cloudshop-facturas --key f-1042.pdf /dev/null >/dev/null && echo ok
ok                                               ✓ acceso legítimo intacto
```

Y se añade retención inmutable, que el versionado por sí solo no daba:

```
bloqueo de objetos en modo cumplimiento, 2.555 días (7 años de retención legal)
réplica a bucket en cuenta distinta, con su propio bloqueo
restauración probada: objeto recuperado y sha256 idéntico al original  ✓
```

Resultado:

```
coste de almacenamiento            67,69 USD    29,51 USD   (−56 %)
facturas accesibles sin credencial     sí           no
borrado de versiones posible           sí           no (cumplimiento)
restauración probada                   nunca      sí, con hash verificado
subidas abandonadas                  1.247          0
```

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/030-s3-objetos-versionado-lifecycle-y-replicacion/lab.py
```

El laboratorio selecciona el motor de práctica storage y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa bucket-gobernado-aws en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una política de durabilidad, acceso, retención y costo. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye bucket-gobernado-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

■■ Errores frecuentes

Síntoma	Causa probable	Corrección
La factura de almacenamiento no baja después de borrar objetos	En un bucket versionado, DELETE crea un marcador y las versiones siguen facturándose	Ciclo de vida con NoncurrentVersionExpiration y limpieza de marcadores caducados.
Se factura almacenamiento que no aparece en ningún listado	Subidas multiparte abandonadas: invisibles en s3 ls y facturadas indefinidamente	Añade AbortIncompleteMultipartUpload al ciclo de vida.
Mover datos a una clase más fría encarece la factura	El coste de recuperación y el mínimo de duración superan el ahorro por GB	Calcula el umbral de lecturas; con acceso frecuente u objetos pequeños, la clase fría pierde.
Se activa el bloqueo de acceso público y el bucket sigue expuesto	Solo se activaron los parámetros de «crear», no los de «ignorar» y «restringir»	Activa los cuatro y a nivel de cuenta; verifica con petición sin credenciales.
Un borrado malicioso se propaga al bucket réplica	La replicación copia cambios, y borrar versiones es un cambio	Réplica en cuenta distinta con bloqueo de objetos propio; la replicación no sustituye a la copia de seguridad.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué riesgo cubre el bloqueo de objetos que el versionado no cubre, y por qué el modo cumplimiento es el único que sirve ahí?
2. Tras aws s3 rm, el objeto no aparece en el listado. ¿Se está pagando? Justifica.
3. 1 TB en Standard-IA se lee 1,5 TB al mes. ¿Ahorra frente a Standard? Haz el cálculo.
4. ¿Cuál de los cuatro parámetros de bloqueo público cierra un bucket que YA estaba expuesto?
5. ¿Por qué una réplica en la misma cuenta no protege de unas credenciales comprometidas?

Referencias

- AWS (2024). Using versioning in S3 buckets — versiones, marcadores de borrado y facturación. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/Versioning.html>
- AWS (2024). S3 Object Lock — modos gobernanza y cumplimiento, y sus límites. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock.html>
- AWS (2024). Blocking public access to your Amazon S3 storage — semántica de los cuatro parámetros. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/access-control-block-public-access.html>
- AWS (2024). Managing your storage lifecycle — transiciones, mínimos de duración y limpieza de multiparte. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lifecycle-mgmt.html>
- AWS (2024). S3 storage classes — precios, latencias de recuperación y umbrales de tamaño. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/storage-class-intro.html>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 02 en PDF · Recorrido de AWS en PDF · Manual integral

Anterior	Índice	Siguiente
← 029 · Elastic Load Balancing y Auto Scaling	Parte 02 · Programa	031 · RDS, DynamoDB y ElastiCache: decisión de datos →

Evaluación — 030 S3: objetos, versionado, lifecycle y replicación

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega bucket-gobernado-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

031 — RDS, DynamoDB y ElastiCache: decisión de datos

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: data · Estado: EXECUTABLECORE

Propósito

Elegir motor de datos a partir de los patrones de acceso, no del catálogo del proveedor. Es la decisión más irreversible de la parte —cambiar de motor con terabytes en producción es una migración, no un ajuste— y la que más determina el coste y la latencia del sistema durante años.

Resultados de aprendizaje

Al finalizar podrás:

1. Derivar el modelo de datos desde los patrones de acceso en lugar de normalizar primero y consultar después.
2. Distinguir cuándo una carga necesita transacciones multiclave y cuándo no, con consecuencias de elección.
3. Dimensionar capacidad en DynamoDB y detectar una clave de partición mal elegida antes de que sature.
4. Justificar una caché por el patrón de lectura y explicar sus dos modos de invalidación.
5. Calcular el coste de las tres opciones para una carga concreta y no solo su idoneidad técnica.

Conceptos centrales

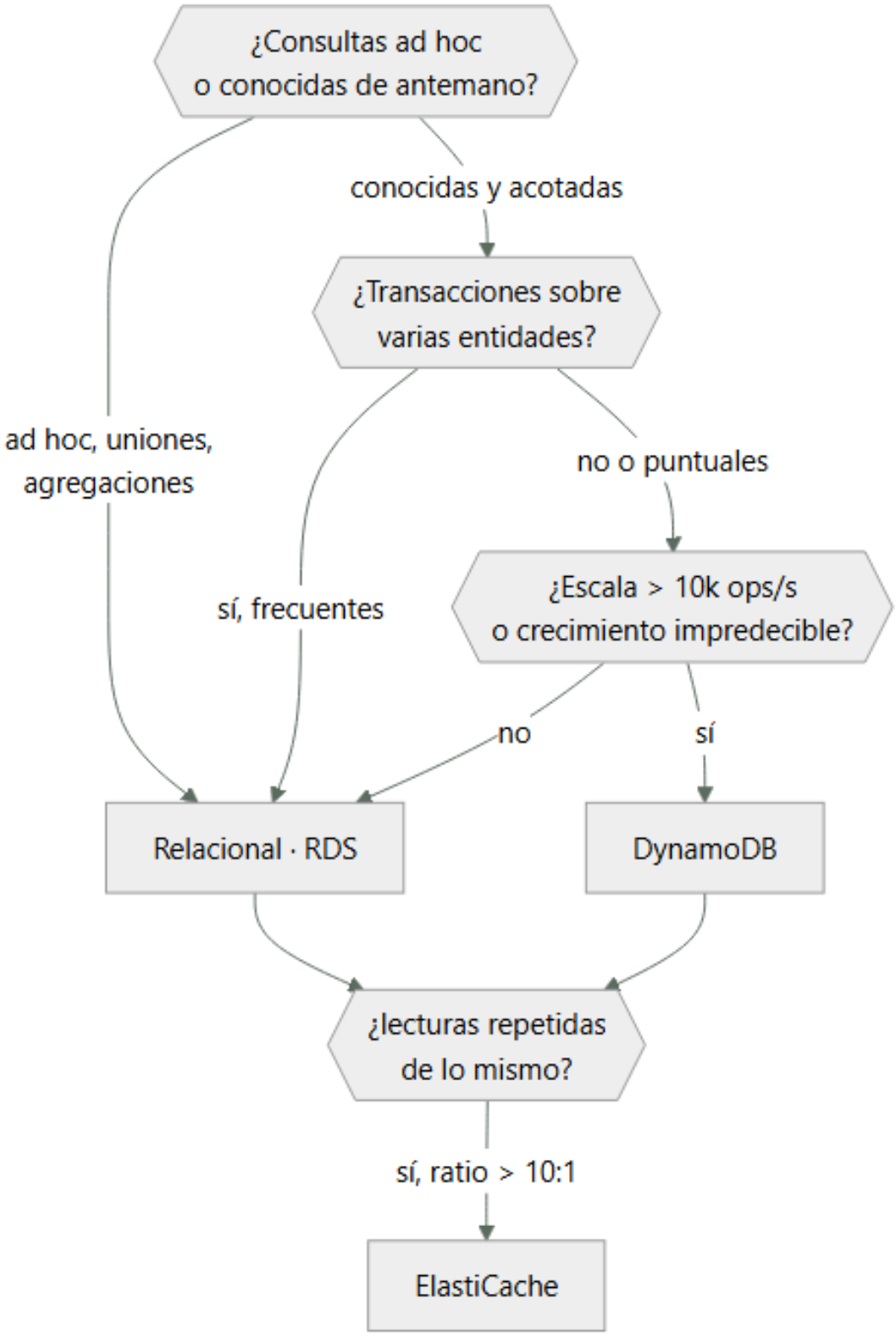
Concepto	Comprensión verificable
patrón de acceso	Consulta concreta que la aplicación hará, con su frecuencia y su latencia exigida. En almacenes no relacionales es la entrada del diseño; enumerarlos mal produce un modelo que no se puede consultar.
clave de partición	Atributo que determina en qué partición física vive un elemento. Una clave con pocos valores distintos concentra el tráfico en pocas particiones y produce una partición caliente.
partición caliente	Partición que recibe una fracción desproporcionada del tráfico y satura antes que el resto. Se manifiesta como limitación de tasa mientras la capacidad agregada parece sobrada.
índice secundario global	Proyección de una tabla con otra clave, que permite consultas por atributos distintos. Tiene su propia capacidad y se actualiza de forma asíncrona: es coherente en último término.
invalidación de caché	Mecanismo para que la copia deje de servirse cuando el origen cambia. Los dos modos —caducidad y escritura activa— tienen fallos distintos: datos obsoletos frente a complejidad y acoplamiento.

Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Los patrones de acceso van primero

En un motor relacional se normaliza el modelo y después se consulta como haga falta: el optimizador se encarga. En un almacén de clave-valor no hay optimizador, así que el modelo debe derivarse de las consultas.

La disciplina es enumerar los patrones antes de diseñar nada:

```
P1  obtener un pedido por su id                        12.000/min   < 10 ms
P2  listar los pedidos de un cliente, más recientes     3.400/min   < 30 ms
P3  listar los pedidos de un estado en una fecha           40/min   < 2 s
P4  total facturado por mes                                2/día    < 30 s
```

Con esa lista, la elección se decide sola:

- P1 y P2 son consultas por clave conocida: un almacén de clave-valor las sirve con latencia de un dígito de milisegundos y escala sin límite práctico.
- P3 necesita un índice secundario.
- P4 es una agregación analítica: no pertenece al almacén operativo, y forzarla ahí degrada P1 y P2.

El error clásico es diseñar el modelo relacional primero, migrarlo tal cual a un almacén no relacional y descubrir que P2 requiere escanear la tabla entera. Un escaneo completo en un almacén de clave-valor no es una consulta lenta: es una consulta que cuesta dinero proporcional al tamaño y que empeora cada mes.

Y el criterio inverso también aplica: si los patrones de acceso no se pueden enumerar —porque el negocio hace preguntas nuevas cada semana—, un motor relacional es la elección correcta, y no una concesión.

2. Transacciones: qué se pierde y qué cuesta recuperarlo

Un motor relacional da atomicidad sobre varias tablas con una sola sentencia. En un almacén de clave-valor eso existe pero con límites duros que hay que conocer antes de diseñar:

```
DynamoDB TransactWriteItems:
  hasta 100 elementos por transacción
  máximo 4 MB en total
  consume el DOBLE de capacidad de escritura
  no puede abarcar regiones
```

La consecuencia de diseño: si la operación central del negocio toca cinco entidades y ocurre miles de veces por segundo, el coste doble y el límite de 100 elementos importan. Si ocurre decenas de veces por minuto, no.

La alternativa cuando la transacción no cabe es el patrón de saga: descomponer en pasos con compensación. No es gratis:

```
transacción     todo o nada, garantizado por el motor
saga            pasos independientes con compensación explícita
                → hay estados intermedios visibles
                → la compensación puede fallar y hay que reintentarla
                → el código de compensación se ejercita poco y suele estar mal probado
```

La regla práctica: elegir saga por diseño, no por descubrir tarde que la transacción no cabía. Un sistema que necesita compensación en su camino principal está pagando complejidad permanente por una decisión de motor.

Y una precisión sobre consistencia: DynamoDB ofrece lectura fuertemente consistente en la tabla base —cuesta el doble de capacidad de lectura— pero los índices secundarios globales son siempre coherentes en último término. Una escritura seguida de una consulta por índice puede no ver el cambio. Ese retraso es de milisegundos en régimen normal y puede crecer bajo carga.

3. La clave de partición decide si escala

DynamoDB reparte los datos por el hash de la clave de partición. Una clave con pocos valores distintos concentra el tráfico:

```
clave = estado_pedido    (5 valores: nuevo, pagado, enviado, entregado, cancelado)
  → 5 particiones lógicas para toda la tabla
  → el 80 % del tráfico va a "nuevo"
  → esa partición satura mientras la capacidad agregada parece sobrada
```

El síntoma es característico y desconcierta: ProvisionedThroughputExceededException con la capacidad global muy por debajo del límite. La partición está limitada a 3.000 unidades de lectura y 1.000 de escritura por segundo, sea cual sea la capacidad total de la tabla.

Una clave adecuada tiene alta cardinalidad y reparto uniforme:

```
clave = pedido_id (UUID)          → millones de valores, reparto uniforme  ✓
clave = cliente_id                → miles de valores, pero un cliente grande
                                    puede concentrar tráfico              ~
```

```
clave = fecha (2026-08-01)        → todo el tráfico del día en una        ✗
```

El segundo caso es el más traicionero: funciona en pruebas y falla cuando un cliente crece. La mitigación es añadir sufijo de dispersión:

```
clave = f"{cliente_id}#{hash(pedido_id) % 10}"
  → reparte cada cliente en 10 particiones
  → coste: consultar todos los pedidos de un cliente exige 10 consultas
```

Es un intercambio explícito: se gana reparto y se pierde simplicidad de consulta. Como todo en esta clase, la decisión debe estar escrita.

Y el modo bajo demanda no elimina el problema: escala automáticamente la capacidad total, pero el límite por partición sigue existiendo. Una clave mal elegida satura igual, solo que la factura crece antes de que se note.

4. Caché: cuándo aporta y cómo se invalida

Una caché solo aporta si el mismo dato se lee muchas más veces de las que se escribe. El indicador es la relación lectura-escritura:

```
ratio < 3:1     la caché añade complejidad y poco beneficio
ratio 10:1      empieza a compensar
ratio > 100:1   claramente rentable
```

Y el ahorro se calcula, no se supone:

```
lecturas/mes                    840.000.000
tasa de acierto medida                 92 %
lecturas evitadas al origen     772.800.000
coste evitado (0,25 USD por millón de lecturas fuertes) = 193 USD/mes
coste de la caché (cache.r7g.large × 2)                 = 208 USD/mes
```

No compensa por coste con esos números; compensa por latencia, que es otra justificación válida y que hay que declarar como tal:

```
latencia desde el origen    8-12 ms
latencia desde la caché    0,3-0,8 ms
```

Los dos modos de invalidación tienen fallos distintos:

Modo	Cómo	Fallo característico
Caducidad	TTL fijo; se relee al expirar	Datos obsoletos hasta el TTL
Escritura activa	La escritura actualiza caché y origen	Se desincroniza si una de las dos falla

El segundo parece mejor y acopla la escritura a la disponibilidad de la caché. Si la caché no responde, ¿falla la escritura o se queda desincronizada? Ninguna respuesta es buena, y por eso la caducidad suele ser preferible salvo que la obsolescencia sea inaceptable.

Hay un tercer fallo que afecta a ambos, la estampida: cuando una clave muy consultada caduca, todas las peticiones simultáneas van al origen a la vez. Se mitiga con bloqueo de recálculo o con caducidad aleatorizada —el mismo jitter de la clase 004—.

5. El coste también decide, y a veces al revés

Comparar motores solo por idoneidad técnica lleva a decisiones caras. Los tres modelos de precio son estructuralmente distintos:

```
RDS         por hora de instancia + almacenamiento + E/S
            → coste fijo, predecible, independiente del tráfico

DynamoDB    por unidades de lectura y escritura consumidas + almacenamiento
            → coste proporcional al tráfico, cero si no hay uso

ElastiCache por hora de nodo
            → coste fijo
```

Para una carga con tráfico muy variable, DynamoDB bajo demanda puede ser mucho más barato:

```
carga con 2 horas de pico al día y el resto casi inactiva
RDS db.r6g.xlarge Multi-AZ:   730 h × 0,96 USD          = 700,80 USD/mes
DynamoDB bajo demanda:
```

```
escrituras 42 M × 1,25 USD/millón                    =  52,50
lecturas  310 M × 0,25 USD/millón                    =  77,50
almacenamiento 180 GB × 0,25                         =  45,00
                                                       --------
                                                       175,00 USD/mes
```

Pero con tráfico alto y sostenido se invierte:

```
tráfico sostenido de 4.000 escrituras/s
DynamoDB bajo demanda: 10.368 M escrituras × 1,25/millón = 12.960 USD/mes
DynamoDB aprovisionado con autoescalado                ≈  2.600 USD/mes
RDS db.r6g.4xlarge Multi-AZ                            ≈  2.800 USD/mes
```

Bajo demanda cuesta cinco veces más que aprovisionado con tráfico predecible. La regla: bajo demanda para tráfico variable o desconocido; aprovisionado con autoescalado cuando el patrón se estabiliza.

Y una partida que se olvida: en RDS, la E/S se factura aparte en algunas clases de almacenamiento. Una consulta sin índice que escanea millones de filas no solo es lenta: aparece en la factura.

Ejemplo trabajado

CloudShop tiene los pedidos en RDS PostgreSQL. La tabla ha llegado a 180 GB, el p95 de la consulta principal es de 340 ms y la factura de base de datos es de 4.900 USD/mes. Se evalúa el cambio.

Primero, los patrones de acceso reales, medidos en 30 días:

```
SELECT queryid, calls, mean_exec_time, rows
FROM pg_stat_statements ORDER BY calls DESC LIMIT 4;
```

```
P1  SELECT * FROM pedidos WHERE id = $1          17.280.000    2,1 ms      1
P2  SELECT * FROM pedidos WHERE cliente_id = $1
      ORDER BY creado DESC LIMIT 20               4.896.000  312,0 ms     20
P3  SELECT * FROM pedidos WHERE estado=$1
      AND creado > $2                                57.600  890,0 ms  ~4000
P4  agregación mensual por mes                           60  8.400 ms      1
```

P2 domina el problema: 4,9 millones de llamadas diarias a 312 ms. Antes de cambiar de motor se comprueba si es un problema de índice:

```
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM pedidos
  WHERE cliente_id = 8241 ORDER BY creado DESC LIMIT 20;
```

```
Index Scan using idx_pedidos_cliente on pedidos  (cost=0.56..18402.11)
  Rows Removed by Filter: 0
  Buffers: shared hit=142 read=8891          ← 8.891 bloques de disco
  Execution Time: 309.442 ms
```

El índice existe pero es solo sobre clienteid: PostgreSQL lo usa para filtrar y después ordena 12.400 filas para devolver 20. Se prueba un índice compuesto:

```
CREATE INDEX CONCURRENTLY idx_pedidos_cliente_creado
  ON pedidos (cliente_id, creado DESC);
```

```
Execution Time: 1.284 ms      ← de 312 ms a 1,3 ms
Buffers: shared hit=24 read=0
```

Un índice compuesto resolvió el 96 % del problema. El motor no era el problema: el modelo de índices sí.

Se reevalúa la situación con ese dato:

```
p95 global                340 ms          38 ms
E/S facturada          1,2 TB/mes      0,18 TB/mes
coste                 4.900 USD       4.130 USD
```

Ahora sí se compara con la alternativa, con los patrones ya optimizados:

```
P1 por id                              2,1 ms           3 ms
P2 por cliente, ordenado               1,3 ms           4 ms   (clave compuesta)
P3 por estado y fecha                  890 ms          GSI, ~40 ms
P4 agregación mensual                 8.400 ms         NO SOPORTADO
transacciones multiclave                 sí             límite 100 elementos
coste mensual estimado              4.130 USD        1.980 USD
migración                                —           ~3 meses + reescritura
```

P4 es el bloqueo: la agregación mensual no existe en DynamoDB sin exportar a otro sistema. Y la migración cuesta tres meses para ahorrar 2.150 USD al mes, con recuperación en 4 meses de trabajo de dos personas.

Decisión: quedarse en RDS. Se añaden dos mejoras acotadas:

```
1. Caché para P1, que es el 78 % del tráfico
   ratio lectura/escritura medido: 47:1  → compensa
   tasa de acierto esperada 90 %
   latencia 2,1 ms → 0,4 ms
   coste: +104 USD/mes

2. P4 a una réplica de lectura, para que la agregación de 8,4 s
   deje de competir con el tráfico operativo
   coste: +380 USD/mes
```

```
p95                          340 ms     31 ms
coste                      4.900 USD  4.614 USD
esfuerzo                      —       2 días
```

La lección: se estaba evaluando un cambio de motor de tres meses para resolver un problema que era un índice mal definido. El análisis de patrones de acceso —que se hizo para elegir motor— acabó descartando el cambio.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/031-rds-dynamodb-y-elasticache-decision-de-datos/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa matriz-datos-aws en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye matriz-datos-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

■■ Errores frecuentes

Síntoma	Causa probable	Corrección
Una consulta usa el índice y aun así tarda cientos de milisegundos	El índice filtra pero no cubre el orden, así que el motor ordena miles de filas	Índice compuesto que incluya las columnas de ordenación; comprueba con EXPLAIN ANALYZE y los bloques leídos.
DynamoDB limita la tasa con la capacidad global muy por debajo del límite	Partición caliente: la clave tiene poca cardinalidad o un valor concentra el tráfico	Elige una clave de alta cardinalidad o añade sufijo de dispersión, asumiendo el coste de consulta múltiple.
Una escritura seguida de consulta por índice secundario no ve el cambio	Los índices secundarios globales son coherentes en último término	Consulta la tabla base cuando necesites leer lo que acabas de escribir.
El coste de DynamoDB se dispara al estabilizarse el tráfico	Bajo demanda cuesta varias veces más que aprovisionado con carga predecible	Cambia a aprovisionado con autoescalado en cuanto el patrón sea estable.
Al caducar una clave muy consultada, el origen recibe una avalancha	Estampida de caché: todas las peticiones recalculan a la vez	Caducidad aleatorizada o bloqueo de recálculo, igual que el jitter en los reintentos.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Por qué en un almacén de clave-valor los patrones de acceso van antes que el modelo, y qué pasa si se invierte el orden?
2. Una tabla tiene la clave de partición fecha. ¿Qué ocurrirá y por qué el modo bajo demanda no lo resuelve?
3. ¿Qué le cuesta a una transacción de DynamoDB en capacidad, y cuál es su límite de elementos?
4. Con ratio lectura/escritura de 3:1, ¿conviene una caché? ¿Y qué otra justificación podría hacerla válida igualmente?
5. ¿En qué caso bajo demanda es cinco veces más caro que aprovisionado, y por qué?

Referencias

- AWS (2024). DynamoDB best practices: partition key design — cardinalidad, particiones calientes y dispersión. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/bp-partition-key-design.html>
- AWS (2024). DynamoDB transactions — límites de elementos, tamaño y consumo de capacidad. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/transaction-apis.html>
- Kleppmann, M. (2017). Designing Data-Intensive Applications, caps. 2-3 — modelos de datos, índices y motores de almacenamiento.
- PostgreSQL (2024). Using EXPLAIN — lectura de planes y coste de E/S. <https://www.postgresql.org/docs/current/using-explain.html>
- AWS (2024). ElastiCache caching strategies — caducidad, escritura activa y mitigación de estampidas. <https://docs.aws.amazon.com/AmazonElastiCache/latest/dg/Strategies.html>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 02 en PDF · Recorrido de AWS en PDF · Manual integral

Anterior	Índice	Siguiente
← 030 · S3: objetos, versionado, lifecycle y replicación	Parte 02 · Programa	032 · Lambda, API Gateway y Step Functions →

Evaluación — 031 RDS, DynamoDB y ElastiCache: decisión de datos

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega matriz-datos-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

032 — Lambda, API Gateway y Step Functions

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: serverless · Estado: EXECUTABLECORE

Propósito

Construir una API serverless sabiendo dónde están sus límites duros y cuáles son negociables. La clase 015 dio el criterio para elegir el modelo; aquí se aplica a AWS y se añade lo que decide si funciona en producción: la concurrencia como recurso compartido de la cuenta, el arranque en frío frente al percentil del SLO, y por qué una función que llama a una base relacional agota sus conexiones.

Resultados de aprendizaje

Al finalizar podrás:

1. Calcular la concurrencia que consumirá una función a partir de su tasa de invocación y su duración.
2. Anticipar el efecto de una función que agota la concurrencia de la cuenta sobre las demás.
3. Decidir entre concurrencia aprovisionada y optimización del paquete según el percentil del SLO.
4. Resolver el agotamiento de conexiones al llamar a una base de datos relacional desde funciones.
5. Elegir entre orquestación con máquina de estados y coreografía por eventos según trazabilidad y acoplamiento.

Conceptos centrales

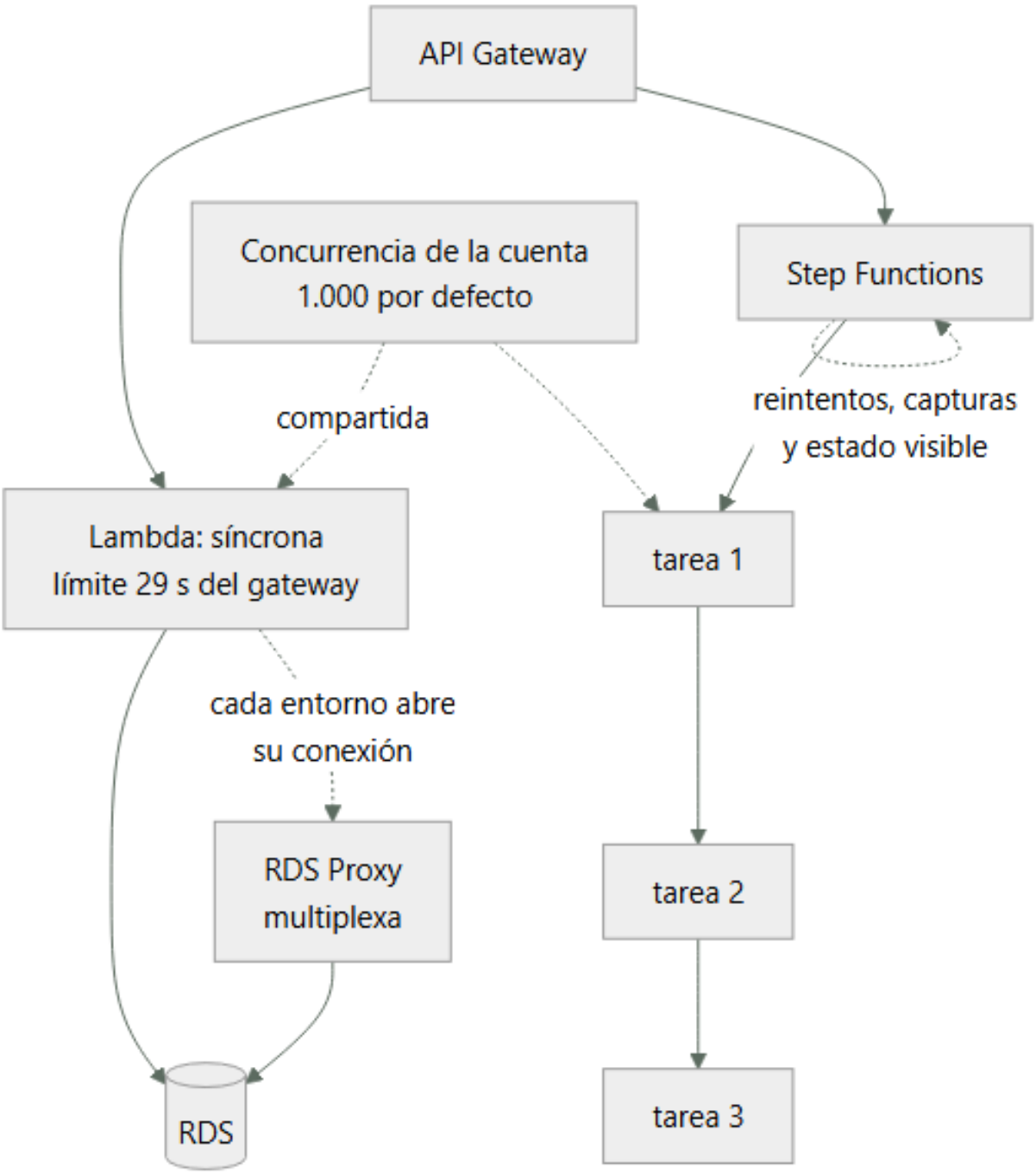
Concepto	Comprensión verificable
concurrencia	Invocaciones ejecutándose simultáneamente. Es el recurso limitado de Lambda y se comparte entre todas las funciones de la cuenta y región: 1.000 por defecto.
concurrencia reservada	Porción de la concurrencia de la cuenta apartada para una función. Actúa a la vez como garantía —nadie se la quita— y como techo —no puede superarla—.
concurrencia aprovisionada	Entornos inicializados y mantenidos calientes. Elimina el arranque en frío y elimina también el ahorro de escalar a cero: se paga aunque no se invoque.
proxy de conexiones	Capa que multiplexa conexiones hacia una base de datos relacional. Resuelve que cada entorno de ejecución abra la suya y agote el límite del motor.
máquina de estados	Orquestador que define el flujo de forma explícita, con reintentos, capturas y estado visible. Se opone a la coreografía, donde cada componente reacciona a eventos sin visión global.

Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. La concurrencia es el recurso que se agota, no la CPU

Lambda no escala por CPU ni por memoria: escala creando entornos de ejecución. El número de entornos simultáneos es la concurrencia, y se calcula con la ley de Little de la clase 011:

```
concurrencia = invocaciones por segundo × duración en segundos
```

```
800 inv/s × 0,250 s = 200 concurrentes
```

El límite por defecto es 1.000 por cuenta y región, compartido entre todas las funciones. De ahí el modo de fallo más desconcertante del modelo:

```
función A: procesamiento por lotes, 900 concurrentes durante 10 minutos
función B: API de pagos, necesita 120
→ B recibe TooManyRequestsException y devuelve 429 a los usuarios
→ B no cambió, no tiene errores, y su código está perfecto
```

Es exactamente el problema de vecino ruidoso de la clase 017, dentro de la misma cuenta. Y la solución es la misma: acotar.

```
# Techo para el proceso por lotes: no puede consumir más de 200
$ aws lambda put-function-concurrency --function-name lotes --reserved-concurrent-executions 200
# Garantía para la API crítica: 300 apartados, nadie se los quita
$ aws lambda put-function-concurrency --function-name pagos-api --reserved-concurrent-executions 300
```

La concurrencia reservada tiene un doble filo que hay que entender: es garantía y es techo. La función con 300 reservados nunca bajará de 300 disponibles y nunca pasará de 300, aunque el resto de la cuenta esté ocioso. Reservar de menos convierte la protección en una limitación.

Y hay un límite adicional sobre el ritmo de crecimiento: la concurrencia sube en incrementos de 1.000 por minuto por región tras la ráfaga inicial. Un pico que necesite 5.000 concurrentes en 30 segundos no se puede absorber escalando, igual que en la clase 016.

2. Arranque en frío: comparar con el percentil del SLO

La pregunta útil no es «¿hay arranque en frío?» sino «¿qué fracción de invocaciones lo sufre y dónde cae el percentil de mi SLO?».

```
invocaciones/mes         2.000.000
arranques en frío           12.000   → 0,6 %
latencia p50 en caliente        45 ms
latencia en frío               820 ms

SLO al p95  → el 0,6 % no lo alcanza; el p95 sigue siendo ~90 ms   ✓
SLO al p99  → el 0,6 % está dentro del 1 % peor; el p99 ES el frío  ✗
```

Factores que determinan la latencia de arranque, por impacto:

```
tamaño del paquete       250 MB descomprimido tarda mucho más que 5 MB
runtime                  JVM y .NET inicializan máquina virtual
inicialización propia    conexiones, carga de configuración, SDK
```

La inicialización propia es la más controlable y la peor aprovechada. El código fuera del manejador se ejecuta una vez por entorno, no por invocación:

```
# Fuera del manejador: se paga una vez por entorno
import boto3
cliente = boto3.client("dynamodb")      # reutilizado en invocaciones siguientes
config = cargar_configuracion()

def handler(event, context):
    return cliente.get_item(...)         # sin coste de inicialización
```

Ponerlo dentro del manejador multiplica el coste por cada invocación, no solo por cada arranque.

La concurrencia aprovisionada elimina el frío y cambia la economía por completo:

```
10 entornos aprovisionados de 512 MB:
  10 × 0,5 GB × 730 h × 0,0000097222 USD/GB-s × 3600 = 127,80 USD/mes
  se paga estén o no invocándose
```

Con eso, el cálculo de la clase 015 cambia: ya no se escala a cero, así que el umbral frente a un contenedor permanente se desplaza a favor del contenedor.

3. Funciones y bases de datos relacionales: el choque de modelos

Una función escala a cientos de entornos y cada entorno abre su propia conexión. Un motor relacional tiene un límite de conexiones muy inferior:

```
concurrencia de la función       400 entornos
conexiones por entorno             1
max_connections de la instancia  200      ← se agota al 50 % de la concurrencia
```

El síntoma: FATAL: sorry, too many clients already. Y no se arregla subiendo maxconnections, porque cada conexión de PostgreSQL consume memoria del servidor y a partir de cierto punto degrada el motor entero.

Las tres respuestas, de peor a mejor:

```
1. Reservar concurrencia por debajo del límite de conexiones
   → funciona y desperdicia la elasticidad que motivó usar funciones
```

```
2. Cerrar la conexión al final de cada invocación
   → añade el coste de establecerla a cada llamada (decenas de ms)

3. Proxy de conexiones
   → multiplexa: 400 entornos comparten un pool de 50 conexiones reales
```

La tercera es la correcta, y tiene un matiz que decide si funciona: el proxy solo puede reutilizar conexiones si la sesión no tiene estado. Una transacción abierta, una tabla temporal o una variable de sesión fuerzan a fijar la conexión a ese cliente hasta que termine, y con suficientes clientes fijados el pool se agota igual.

```
sin estado de sesión   reutilización alta, el proxy cumple su función
con transacciones largas  fijación de conexión, el proxy no ayuda
```

Y una alternativa estructural: si la carga es de funciones, un almacén de clave-valor encaja mejor que uno relacional, porque su modelo de acceso es sin conexión persistente. Es la decisión de la clase 031 vista desde el otro lado.

4. Los límites del gateway y cómo se rodean

API Gateway impone restricciones que no dependen de Lambda y que hay que conocer antes de diseñar:

Límite	Valor	Negociable
Tiempo de espera de integración	29 s	No
Tamaño de carga útil	10 MB	No
Peticiones por segundo	10.000 por región	Sí, con solicitud
Ráfaga	5.000	Sí

Los dos primeros son duros y determinan la arquitectura:

29 segundos es menos que los 15 minutos de Lambda. Una función que tarda 2 minutos funciona invocada directamente y falla siempre a través del gateway. El patrón correcto es asíncrono:

```
POST /informes  → 202 Accepted, devuelve id de trabajo
                  encola el trabajo
GET /informes/{id} → 200 con estado, o 303 al resultado cuando termina
```

10 MB de carga útil hace inviable subir ficheros grandes por la API. La solución es no pasarlos por ella:

```
POST /subidas   → devuelve una URL prefirmada de S3
cliente         → sube directamente a S3, sin límite del gateway
S3              → emite evento que dispara el procesamiento
```

Esto además ahorra dinero: los bytes no atraviesan el gateway ni la función.

Y una decisión que se toma al principio y cuesta cambiar: HTTP API frente a REST API. La primera cuesta aproximadamente un 70 % menos y tiene menos funciones —sin planes de uso, sin caché, sin validación de solicitud—. Empezar por REST API «por si acaso» es una de las formas más comunes de pagar de más sin usar nada de lo que se paga.

5. Orquestación frente a coreografía

Un flujo de varios pasos se puede construir de dos formas, y la elección determina cuánto cuesta diagnosticar un fallo:

	Orquestación (máquina de estados)	Coreografía (eventos)
Flujo	Explícito en un sitio	Emergente, repartido
Estado	Visible y consultable	Hay que reconstruirlo
Reintentos y compensación	Declarativos	En cada componente
Acoplamiento	Mayor: el orquestador conoce los pasos	Menor
Diagnóstico	Se ve dónde falló	Hay que correlacionar trazas
Coste	Por transición de estado	Por evento

La orquestación se declara y el motor se encarga de reintentar:

```
{
```

```
  "Type": "Task",
  "Resource": "arn:aws:lambda:...:function:cobrar",
  "Retry": [{
    "ErrorEquals": ["States.TaskFailed"],
    "IntervalSeconds": 2, "MaxAttempts": 3, "BackoffRate": 2.0
  }],
  "Catch": [{"ErrorEquals": ["States.ALL"], "Next": "CompensarReserva"}]
}
```

Esas nueve líneas sustituyen a código de reintento con retroceso exponencial repartido por varios servicios, y —más importante— hacen visible la compensación, que en coreografía suele estar implícita y sin probar.

El criterio práctico:

```
orquestación   flujos de negocio con compensación, pasos con orden,
               necesidad de auditar dónde quedó cada ejecución
coreografía    reacciones independientes, muchos consumidores del mismo hecho,
               equipos que no deben coordinarse para añadir un consumidor
```

Y sobre coste: Step Functions Standard cobra por transición de estado, lo que en flujos muy repetitivos se acumula. El modo Express cuesta por duración y es mucho más barato para flujos cortos de alto volumen, a cambio de garantía «al menos una vez» en vez de «exactamente una vez» —lo que exige idempotencia en los pasos, como en la clase 009—.

Ejemplo trabajado

La API de pagos de CloudShop empieza a devolver 429 sin que su tráfico haya cambiado, y en el mismo día aparecen errores de conexión contra la base de datos.

Síntoma 1 — 429 sin cambio de tráfico:

```
$ aws cloudwatch get-metric-statistics --namespace AWS/Lambda \
    --metric-name Throttles --dimensions Name=FunctionName,Value=pagos-api \
    --start-time 2026-08-01T09:00:00Z --end-time 2026-08-01T12:00:00Z \
    --period 300 --statistics Sum --query 'sum(Datapoints[].Sum)'
8412.0
$ aws lambda get-account-settings --query 'AccountLimit.ConcurrentExecutions'
1000
```

Se busca quién consume la concurrencia:

```
$ for f in pagos-api catalogo informes-lotes notificaciones; do
    printf "%-18s " $f
    aws cloudwatch get-metric-statistics --namespace AWS/Lambda \
      --metric-name ConcurrentExecutions --dimensions Name=FunctionName,Value=$f \
      --start-time 2026-08-01T10:00:00Z --end-time 2026-08-01T11:00:00Z \
      --period 3600 --statistics Maximum --query 'Datapoints[0].Maximum'
  done
pagos-api          118.0
catalogo            94.0
informes-lotes     742.0        ←
notificaciones      46.0
```

informes-lotes consume 742 de los 1.000. Se lanzó un reproceso histórico esa mañana. La API de pagos no cambió: se quedó sin sitio.

Verificación con la ley de Little de lo que cada una necesita:

```
pagos-api      340 inv/s × 0,32 s = 109 concurrentes → observado 118  ✓
informes-lotes  62 inv/s × 12,0 s = 744              → observado 742  ✓
```

Corrección con techo y garantía:

```
$ aws lambda put-function-concurrency --function-name informes-lotes \
    --reserved-concurrent-executions 200
$ aws lambda put-function-concurrency --function-name pagos-api \
    --reserved-concurrent-executions 250      # 118 observados + margen
```

```
con 200 reservados, informes-lotes tarda 3,7× más (62 → 16,7 inv/s efectivas)
y es trabajo por lotes sin plazo: aceptable y declarado
```

Síntoma 2 — conexiones agotadas. Aparece al aumentar la concurrencia:

```
FATAL: sorry, too many clients already
```

```
$ aws rds describe-db-parameters --db-parameter-group-name cloudshop-pg \
    --query 'Parameters[?ParameterName==`max_connections`].ParameterValue' --output text
```

```
200

concurrencia combinada de las funciones que tocan la base:  118 + 94 + 200 = 412
conexiones disponibles                                          200
→ se agotan al 48 % de la concurrencia
```

Se evalúan las tres salidas:

```
1. bajar la concurrencia por debajo de 200 → renuncia a la elasticidad
2. cerrar conexión por invocación         → +18 ms medidos por llamada
3. proxy de conexiones                    → multiplexa
```

Se elige el proxy y se comprueba la condición que decide si sirve:

```
$ aws rds create-db-proxy --db-proxy-name cloudshop-proxy \
    --engine-family POSTGRESQL --require-tls ...
$ aws cloudwatch get-metric-statistics --namespace AWS/RDS \
    --metric-name DatabaseConnectionsCurrentlySessionPinned \
    --dimensions Name=ProxyName,Value=cloudshop-proxy \
    --period 300 --statistics Maximum --query 'Datapoints[0].Maximum'
3.0
```

Solo 3 conexiones fijadas de 412 clientes: el código no usa transacciones largas ni estado de sesión, así que la multiplexación funciona.

```
conexiones reales al motor          412 (falla)   47
concurrencia sostenible             ~190         412
latencia p95 de pagos-api           340 ms      112 ms
throttles/hora                      2.804          0
```

Los dos síntomas tenían la misma raíz: recursos compartidos sin acotar —concurrencia de la cuenta y conexiones del motor—. Ninguno era un problema de la función que fallaba.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/032-lambda-api-gateway-y-step-functions/lab.py
```

El laboratorio selecciona el motor de práctica serverless y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa api-serverless-aws en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una función con límites, reintentos e idempotencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye api-serverless-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.

- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

■■ Errores frecuentes

Síntoma	Causa probable	Corrección
Una función devuelve 429 sin que su tráfico haya cambiado	Otra función consumió la concurrencia compartida de la cuenta	Reserva concurrencia como techo para lo interrumpible y como garantía para lo crítico.
Reservar concurrencia protege la función pero la limita en el pico	La concurrencia reservada es garantía y también techo	Dimensiona el reservado por encima del máximo observado más margen; reservar de menos es limitarse.
Errores de conexión contra la base de datos al escalar la función	Cada entorno abre su conexión y el motor tiene un límite muy inferior	Usa un proxy de conexiones y comprueba la métrica de conexiones fijadas para saber si multiplexa.
Una función de 2 minutos falla siempre a través del gateway	El tiempo de espera de integración es de 29 s y no es negociable	Patrón asíncrono: 202 con identificador de trabajo y consulta posterior del estado.
El p99 se dispara aunque el p50 sea excelente	La fracción de arranques en frío cae dentro del percentil del SLO	Compara la fracción medida con el percentil exigido; reduce el paquete o aprovisiona solo la ruta crítica.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. Una función se invoca 500 veces por segundo y dura 400 ms. ¿Cuánta concurrencia consume y qué fracción del límite por defecto?
2. ¿Por qué la concurrencia reservada puede empeorar el rendimiento de la función que pretende proteger?
3. ¿Por qué subir maxconnections no resuelve el agotamiento de conexiones desde funciones?
4. ¿Qué límite del gateway hace imposible una integración síncrona de 2 minutos, y qué patrón lo rodea?
5. ¿Qué garantiza el modo Express de una máquina de estados y qué exige a cambio a los pasos?

Referencias

- AWS (2024). Lambda function scaling and concurrency — límites de cuenta, reserva y ritmo de crecimiento. <https://docs.aws.amazon.com/lambda/latest/dg/lambda-concurrency.html>
- AWS (2024). Provisioned concurrency — coste y efecto sobre el arranque en frío. <https://docs.aws.amazon.com/lambda/latest/dg/provisioned-concurrency.html>
- AWS (2024). Using Amazon RDS Proxy — multiplexación, fijación de sesión y sus causas. <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/rds-proxy.html>
- AWS (2024). API Gateway quotas and important notes — tiempo de espera de 29 s y tamaño de carga útil. <https://docs.aws.amazon.com/apigateway/latest/developerguide/limits.html>
- AWS (2024). Step Functions: Standard vs Express workflows — garantías de ejecución y modelo de precio. <https://docs.aws.amazon.com/step-functions/latest/dg/concepts-standard-vs-express.html>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 02 en PDF · Recorrido de AWS en PDF · Manual integral

Evaluación — 032 Lambda, API Gateway y Step Functions

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega api-serverless-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

033 — SQS, SNS y EventBridge

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: messaging · Estado: EXECUTABLECORE

Propósito

Diseñar comunicación asíncrona sabiendo qué garantiza cada servicio y qué hay que construir encima. La entrega «exactamente una vez» no existe en la red —lo estableció la clase 009— así que el receptor debe ser idempotente. Aquí se elige entre cola, tema y bus por acoplamiento y garantías, y se construye la cola de mensajes fallidos que evita perder trabajo en silencio.

Resultados de aprendizaje

Al finalizar podrás:

1. Elegir entre cola, tema y bus a partir del número de consumidores y del acoplamiento deseado.
2. Configurar el tiempo de invisibilidad coherente con la duración del procesamiento y el número de reintentos.
3. Construir una cola de mensajes fallidos con alerta y procedimiento de reproceso.
4. Explicar por qué una cola FIFO limita el rendimiento y cuándo ese coste se justifica.
5. Diagnosticar mensajes procesados varias veces distinguiendo duplicado de entrega de fallo de idempotencia.

Conceptos centrales

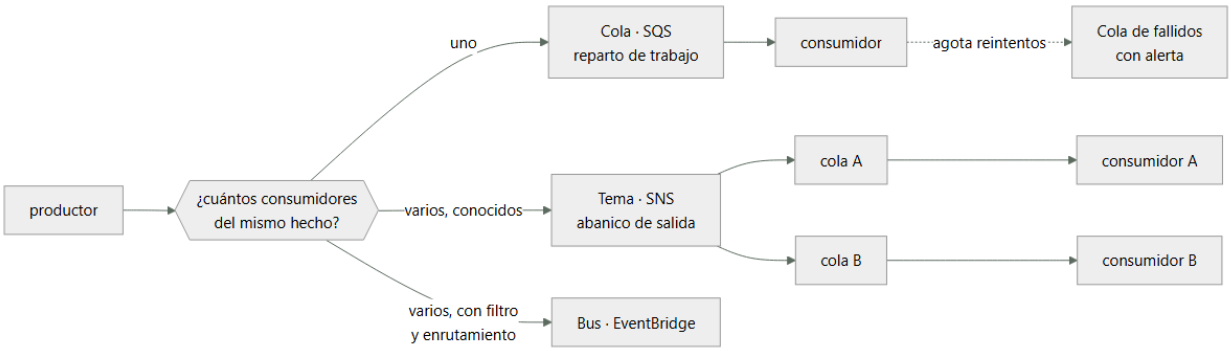
Concepto	Comprensión verificable
tiempo de invisibilidad	Periodo durante el cual un mensaje leído no es visible para otros consumidores. Si expira antes de que el consumidor termine, otro lo recibe y el mensaje se procesa dos veces.
cola de mensajes fallidos	Destino de los mensajes que agotaron sus reintentos. Sin ella, un mensaje envenenado se reintenta indefinidamente y bloquea la cola; con ella, el fallo queda aislado y visible.
abanico de salida	Patrón en el que un mensaje llega a varios consumidores independientes. Un tema lo hace en el momento; una cola, no: el mensaje lo consume uno solo.
entrega al menos una vez	Garantía de que un mensaje llegará, posiblemente más de una vez. Es lo que ofrecen las colas estándar, y obliga a que el receptor sea idempotente.
mensaje envenenado	Mensaje que provoca un fallo determinista en el consumidor. Sin cola de fallidos, se reintenta para siempre y consume capacidad que otros mensajes necesitan.

Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Cola, tema y bus: tres formas de desacoplar

	Cola (SQS)	Tema (SNS)	Bus (EventBridge)
Consumidores del mismo mensaje	Uno	Todos los suscritos	Los que casen con la regla
Persistencia	Hasta 14 días	Ninguna: si no hay suscriptor, se pierde	Reproducible con archivo
Filtrado	No	Por atributos	Por contenido completo
Reintentos	Del consumidor	Del suscriptor	Con cola de fallidos por regla
Latencia típica	ms	ms	0,5 s

La primera fila es la distinción fundamental: una cola reparte trabajo, un tema difunde un hecho. Si diez consumidores leen de la misma cola, cada mensaje lo procesa uno solo; eso es correcto para repartir carga y erróneo si los diez deben enterarse.

La segunda fila es la que produce pérdidas silenciosas: SNS no persiste. Si un suscriptor está caído cuando llega el mensaje, lo pierde. Por eso el patrón robusto es siempre tema seguido de cola por consumidor:

```
SNS tema "pedido-creado"
  ■■■ cola facturacion   → consumidor de facturación
  ■■■ cola inventario    → consumidor de inventario
  ■■■ cola analitica     → consumidor de analítica
```

Cada consumidor tiene su propia cola con su propio ritmo, sus propios reintentos y su propia cola de fallidos. Si el de analítica cae dos horas, sus mensajes esperan en su cola sin afectar a los demás.

EventBridge añade filtrado por contenido y enrutamiento declarativo, a cambio de más latencia:

```
{"detail-type": ["pedido-creado"], "detail": {"importe": [{"numeric": [">", 1000]}]}}
```

Esa regla entrega solo los pedidos por encima de 1.000. Con SNS habría que filtrar en el consumidor y pagar por procesar lo que se descarta.

2. El tiempo de invisibilidad es la causa de los duplicados

Cuando un consumidor lee un mensaje, este se vuelve invisible durante un tiempo configurable. Si el consumidor no lo borra antes de que expire, el mensaje vuelve a estar disponible y otro lo procesa.

```
tiempo de invisibilidad   30 s
duración del procesado    45 s      ← MAYOR

t+0   consumidor A lee el mensaje
t+30  expira la invisibilidad; el mensaje reaparece
t+30  consumidor B lo lee y empieza a procesarlo
t+45  A termina y lo borra
t+75  B termina: el efecto se aplicó DOS VECES
```

La regla de dimensionado:

```
invisibilidad ≥ duración del p99 del procesamiento × margen
```

Con un p99 de 45 s, un valor de 120 s es razonable. Y cuando la duración es muy variable, el consumidor puede extenderla mientras trabaja:

```
$ aws sqs change-message-visibility --queue-url $URL \
    --receipt-handle $RH --visibility-timeout 300
```

El efecto compuesto con el número de reintentos es el que sorprende:

```
invisibilidad 120 s, maxReceiveCount 5
→ un mensaje envenenado tarda 120 × 5 = 10 minutos en llegar a la cola de fallidos
→ durante ese tiempo ocupa capacidad de consumo 5 veces
```

Y el límite superior: la invisibilidad máxima es de 12 horas. Un procesamiento que pueda tardar más no cabe en el modelo y necesita otro diseño —un trabajo asíncrono con estado propio—.

Un último detalle que confunde al depurar: si el consumidor falla y no borra el mensaje, no hace falta esperar a la invisibilidad para reintentarlo si el consumidor la reduce a cero explícitamente al capturar el error. Eso acelera el reintento de fallos transitorios sin tocar la configuración.

3. La cola de mensajes fallidos no es opcional

Sin ella, un mensaje que provoca un fallo determinista se reintenta indefinidamente. Consume capacidad de consumo, llena los logs de errores repetidos y retrasa a los mensajes correctos que están detrás.

```
$ aws sqs set-queue-attributes --queue-url $URL --attributes '{
    "RedrivePolicy": "{\"deadLetterTargetArn\":\"arn:aws:sqs:...:pedidos-dlq\",
                      \"maxReceiveCount\":\"5\"}",
    "VisibilityTimeout": "120",
    "MessageRetentionPeriod": "1209600"
  }'
```

Tres decisiones en esa configuración:

maxReceiveCount. Demasiado bajo (2) envía a fallidos por errores transitorios de red; demasiado alto (20) tarda horas en aislar un envenenado. Entre 3 y 5 cubre los fallos transitorios sin retrasar el aislamiento.

La retención de la cola de fallidos debe ser mayor que la de origen. Si ambas son de 14 días, un mensaje que llega a fallidos el día 13 solo se conserva un día más. La cola de fallidos debe tener 14 días contados desde su llegada, y por eso conviene revisar que el mensaje no envejezca antes de que alguien lo mire.

La alerta. Una cola de fallidos sin alerta es un cubo donde el trabajo desaparece en silencio, que es peor que no tenerla porque da falsa sensación de control:

```
$ aws cloudwatch put-metric-alarm --alarm-name pedidos-dlq-no-vacia \
    --metric-name ApproximateNumberOfMessagesVisible --namespace AWS/SQS \
    --dimensions Name=QueueName,Value=pedidos-dlq \
    --statistic Maximum --period 300 --threshold 0 \
    --comparison-operator GreaterThanThreshold --evaluation-periods 1
```

El umbral es cero: cualquier mensaje en fallidos merece atención. Y hace falta un procedimiento de reproceso escrito, porque el momento de improvisarlo no puede ser durante un incidente.

4. FIFO: qué garantiza y qué cuesta

Las colas FIFO añaden orden estricto y deduplicación, y ambas cosas se pagan:

	Estándar	FIFO
Rendimiento	Prácticamente ilimitado	300 msg/s, 3.000 con lotes
Orden	Mejor esfuerzo	Estricto dentro del grupo
Entrega	Al menos una vez	Exactamente una vez, ventana de 5 min
Alto rendimiento	—	Hasta 70.000/s con particionado por grupo

La frase clave está en la fila del orden: el orden es estricto dentro de un grupo de mensajes, no en toda la cola. El identificador de grupo es lo que decide el paralelismo:

```
grupo = "global"        → orden total, un solo consumidor efectivo, 300 msg/s
grupo = pedido_id       → orden por pedido, miles de grupos en paralelo
```

La segunda opción es casi siempre la correcta: rara vez se necesita orden global. Lo que se necesita es que los eventos de un mismo pedido lleguen en orden, y para eso el grupo debe ser el identificador del pedido.

La deduplicación tiene un límite temporal que hay que conocer: 5 minutos. Dos mensajes idénticos separados por 6 minutos se entregan ambos. Por eso la deduplicación de FIFO no sustituye a la idempotencia del receptor, solo reduce su frecuencia de activación.

La pregunta antes de elegir FIFO:

```
¿el orden importa de verdad, o solo lo parece?
  "el pago debe ir después de la reserva"  → sí, importa
  "prefiero que lleguen en orden"          → no justifica el coste
```

Muchos sistemas que creen necesitar orden lo que necesitan es idempotencia y detección de eventos fuera de orden —descartar una actualización con marca de tiempo anterior a la ya aplicada—, que escala sin límite.

5. Duplicado de entrega frente a fallo de idempotencia

Cuando aparece un efecto duplicado hay dos causas posibles y se corrigen en sitios distintos:

```
duplicado de ENTREGA      el mismo mensaje se entregó dos veces
  → normal en entrega al menos una vez
  → se corrige con idempotencia en el consumidor

fallo de IDEMPOTENCIA     el consumidor no reconoció que ya lo había procesado
  → es un defecto
  → se corrige en la lógica del consumidor
```

Distinguirlas exige registrar el identificador del mensaje:

```
def handler(event, context):
    for registro in event["Records"]:
        mid = registro["messageId"]
        veces = int(registro["attributes"]["ApproximateReceiveCount"])
        log.info("procesando", extra={"message_id": mid, "recepcion": veces})
        if ya_procesado(mid):
            log.info("duplicado descartado", extra={"message_id": mid})
            continue
        procesar(registro)
        marcar_procesado(mid)     # en la MISMA transacción que el efecto
```

El comentario de la última línea es el mismo punto de la clase 009: si marcar y aplicar el efecto no son atómicos, existe una ventana en la que el efecto ocurrió y la marca no, y el reintento duplica.

ApproximateReceiveCount es la métrica que separa los diagnósticos:

```
contador = 1 y efecto duplicado   → fallo de idempotencia: el consumidor falló
contador > 1 y efecto duplicado   → duplicado de entrega no manejado
contador > 1 y sin duplicado      → funcionando como debe
```

Y un consejo operativo: alertar cuando ApproximateReceiveCount supere 2 de forma sostenida. No es un fallo por sí mismo, pero indica que algo está reintentando más de lo normal —invisibilidad corta, consumidor lento o errores transitorios frecuentes— y precede a problemas mayores.

Ejemplo trabajado

Durante una promoción, 1.847 pedidos de CloudShop se facturan dos veces y 312 no se facturan en absoluto. Dos síntomas opuestos, causas distintas.

Síntoma 1 — facturación duplicada.

```
$ aws sqs get-queue-attributes --queue-url $URL_FACTURACION \
    --attribute-names VisibilityTimeout ApproximateNumberOfMessagesNotVisible
{"VisibilityTimeout": "30", "ApproximateNumberOfMessagesNotVisible": "412"}
```

Se mide cuánto tarda el consumidor:

```
p50   8,2 s
p95  28,4 s
p99  51,7 s      ← mayor que la invisibilidad de 30 s
```

```
invocaciones en el pico          42.000
fracción por encima de 30 s        4,4 %  → 1.848 mensajes
duplicados observados                     1.847
```

Cuadra exactamente. El 4,4 % de los mensajes tarda más que la invisibilidad, reaparece y lo procesa otro consumidor. No es un fallo del código: es un parámetro mal dimensionado.

Se comprueba si además había fallo de idempotencia:

```
$ aws logs filter-log-events --log-group-name /aws/lambda/facturacion \
    --filter-pattern '"duplicado descartado"' --start-time 1785540000000 \
    --query 'length(events)'
0
```

Cero descartes: el consumidor no comprobaba duplicados en absoluto. Los dos arreglos son necesarios y ninguno basta solo.

```
$ aws sqs set-queue-attributes --queue-url $URL_FACTURACION \
    --attributes VisibilityTimeout=180        # 3,5× el p99

# y en el consumidor, marca y efecto en la misma transacción
with db.transaction():
    if db.execute("INSERT INTO procesados(message_id) VALUES(%s) "
                  "ON CONFLICT DO NOTHING", [mid]).rowcount == 0:
        return            # ya procesado
    facturar(pedido)
```

Síntoma 2 — 312 pedidos sin facturar.

```
$ aws sqs get-queue-attributes --queue-url $URL_FACTURACION \
    --attribute-names RedrivePolicy
{}
```

No había cola de mensajes fallidos. Se reconstruye qué pasó con los logs:

```
$ aws logs filter-log-events --log-group-name /aws/lambda/facturacion \
    --filter-pattern 'ERROR' --query 'events[0].message' --output text
ERROR ValueError: importe negativo en pedido A-88213
```

312 pedidos de una promoción con descuento superior al importe generaban un valor negativo. El consumidor fallaba, el mensaje volvía a la cola, y así indefinidamente:

```
mensajes envenenados                    312
reintentos por mensaje en 4 horas    ~480   (cada 30 s)
invocaciones desperdiciadas       ~149.760
```

Además de perder esos pedidos, consumían capacidad que retrasaba a los buenos, lo que agravó el síntoma 1 al alargar los tiempos de procesamiento.

```
$ aws sqs create-queue --queue-name facturacion-dlq \
    --attributes MessageRetentionPeriod=1209600
$ aws sqs set-queue-attributes --queue-url $URL_FACTURACION --attributes '{
    "RedrivePolicy":"{\"deadLetterTargetArn\":\"arn:...:facturacion-dlq\",
                      \"maxReceiveCount\":\"4\"}"}'
$ aws cloudwatch put-metric-alarm --alarm-name facturacion-dlq-no-vacia \
    --metric-name ApproximateNumberOfMessagesVisible --namespace AWS/SQS \
    --dimensions Name=QueueName,Value=facturacion-dlq \
    --statistic Maximum --period 300 --threshold 0 \
    --comparison-operator GreaterThanThreshold --evaluation-periods 1
```

Y se reprocesan los 312 tras corregir la validación:

```
$ aws sqs start-message-move-task \
    --source-arn arn:aws:sqs:...:facturacion-dlq \
    --destination-arn arn:aws:sqs:...:facturacion
```

Resultado en la promoción siguiente:

```
pedidos duplicados                1.847         0
pedidos sin facturar                312         0
invocaciones desperdiciadas     149.760         0
p95 de la cola                   28,4 s     9,1 s
mensajes en cola de fallidos       n/a         7  (con alerta y reproceso)
```

Los 7 de la cola de fallidos son el resultado correcto: fallos reales, aislados, visibles y recuperables, en vez de trabajo que desaparece.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/033-sqs-sns-y-eventbridge/lab.py
```

El laboratorio selecciona el motor de práctica messaging y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa flujo-eventos-aws en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un flujo que declara orden, entrega y manejo de errores. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye flujo-eventos-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

■■ Errores frecuentes

Síntoma	Causa probable	Corrección
Un porcentaje de mensajes produce efecto duplicado	El procesamiento supera el tiempo de invisibilidad y el mensaje reaparece	Dimensiona la invisibilidad por encima del p99 del procesamiento, o extiéndela mientras trabajas.
Trabajo que desaparece sin dejar rastro	No hay cola de mensajes fallidos: los envenenados se reintentan hasta caducar	Configura cola de fallidos con maxReceiveCount entre 3 y 5 y alerta con umbral cero.
Un suscriptor pierde mensajes mientras está caído	SNS no persiste: si no hay quien reciba, el mensaje se pierde	Patrón tema seguido de cola por consumidor; cada uno con su ritmo y sus reintentos.
Una cola FIFO no supera 300 mensajes por segundo	Todos los mensajes usan el mismo identificador de grupo, así que no hay paralelismo	Usa el identificador de la entidad como grupo; el orden estricto es por grupo, no por cola.
Aparece un duplicado con ApproximateReceiveCount igual a 1	No es duplicado de entrega: es un fallo de idempotencia en el consumidor	Marca el mensaje como procesado en la misma transacción que el efecto.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Por qué el patrón robusto es tema seguido de cola por consumidor en vez de suscribir consumidores directamente al tema?
2. El procesamiento tiene un p99 de 50 s y la invisibilidad es de 30 s. ¿Qué fracción se duplicará y por qué?
3. Con invisibilidad de 120 s y maxReceiveCount de 5, ¿cuánto tarda un mensaje envenenado en aislarse?
4. ¿Por qué la deduplicación de FIFO no sustituye a la idempotencia del receptor?
5. Ves un efecto duplicado con ApproximateReceiveCount = 1. ¿Dónde está el fallo?

Referencias

• AWS (2024). Amazon SQS visibility timeout — semántica, extensión y límite de 12 horas. <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-visibility-timeout.html>

• AWS (2024). Amazon SQS dead-letter queues — política de reenvío y reproceso. <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-dead-letter-queues.html>

• AWS (2024). FIFO queues: message groups and throughput — orden por grupo y límites. <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/FIFO-queues.html>

• AWS (2024). EventBridge event patterns — filtrado por contenido y enrutamiento declarativo. <https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-event-patterns.html>

• Hohpe, G. y Woolf, B. (2003). Enterprise Integration Patterns — canal punto a punto, publicación-suscripción y canal de mensajes inválidos.

• Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 02 en PDF · Recorrido de AWS en PDF · Manual integral

Anterior	Índice	Siguiente
← 032 · Lambda, API Gateway y Step Functions	Parte 02 · Programa	034 · CloudWatch, CloudTrail, Config y Systems Manager →

Evaluación — 033 SQS, SNS y EventBridge

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega flujo-eventos-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

• [ ] Resultado reproducible por una segunda persona.

• [ ] Evidencia vinculada a cada afirmación técnica.

• [ ] Prueba negativa o de fallo incluida.

- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

034 — CloudWatch, CloudTrail, Config y Systems Manager

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: observability · Estado: EXECUTABLECORE

Propósito

Construir la evidencia operativa que permite responder tres preguntas distintas durante un incidente: qué está pasando, quién hizo qué y qué cambió. Cada una se responde con un servicio diferente, y confundirlos hace que se busque durante horas en el sitio equivocado. Aquí se aterriza en AWS lo que la clase 012 estableció sobre señales.

Resultados de aprendizaje

Al finalizar podrás:

1. Asignar cada pregunta forense al servicio que la responde y saber qué no responde ninguno.
2. Diseñar métricas y filtros que no multipliquen el coste por la dimensionalidad.
3. Consultar registros de auditoría para reconstruir quién ejecutó una acción y desde dónde.
4. Detectar desviaciones de configuración con reglas que se evalúan de forma continua.
5. Ejecutar una operación en una flota sin abrir puertos ni mantener claves de acceso.

Conceptos centrales

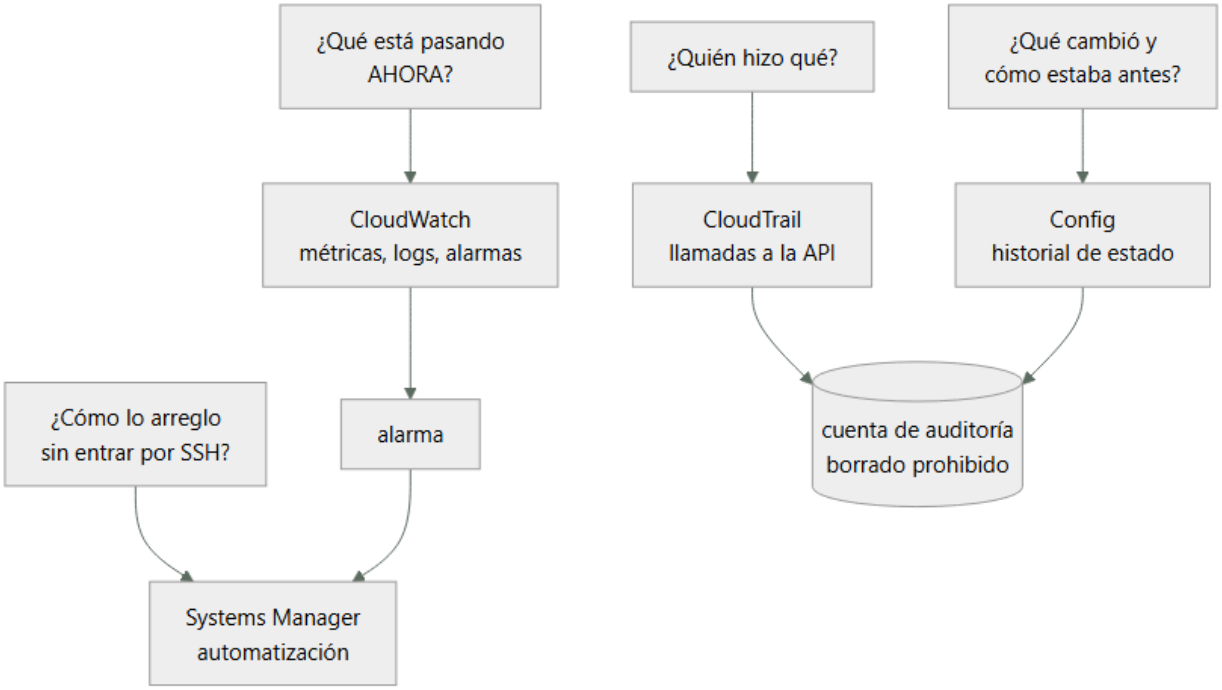
Concepto	Comprensión verificable
cardinalidad	Número de combinaciones distintas de dimensiones de una métrica. Cada combinación es una serie temporal facturada por separado, así que una dimensión con muchos valores multiplica el coste.
métrica de filtro	Métrica derivada de patrones en los logs. Convierte texto en series temporales sin instrumentar el código, a cambio de depender del formato del mensaje.
registro de auditoría	Historial de llamadas a la API con identidad, origen, parámetros y resultado. Responde «quién hizo qué», que ninguna métrica ni log de aplicación responde.
elemento de configuración	Instantánea del estado de un recurso en un momento dado. Permite comparar cómo estaba antes y después de un cambio, que es distinto de saber quién lo hizo.
documento de automatización	Procedimiento declarativo ejecutable sobre una flota, con parámetros y control de errores. Convierte un runbook escrito en uno ejecutable y auditable.

Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Cuatro preguntas, cuatro servicios

Durante un incidente se hacen preguntas distintas y cada una tiene su fuente. Buscar en la equivocada es la causa más común de que un diagnóstico tarde horas:

Pregunta	Servicio	Qué no responde
¿Qué está pasando ahora?	CloudWatch	Quién lo provocó
¿Quién hizo qué y desde dónde?	CloudTrail	Cómo estaba antes
¿Qué cambió y cuál era el estado previo?	Config	Quién lo cambió
¿Cómo actúo sobre la flota?	Systems Manager	—

Las dos filas centrales se confunden constantemente y son complementarias:

```
CloudTrail dice:  "a las 14:22, el rol ci-deploy llamó a ModifyDBInstance
                   desde 203.0.113.40 con estos parámetros"
Config dice:      "a las 14:22, la instancia pasó de BackupRetentionPeriod=7
                   a BackupRetentionPeriod=0"
```

CloudTrail da el actor y la intención; Config da el estado antes y después. Para reconstruir un incidente hacen falta los dos, y por eso ambos deben escribir en la cuenta de auditoría de la clase 025.

Y hay una pregunta que ninguno responde: por qué. Eso solo está en el ADR, el ticket o la conversación. Es la razón por la que la clase 024 insistía en registrar el razonamiento: la telemetría reconstruye el qué con precisión y nunca el motivo.

2. La cardinalidad multiplica la factura

Cada combinación distinta de dimensiones es una métrica personalizada facturada por separado, a unos 0,30 USD al mes. La aritmética se dispara rápido:

```
métrica: latencia_peticion
dimensiones: servicio (8) × endpoint (40) × código (12) × instancia (24)

series = 8 × 40 × 12 × 24 = 92.160
coste  = 92.160 × 0,30 = 27.648 USD/mes
```

Casi 28.000 dólares al mes en métricas, por añadir dos dimensiones que parecían útiles. Y la mayoría de esas series no se consulta nunca.

La dimensión culpable suele ser identificable: instancia tiene 24 valores hoy y crecerá con el autoescalado, además de generar series que mueren en cada despliegue. Un identificador efímero nunca debe ser dimensión de una métrica.

```
sin la dimensión instancia: 8 × 40 × 12 = 3.840 series → 1.152 USD/mes
agrupando códigos en clases (2xx, 4xx, 5xx): 8 × 40 × 3 = 960 → 288 USD/mes
```

Un factor de 96 respecto al diseño inicial, sin perder capacidad de diagnóstico: si hace falta saber qué instancia concreta falló, eso está en los logs, que se cobran por volumen ingerido y no por combinación.

La regla práctica:

```
métrica  → pocas dimensiones, valores acotados y estables
log      → todo el detalle, incluidos identificadores
traza    → la relación entre servicios de una petición concreta
```

Y una forma económica de obtener métricas sin instrumentar: el formato de métrica embebida, que permite emitir un log estructurado del que CloudWatch extrae métricas automáticamente, pagando la ingesta del log en vez de la publicación de cada métrica.

3. Consultar registros de auditoría para reconstruir un incidente

CloudTrail responde quién hizo qué, y la consulta directa por atributos es limitada. Para investigaciones reales conviene usar el lago de datos o consultas sobre el bucket:

```
# Búsqueda rápida por evento, últimos 90 días
$ aws cloudtrail lookup-events \
    --lookup-attributes AttributeKey=EventName,AttributeValue=DeleteBucketPolicy \
    --query 'Events[].[EventTime,Username,CloudTrailEvent]' --output text | head -3
```

Y el detalle completo del evento, que es donde está lo útil:

```
{
  "eventTime": "2026-08-01T14:22:08Z",
  "userIdentity": {
    "type": "AssumedRole",
    "arn": "arn:aws:sts::123456789012:assumed-role/ci-deploy/despliegue-4821",
    "sessionContext": {"attributes": {"mfaAuthenticated": "false"}}
  },
  "sourceIPAddress": "203.0.113.40",
  "userAgent": "aws-cli/2.15.0",
  "requestParameters": {"bucketName": "cloudshop-facturas"},
  "errorCode": null
}
```

Tres campos que suelen decidir la investigación:

- sessionContext indica si hubo MFA. Una acción sensible sin MFA es un hallazgo por sí misma.
- sourceIPAddress distingue una acción desde el pipeline de una desde un portátil.
- errorCode revela los intentos fallidos, que son la señal más útil de un ataque en curso: alguien probando permisos que no tiene.

Dos límites que conviene conocer: el historial de consulta directa cubre 90 días; más atrás hay que ir al bucket. Y los eventos de datos no se registran por defecto —lecturas de objetos, invocaciones de funciones—: si hacen falta, hay que activarlos explícitamente y su volumen puede ser enorme.

Esa segunda limitación es exactamente lo que impidió, en la brecha de la clase 010, saber cuántas veces se descargaron las facturas: los eventos de gestión estaban registrados y los de datos no.

4. Config: qué cambió y detección continua de desviaciones

Config graba el estado de cada recurso a lo largo del tiempo, lo que permite responder cómo estaba antes de un cambio:

```
$ aws configservice get-resource-config-history \
    --resource-type AWS::RDS::DBInstance --resource-id db-ABCD1234 \
    --query 'configurationItems[0:2].[configurationItemCaptureTime,
             configuration.backupRetentionPeriod]' --output text
2026-08-01T14:22:31Z    0
```

```
2026-07-15T09:11:02Z    7
```

Y sobre ese historial se evalúan reglas de forma continua:

```
$ aws configservice put-config-rule --config-rule '{
    "ConfigRuleName": "rds-retencion-minima",
    "Source": {"Owner": "AWS", "SourceIdentifier": "DB_INSTANCE_BACKUP_ENABLED"},
    "InputParameters": "{\"backupRetentionPeriod\":\"7\"}"
  }'
```

La distinción con las SCP de la clase 025 importa:

```
SCP           preventiva: impide que la acción ocurra
regla Config  detectiva: la acción ocurre y se marca como no conforme
```

Son complementarias y ninguna sustituye a la otra. Hay controles que no se pueden prevenir sin bloquear trabajo legítimo, y ahí la detección con corrección automática es la respuesta:

```
$ aws configservice put-remediation-configurations --remediation-configurations '[{
    "ConfigRuleName": "rds-retencion-minima",
    "TargetType": "SSM_DOCUMENT",
    "TargetId": "AWSConfigRemediation-ModifyRDSInstanceBackupRetention",
    "Automatic": true,
    "MaximumAutomaticAttempts": 3
  }]'
```

Una advertencia sobre el coste: Config cobra por elemento de configuración registrado. En un entorno con autoescalado agresivo, cada instancia creada y destruida genera varios elementos, y la factura puede crecer de forma inesperada. Se acota limitando los tipos de recurso grabados a los que de verdad importan.

5. Operar la flota sin puertos abiertos ni claves

Systems Manager permite ejecutar comandos y sesiones interactivas sin abrir el puerto 22, sin claves SSH y sin IP pública. El agente inicia una conexión saliente, así que la instancia puede vivir en una subred aislada —la de la clase 027—.

```
# Sesión interactiva, auditada y sin SSH
$ aws ssm start-session --target i-0a1b2c3d

# Comando en toda una flota por etiqueta
$ aws ssm send-command --document-name AWS-RunShellScript \
    --targets Key=tag:Entorno,Values=produccion \
    --parameters 'commands=["systemctl restart cloudshop"]' \
    --max-concurrency 10% --max-errors 2
```

Los dos últimos parámetros son la diferencia entre una operación y un incidente: max-concurrency limita cuántas instancias se tocan a la vez y max-errors detiene la ejecución si demasiadas fallan. Sin ellos, un comando erróneo se aplica a las 200 instancias simultáneamente.

La ventaja sobre SSH no es la comodidad, son tres propiedades:

```
1. Auditado: cada sesión y cada comando quedan en CloudTrail
2. Sin credenciales de larga vida: usa el rol de instancia
3. Sin superficie de red: ningún puerto entrante abierto
```

Y los runbooks se vuelven ejecutables en vez de escritos:

```
schemaVersion: '0.3'
parameters:
  InstanceId: {type: String}
mainSteps:
  - name: DrenarDelBalanceador
    action: aws:executeAwsApi
    inputs: {Service: elbv2, Api: DeregisterTargets, ...}
  - name: EsperarDrenado
    action: aws:sleep
    inputs: {Duration: PT90S}
  - name: Reiniciar
    action: aws:runCommand
    inputs: {DocumentName: AWS-RunShellScript, ...}
```

La diferencia con un runbook en una wiki: este se ejecuta igual a las 3 de la madrugada que en un simulacro, no depende de que alguien recuerde el orden, y su ejecución queda registrada.

Ejemplo trabajado

A las 14:31 el equipo de CloudShop recibe una alerta: la retención de copias de la base de datos de producción es cero. Se reconstruye el incidente completo con los cuatro servicios.

¿Qué está pasando? — CloudWatch dio la alerta, pero no dice más:

```
$ aws cloudwatch describe-alarms --alarm-names config-rds-retencion \
    --query 'MetricAlarms[0].[StateValue,StateUpdatedTimestamp]' --output text
ALARM   2026-08-01T14:31:12Z
```

¿Qué cambió y cómo estaba antes? — Config:

```
$ aws configservice get-resource-config-history --resource-type AWS::RDS::DBInstance \
    --resource-id db-ABCD1234 --limit 2 \
    --query 'configurationItems[].[configurationItemCaptureTime,
             configuration.backupRetentionPeriod,configuration.multiAZ]' --output text
2026-08-01T14:22:31Z    0     true
2026-07-15T09:11:02Z    7     true
```

De 7 días a 0 a las 14:22. Nueve minutos antes de la alerta.

¿Quién lo hizo? — CloudTrail:

```
$ aws cloudtrail lookup-events \
    --lookup-attributes AttributeKey=ResourceName,AttributeValue=db-ABCD1234 \
    --start-time 2026-08-01T14:00:00Z \
  | jq -r '.Events[0].CloudTrailEvent | fromjson
           | [.eventTime, .userIdentity.arn, .sourceIPAddress,
              .requestParameters.backupRetentionPeriod] | @tsv'
2026-08-01T14:22:08Z  arn:aws:sts::...:assumed-role/ci-deploy/despliegue-4821  203.0.113.40  0
```

Fue el pipeline, no una persona. Se busca el origen en el cambio de infraestructura:

```
el módulo de Terraform tenía backup_retention_period sin declarar
→ el proveedor aplicó su valor por defecto: 0
→ el plan mostraba el cambio y nadie lo leyó
```

Se comprueba si hubo más recursos afectados por la misma causa:

```
$ aws configservice select-resource-config \
    --expression "SELECT resourceId WHERE resourceType='AWS::RDS::DBInstance'
                  AND configuration.backupRetentionPeriod = 0"
{"Results": ["{\"resourceId\":\"db-ABCD1234\"}", "{\"resourceId\":\"db-EFGH5678\"}"]}
```

Dos instancias, no una. La segunda no tenía alerta porque la regla solo cubría producción.

Cuánto tiempo estuvo sin protección:

```
cambio                14:22:08
detección              14:31:12   (9 min: la regla se evalúa cada 10 min)
corrección             14:38:00
ventana sin copias      15 min 52 s
copias perdidas         ninguna: las anteriores se conservan 7 días desde su creación
```

Correcciones, separadas por tipo de control:

```
PREVENTIVO   SCP que deniega ModifyDBInstance con BackupRetentionPeriod=0
             probado con prueba negativa:
             $ aws rds modify-db-instance --backup-retention-period 0 ...
             AccessDenied ... explicit deny in a service control policy  ✓

DETECTIVO    regla de Config ampliada a todas las cuentas, no solo producción
             corrección automática que restaura 7 días

ORIGEN       el módulo de Terraform declara el valor explícitamente
             y una prueba de política rechaza el plan si es menor que 7

OPERATIVO    documento de automatización para verificar retención en la flota,
             ejecutable en simulacro
```

Y se detecta un hueco al revisar el registro: los eventos de datos no estaban activados, así que no se puede saber si alguien accedió a las copias durante la ventana. Se activan para los buckets de copias, con el coste declarado:

```
eventos de datos en 3 buckets: ~2,4 M eventos/mes × 0,10 USD/100k = 2,40 USD/mes
```

Resultado: el incidente se reconstruyó completo en 12 minutos porque cada pregunta tenía su fuente. Sin Config no se habría sabido el estado anterior; sin CloudTrail, que fue el pipeline y no una persona; y sin la consulta agregada, que había una segunda instancia afectada.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/034-cloudwatch-cloudtrail-config-y-systems-manager/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa evidencia-operativa-aws en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye evidencia-operativa-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

■■ Errores frecuentes

Síntoma	Causa probable	Corrección
La factura de métricas personalizadas es de miles de dólares	Una dimensión de alta cardinalidad multiplica las series facturadas	Saca los identificadores efímeros de las dimensiones; ese detalle va en los logs, que se cobran por volumen.
Tras un incidente no se sabe si alguien accedió a los datos	Los eventos de datos de CloudTrail no se registran por defecto	Actívalos para los recursos sensibles y asume su coste; los de gestión no cubren lecturas de objetos.
Se sabe qué cambió pero no quién, o al revés	Se consultó un solo servicio: Config da el estado y CloudTrail el actor	Usa ambos: son complementarios y ninguno responde la pregunta del otro.
Un comando erróneo se aplica a toda la flota a la vez	Se ejecutó sin limitar concurrencia ni tolerancia a errores	Usa max-concurrency y max-errors en toda ejecución sobre flota.
La factura de Config crece de forma inesperada	Se graban todos los tipos de recurso y el autoescalado genera elementos continuamente	Limita los tipos grabados a los que importan para el cumplimiento y el diagnóstico.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué servicio responde «quién lo hizo» y cuál «cómo estaba antes»? ¿Por qué hacen falta los dos?
2. Una métrica con 4 dimensiones de 8, 40, 12 y 24 valores. ¿Cuántas series genera y cuál eliminarías primero?
3. ¿Qué campo de un evento de auditoría revela un ataque en curso mejor que las acciones exitosas?
4. ¿Qué diferencia hay entre una SCP y una regla de Config, y por qué ninguna sustituye a la otra?
5. ¿Qué tres propiedades aporta una sesión gestionada frente a SSH, más allá de la comodidad?

Referencias

- AWS (2024). CloudWatch: publishing custom metrics and dimensions — cardinalidad y facturación por serie. <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/publishingMetrics.html>
- AWS (2024). CloudWatch Embedded Metric Format — métricas desde logs estructurados. <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/CloudWatchEmbeddedMetricFormat.html>
- AWS (2024). CloudTrail: logging data events — qué se registra por defecto y qué hay que activar. <https://docs.aws.amazon.com/awscloudtrail/latest/userguide/logging-data-events-with-cloudtrail.html>
- AWS (2024). AWS Config rules and remediation — evaluación continua y corrección automática. <https://docs.aws.amazon.com/config/latest/developerguide/evaluate-config.html>
- AWS (2024). Systems Manager Session Manager — acceso sin puertos abiertos ni claves, con auditoría. <https://docs.aws.amazon.com/systems-manager/latest/userguide/session-manager.html>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 02 en PDF · Recorrido de AWS en PDF · Manual integral

Anterior	Índice	Siguiente
← 033 · SQS, SNS y EventBridge	Parte 02 · Programa	035 · KMS, Secrets Manager, WAF y controles de seguridad →

Evaluación — 034 CloudWatch, CloudTrail, Config y Systems Manager

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega evidencia-operativa-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

035 — KMS, Secrets Manager, WAF y controles de seguridad

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

Propósito

Aplicar defensa en profundidad a la plataforma AWS: cifrado con control de claves propio, secretos que rotan sin desplegar, y filtrado en el borde. La clase 026 cubrió quién puede hacer qué; esta cubre qué ocurre cuando esa barrera falla, que es la única suposición razonable a la hora de diseñar seguridad.

Resultados de aprendizaje

Al finalizar podrás:

1. Distinguir clave gestionada por el proveedor, gestionada por el cliente y material importado, con sus consecuencias.
2. Escribir una política de clave que impida el acceso incluso a quien tenga permisos sobre el recurso cifrado.
3. Rotar un secreto sin desplegar la aplicación, entendiendo la fase de dos versiones.
4. Configurar reglas de borde que corten ataques volumétricos y de aplicación sin bloquear tráfico legítimo.
5. Verificar cada control con prueba negativa, no con inspección de la configuración.

Conceptos centrales

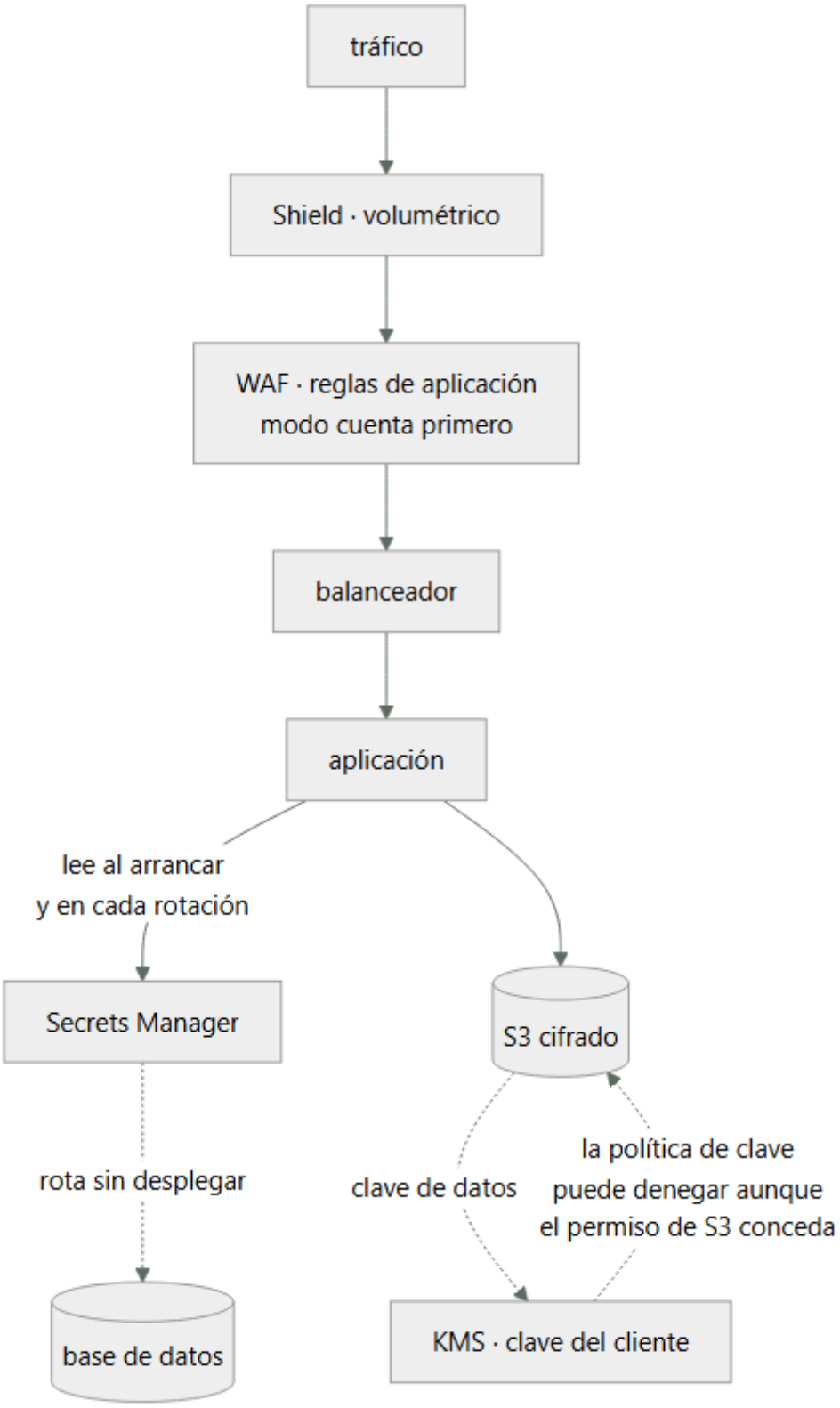
Concepto	Comprensión verificable
cifrado de sobre	Cifrar los datos con una clave de datos y esa clave con una clave maestra. Permite cifrar volúmenes grandes sin llamar al servicio de claves por cada operación y rotar la maestra sin recifrar los datos.
política de clave	Documento que gobierna quién puede usar y administrar una clave. Es independiente de los permisos sobre el recurso cifrado: sin acceso a la clave, el acceso al dato no sirve.
rotación de secretos	Sustitución periódica de una credencial sin intervención humana. Exige una fase en la que dos versiones son válidas, o la rotación corta las conexiones en curso.
regla gestionada	Conjunto de reglas de filtrado mantenido por el proveedor o un tercero. Ahorra escribirlas y exige medirlas en modo cuenta antes de bloquear, porque generan falsos positivos.
modo cuenta	Configuración en la que una regla registra las coincidencias sin bloquear. Es el único modo seguro de introducir reglas nuevas en un servicio con tráfico real.

Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento


tráfico
Shield · volumétrico
WAF · reglas de aplicación
modo cuenta primero
balanceador
aplicación
lee al arrancar
y en cada rotación
Secrets Manager
S3 cifrado
rota sin desplegar
clave de datos
la política de clave
puede denegar aunque
el permiso de S3 conceda
base de datos
KMS · clave del cliente

Desarrollo

1. Tres tipos de clave, tres niveles de control

Tipo	Quién controla	Se puede	Coste
Gestionada por AWS	AWS	Nada: no se ve ni se audita su uso por separado	Gratis
Gestionada por el cliente	Tú	Política propia, rotación, desactivación, auditoría	1 USD/mes + uso
Material importado	Tú, fuera de AWS	Todo lo anterior más control del material	1 USD/mes + uso

La diferencia práctica entre la primera y la segunda es mayor de lo que sugiere la tabla:

```
clave gestionada por AWS
  no puedes escribir su política
  no puedes desactivarla
  no puedes impedir que otro principal de la cuenta la use
  → el cifrado protege del disco físico y de poco más

clave del cliente
  su política es una segunda barrera independiente de la del recurso
  se puede desactivar y con ello hacer ilegibles los datos al instante
  su uso se audita como evento propio
```

Esa segunda barrera es lo que convierte el cifrado en un control real. Con clave del cliente, alguien con permiso de lectura sobre el bucket pero sin permiso sobre la clave no puede leer nada:

```
{
  "Sid": "Solo el rol de la aplicación descifra",
  "Effect": "Allow",
  "Principal": {"AWS": "arn:aws:iam::123456789012:role/cloudshop-app"},
  "Action": ["kms:Decrypt", "kms:GenerateDataKey"],
  "Resource": "*",
  "Condition": {"StringEquals": {"kms:ViaService": "s3.sa-east-1.amazonaws.com"}}
}
```

La condición kms:ViaService añade una restricción útil: la clave solo se puede usar a través de S3, no directamente. Alguien que consiga el permiso no puede descifrar objetos que haya copiado a otro sitio.

Y el cifrado de sobre explica por qué esto no arruina el rendimiento: KMS no cifra los datos, cifra la clave de datos que sí los cifra. Un objeto de 5 GB implica una llamada a KMS, no cinco mil.

2. Rotación de claves: lo que rota y lo que no

La rotación automática de una clave de KMS crea material criptográfico nuevo y conserva el anterior:

```
los datos cifrados con el material antiguo NO se recifran
KMS guarda todas las versiones y usa la correcta al descifrar
las escrituras nuevas usan el material nuevo
```

Eso tiene dos consecuencias que sorprenden:

1. La rotación no invalida el material antiguo. Si alguien lo obtuvo, sigue sirviendo para lo cifrado antes. Rotar reduce la exposición futura, no la pasada.
2. No hay que hacer nada al rotar. No hay migración, no hay ventana, no hay recifrado. Por eso activarla no tiene coste operativo y no activarla no tiene excusa.

```
$ aws kms enable-key-rotation --key-id 1a2b3c4d --rotation-period-in-days 365
$ aws kms get-key-rotation-status --key-id 1a2b3c4d --query 'KeyRotationEnabled'
true
```

La acción que sí invalida el acceso es desactivar la clave:

```
$ aws kms disable-key --key-id 1a2b3c4d
# a partir de aquí, ningún descifrado funciona
```

Es la respuesta correcta ante un compromiso confirmado, y por eso la clave debe estar en una cuenta donde el atacante no tenga permisos. Si la clave y los datos comparten cuenta y credenciales, desactivarla no es una opción disponible durante el incidente.

Y el borrado de una clave tiene una salvaguarda deliberada: un periodo de espera de entre 7 y 30 días, irreversible después. Borrar una clave hace ilegibles todos los datos cifrados con ella, para siempre. No hay recuperación, ni por soporte. Es el equivalente criptográfico de un borrado seguro y hay que tratarlo con el mismo cuidado.

3. Rotar secretos sin cortar conexiones

Un secreto que rota mal corta las conexiones activas. La estrategia de cuatro pasos existe para evitarlo:

```
createSecret   genera la credencial nueva, la guarda como AWSPENDING
setSecret      la aplica en el servicio destino (crea el usuario nuevo)
testSecret     comprueba que funciona
finishSecret   mueve la etiqueta AWSCURRENT a la nueva versión
```

Durante los pasos 1 a 3 ambas credenciales son válidas. Esa ventana es lo que permite que las conexiones en curso terminen con la antigua mientras las nuevas usan la nueva.

La estrategia de usuario alterno lo lleva más lejos y es la recomendada para bases de datos:

```
usuario_a  activo, contraseña vigente
usuario_b  se rota este; cuando termina, se conmuta la etiqueta
siguiente rotación: se rota usuario_a
```

Así nunca se toca la credencial que la aplicación está usando en ese momento.

El error que anula todo esto: cachear el secreto indefinidamente en la aplicación.

```
# mal: se lee una vez al arrancar y nunca más
SECRETO = obtener_secreto("cloudshop/db")

# bien: caché con caducidad y reintento ante fallo de autenticación
_cache = {"valor": None, "expira": 0}
def secreto():
    if time.time() > _cache["expira"]:
        _cache.update(valor=obtener_secreto("cloudshop/db"), expira=time.time() + 300)
    return _cache["valor"]

def conectar():
    try:
        return db.connect(**secreto())
    except AuthenticationError:
        _cache["expira"] = 0          # forzar relectura: quizá rotó
        return db.connect(**secreto())
```

El bloque except es la pieza que hace la rotación transparente: si la autenticación falla, se relee el secreto antes de rendirse. Sin él, la aplicación falla hasta que alguien la reinicie, y la rotación automática se convierte en una fuente de incidentes programados.

4. Filtrado en el borde: medir antes de bloquear

Las reglas gestionadas de WAF cubren categorías conocidas —inyección SQL, cruce de sitios, entradas maliciosas— y generan falsos positivos con tráfico real. Activarlas en modo bloqueo directamente es una forma fiable de cortar usuarios legítimos.

El procedimiento seguro:

```
1. Añadir la regla en modo cuenta
2. Observar 7-14 días de tráfico real, incluido un cierre de mes
3. Revisar las coincidencias: ¿son ataques o tráfico propio?
4. Excluir las reglas concretas que producen falsos positivos
5. Pasar a bloqueo
```

El paso 3 es donde aparecen las sorpresas: un campo de texto libre donde los usuarios escriben comillas o guiones dispara reglas de inyección; una carga de fichero grande dispara reglas de tamaño de cuerpo.

```
$ aws wafv2 get-sampled-requests --web-acl-arn $ACL --rule-metric-name AWSManagedRulesCommonRuleSet \
    --scope REGIONAL --time-window StartTime=...,EndTime=... --max-items 100 \
  | jq -r '.SampledRequests[] | [.Request.URI, .RuleNameWithinRuleGroup] | @tsv' \
  | sort | uniq -c | sort -rn | head -5
```

La regla más útil y la más barata de todas no es de contenido, es de tasa:

```
{
  "Name": "limite-por-ip",
  "Priority": 1,
```

```
  "Statement": {"RateBasedStatement": {"Limit": 2000, "AggregateKeyType": "IP"}},
  "Action": {"Block": {}}
}
```

Corta la fuerza bruta y el rastreo agresivo sin necesidad de entender el contenido. Dos matices: la ventana de evaluación es de 5 minutos, así que un pico corto puede pasar; y agregando por IP se penaliza a redes con NAT compartida —una oficina entera sale por una IP—, para lo que conviene agregar por otra clave, como una cabecera de sesión.

Y la capa anterior: la protección volumétrica estándar cubre ataques de red sin coste. El nivel avanzado añade respuesta gestionada y protección de costes frente al escalado provocado por un ataque, que es un riesgo real: un ataque que no tumba el servicio pero dispara el autoescalado produce una factura, no una caída.

5. Cada control necesita su prueba negativa

Igual que en la clase 025, la configuración no demuestra el efecto. Cada control de esta clase tiene su comprobación:

```
# 1. La política de clave deniega a quien no debe
$ aws s3api get-object --bucket cloudshop-facturas --key f-1042.pdf /tmp/x \
    --profile rol-sin-permiso-de-clave
An error occurred (AccessDenied) ... not authorized to perform: kms:Decrypt   ✓

# 2. El secreto rotado sigue funcionando sin desplegar
$ aws secretsmanager rotate-secret --secret-id cloudshop/db
$ sleep 60 && curl -s -o /dev/null -w '%{http_code}\n' https://api.cloudshop.cl/health/ready
200                                                                            ✓

# 3. La regla de tasa corta el exceso
$ for i in $(seq 1 2500); do curl -s -o /dev/null https://api.cloudshop.cl/productos; done
$ curl -s -o /dev/null -w '%{http_code}\n' https://api.cloudshop.cl/productos
403                                                                            ✓

# 4. El tráfico legítimo NO se bloquea
$ curl -s -o /dev/null -w '%{http_code}\n' -X POST https://api.cloudshop.cl/opiniones \
    -d '{"texto":"El envío llegó rápido; muy bien -- lo recomiendo"}'
200                                                                            ✓
```

La cuarta es tan importante como las tres primeras y casi nunca se hace: ese texto contiene -- y ;, que las reglas de inyección marcan. Un control que bloquea tráfico legítimo es un incidente igual que uno que deja pasar un ataque, con la diferencia de que se descubre cuando los usuarios se quejan.

Y todas deben ser automáticas y periódicas. Una política de clave correcta hoy puede quedar anulada mañana por otra concesión; una exclusión de WAF puede volverse innecesaria o insuficiente cuando cambia el tráfico.

Ejemplo trabajado

Una revisión de seguridad de CloudShop encuentra tres huecos: cifrado con clave gestionada por AWS, la contraseña de la base de datos en una variable de entorno desde hace 14 meses, y ninguna protección de aplicación en el borde.

Hueco 1 — el cifrado no protegía de nada útil.

```
$ aws s3api get-bucket-encryption --bucket cloudshop-facturas \
    --query 'ServerSideEncryptionConfiguration.Rules[0].ApplyServerSideEncryptionByDefault'
{"SSEAlgorithm": "AES256"}          # clave gestionada por AWS
```

Con esa configuración, cualquiera con s3:GetObject lee los objetos: no hay segunda barrera. Se migra a clave del cliente:

```
$ aws kms create-key --description "cloudshop facturas" --policy file://politica-clave.json
$ aws s3api put-bucket-encryption --bucket cloudshop-facturas \
    --server-side-encryption-configuration '{"Rules":[{
      "ApplyServerSideEncryptionByDefault":{"SSEAlgorithm":"aws:kms",
        "KMSMasterKeyID":"arn:aws:kms:...:key/1a2b3c4d"},
      "BucketKeyEnabled":true}]}'
```

BucketKeyEnabled reduce las llamadas a KMS reutilizando una clave a nivel de bucket:

```
sin clave de bucket: 1 llamada por objeto → 8,4 M llamadas/mes × 0,03/10k = 25,20 USD
con clave de bucket: ~99 % menos          →                                 0,28 USD
```

Prueba negativa con un rol que tiene permiso sobre S3 y no sobre la clave:

```
AccessDenied ... not authorized to perform: kms:Decrypt   ✓
```

Hueco 2 — la contraseña en variable de entorno.

```
$ aws ecs describe-task-definition --task-definition cloudshop-api:41 \
    --query 'taskDefinition.containerDefinitions[0].environment[?name==`DB_PASSWORD`].name' --output
text
DB_PASSWORD
```

Una variable de entorno es visible en la definición de tarea, en los logs de despliegue y para cualquiera que pueda describirla. Se migra:

```
$ aws secretsmanager create-secret --name cloudshop/db \
    --secret-string '{"username":"cloudshop_a","password":"..."}'
$ aws secretsmanager rotate-secret --secret-id cloudshop/db \
    --rotation-lambda-arn arn:...:function:SecretsManagerRDSPostgreSQLRotationMultiUser \
    --rotation-rules AutomaticallyAfterDays=30
```

La primera rotación cortó el servicio 4 minutos. La causa:

```
SECRETO = obtener_secreto("cloudshop/db")   # leído una vez al arrancar
```

La aplicación cacheaba el secreto indefinidamente. Se corrige con caducidad y relectura ante fallo de autenticación, y se repite la prueba:

```
rotación #2:  0 s de indisponibilidad, 0 errores    ✓
```

Hueco 3 — nada en el borde. Se añaden reglas gestionadas en modo cuenta:

```
tras 10 días en modo cuenta:
  coincidencias totales                   18.412
  ataques reales (inyección, rastreo)     17.903
  FALSOS POSITIVOS                           509
```

Los 509 se concentran en un sitio:

```
$ aws wafv2 get-sampled-requests ... | jq -r '.SampledRequests[].Request.URI' \
  | sort | uniq -c | sort -rn | head -2
    486 /opiniones
     23 /soporte/adjuntar
```

Opiniones de clientes con guiones y comillas disparaban SQLiBODY. Se excluye esa regla solo para esa ruta, no globalmente:

```
scope-down statement: NOT (URI = "/opiniones")
→ la regla sigue activa en todo lo demás
```

Y se añade la regla de tasa, que resultó ser la más efectiva:

```
límite 2.000 peticiones / 5 min por IP
bloqueos en la primera semana: 41 IP
tráfico malicioso cortado: 63 % del total de bloqueos
```

Resultado, con las cuatro pruebas:

```
cifrado con segunda barrera                no        sí       kms:Decrypt denegado ✓
secreto en variable de entorno             sí        no       rotación sin caída ✓
rotación automática                        no        30 días  ejecutada 2 veces ✓
filtrado de aplicación                     no        sí       exceso bloqueado ✓
tráfico legítimo con caracteres especiales n/a       pasa     opinión con -- : 200 ✓
coste añadido                               —      +38 USD/mes
```

Los 509 falsos positivos son el dato que justifica el modo cuenta: activadas en bloqueo desde el principio, habrían impedido escribir opiniones a 486 clientes sin que nadie relacionara la causa.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/035-kms-secrets-manager-waf-y-controles-de-seguridad/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.

2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa controles-aws en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye controles-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

■■ Errores frecuentes

Síntoma	Causa probable	Corrección
El cifrado está activado y cualquiera con permiso de lectura ve los datos	Clave gestionada por AWS: no hay política propia que actúe como segunda barrera	Usa clave del cliente con política que restrinja el descifrado a los roles previstos.
La primera rotación automática corta el servicio	La aplicación lee el secreto una vez al arrancar y lo cachea indefinidamente	Caché con caducidad y relectura ante fallo de autenticación.
Activar reglas de WAF bloquea a usuarios legítimos	Se pasó a bloqueo sin medir falsos positivos con tráfico real	Modo cuenta 7-14 días, excluye reglas concretas por ruta y solo entonces bloquea.
La factura de KMS crece con el volumen de objetos	Una llamada a KMS por objeto sin clave de bucket	Activa la clave de bucket: reduce las llamadas en torno al 99 %.
Durante un compromiso no se puede revocar el acceso a los datos	La clave vive en la misma cuenta que el atacante controla	Sitúa la clave en una cuenta separada; desactivarla es la palanca de emergencia.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué aporta una clave gestionada por el cliente que una gestionada por AWS no puede aportar?
2. ¿Recifra la rotación de una clave de KMS los datos existentes? ¿Qué implica eso si el material antiguo se filtró?
3. ¿Por qué la rotación de secretos necesita una fase con dos versiones válidas?
4. ¿Qué código debe tener una aplicación para que una rotación automática le resulte transparente?

5. ¿Por qué una prueba de que el tráfico legítimo pasa es tan necesaria como la de que el ataque se bloquea?

Referencias

- AWS (2024). AWS KMS: key policies — segunda barrera independiente de los permisos sobre el recurso. <https://docs.aws.amazon.com/kms/latest/developerguide/key-policies.html>
- AWS (2024). Rotating AWS KMS keys — qué rota, qué se conserva y por qué no hay recifrado. <https://docs.aws.amazon.com/kms/latest/developerguide/rotate-keys.html>
- AWS (2024). Secrets Manager rotation — los cuatro pasos y la estrategia de usuario alterno. <https://docs.aws.amazon.com/secretsmanager/latest/userguide/rotating-secrets.html>
- AWS (2024). AWS WAF managed rule groups — modo cuenta, exclusiones y sentencias de reducción de alcance. <https://docs.aws.amazon.com/waf/latest/developerguide/waf-managed-rule-groups.html>
- AWS (2024). S3 Bucket Keys — reducción de llamadas a KMS y su efecto en el coste. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/bucket-key.html>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 02 en PDF · Recorrido de AWS en PDF · Manual integral

Anterior	Índice	Siguiente
← 034 · CloudWatch, CloudTrail, Config y Systems Manager	Parte 02 · Programa	036 · Proyecto: aplicación de tres capas en AWS →

Evaluación — 035 KMS, Secrets Manager, WAF y controles de seguridad

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega controles-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

036 — Proyecto: aplicación de tres capas en AWS

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Integrar las once clases anteriores en una plataforma que funcione, se observe, falle de forma controlada y cueste lo que se decidió que costara. No es un ejercicio de montar servicios: es demostrar con evidencia que cada decisión de las clases 025 a 035 produce el efecto que se le atribuyó, incluidas las que resulten estar mal.

Resultados de aprendizaje

Al finalizar podrás:

1. Ensamblar una aplicación de tres capas con las decisiones de las once clases previas, cada una justificada.
2. Demostrar con prueba negativa que los controles de identidad, red y cifrado deniegan lo que deben.
3. Medir una línea base de latencia y coste unitario que sirva de referencia para las partes siguientes.
4. Provocar tres fallos —zona, dependencia y credencial— y documentar el comportamiento observado.
5. Entregar un paquete de evidencia que otra persona pueda verificar sin conocimiento tácito.

Conceptos centrales

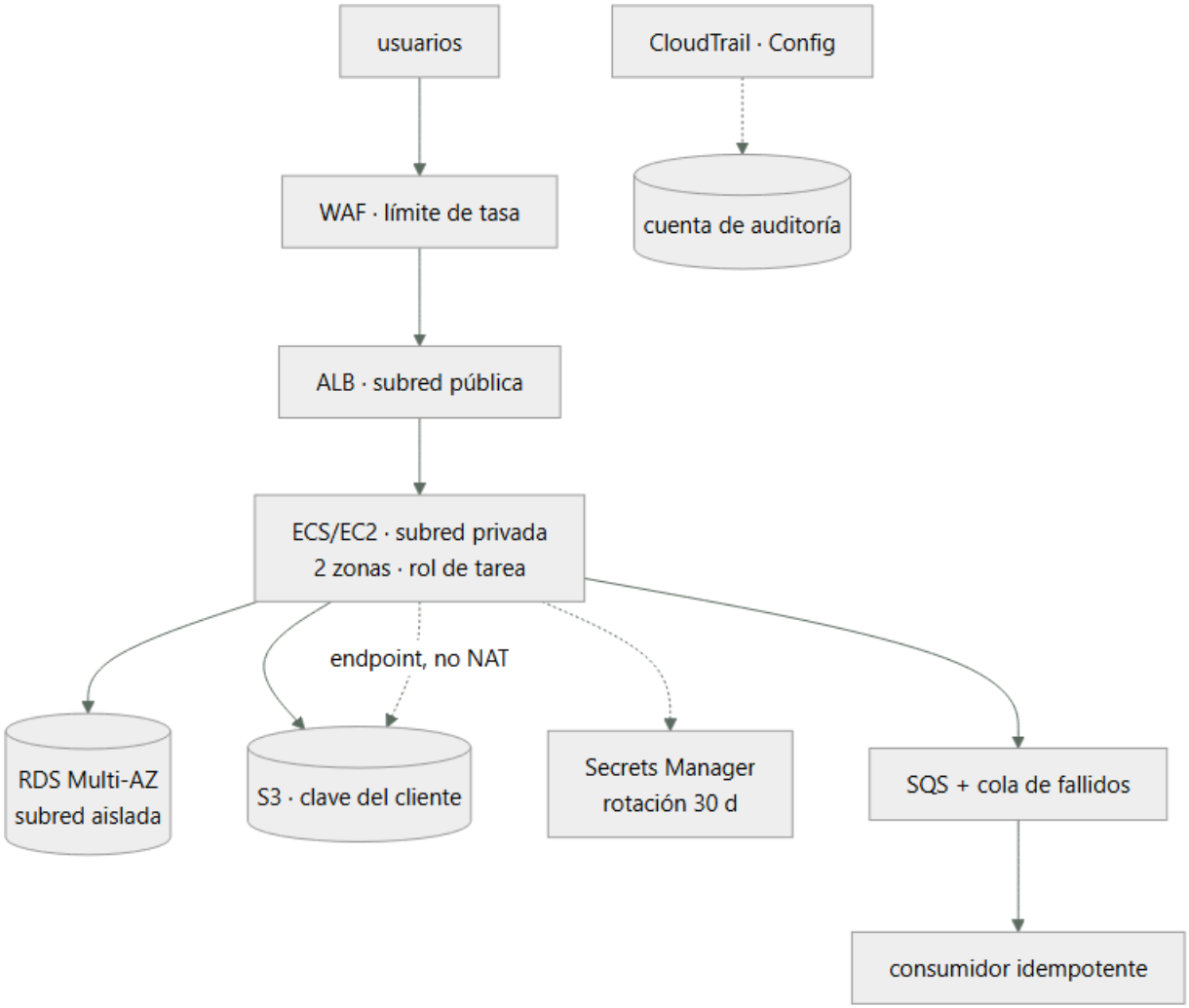
Concepto	Comprensión verificable
paquete de evidencia	Conjunto de salidas reproducibles que respaldan cada afirmación del diseño. Sustituye a «está bien configurado» por el comando ejecutado y su resultado.
prueba de fallo	Inyección deliberada de una avería para observar el comportamiento real. Un diseño resiliente que nunca se probó es una hipótesis, no una propiedad.
línea base	Medición inicial de latencia, rendimiento y coste unitario contra la que se comparará todo cambio posterior. Sin ella, «mejoró» es una opinión.
criterio de aceptación	Condición verificable que decide si el trabajo está terminado. Debe poder evaluarla alguien distinto de quien lo construyó.
riesgo residual	Lo que sigue siendo vulnerable tras aplicar los controles. Nombrarlo es parte de la entrega; omitirlo convierte un límite conocido en una sorpresa futura.

Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. La arquitectura y de dónde sale cada decisión

Nada en este capstone es una elección nueva: todo procede de una clase anterior, y esa trazabilidad es lo que hay que poder defender.

Componente	Decisión	Viene de
Cuentas separadas por entorno	Cuotas y radio de impacto	017, 025
Rol federado para el pipeline	Cero credenciales de larga vida	018, 026
Subred aislada para la base	Sin ruta a internet	027
Endpoints en vez de NAT	86 % del tráfico no va a internet	027
m7g en vez de t3	La media supera la línea base	028
Seguimiento de objetivo	Histéresis automática	016, 029
Drenado de 60 s	Mayor que la petición más larga	029
Bloqueo de objetos	El versionado no frena un borrado con permisos	010, 030
Índice compuesto	El orden estaba fuera del índice	031
Cola con fallidos e idempotencia	Entrega al menos una vez	009, 033
Clave del cliente	Segunda barrera independiente	035
Modo cuenta antes de bloquear	509 falsos positivos medidos	035

La columna derecha es el entregable real. Una arquitectura sin esa columna es una lista de servicios, y es exactamente lo que la clase 021 identificaba como diseño sin compromisos declarados.

Y hay decisiones que se toman en contra de lo que parece mejor, y también se registran: dos zonas en vez de tres porque el requisito es 21,6 min/mes y con dos ya sobran 20; RDS en vez de DynamoDB porque la agregación mensual no tiene equivalente. Justificar lo que no se hizo es tan parte de la defensa como lo que sí.

2. Cada control necesita su prueba negativa

La configuración no demuestra el efecto. El paquete de evidencia se construye ejecutando intentos que deben fallar y intentos que deben funcionar:

```
# Identidad: un repositorio ajeno no puede asumir el rol
$ (desde repo externo) aws sts assume-role-with-web-identity ...
AccessDenied                                                    ✓

# Red: la base de datos no alcanza internet
$ aws ssm start-session --target $ID_BASTION
$ curl -m 5 https://example.com
curl: (28) Connection timed out                                  ✓
$ aws secretsmanager get-secret-value --secret-id cloudshop/db --query Name
"cloudshop/db"                                                  ✓ el endpoint sí

# Cifrado: sin permiso de clave no se lee, aunque haya permiso de S3
$ aws s3api get-object --bucket cloudshop-facturas --key f.pdf /tmp/x --profile sin-kms
not authorized to perform: kms:Decrypt                           ✓

# Almacenamiento: no se puede reabrir al público
$ aws s3api put-bucket-acl --bucket cloudshop-facturas --acl public-read
AccessDenied ... blocked by the public access block              ✓

# Guardarraíl: no se puede desactivar la auditoría
$ aws cloudtrail stop-logging --name org-trail
AccessDenied ... explicit deny in a service control policy       ✓
```

Y las positivas, que son las que se olvidan:

```
$ curl -s -o /dev/null -w '%{http_code}\n' https://api.cloudshop.cl/health/ready
200                                                              ✓
$ curl -s -X POST https://api.cloudshop.cl/opiniones \
    -d '{"texto":"llegó rápido -- muy bien"}' -o /dev/null -w '%{http_code}\n'
200                                                              ✓ WAF no bloquea
```

Cinco negativas y dos positivas. Un control que deniega todo también «pasa» las cinco primeras, y solo las positivas distinguen seguridad de indisponibilidad.

3. La línea base es el entregable que más dura

Las partes 03 a 23 compararán contra estos números, así que hay que medirlos bien y guardarlos versionados.

```
$ hey -z 120s -c 50 https://api.cloudshop.cl/productos/A-1042

Summary:
  Requests/sec: 987.42
Latency distribution:
  50% in 0.0384 secs
  95% in 0.0912 secs
  99% in 0.1847 secs
Status code distribution:
  [200] 118496 responses
```

Se verifica la coherencia con la ley de Little de la clase 011:

```
L = λ × W = 987 × 0,0384 = 37,9 concurrentes
solicitados: 50 → hay holgura; la medida no está limitada por el generador  ✓
```

Si L hubiera dado 50 exactos, el número mediría el generador de carga y no el servicio.

Y el coste unitario, con el denominador de la clase 011:

```
cómputo (3 × m7g.large)             139,00
base de datos (Multi-AZ)            412,00
almacenamiento y endpoints           58,80
```

```
balanceador y WAF                     47,20
observabilidad                        31,40
                                     -------
total                                688,40

pedidos/mes                          980.000
coste unitario            688,40 / 980.000 = 0,000702 USD/pedido
```

Se registra como contrato comparable, con el escenario incluido:

```
{"escenario":"productos-lectura","concurrencia":50,"duracion_s":120,
 "rps":987.4,"p50_ms":38.4,"p95_ms":91.2,"p99_ms":184.7,"errores":0,
 "coste_mensual_usd":688.40,"coste_unitario_usd":0.000702,
 "zonas":2,"instancias":3,"fecha":"2026-08-01"}
```

La relación p99/p50 = 4,8× es la métrica a vigilar entre versiones: si crece, la cola se alarga aunque la media no se mueva.

4. Tres fallos provocados y lo que enseña cada uno

Fallo 1 — pérdida de una zona.

```
$ aws ec2 modify-network-acl-entry --network-acl-id $ACL_ZONA_B \
    --rule-number 100 --egress --protocol -1 --rule-action deny --cidr-block 0.0.0.0/0
```

```
t+0    se aísla la zona B
t+31   el balanceador marca insanos sus destinos (10 s × 3)
t+31   el tráfico se concentra en la zona A
t+34   RDS conmuta a la réplica de la zona A
t+96   el autoescalado añade 2 instancias en A
p95 durante la ventana:  91 ms → 214 ms → 97 ms
errores:  0
```

Los 31 segundos de detección son el coste de los parámetros de la clase 029. Bajarlos aumentaría los falsos positivos.

Fallo 2 — dependencia opcional degradada. No caída, sino lenta, que es el caso que rompe sistemas:

```
$ aws ssm send-command --targets Key=tag:Servicio,Values=recomendaciones \
    --document-name AWS-RunShellScript \
    --parameters 'commands=["tc qdisc add dev eth0 root netem delay 800ms"]'
```

```
p95 sin degradación   91,2 ms
p95 con la dependencia a 800 ms   94,7 ms
```

El plazo de 150 ms corta la llamada. Sin él, cada petición ocuparía un trabajador 800 ms y por la ley de Little la concurrencia necesaria se multiplicaría por ocho.

Fallo 3 — credencial comprometida. Se simula qué obtiene un atacante con el rol de tarea:

```
$ aws iam list-users                    AccessDenied     ✓
$ aws rds delete-db-instance ...        AccessDenied     ✓
$ aws s3 rm s3://cloudshop-facturas/... AccessDenied     ✓ (bloqueo de objetos)
$ aws s3 ls s3://cloudshop-facturas/    lista objetos    ← SÍ puede
$ aws s3 cp s3://cloudshop-facturas/f-1042.pdf .   descarga    ← SÍ puede
```

El rol puede leer y exfiltrar las facturas. No es un fallo de configuración: es el permiso que la aplicación necesita. El control que queda es detectivo —alerta por volumen anómalo de lecturas— y hay que declararlo como riesgo residual, no ocultarlo.

5. La entrega: sin conocimiento tácito

El criterio de aceptación no es que funcione, sino que otra persona lo verifique sin preguntarte nada.

```
capstone-aws/
■■■ README.md          qué es, cómo se despliega, cómo se comprueba, cómo se retira
■■■ infra/             la infraestructura declarada, no clicada
■■■ evidencia/
■   ■■■ negativas.md   los 5 intentos que deben fallar, con su salida
■   ■■■ positivas.md   los 2 que deben funcionar
■   ■■■ baseline.json  la línea base con su escenario
■   ■■■ fallo-zona.md  cronología medida
■   ■■■ fallo-dep.md   p95 con y sin degradación
■   ■■■ fallo-cred.md  qué obtiene un atacante con el rol de tarea
■■■ adr/               las decisiones con sus alternativas descartadas
■■■ verify.sh          ejecuta todas las comprobaciones y sale != 0 si alguna falla
```

verify.sh es la pieza que convierte la evidencia en algo vivo:

```
$ ./verify.sh
✓ rol federado deniega desde repositorio ajeno
✓ base de datos sin salida a internet
✓ objeto no legible sin permiso de clave
✓ bucket no reabrible al público
✓ CloudTrail no desactivable
✓ salud del servicio: 200
✓ opinión con caracteres especiales: 200
7/7 comprobaciones correctas
```

Un script que imprime «OK» y siempre sale 0 no verifica nada: el código de salida es el contrato, como en la clase 012. Y con él, estas mismas comprobaciones se ejecutan en CI en la parte 08 sin cambiar una línea.

Riesgos residuales que se entregan declarados:

```
1. El rol de tarea puede leer y exfiltrar facturas. Mitigación detectiva
   (alerta por volumen), no preventiva. Aceptado por el responsable de datos.
2. No sobrevive a un fallo regional completo. Aceptado: no es requisito hoy.
3. La línea base se midió con datos sintéticos; la distribución real de
   tamaños de pedido puede diferir.
4. El WAF excluye la regla de inyección en /opiniones. Compensado con
   validación y consultas parametrizadas en esa ruta.
```

Esa lista es la parte más valiosa de la entrega. Un capstone sin riesgos residuales declarados no es que no los tenga: es que no los buscó.

Ejemplo trabajado

Entrega del capstone de la parte 02, con las cifras que se llevan a la parte 03.

Verificación completa:

```
$ ./verify.sh
✓ rol federado deniega desde repositorio ajeno       AccessDenied
✓ base de datos sin salida a internet                timeout a los 5 s
✓ objeto no legible sin permiso de clave             kms:Decrypt denegado
✓ bucket no reabrible al público                     bloqueado
✓ CloudTrail no desactivable                         deny de SCP
✓ salud del servicio                                 200
✓ opinión con " -- " y ";"                           200
7/7 correctas
```

Línea base:

```
rps 987,4 · p50 38,4 ms · p95 91,2 ms · p99 184,7 ms · errores 0
L = 987 × 0,0384 = 37,9 de 50 solicitados → medida válida
p99/p50 = 4,8×
coste 688,40 USD/mes · 0,000702 USD/pedido
```

Los tres fallos, con lo aprendido en cada uno:

```
zona aislada               31 s        214 ms        0       el coste de detección
                                                               son los parámetros de salud
dependencia a 800 ms        —          94,7 ms       0       el plazo hizo el trabajo;
                                                               sin él, ×8 de concurrencia
credencial comprometida     —            —           —       lee facturas: riesgo residual
```

Un hallazgo que la prueba de fallo destapó y la configuración no. Durante el aislamiento de la zona B:

```
$ aws logs filter-log-events --log-group-name /aws/ecs/cloudshop \
    --filter-pattern 'ERROR' --start-time ... --query 'length(events)'
0
$ aws sqs get-queue-attributes --queue-url $URL_DLQ \
    --attribute-names ApproximateNumberOfMessagesVisible
{"ApproximateNumberOfMessagesVisible": "14"}
```

14 mensajes en la cola de fallidos sin ningún error en los logs. El consumidor vivía solo en la zona B; al aislarla, sus mensajes agotaron reintentos mientras el servicio HTTP —que sí estaba en dos zonas— seguía en verde.

```
síntoma observable   ninguno: cero errores, p95 aceptable, alarmas en verde
consecuencia real    14 pedidos sin facturar
causa                el consumidor asíncrono no estaba en dos zonas
```

Se corrige y se añade lo que faltaba:

```
1. consumidor desplegado en dos zonas
2. alerta sobre la cola de fallidos con umbral cero (clase 033)
3. la prueba de fallo de zona pasa a incluir la ruta asíncrona,
   no solo la síncrona
```

El valor del capstone fue este hallazgo. Todo estaba correctamente configurado, todas las pruebas negativas pasaban, y había un camino —el asíncrono— que nadie había probado porque el fallo no era visible desde fuera.

Se entrega a la parte 03 con:

```
línea base       para comparar la misma aplicación en Azure
ADR              9 decisiones con sus alternativas descartadas
4 riesgos residuales declarados y aceptados por su responsable
verify.sh        reutilizable, con código de salida
```

Y la pregunta que abre la parte siguiente: de estas nueve decisiones, ¿cuáles son de arquitectura y cuáles son de proveedor? Las primeras deberían reaparecer en Azure con otro nombre; las segundas, no reaparecer en absoluto.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/036-proyecto-aplicacion-de-tres-capas-en-aws/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa plataforma-aws en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye plataforma-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

■■ Errores frecuentes

Síntoma	Causa probable	Corrección
El capstone es una lista de servicios desplegados	Falta la trazabilidad de cada decisión a la clase y al requisito que la motivó	Cada componente debe citar qué requisito satisface y qué alternativa se descartó.
Todas las pruebas negativas pasan y el sistema no funciona	Un control que deniega todo también las pasa; faltan las pruebas positivas	Empareja cada prueba negativa con una positiva del acceso legítimo.
Un fallo de zona deja trabajo asíncrono sin procesar y nada lo detecta	Solo se probó el camino síncrono, que sí estaba en dos zonas	Incluye colas y consumidores en la prueba de fallo, y alerta sobre la cola de fallidos.
No se puede afirmar si un cambio posterior mejoró o empeoró	No hay línea base con escenario, percentiles y coste unitario	Registra la medición versionada junto al código, con la carga y la duración usadas.
La entrega depende de que su autor explique algo	Conocimiento tácito no escrito y verificación no automatizada	Exige que otra persona ejecute verify.sh sin ayuda; lo que pregunte es lo que falta en el README.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. Elige tres componentes de tu capstone y di de qué clase y de qué requisito procede cada decisión.
2. ¿Por qué cinco pruebas negativas correctas no demuestran que el sistema sea seguro Y usable?
3. Tu medición da L = 50 con concurrencia solicitada de 50. ¿Qué estás midiendo en realidad?
4. Durante un fallo de zona no hay errores ni alarmas, pero se pierden pedidos. ¿Dónde buscarías?
5. De las decisiones de tu capstone, ¿cuáles esperas volver a tomar en Azure y cuáles no reaparecerán?

Referencias

- AWS (2024). Well-Architected Framework: the review process — cómo estructurar la revisión de una carga. <https://docs.aws.amazon.com/wellarchitected/latest/framework/the-review-process.html>
- AWS (2024). Fault Injection Service — inyección controlada de fallos de zona, red y recursos. <https://docs.aws.amazon.com/fis/latest/userguide/what-is.html>
- Beyer, B. et al., eds. (2018). The Site Reliability Workbook, cap. 5 — pruebas de fiabilidad y game days. <https://sre.google/workbook/testing-reliability/>
- Nygard, M. (2018). Release It!, 2.ª ed., cap. 5 — plazos, mamparos y degradación bajo dependencia lenta.
- Nygard, M. (2011). Documenting Architecture Decisions — formato de los ADR que acompañan la entrega. <https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 02 en PDF · Recorrido de AWS en PDF · Manual integral

Anterior	Índice	Siguiente
← 035 · KMS, Secrets Manager, WAF y controles de seguridad	Parte 02 · Programa	037 · Tenant, management groups, suscripciones y resource groups →

Evaluación — 036 Proyecto: aplicación de tres capas en AWS

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega plataforma-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 06 — Kubernetes y plataformas administradas

083 — EKS, AKS y GKE: similitudes y diferencias

Parte: 06 — Kubernetes y plataformas administradas Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: decision · Estado: EXECUTABLECORE

Propósito

Comparar las tres plataformas gestionadas de Kubernetes por lo que de verdad diferencia la operación, y no por una tabla de funciones. Tres ejes deciden casi todo: quién decide cuándo actualizas, cuántos pods caben en tu plan de direcciones y cómo obtiene una carga la identidad de la nube. Y la conclusión que interesa al programa es cuantificable: qué parte del trabajo de las clases 073 a 082 se conserva al cambiar de proveedor y qué parte hay que rehacer.

Resultados de aprendizaje

Al finalizar podrás:

1. Delimitar qué gestiona el proveedor y qué sigue siendo trabajo propio.
2. Anticipar una actualización forzada por fin de soporte y lo que puede bloquearla.
3. Calcular cuántos pods caben según el modelo de red de cada plataforma.
4. Federar la identidad de una carga con la nube, acotando la confianza correctamente.
5. Estimar qué se conserva y qué se rehace al cambiar de plataforma gestionada.

Conceptos centrales

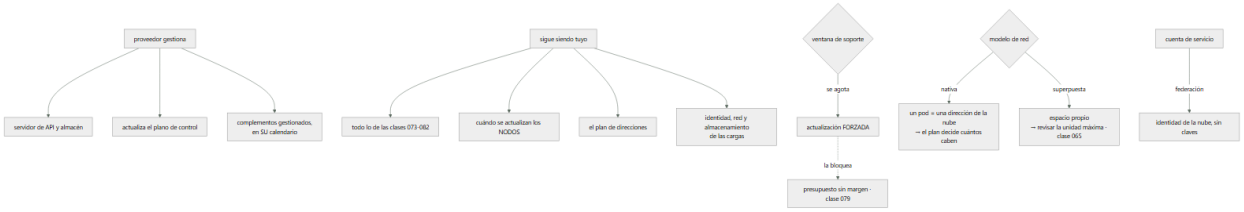
Concepto	Comprensión verificable
plano de control gestionado	El proveedor opera el servidor de API y el almacén de estado. No opera tus cargas ni tus complementos, que es donde está casi todo el trabajo de esta parte.
ventana de soporte	Periodo durante el cual una versión recibe correcciones. Al agotarse, el proveedor actualiza el clúster aunque nadie lo pida.
dirección de pod nativa de la red	Cada pod recibe una dirección del espacio de la nube. Simplifica el enrutamiento y consume el plan de direcciones de las clases 039 y 051.
red superpuesta	Los pods usan un espacio propio y su tráfico se encapsula. Ahorra direcciones y añade una capa —con la trampa de la unidad máxima de transmisión de la clase 065.
identidad federada de carga	La cuenta de servicio del clúster obtiene credenciales de la nube sin ninguna clave. Séptima aparición del mismo contrato del programa.
complemento gestionado	Pieza que el proveedor instala y actualiza en su calendario. Quita trabajo y quita control: su versión puede cambiar sin que nadie lo decida.
desviación de versiones	Diferencia admitida entre el plano de control y los nodos. Permite actualizar por fases y tiene un límite que, si se cruza, deja nodos sin soporte.

Modelo mental

Kubernetes es un sistema de reconciliación: declaras estado deseado y controladores reducen continuamente la diferencia con el estado observado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Qué gestiona el proveedor, y qué no

«Gestionado» se entiende como más de lo que es, y conviene delimitarlo antes de comparar nada:

```
el proveedor gestiona
  el servidor de API y el almacén de estado: disponibilidad, copias, parches
  la actualización del plano de control
  algunos complementos, en su calendario
  la integración con su red, su almacenamiento y su identidad

sigue siendo trabajo propio
  ABSOLUTAMENTE todo lo de las clases 073 a 082
  cuándo y cómo se actualizan los NODOS
  el plan de direcciones y sus consecuencias
  las políticas de red, de admisión y de permisos
  la observabilidad, las copias de las cargas con estado y su restauración
```

Es decir: el proveedor quita el trabajo que menos se nota y deja el que produce los incidentes de esta parte. Y merece decirse con claridad porque la expectativa contraria lleva a no dedicar equipo:

> Un clúster gestionado no reduce el trabajo de operar aplicaciones en Kubernetes. Reduce el de operar Kubernetes.

Sobre el coste del plano de control, las tres cobran de forma distinta y la cifra rara vez decide:

```
una tarifa por hora y por clúster, con independencia del tamaño
o una tarifa de gestión con algún nivel gratuito acotado
o un nivel gratuito sin acuerdo de servicio y uno de pago con él
```

Lo que sí decide es la consecuencia estructural: un clúster tiene un coste fijo, así que muchos clústeres pequeños cuestan más que uno grande. Y esa aritmética empuja a compartir clúster entre equipos, lo que devuelve al aislamiento de la clase 080 y a su límite: los espacios de nombres no bastan para inquilinos no confiables.

Y la decisión que aparece pronto, con la misma forma en las tres plataformas:

```
un clúster por entorno            frontera clara, coste fijo × entornos
un clúster con espacios por equipo  más barato, aislamiento más débil
un clúster por equipo y entorno     lo más aislado y lo más caro de operar
```

La respuesta razonable en la mayoría de las organizaciones es un clúster por entorno con espacios de nombres por equipo, y producción separada de todo lo demás — que es la misma conclusión a la que llegaron las clases 025, 037 y 049 con cuentas, suscripciones y proyectos. Cuarta aparición del mismo criterio.

2. Quién decide cuándo actualizas

Este es el eje que más cambia la operación y el que menos aparece en las comparativas.

Cada versión tiene una ventana de soporte de aproximadamente un año, y al agotarse el proveedor actualiza el clúster:

```
se publica una versión              cada ~4 meses
soporte estándar                    ~12-14 meses desde su publicación
soporte extendido, si existe         más meses, con sobrecoste
al agotarse                         ACTUALIZACIÓN AUTOMÁTICA por el proveedor
```

De ahí salen tres consecuencias operativas:

Actualizar es una tarea recurrente, no un proyecto. Con una ventana de un año y una versión cada cuatro meses, un clúster se actualiza dos o tres veces al año, siempre. Un equipo que trate cada actualización como un proyecto excepcional vive permanentemente en uno.

Lo que bloquea la actualización lo has puesto tú. El proveedor respeta los presupuestos de interrupción de la clase 079, así que un presupuesto sin margen bloquea también su actualización automática. Y ahí ocurre lo peor:

```
el proveedor intenta actualizar
un presupuesto sin margen bloquea el vaciado del nodo
la actualización se detiene o se agota su plazo
el clúster se queda con nodos en dos versiones, o sin actualizar
→ y el aviso llega por correo a una dirección que nadie lee
```

Es el incidente de la clase 079 con un origen distinto y una consecuencia mayor, porque termina con nodos ejecutando una versión sin soporte.

La desviación de versiones limita el orden. El plano de control se actualiza primero y los nodos después, y hay un margen máximo entre ambos:

```
los nodos pueden ir por detrás del plano de control, hasta un límite
nunca por delante
→ y saltarse versiones intermedias no está soportado:
  hay que pasar por cada una
```

La última línea es la que convierte un retraso en un problema: un clúster tres versiones por detrás necesita tres actualizaciones encadenadas, cada una con su ventana y su riesgo.

Y las funciones que se retiran son la otra mitad del trabajo. Cada versión elimina versiones de API antiguas, y un manifiesto que las use deja de aplicarse:

```
$ kubectl get --raw /metrics | grep apiserver_requested_deprecated_apis
```

Esa métrica dice qué APIs obsoletas se están usando ahora mismo y quién. Debería revisarse antes de cada actualización, y es la comprobación que evita descubrir el problema con el clúster ya actualizado.

Y una recomendación que se paga sola: un clúster de pruebas una versión por delante del de producción. Cuesta el plano de control y unos nodos pequeños, y convierte cada actualización en algo ya ensayado.

3. Cuántos pods caben: el modelo de red decide

Aquí está la divergencia real entre plataformas, y tiene consecuencias que se descubren tarde.

Hay dos modelos:

```
DIRECCIONES NATIVAS
  cada pod recibe una dirección del espacio de la nube
  ventaja: el enrutamiento es directo; los cortafuegos y el registro de flujo
           de las clases 039 y 051 ven al pod, no al nodo
  coste:   consume el plan de direcciones, y mucho

RED SUPERPUESTA
  los pods usan un espacio propio y el tráfico se encapsula
  ventaja: no consume direcciones de la nube
  coste:   una capa más, y la unidad máxima de transmisión de la clase 065
```

Y el cálculo que hay que hacer antes de crear el clúster, no después:

```
con direcciones nativas y una subred /22 (1.024 direcciones)
  reservadas por la nube                    ~5
  una por nodo                              × nodos
  una por pod                               × pods
  y direcciones en tránsito durante los despliegues, que no se liberan al instante

40 nodos × 30 pods = 1.200 direcciones necesarias
→ una /22 NO llega; hace falta al menos una /21, con margen
```

La última línea del cálculo es la que se olvida: durante un despliegue coexisten pods viejos y nuevos, así que el pico de direcciones es mayor que el estado estable. Y el síntoma del agotamiento es característico y despista:

```
pods en ContainerCreating de forma intermitente, sobre todo al desplegar
  failed to allocate for range 0: no IP addresses available in range
```

Parece un problema del complemento de red y es un problema de aritmética de la clase 051.

Y hay un segundo límite propio de algunos modelos: el número de pods por nodo puede depender del tipo de máquina, porque las direcciones se asignan al nodo en bloques ligados a sus interfaces. Un nodo pequeño puede admitir muy pocos pods aunque le sobren CPU y memoria:

```
síntoma   nodos con recursos libres y pods en Pending
          el suceso menciona pods insuficientes, no CPU ni memoria
```

Ese caso se corrige con máquinas mayores o con modos que reparten direcciones por prefijo en vez de una a una, y hay que conocerlo porque contradice la intuición de la clase 078: no siempre falta capacidad; a veces faltan direcciones.

Y los rangos secundarios que la clase 051 pedía reservar aparecen aquí con su factura: en las plataformas que los usan, pods y servicios consumen bloques propios, grandes y fijados al crear el clúster. Ampliarlos después no siempre es posible, lo que convierte el plan de direcciones en otra decisión de un solo sentido.

Y la conclusión que enlaza las dos cosas:

```
el tamaño máximo del clúster lo decide, en la práctica,
el plan de direcciones que alguien hizo antes de crearlo
```

4. La identidad, por séptima vez

Las tres plataformas resuelven lo mismo con el mismo mecanismo y distinta sintaxis: una carga del clúster obtiene credenciales de la nube sin ninguna clave.

```
la cuenta de servicio del pod recibe un testigo firmado por el clúster
la nube confía en el emisor del clúster
y cambia ese testigo por credenciales suyas
```

Es el mismo contrato de las clases 026, 038, 050, 069 y 080. Séptima aparición, con la misma pieza crítica:

```
la confianza del lado de la NUBE debe acotarse al espacio de nombres
y a la cuenta de servicio concretos
```

Sin acotar, cualquier pod del clúster puede pedir esa identidad. Y es exactamente el mismo fallo que el sujeto sin acotar de las federaciones anteriores, con un cuarto vocabulario:

```
condición de confianza correcta
  emisor = el del clúster
  sujeto = system:serviceaccount:<espacio>:<cuenta>
```

La comprobación es la misma prueba negativa que este programa ha ejecutado cuatro veces:

```
# desde un pod con la cuenta correcta
$ kubectl exec -n tienda api-x -- /probar-acceso-nube
ok
# desde un pod de otro espacio con otra cuenta
$ kubectl exec -n desarrollo prueba-y -- /probar-acceso-nube
acceso denegado                                                    ✓
```

Y la parte de la sintaxis, que es lo único que cambia entre plataformas:

```
# el mecanismo es el mismo; la anotación y el nombre del recurso, no
apiVersion: v1
kind: ServiceAccount
metadata:
  name: api
  namespace: tienda
  annotations:
    # <clave específica del proveedor>: <identidad de la nube>
```

Y los complementos gestionados merecen su nota porque cambian el reparto de responsabilidades:

```
complemento gestionado
  el proveedor lo instala y lo actualiza en su calendario
  menos trabajo, y su versión puede cambiar sin que nadie lo decida

complemento propio
  se elige la versión y el momento
  más trabajo, y es la única forma de fijar el comportamiento
```

La decisión importa sobre todo en el complemento de red, por lo que la clase 080 mostró: si no implementa políticas, los objetos no filtran nada. Y algunas plataformas ofrecen varios, con soporte distinto de políticas, así que la elección

del complemento al crear el clúster decide si la segmentación de red es posible. Es una decisión de instalación con consecuencias de seguridad, y hay que tomarla con la clase 080 delante.

5. Qué se conserva al cambiar de plataforma

La pregunta que este programa lleva haciendo desde la clase 048 tiene aquí una respuesta cuantificable, y es la mejor noticia de la parte.

```
se conserva SIN CAMBIOS
  todos los manifiestos de carga: despliegues, servicios, configuración,
  presupuestos, políticas de red, perfiles de seguridad, escalado
  las imágenes y su cadena de suministro (clases 061-067)
  el contrato de la aplicación: señales, comprobaciones, apagado (068)
  la instrumentación: el formato de métricas es el mismo (082)
  las herramientas de gestión de manifiestos (081)

hay que REHACER
  la creación del clúster y su plan de direcciones
  la sintaxis de la federación de identidad
  los grupos de nodos y su escalado
  la elección e instalación de complementos
  la integración con el almacenamiento: nombres de clases y parámetros
  la integración con el balanceador de entrada: anotaciones
  la recogida de registros y métricas hacia el sistema del proveedor
```

El primer bloque es grande y el segundo es la capa de plataforma, no las aplicaciones. Expresado como proporción, en una migración real entre dos proveedores:

```
ficheros de manifiesto de aplicaciones      ~95 % sin cambios
capa de plataforma                          ~100 % reescrita
esfuerzo total                              concentrado en 6-8 componentes
```

Y eso confirma con datos lo que la clase 072 predijo: el contrato del contenedor es portable y las fugas están en los bordes. Aquí los bordes tienen nombre: red, almacenamiento, identidad y entrada — las mismas cuatro, por tercera vez.

Y hay una conclusión práctica que conviene sacar y que casi nadie saca: si la capa de plataforma se va a reescribir de todos modos, conviene aislarla desde el principio.

```
repositorio de aplicaciones     manifiestos portables, sin nada del proveedor
repositorio de plataforma       creación del clúster, complementos,
                                clases de almacenamiento, anotaciones de entrada,
                                federación de identidad
```

Con esa separación, cambiar de proveedor toca un repositorio y no doscientos ficheros. Y tiene un beneficio inmediato aunque nunca se cambie: hace visible qué parte del sistema está atada al proveedor, que es una cifra que casi ninguna organización conoce.

Y un aviso honesto sobre la portabilidad, para no exagerarla: que los manifiestos sean portables no significa que el sistema lo sea. Los servicios gestionados que las cargas usan —bases de datos, colas, almacenamiento de objetos— siguen siendo del proveedor, y suelen ser la parte más difícil de mover. Kubernetes hace portable el cómputo, que es la parte que ya era más fácil.

La lista de comprobación para elegir y montar una plataforma gestionada:

- plan de direcciones calculado para el tamaño máximo previsto, con margen de despliegue, y comprobado el límite de pods por nodo
- complemento de red que implementa políticas, verificado con prueba negativa
- federación de identidad acotada a espacio y cuenta, con prueba negativa
- ventana de soporte anotada, con fecha de la próxima actualización obligatoria
- clúster de pruebas una versión por delante
- métrica de APIs obsoletas revisada antes de cada actualización
- presupuestos con margen, para que la actualización del proveedor no se bloquee
- separación entre repositorio de aplicaciones y de plataforma
- complementos: decidido cuáles son gestionados y cuáles propios, y por qué

Ejemplo trabajado

CloudShop opera un clúster en una nube y evalúa mover una parte a otra. El ejercicio se hace en serio —con una migración real de un entorno— y produce cinco datos que ninguna comparativa daba.

Dato 1 — el 94 % de los manifiestos no cambió.

```
manifiestos de aplicaciones                214          13
  de esos 13:
    clase de almacenamiento                  4
    anotaciones del objeto de entrada        5
    anotaciones de cuenta de servicio        3
    tolerancias de nodos especiales          1
capa de plataforma                          31          31
```

Los trece cambios son exactamente las cuatro fugas de la clase 072, y ninguno estaba en la lógica de la aplicación.

```
reescribir la capa de plataforma          9 días-persona
adaptar los 13 manifiestos                1 día-persona
validar y comparar                        4 días-persona
```

Y la conclusión que se documentó: el coste de cambiar de plataforma gestionada está en seis componentes de infraestructura, no en las aplicaciones.

Dato 2 — el plan de direcciones limitaba el clúster a 27 nodos.

Al dimensionar el clúster nuevo:

```
subred asignada                    /22 · 1.024 direcciones
modelo                             direcciones nativas
pods por nodo previstos            30
nodos que caben                    1.024 / 31 ≈ 33
menos margen de despliegue (~20 %)  ≈ 27
```

El clúster original tenía el mismo problema y no lo sabían: llevaban meses sin poder pasar de 24 nodos, y lo atribuían a cuotas del proveedor.

```
$ kubectl describe node nodo-14 | grep -i 'pods:'
  pods:  29        ← límite por tipo de máquina, no por CPU ni memoria
```

```
subred del clúster                          /22              /20
nodos posibles                               27              110
límite de pods por nodo                      29               110 (modo por prefijo)
sucesos "no IP addresses available"     41 en un mes           0
pods en Pending por límite de pods       sí, en los picos       no
```

Dos correcciones distintas: la subred resolvía el techo del clúster y el modo de asignación por prefijo resolvía el techo por nodo. Ninguna se puede aplicar sobre un clúster existente sin recrearlo, lo que confirma que es una decisión de un solo sentido.

Dato 3 — una actualización forzada que llevaba tres meses bloqueada.

```
versión del clúster            fin de soporte hace 6 semanas
intentos del proveedor         4, todos detenidos
nodos actualizados             11 de 24
motivo                         presupuesto sin margen en 2 servicios (clase 079)
aviso                          correo a una lista que nadie leía
```

El clúster llevaba seis semanas con la mitad de los nodos en una versión sin soporte, y con el plano de control ya actualizado por el proveedor.

```
presupuestos que bloquean                    2                0
notificación del proveedor             correo a lista    alerta en el canal
                                                          de guardia
clúster de pruebas por delante           no había          sí, una versión
métrica de APIs obsoletas revisada        nunca      antes de cada actualización
duración de la actualización completa   no terminaba      4 h 10 min
```

Y la revisión de APIs obsoletas, hecha por primera vez, encontró dos manifiestos que la versión siguiente habría rechazado.

Dato 4 — el complemento de red decidía si la segmentación era posible.

La plataforma de destino ofrecía dos complementos, y solo uno implementaba políticas. La elección por defecto del asistente de creación era el otro.

```
políticas de red efectivas                   0               14
direcciones consumidas por pod               1                1
unidad máxima de transmisión           sin cambios      revisada (clase 065)
complemento gestionado por el proveedor      sí               sí
```

Si la migración se hubiera hecho aceptando el valor por defecto, las catorce políticas de la clase 080 habrían vuelto a no filtrar nada — el mismo incidente, en una plataforma nueva, por una casilla del asistente.

Dato 5 — la federación de identidad, sin acotar por defecto.

La primera configuración funcionaba desde el primer intento, lo que hizo sospechar:

```
$ kubectl run prueba -n desarrollo --rm -it --image=… -- /probar-acceso-nube
ok        ← desde OTRO espacio, con OTRA cuenta
```

La condición de confianza del lado de la nube aceptaba cualquier testigo del clúster.

```
condición de confianza              emisor del clúster   emisor + espacio + cuenta
pods que podían obtener la identidad    todos            solo el previsto
prueba negativa                          ninguna       en el guion de verificación
```

Cuarta vez en el programa que esta misma prueba negativa se ejecuta y tercera vez que falla la primera vez.

Resumen de la evaluación:

```
manifiestos de aplicación reescritos         —              13 de 214
capa de plataforma reescrita                 —              31 de 31
nodos que caben en el plan de direcciones    27              110
políticas de red efectivas                   14              14
federación acotada                          sí            sí (tras corregir)
semanas con versión sin soporte              6               0
esfuerzo total de la migración               —          14 días-persona
```

La lección que esta clase traslada al proyecto de la clase 084: la comparación entre plataformas gestionadas no se decide por funciones sino por tres cosas que se fijan al crear el clúster y no se pueden cambiar después — el plan de direcciones, el complemento de red y el modelo de identidad. Las tres son decisiones de un solo sentido, las tres se toman en un asistente de creación en cinco minutos, y las tres deciden el techo de crecimiento, la posibilidad de segmentar y el alcance de una credencial. El resto —el 94 % de los manifiestos— es portable, y esa es la confirmación cuantificada de la hipótesis que abrió la parte 05.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-06-kubernetes-managed-platforms/083-eks-aks-y-gke-similitudes-y-diferencias/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa matriz-kubernetes-administrado en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye matriz-kubernetes-administrado para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.

- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

■■ Errores frecuentes

Síntoma	Causa probable	Corrección
El clúster no puede crecer más allá de cierto número de nodos	El plan de direcciones no da para más pods, y el pico de despliegue consume más que el estado estable	Calcula el tamaño máximo previsto con margen antes de crear el clúster; ampliarlo después suele exigir recrearlo.
Hay nodos con CPU y memoria libres y pods en Pending	El límite de pods por nodo depende del tipo de máquina y del modo de asignación de direcciones	Usa máquinas mayores o el modo de asignación por prefijo; el suceso menciona pods insuficientes, no recursos.
El proveedor intenta actualizar y el clúster queda a medias durante semanas	Un presupuesto de interrupción sin margen bloquea también la actualización automática	Mantén margen en los presupuestos y dirige los avisos del proveedor al canal de guardia, no a una lista.
Tras una actualización, algunos manifiestos dejan de aplicarse	La versión nueva retiró versiones de API que esos manifiestos usaban	Revisa la métrica de APIs obsoletas antes de cada actualización y mantén un clúster de pruebas por delante.
Las políticas de red no filtran en la plataforma nueva	El complemento elegido en el asistente de creación no las implementa	Elige el complemento con soporte de políticas y verifica con una prueba negativa antes de dar por buena la migración.
Cualquier pod del clúster obtiene la identidad de la nube	La condición de confianza acepta cualquier testigo del clúster, sin acotar espacio ni cuenta	Acota la confianza al espacio de nombres y a la cuenta concretos, y comprueba desde otro espacio que se deniega.
Se espera que la plataforma gestionada reduzca el trabajo de operación	Gestiona el plano de control, no las aplicaciones ni los complementos	Dimensiona el equipo para el trabajo de las clases 073 a 082, que es el que produce los incidentes.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué gestiona exactamente el proveedor y qué sigue siendo trabajo propio?
2. ¿Qué ocurre cuando se agota la ventana de soporte y qué puede impedir esa actualización?
3. Calcula cuántos nodos caben en una subred /22 con 30 pods por nodo y margen de despliegue.
4. ¿Por qué la elección del complemento de red al crear el clúster es una decisión de seguridad?
5. ¿Qué proporción de manifiestos se conserva al cambiar de plataforma, y dónde se concentra el esfuerzo?

Referencias

- Kubernetes (2025). Version skew policy — desviación admitida entre plano de control y nodos. <https://kubernetes.io/releases/version-skew-policy/>
- Kubernetes (2025). Deprecated API migration guide — versiones retiradas y cómo detectarlas. <https://kubernetes.io/docs/reference/using-api/deprecation-guide/>

- AWS (2025). Amazon EKS networking and IP address planning — direcciones por nodo y modos de asignación. <https://docs.aws.amazon.com/eks/latest/userguide/eks-networking.html>
- Microsoft (2025). AKS networking concepts — modelos de red y sus consecuencias de direccionamiento. <https://learn.microsoft.com/en-us/azure/aks/concepts-network>
- Google Cloud (2025). GKE cluster networking and IP allocation — rangos secundarios y límites por nodo. <https://cloud.google.com/kubernetes-engine/docs/concepts/alias-ips>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 06 en PDF · Manual integral

Anterior	Índice	Siguiente
← 082 · Logs, métricas, eventos y depuración	Parte 06 · Programa	084 · Proyecto: plataforma Kubernetes portable →

Evaluación — 083 EKS, AKS y GKE: similitudes y diferencias

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega matriz-kubernetes-administrado con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 07 — Infraestructura como código y configuración

093 — CloudFormation, Bicep, Pulumi y Terraform

Parte: 07 — Infraestructura como código y configuración Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: decision · Estado: EXECUTABLECORE

Propósito

Comparar las cuatro familias de herramientas por lo que cambia la operación, que son cuatro ejes y ninguno es la sintaxis: quién guarda el estado, quién ejecuta el motor, qué ocurre cuando una aplicación falla a medias y si el lenguaje es un lenguaje de programación. Las clases 047 y 059 ya midieron el primero; aquí se completan los otros tres y se llega a la conclusión que este programa lleva sugiriendo desde la parte 03: la disciplina es la misma en las cuatro, y es lo que decide si funciona.

Resultados de aprendizaje

Al finalizar podrás:

1. Comparar las cuatro familias por los ejes que cambian la operación, no por funciones.
2. Anticipar qué ocurre cuando una aplicación falla a medias en cada una.
3. Valorar el uso de un lenguaje de programación general con sus dos caras.
4. Elegir herramienta con un criterio defendible y sabiendo qué se hereda con ella.
5. Reconocer la parte de la práctica que no depende de la herramienta.

Conceptos centrales

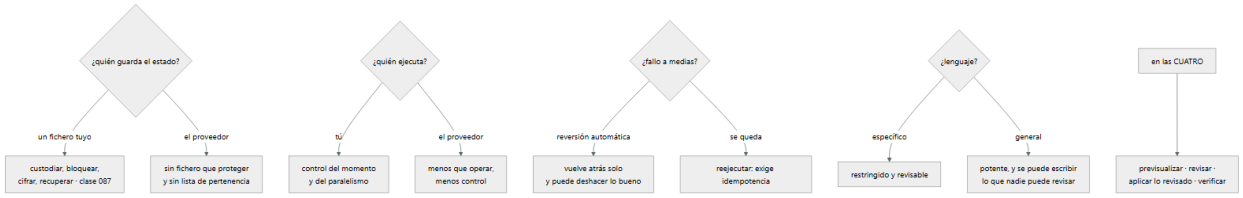
Concepto	Comprensión verificable
estado externo frente a estado del proveedor	O hay un fichero que hay que custodiar, o el propio servicio recuerda lo que gestiona. Decide la mitad de las obligaciones operativas.
motor gestionado	El proveedor ejecuta el despliegue. No hay estado que proteger ni ejecución que orquestar, y se pierde control sobre el momento y sobre el detalle.
comportamiento ante fallo parcial	Qué queda cuando la mitad se aplicó y algo falló: reversión automática, o lo aplicado se queda. Cambia por completo el procedimiento de recuperación.
lenguaje general	Escribir infraestructura en un lenguaje de programación. Da bucles, abstracciones y pruebas conocidas, y quita la restricción que hacía revisable el resultado.
retraso del proveedor	Tiempo entre que una nube publica una función y la herramienta la soporta. Es cero en las herramientas de la propia nube y variable en las demás.
disciplina portable	Previsualizar, revisar, aplicar lo revisado, verificar convergencia, política sobre el resultado y secretos fuera. Es idéntica en las cuatro familias.

Modelo mental

IaC trata la infraestructura como producto versionado: el plan explica intención, el estado conecta intención y realidad, y la revisión reduce cambios accidentales.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Cuatro ejes, y la sintaxis no es ninguno

Las comparativas habituales enumeran funciones. Los cuatro ejes que de verdad cambian el trabajo diario son otros:

	Estado	Motor	Fallo parcial	Lenguaje
Nativa de la nube (plantillas del proveedor)	Del proveedor	Gestionado	Revierte solo	Específico
Terraform	Fichero propio	Tuyo	Se queda	Específico
Lenguaje general con estado	Fichero propio	Tuyo	Se queda	General
Kit de desarrollo que genera plantillas	Del proveedor	Gestionado	Revierte solo	General

Y las consecuencias de cada columna, que son lo que se hereda al elegir.

El estado ya se midió en las clases 047 y 059, y el resumen se sostiene:

```
fichero propio      hay que custodiarlo, bloquearlo, cifrarlo y recuperarlo
                    → toda la clase 087
                    a cambio: se sabe QUÉ gestiona este código
                    y se detecta que alguien BORRÓ algo

del proveedor       no hay fichero que proteger ni secretos que se filtren por ahí
                    a cambio: no hay lista de pertenencia clara,
                    y detectar un borrado externo depende del servicio
```

El motor es el eje que menos se discute y más cambia la operación:

```
motor gestionado    el proveedor despliega, con su ritmo y sus reintentos
                    no hay agentes que operar, ni bloqueos, ni concurrencia
                    y hay poco control: el paralelismo, el orden fino
                    y los tiempos de espera los decide él

motor propio        control total del momento, del paralelismo y del alcance
                    y hay que operar el agente, su identidad y su bloqueo
```

La segunda fila incluye una ventaja que se nota al depurar: con motor propio se puede aplicar una parte, planificar sin refrescar o reintentar una operación concreta (clase 090). Con motor gestionado, esas herramientas no existen.

El retraso del proveedor es un argumento real y suele exagerarse:

```
herramienta de la propia nube   soporte el mismo día del anuncio
otras                           de días a meses, según el servicio
```

Y su importancia depende de un dato que cada equipo puede medir: cuántas veces al año necesita una función publicada esta semana. En la mayoría de las organizaciones la respuesta es pocas, y para esos casos existe la salida de declarar el recurso con la API genérica del proveedor hasta que llegue el soporte.

2. Qué pasa cuando falla a medias

Este es el eje que menos aparece en las comparativas y el que más cambia el procedimiento de recuperación.

Reversión automática. El motor deshace lo que ya había aplicado y devuelve el conjunto a su estado anterior.

```
ventaja   nunca queda un estado intermedio que nadie ha probado
coste     la reversión también deshace lo que SÍ funcionó
          y puede fallar ella misma, dejando un estado del que hay que salir a mano
          y en un despliegue grande, tarda
```

La segunda línea produce una situación conocida: el motor intenta revertir un recurso que no se puede eliminar —tiene protección, tiene dependencias, tiene datos— y el conjunto queda en un estado que exige intervención. Y ahí el procedimiento no es evidente y conviene tenerlo escrito antes.

Lo aplicado se queda. Es el comportamiento de Terraform, y la clase 059 ya lo enunció: no hay transacción.

```
ventaja   lo que funcionó, funcionó; se corrige y se vuelve a aplicar
coste     hay un estado intermedio real, y hay que asumir que existirá
          → EXIGE que las plantillas sean idempotentes y reejecutables
```

Y de ahí sale la propiedad que este programa lleva pidiendo desde la clase 047: la respuesta a un fallo parcial es corregir y volver a aplicar. Una plantilla que solo funciona sobre un entorno vacío no sirve para operar, con ninguna herramienta.

La comparación honesta entre ambos comportamientos:

```
la reversión automática es mejor para        despliegues acotados y frecuentes
                                             de un conjunto pequeño
lo aplicado se queda es mejor para           conjuntos grandes, donde revertir
                                             todo por un fallo al final es peor
                                             que corregir y continuar
```

Y una coincidencia entre las dos familias que conviene señalar: ninguna revierte los efectos que no son recursos. Una migración de datos ejecutada por un gancho, un mensaje publicado, un fichero escrito — nada de eso vuelve atrás. Es el mismo límite que la clase 079 estableció para las vueltas atrás de despliegue, y sigue exigiendo lo mismo: cambios compatibles en ambos sentidos y una pregunta previa —«si esto falla a mitad, ¿qué queda roto?»—.

Y una diferencia operativa que aparece con motores gestionados y sorprende: la detección de desviación es una función del propio servicio en algunas plataformas, con lo que se obtiene sin montar la ejecución programada de la clase 090. Y suele tener límites: no todos los tipos de recurso, no todos los campos. Conviene comprobar la cobertura real antes de darla por buena, con la misma desconfianza que la clase 080 aplicó a las políticas de red.

3. El lenguaje general: dos caras

Escribir infraestructura en un lenguaje de programación completo resuelve problemas reales y quita una restricción que era útil. Merece las dos columnas.

Lo que resuelve:

```
abstracciones de verdad     clases, funciones, herencia
                            un módulo con lógica compleja se expresa mejor
bucles y condicionales      sin las contorsiones de un lenguaje declarativo
pruebas con las herramientas del lenguaje
                            unitarias, sin desplegar nada
reutilización de código     bibliotecas ya existentes, tipos compartidos
                            con la aplicación
un solo lenguaje            el equipo no aprende una sintaxis más
```

La tercera fila es la más valiosa y la menos citada: se pueden escribir pruebas unitarias sobre la lógica que decide la infraestructura, sin crear nada.

Lo que quita:

```
la restricción que hacía revisable el resultado
```

Un lenguaje declarativo limitado obliga a que el código se parezca al resultado. Con un lenguaje general se puede escribir esto:

```
una función que lee una base de datos y decide cuántos recursos crear
un bucle cuyo número de iteraciones depende de la hora
una jerarquía de clases de cinco niveles para tres tipos de servicio
```

Y nada de eso es revisable leyendo el código: hay que ejecutarlo para saber qué hace.

Y aquí está la parte que reconcilia las dos columnas y que conviene tener clara antes de elegir:

> El artefacto de revisión no es el código, es el plan. Con un lenguaje general, el plan sigue existiendo y sigue siendo el sitio donde se revisa, se aplica la política y se aprueban los borrados.

Eso reduce mucho el riesgo, y no lo elimina: un plan de doscientos recursos generado por un bucle cuyo criterio nadie entiende es revisable en su resultado y no en su intención. Por eso la disciplina que hay que añadir es de estilo:

```
la lógica que decide QUÉ existe: simple, plana, y probada
la complejidad, en las bibliotecas de apoyo, no en la definición
ninguna consulta a sistemas externos en tiempo de definición
  → hace que el resultado dependa de cuándo se ejecute
```

La tercera es la más importante: una definición que consulta una base de datos o una API para decidir qué crear no es reproducible, y rompe la propiedad que hace útil todo lo demás.

Y un matiz sobre los kits de desarrollo que generan plantillas nativas: dan el lenguaje general y conservan el motor gestionado, así que heredan la reversión automática y la ausencia de estado propio. Es una combinación coherente y con un coste conocido: el resultado es una plantilla generada, a veces grande y poco legible, y depurar exige leerla.

4. Elegir, y qué se hereda con cada elección

Un criterio defendible en cinco preguntas, en orden:

```
1. ¿una nube o varias?
   una sola y sin previsión de cambiar  → la nativa es una opción seria
   varias, o Kubernetes, o servicios de terceros → una herramienta común

2. ¿quién va a operarlo?
   equipo pequeño sin capacidad de operar agentes → motor gestionado
   equipo de plataforma con canalizaciones → motor propio

3. ¿el equipo ya tiene un lenguaje?
   sí, y la infraestructura la escriben desarrolladores → lenguaje general
   la escribe un equipo de plataforma → específico, más restringido y más legible

4. ¿cuánta lógica hace falta de verdad?
   poca, casi siempre → un lenguaje declarativo basta y se lee mejor

5. ¿qué se hereda?
   → la tabla del primer apartado, entera
```

Y la respuesta más común en organizaciones medianas, dicha sin adornos: una herramienta común con motor propio y lenguaje específico, porque cubre varias nubes, Kubernetes y servicios de terceros con un solo flujo, y porque la lógica que hace falta es poca.

Y dos situaciones en las que la nativa gana con claridad:

```
una sola nube, equipo pequeño, sin canalización de plataforma
  → el motor gestionado quita la mitad del trabajo de las clases 087 y 090

ofrecer plantillas a clientes o a otras organizaciones
  → el formato nativo lo entiende cualquiera de esa nube sin instalar nada
```

Y la situación que aparece de verdad y que ninguna comparativa contempla: una organización con tres herramientas a la vez, porque cada equipo eligió la suya. Consolidar cuesta y no consolidar también, y la decisión se toma con dos cifras:

```
coste de consolidar    reescribir e importar, medible en días-persona
coste de no hacerlo    tres canalizaciones, tres políticas, tres auditorías,
                       tres formaciones y ninguna revisión cruzada posible
```

Y una salida intermedia que funciona mejor de lo que parece: consolidar la disciplina antes que la herramienta. Las mismas políticas, la misma secuencia de revisión, el mismo tratamiento de secretos y la misma detección de desviación, con tres herramientas distintas. Eso captura la mayor parte del beneficio y no exige reescribir nada.

Y para migrar de una herramienta a otra, cuando se decide, el procedimiento tiene una forma conocida:

```
1. la herramienta nueva IMPORTA lo que existe (clase 087)
2. se verifica con una previsualización sin cambios
3. la herramienta antigua deja de gestionarlo, sin destruirlo (clase 090)
4. se retira la antigua cuando ya no gestione nada
```

El paso 3 es el que evita el desastre: retirar de la gestión, no destruir. Y el 2 es la comprobación que decide si el paso 1 se hizo bien, exactamente como en la clase 087.

5. Lo que no depende de la herramienta

Con cuatro familias comparadas, lo que queda es la conclusión que este programa lleva sugiriendo desde la parte 03: la mayor parte de la práctica es idéntica.

```
previsualizar antes de aplicar             047 · 059 · 081 · 090
aplicar exactamente lo previsualizado      059 · 090
verificar la convergencia después          073 · 085 · 090
revisión legible: sin ruido                047 · 081 · 085 · 090
política sobre el RESULTADO, no el código  091
secretos fuera, y rotar lo expuesto        047 · 059 · 061 · 087 · 092
versiones fijadas de todo                  047 · 059 · 062 · 081 · 088
un estado o alcance por radio de impacto   047 · 059 · 087
proteger lo que no se puede perder         042 · 059 · 077 · 086
identidad federada, sin claves             026 · 038 · 050 · 059 · 092
refactorizar sin destruir                  087 · 090
detección de desviación con señal          085 · 090
```

Doce prácticas, todas presentes en las cuatro familias con nombres distintos, y todas responsables de más incidentes que cualquier decisión de herramienta.

Y los tres criterios que la clase 085 fijó para juzgar una solución de esta parte siguen siendo los mismos y siguen sin depender de la herramienta:

```
1. ¿se puede ver el cambio antes de aplicarlo, y es legible?
2. ¿lo que se revisó es exactamente lo que se aplica?
3. ¿alguien sabría que el sistema dejó de converger?
```

Y una observación que conviene decir con claridad, porque ahorra discusiones improductivas:

> Un equipo con una herramienta mediocre y las doce prácticas tiene una infraestructura operable. Un equipo con la mejor herramienta y sin ellas tiene los mismos incidentes que tenía antes, con más ficheros.

Y la lista de comprobación de la clase, que es de decisión y no de configuración:

- los cuatro ejes evaluados para el caso concreto, no una tabla de funciones
- el comportamiento ante fallo parcial, conocido y con procedimiento escrito
- si el lenguaje es general: reglas de estilo sobre la lógica de definición
- ninguna consulta a sistemas externos en tiempo de definición
- si hay varias herramientas: la disciplina consolidada aunque no lo esté la herramienta
- si se migra: importar, verificar sin cambios, retirar de la gestión, destruir nunca
- las doce prácticas portables, presentes con independencia de la elección

Ejemplo trabajado

CloudShop tiene tres herramientas de infraestructura como código y una discusión anual sobre cuál usar. El ejercicio de este año se hace midiendo en vez de opinando, y la conclusión no es la que nadie esperaba.

El punto de partida.

```
equipo de plataforma      Terraform, 4 estados, 88 % de la infraestructura
equipo de datos           plantillas nativas de la nube, 9 %
equipo de aplicación      un kit de desarrollo en el lenguaje de la aplicación, 3 %
```

Y los tres argumentos de la discusión anual, siempre los mismos:

```
"las plantillas nativas no necesitan estado"
"con el kit escribimos infraestructura en el mismo lenguaje"
"tener tres herramientas es insostenible"
```

La medición: incidentes por causa, últimos doce meses.

```
causa                                  incidentes   herramienta
falta de previsualización revisada           6      las tres
plantilla no idempotente                     4      las tres
secreto en un rastro no auditado             3      las tres
desviación no detectada                      5      las tres
recurso destruido sin querer                 2      Terraform, kit
estado perdido o bloqueado                   2      Terraform
```

```
reversión automática que falló a medias              1      nativa
lógica de definición imposible de revisar            1      kit
```

Veinticuatro incidentes. Dieciocho de los veinticuatro son de las doce prácticas portables y no de la herramienta: ocurrieron con las tres, y habrían ocurrido con cualquier otra.

Los seis restantes sí son atribuibles:

```
dos por destrucción         falta de protección contra el borrado (clases 059, 086)
dos por el estado           custodia y bloqueo (clase 087)
uno por reversión           el motor gestionado no pudo revertir y quedó a medias
uno por lógica              un bucle en el kit cuyo criterio nadie entendía
```

La decisión, que no fue consolidar.

```
consolidar en una herramienta        34 días-persona
consolidar la DISCIPLINA               9 días-persona
```

Y lo que cubría cada opción:

```
consolidar la herramienta   los 6 incidentes atribuibles, y de rebote
                            facilitaría las prácticas
consolidar la disciplina    los 18 incidentes portables, en las tres herramientas
```

Se hizo la segunda primero. Y las nueve jornadas se repartieron así:

```
política común sobre el resultado, para las tres            3 días
detección de desviación programada, para las tres           2 días
auditoría de los cinco rastros de secretos (clase 092)      2 días
protección contra destrucción en lo que no se puede perder  1 día
procedimiento escrito de fallo parcial, uno por herramienta 1 día
```

La política común fue lo más interesante del ejercicio: las tres herramientas producen una representación estructurada de lo que van a hacer, así que las mismas diecinueve reglas se pudieron aplicar a las tres con adaptadores pequeños.

```
reglas de política                    19, solo en Terraform   19, en las tres
detección de desviación                1 de 3 herramientas    3 de 3
audita rastros de secretos             1 de 3                 3 de 3
procedimiento de fallo parcial escrito 0 de 3                 3 de 3
recursos con protección de borrado     6 de 41               41 de 41
```

Los seis meses siguientes.

```
incidentes de causa portable          18 → 2
incidentes atribuibles a la herramienta 6 → 3
```

Los dos portables restantes fueron plantillas no idempotentes en el kit, corregidas.

Y con esos datos, la decisión sobre consolidar la herramienta se tomó por fin con criterio:

```
las plantillas nativas se quedan       el equipo de datos es pequeño,
                                       no opera canalizaciones, una sola nube
                                       y el motor gestionado le quita trabajo real
el kit se retira                       3 % de la infraestructura, un incidente
                                       por lógica irrevisable, y el equipo
                                       prefería no mantener dos flujos
Terraform sigue                        cubre varias nubes y Kubernetes
```

La migración del kit se hizo con el procedimiento de esta clase:

```
recursos importados                    31
verificados con previsualización vacía 31 de 31, a la tercera iteración
retirados de la gestión del kit        31, sin destruir
recursos destruidos en el proceso       0
esfuerzo                               4 días-persona
```

La conclusión que se escribió, y que cerró la discusión anual.

```
de 24 incidentes en un año, 18 no dependían de la herramienta
consolidar la disciplina costó 9 días y eliminó 16 de esos 18
consolidar la herramienta habría costado 34 días y eliminado 6
y la decisión final NO fue una sola herramienta:
  dos, cada una donde su modelo operativo encaja
```

La lección que esta clase traslada al resto de la parte 07: la discusión sobre herramientas consume tiempo desproporcionado respecto de lo que decide. Tres de cada cuatro incidentes eran de las doce prácticas portables, presentes o ausentes con independencia de la elección. Y cuando por fin se eligió, el criterio no fue cuál es mejor sino

qué modelo operativo encaja con cada equipo: motor gestionado donde no hay capacidad de operar canalizaciones, motor propio donde hay varias nubes que cubrir.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-07-infrastructure-as-code-configuration/093-cloudformation-bicep-pulumi-y-terraform/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa adr-herramienta-iac en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye adr-herramienta-iac para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

■■ Errores frecuentes

Síntoma	Causa probable	Corrección
La discusión sobre qué herramienta usar se repite cada año sin resolverse	Se compara por funciones en vez de por los ejes que cambian la operación	Mide los incidentes del último año por causa: la mayoría suele ser de prácticas portables, no de la herramienta.
Una aplicación falla a medias y el motor no consigue revertir	La reversión automática intenta eliminar algo protegido o con dependencias	Ten escrito el procedimiento de salida antes de necesitarlo; el comportamiento ante fallo parcial se conoce, no se descubre.
Un cambio en el kit genera doscientos recursos y nadie sabe por qué	La lógica de definición usa bucles o consultas cuyo criterio no se puede leer	Reglas de estilo: lógica de definición simple y plana; la complejidad, en bibliotecas de apoyo probadas.
El resultado de una ejecución depende de cuándo se ejecute	La definición consulta un sistema externo para decidir qué crear	Prohíbe las consultas en tiempo de definición: rompen la reproducibilidad, que es la propiedad que sostiene todo lo demás.
Migrar entre herramientas destruye recursos	Se retiró la antigua antes de que la nueva los gestionara, o se destruyó en vez de retirar de la gestión	Importar, verificar con previsualización vacía, retirar de la gestión sin destruir, y solo entonces retirar la antigua.
Se adopta la mejor herramienta y los incidentes siguen igual	Las doce prácticas portables no estaban antes y siguen sin estar	Consolida la disciplina antes que la herramienta: cubre más incidentes por menos esfuerzo.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Cuáles son los cuatro ejes que cambian la operación, y qué se hereda con cada opción?
2. ¿Qué ventaja y qué coste tiene la reversión automática frente a que lo aplicado se quede?
3. ¿Qué resuelve y qué quita usar un lenguaje de programación general, y qué reduce el riesgo?
4. ¿Qué procedimiento sigue una migración entre herramientas sin destruir nada?
5. Enumera cinco de las doce prácticas que no dependen de la herramienta.

Referencias

- AWS (2025). CloudFormation stack failure options and rollback — comportamiento ante fallo parcial. <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/stack-failure-options.html>
- Microsoft (2025). Bicep vs ARM templates and deployment behaviour — motor gestionado y modos de despliegue. <https://learn.microsoft.com/en-us/azure/azure-resource-manager/bicep/overview>
- HashiCorp (2025). Terraform: no partial rollback — por qué las plantillas deben ser reejecutables. <https://developer.hashicorp.com/terraform/language/resources/behavior>
- Pulumi (2025). Programming model and preview — lenguajes generales con previsualización del cambio. <https://www.pulumi.com/docs/concepts/>
- AWS (2025). CDK: synthesized templates — generar plantillas nativas desde un lenguaje general. <https://docs.aws.amazon.com/cdk/v2/guide/home.html>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 07 en PDF · Manual integral

Evaluación — 093 CloudFormation, Bicep, Pulumi y Terraform

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega adr-herramienta-iac con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 17 — AWS: arquitectura, automatización y operación en producción

205 — Hosting progresivo con Amplify, S3 y CloudFront

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: delivery · Estado: EXECUTABLECORE

Propósito

Publicar una aplicación web en AWS eligiendo entre el camino gestionado y el montado a mano, y entender qué se gana y qué se pierde con cada uno. La clase compara Amplify Hosting con el montaje S3 + CloudFront pieza a pieza, fija los valores por defecto que hay que cambiar para producción —que son más de los que parece—, y trata los tres asuntos que rompen estos despliegues: el enrutado de aplicaciones de una sola página, la invalidación de caché en cada publicación y el acceso al bucket.

Resultados de aprendizaje

Al finalizar podrás:

1. Elegir entre alojamiento gestionado y montaje propio con criterios claros.
2. Configurar S3 y CloudFront con los valores correctos para producción.
3. Resolver el enrutado de una aplicación de una sola página sin romper los 404 reales.
4. Publicar sin invalidaciones masivas, usando huellas en los nombres.
5. Cerrar el acceso al origen para que solo la distribución pueda leerlo.

Conceptos centrales

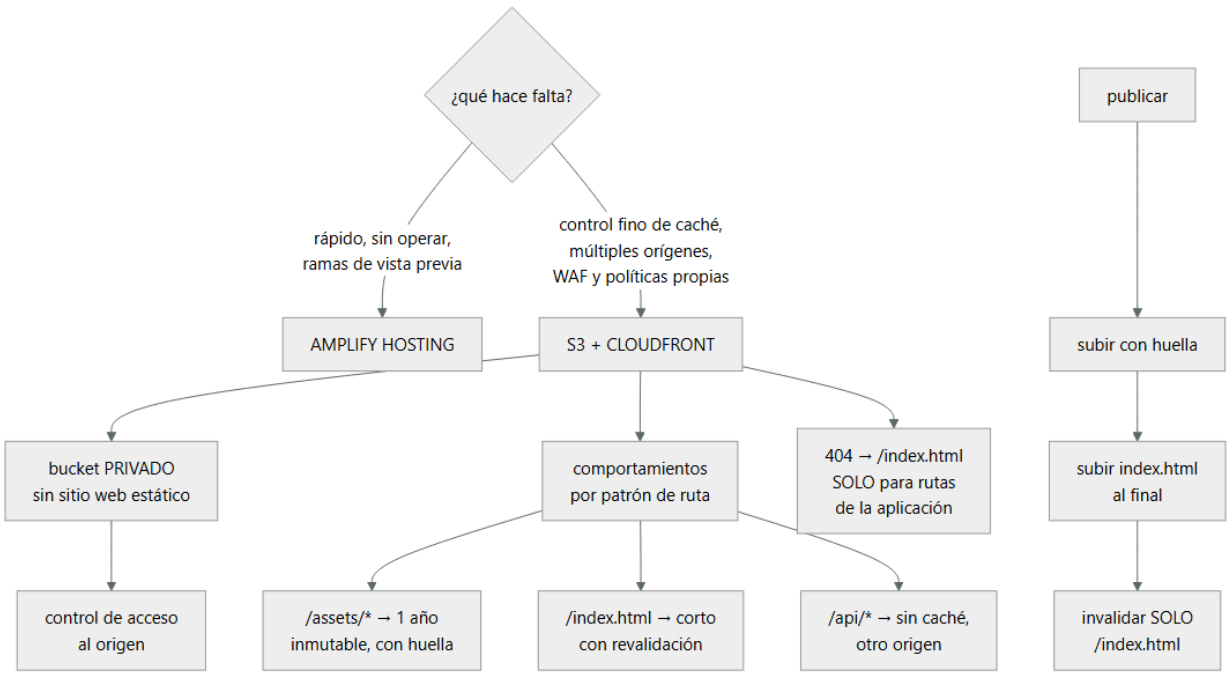
Concepto	Comprensión verificable
Amplify Hosting	Servicio gestionado que construye y publica una aplicación web desde un repositorio, con distribución incluida.
distribución	Recurso de CloudFront que sirve el contenido desde el borde y define caché, orígenes y comportamiento.
control de acceso al origen	Mecanismo por el que solo la distribución puede leer el bucket, que queda privado.
comportamiento de caché	Regla por patrón de ruta que fija clave de caché, validez y política de origen.
función de borde	Código ligero que se ejecuta en el borde para reescribir peticiones o añadir cabeceras.
huella en el nombre	Sufijo derivado del contenido que hace innecesaria la invalidación al publicar.

Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Gestionado o montado a mano

Las dos opciones sirven para lo mismo y se diferencian en control y en operación.

```
AMPLIFY HOSTING
  construye desde el repositorio y publica
  da   ramas de vista previa por cada propuesta de cambio
       certificado y dominio gestionados
       reversión a una publicación anterior
       reescrituras y redirecciones por configuración
       distribución por debajo, sin gestionarla
  no da
       control fino de la clave de caché por ruta
       comportamientos con orígenes distintos
       políticas de origen propias
       WAF con reglas propias en muchos casos
       acceso a los registros de la distribución

S3 + CLOUDFRONT MONTADO
  da   control total: comportamientos, claves, políticas,
       WAF, registros, varios orígenes
  cuesta
       montarlo y mantenerlo
       y aprender los valores por defecto que hay que cambiar
```

Y el criterio:

```
¿el sitio es estático o casi, y no hay requisitos de caché
 ni de seguridad especiales?
  → Amplify; se monta en una tarde y se opera solo

¿hay API en el mismo dominio, varias rutas con caché
 distinta, WAF propio o necesidad de registros?
  → S3 + CloudFront

¿el equipo ya opera CloudFront para otras cosas?
  → montarlo, por coherencia
```

Y una vía intermedia que funciona bien:

```
Amplify para las ramas de vista previa (donde su valor es
mayor y los requisitos son bajos)
S3 + CloudFront para producción
→ y así se tiene rapidez donde importa la velocidad y
```

```
  control donde importa el control
```

Y la advertencia de coste que aparece con Amplify:

```
factura por minuto de construcción y por gigabyte servido
→ con muchas ramas de vista previa y construcciones
  frecuentes, la partida de construcción sube deprisa
→ y las vistas previa no caducan solas                ley 25
```

2. Los valores por defecto que hay que cambiar

Montar S3 + CloudFront con los valores por defecto produce algo que funciona y no es apto para producción. Esta es la lista.

```
EL BUCKET
  ✗ por defecto   se tiende a activar «alojamiento de sitio
                  web estático» y hacerlo público
  ✓ correcto      bucket PRIVADO, sin alojamiento estático,
                  accesible solo por control de acceso al
                  origen desde la distribución
  por qué         un bucket público es alcanzable saltándose
                  la distribución: sin WAF, sin registros,
                  sin caché y pagando salida  clases 189, 200

  ✓ además        bloqueo de acceso público activado
  ✓               cifrado en reposo
  ✓               versionado, para poder revertir una
                  publicación

LA DISTRIBUCIÓN
  ✗ por defecto   política de caché heredada que reenvía
                  cabeceras y cookies innecesarias
  ✓ correcto      política que NO incluye cookies ni
                  cabeceras salvo las que varían   clase 197

  ✗ por defecto   un solo comportamiento para todo
  ✓ correcto      comportamientos por patrón de ruta

  ✗ por defecto   HTTP permitido
  ✓ correcto      redirección a HTTPS y versión mínima de TLS

  ✗ por defecto   sin cabeceras de seguridad
  ✓ correcto      política de cabeceras de respuesta

  ✗ por defecto   sin compresión negociada en algunos casos
  ✓ correcto      compresión activada

  ✗ por defecto   registros desactivados
  ✓ correcto      registros a un bucket propio, con caducidad
```

Y los comportamientos que hay que definir, con sus valores:

```
/assets/*   o /_next/static/*  (recursos con huella)
  validez           1 año, inmutable
  clave de caché    solo la ruta
  → nunca se invalidan: cambian de nombre       clase 197

/index.html  y las rutas de la aplicación
  validez           0 o muy corta, con revalidación
  → es el único fichero que hay que invalidar al publicar

/api/*
  otro origen (API Gateway o balanceador)
  sin caché, reenviando las cabeceras necesarias
  → tener la API en el mismo dominio evita CORS y cookies
    de terceros

/media/*  (imágenes subidas)
  validez media, con nombre versionado si se pueden
  reemplazar
```

Y una decisión de arquitectura que ahorra problemas:

```
servir la aplicación y la API bajo el MISMO dominio, con
comportamientos distintos
```

```
→ sin CORS, sin cookies de terceros y con una sola
  distribución que registrar y proteger
```

3. Las tres cosas que rompen

1 · El enrutado de una aplicación de una sola página.

```
EL PROBLEMA
  la aplicación tiene rutas como /pedidos/4471
  ese objeto NO existe en el bucket
  → S3 devuelve 404 y el usuario ve un error al recargar o
    al entrar por un enlace directo

LA SOLUCIÓN HABITUAL, Y SU DEFECTO
  configurar «404 → /index.html con código 200»
  funciona… y convierte TODOS los 404 en 200
  → incluidos los de recursos que faltan de verdad
  → los buscadores indexan páginas inexistentes
  → y los errores de despliegue se vuelven invisibles
                                                  ley 13

LA SOLUCIÓN CORRECTA
  una función de borde que reescribe la petición a
  /index.html SOLO si la ruta no tiene extensión de fichero
  → /pedidos/4471   → index.html, 200
  → /assets/x.js    → 404 real si no existe
  → y para rutas conocidas de la aplicación, mejor aún:
    lista explícita
```

2 · La invalidación en cada publicación.

```
EL ANTIPATRÓN
  publicar y ejecutar «invalidar /*»
  → tira toda la caché en cada despliegue
  → avalancha contra el origen                  clase 197
  → y por encima de cierto número, las invalidaciones se
    facturan

EL PATRÓN CORRECTO
  1  construir con huella en el nombre de cada recurso
     app.7f3a91.js, estilos.2b91cd.css
  2  subir los recursos nuevos PRIMERO
  3  subir index.html AL FINAL
     → así nunca hay un index que apunte a algo que aún no
       está
  4  invalidar SOLO /index.html (y las rutas HTML)
  5  no borrar los recursos antiguos de inmediato
     → los usuarios con la página abierta siguen pidiéndolos
     → borrarlos con una regla de caducidad de 30 días
```

Y el fallo clásico que evita el paso 5:

```
publicar y borrar lo antiguo a la vez
→ quien tenía la aplicación abierta pide un recurso que ya
  no existe y la página se rompe hasta recargar
```

3 · El acceso al bucket.

```
CON CONTROL DE ACCESO AL ORIGEN
  la distribución firma sus peticiones al bucket
  la política del bucket permite SOLO a esa distribución
  el bucket no tiene acceso público
  → y el contenido solo se puede obtener por la
    distribución, con su WAF, sus registros y su caché

LO QUE HAY QUE COMPROBAR, NO SUPONER
  intentar leer un objeto directamente por la URL del bucket
  → debe fallar                                    ley 22
```

Y el fallo de configuración que deja el hueco:

```
dejar activado el alojamiento de sitio web estático del
bucket «por si acaso»
→ expone un punto de entrada alternativo, público y sin
  control                                      clase 189
```

4. Publicar y operar

La canalización de publicación, que debe hacer lo mismo siempre:

```
1  construir, con huella en los nombres
2  autenticarse sin secretos                        clase 206
3  subir recursos nuevos
4  subir HTML
5  invalidar solo HTML
6  comprobar que el sitio responde y sirve la versión nueva
7  y si no, revertir
```

Y la reversión, que hay que tener resuelta antes de necesitarla:

```
con versionado del bucket y huellas
  revertir = volver a subir el index.html anterior e
  invalidarlo
  → los recursos antiguos siguen ahí (por eso no se borran)
  → tiempo de reversión: segundos

sin huellas ni versionado
  → reconstruir y volver a publicar: minutos u horas
```

Lo que hay que vigilar:

```
tasa de aciertos por comportamiento, no global  clase 197
errores 4xx y 5xx desde el origen
bytes servidos desde el borde y desde el origen
latencia en el borde, por región
certificado: días hasta caducar y días desde la renovación
                                                clase 196
y el coste: peticiones, transferencia e invalidaciones
```

Y dos alertas que evitan sorpresas:

```
«la tasa de aciertos ha caído más de 10 puntos»
  → suele significar que alguien cambió la clave de caché
    o que se está invalidando de más

«hay peticiones directas al bucket»
  → si el control de acceso al origen está bien, deben ser
    cero; cualquier cosa distinta es un hueco
```

Y una nota sobre el dominio y el certificado:

```
el certificado de una distribución de CloudFront debe estar
en la región us-east-1, independientemente de dónde esté
todo lo demás
→ es una de esas particularidades que cuestan una tarde la
  primera vez
```

Y la lista de comprobación de la clase:

- ■ la elección entre gestionado y montado está justificada
- ■ el bucket es privado y sin alojamiento estático activado
- ■ el acceso solo es posible por la distribución, comprobado
- ■ hay bloqueo de acceso público y versionado
- ■ hay comportamientos por patrón de ruta
- ■ la clave de caché no incluye cookies ni cabeceras innecesarias
- ■ los recursos llevan huella y validez de un año
- ■ el HTML tiene validez corta
- ■ el enrutado de la aplicación no convierte todos los 404 en 200
- ■ la publicación sube HTML al final e invalida solo HTML
- ■ los recursos antiguos caducan a 30 días, no se borran
- ■ hay redirección a HTTPS y cabeceras de seguridad
- ■ los registros están activados con caducidad
- ■ la reversión está probada y tarda segundos
- ■ hay alerta si aparecen peticiones directas al bucket

Y el cierre que enlaza con la clase siguiente: la canalización que publica necesita permisos en AWS, y el modo habitual de dárselos —una clave de acceso guardada como secreto— es exactamente lo que este programa lleva desaconsejando desde la parte 11. Federación desde el repositorio, sin secretos, es la materia de la clase 206.

Ejemplo trabajado

CloudShop publica su tienda web. Lo que sigue es la comparación de las dos opciones con cifras, el montaje que eligieron, y los tres problemas que aparecieron —dos de ellos por valores por defecto.

La comparación, hecha con datos:

```
requisitos
  la tienda y la API deben estar bajo el mismo dominio
  WAF con reglas propias                          clase 209
  registros de la distribución para análisis de coste
  caché distinta por ruta: 4 comportamientos
  ramas de vista previa para cada propuesta de cambio

Amplify
  cubre    ramas de vista previa, certificado, reversión
  no cubre WAF propio, registros, comportamientos por ruta
           y otro origen para /api/*

decisión
  producción            S3 + CloudFront montado
  vistas previa         Amplify, con caducidad automática a
                        los 14 días
  motivo de la caducidad   las ramas de vista previa no se
                        borran solas y facturan       ley 25
```

El montaje.

```
bucket
  privado, sin alojamiento estático
  bloqueo de acceso público activado
  versionado activado
  cifrado con clave gestionada
  regla de caducidad: versiones antiguas a 30 días

distribución
  origen 1   el bucket, con control de acceso al origen
  origen 2   API Gateway, para /api/*             clase 207
  TLS mínimo 1.2, redirección a HTTPS
  compresión activada
  registros a un bucket propio, caducidad de 90 días
  WAF asociado                                    clase 209

comportamientos
  /assets/*      caché 1 año, inmutable; clave = ruta
  /media/*       caché 7 días; clave = ruta
  /api/*         sin caché; origen 2; reenvía
                 Authorization y Content-Type
  por defecto    caché 0 con revalidación; función de borde
                 de enrutado
```

Problema 1 · Todos los 404 devolvían 200.

```
montaje inicial
  página de error personalizada: 404 → /index.html con 200
  funcionaba: las rutas de la aplicación cargaban bien

lo que se descubrió tres semanas después
  · un despliegue subió el HTML sin uno de los recursos
    el navegador pidió /assets/app.9c2f1a.js
    la distribución devolvió index.html con código 200
    el navegador intentó ejecutar HTML como JavaScript
    → pantalla en blanco, sin ningún error en los paneles
    → la tasa de errores era del 0 %               ley 13

  · el buscador había indexado 1.900 rutas inexistentes
    generadas por enlaces rotos de una campaña

corrección
  función de borde con esta lógica
    si la ruta contiene un punto (tiene extensión) → pasa
      tal cual; si no existe, 404 real
    si no → reescribe a /index.html, código 200
```

```
  y una alerta nueva
    «peticiones a /assets/* que devuelven 404»
    → habría detectado el despliegue incompleto en 30 s
```

Problema 2 · La invalidación masiva en cada publicación.

```
la canalización hacía
  aws s3 sync ... --delete
  aws cloudfront create-invalidation --paths "/*"

efectos medidos
  tasa de aciertos tras cada publicación   97 % → 12 %
  tiempo en recuperarla                    ~40 min
  peticiones al origen en ese periodo      ×31
  y dos veces, con despliegues en hora punta, el origen se
    saturó                                       clase 197
  invalidaciones facturadas al mes         1.100 rutas

y el efecto del --delete
  los recursos antiguos se borraban al publicar
  → quien tenía la tienda abierta veía la página romperse
  → 3 incidentes reportados por atención al cliente,
    clasificados como «error del navegador»

corrección
  construcción con huella en todos los recursos
  publicación en tres pasos
    1  subir /assets/* nuevos (sin borrar)
    2  subir HTML
    3  invalidar solo /index.html y /
  recursos antiguos: regla de caducidad a 30 días

resultado
  tasa de aciertos tras publicar    97 % → 96,4 %
  invalidaciones al mes             1.100 → 62
  incidentes por publicación             3 → 0
  tiempo de publicación             4 min → 1 min
```

Problema 3 · El bucket seguía siendo alcanzable.

```
la prueba negativa, ejecutada en la revisión de seguridad
  pedir un objeto por la URL directa del bucket
  → FUNCIONÓ

causa
  el alojamiento de sitio web estático se había activado
  durante el montaje inicial y no se desactivó
  la política del bucket tenía una regla antigua que
  permitía lectura pública, añadida «para probar»  ley 25

qué implicaba
  se podía descargar todo el contenido saltándose
    el WAF
    los registros
    la caché → pagando salida a precio de S3   clase 200
  y en los registros del bucket aparecían 41.000 peticiones
    al mes por esa vía, de rastreadores

corrección
  alojamiento estático desactivado
  política del bucket reducida al control de acceso al
    origen
  bloqueo de acceso público activado
  alerta: «peticiones directas al bucket > 0»
  y la prueba negativa añadida al calendario trimestral
```

La reversión, probada:

```
ensayo
  publicar una versión con un fallo evidente
  revertir subiendo el index.html anterior desde el
  versionado e invalidándolo

  tiempo medido                          38 s
  recursos antiguos disponibles          sí (no se borran)
```

```
   usuarios afectados durante la reversión  los que cargaron
                                          en esos 38 s
```

El resultado, tres meses después:

```
tasa de aciertos en régimen                97 %       97,4 %
tasa de aciertos tras publicar             12 %       96,4 %
invalidaciones al mes                     1.100          62
peticiones directas al bucket            41.000/mes       0
404 reales que se veían como 200            todos         0
tiempo de reversión                      reconstruir     38 s
incidentes por publicación                   3/trim        0
coste mensual de la distribución          1.240 €       890 €
```

La lección que esta clase deja: dos de los tres problemas venían de valores por defecto que funcionaban: convertir los 404 en 200 hacía que las rutas de la aplicación cargasen, y el alojamiento estático del bucket hacía que el contenido se sirviera. Los dos escondían algo: un despliegue incompleto que no generaba ningún error y cuarenta y un mil peticiones al mes por un camino sin WAF ni registros. Y el tercero —invalidar todo en cada publicación— costaba cuarenta minutos de caché fría por despliegue y se resolvió cambiando el orden de tres comandos.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/205-hosting-progresivo-con-amplify-s3-y-cloudfront/
lab.py
```

El laboratorio selecciona el motor de práctica delivery y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa aws-static-platform en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un pipeline con gates, promoción y rollback. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye aws-static-platform para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

Errores frecuentes

Síntoma	Causa probable	Corrección
Los errores 404 reales se ven como respuestas correctas	La página de error personalizada convierte todos los 404 en 200 con index.html	Reescribe a index.html solo las rutas sin extensión, con una función de borde, y alerta sobre 404 en recursos.
La tasa de aciertos se hunde tras cada publicación	Se invalida con comodín en cada despliegue	Construye con huella en los nombres, sube el HTML al final e invalida solo el HTML.
La página se rompe para quien la tenía abierta al publicar	Se borran los recursos antiguos al sincronizar	No borres al publicar; retira los recursos viejos con una regla de caducidad a 30 días.
Se puede descargar el contenido saltándose la distribución	El bucket tiene alojamiento estático o una política pública heredada de las pruebas	Bucket privado con control de acceso al origen, bloqueo de acceso público, y prueba negativa que compruebe que la URL directa falla.
La factura de construcción sube sin control	Las ramas de vista previa no caducan y siguen construyendo	Pon caducidad automática a las vistas previa; lo provisional sin fecha se queda para siempre.
El certificado no se puede asociar a la distribución	Se emitió en una región distinta de us-east-1	Emite el certificado de una distribución de CloudFront en us-east-1, aunque el resto del sistema esté en otra región.

Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

Preguntas de comprobación

1. ¿En qué casos compensa el alojamiento gestionado y en cuáles el montaje propio?
2. ¿Qué valores por defecto del bucket y de la distribución hay que cambiar para producción?
3. ¿Cuál es el defecto de resolver el enrutado con la página de error 404?
4. ¿Qué cinco pasos evitan invalidar la caché entera en cada publicación?
5. ¿Qué prueba negativa comprueba que el bucket está realmente cerrado?

Referencias

- AWS (2025). Amplify Hosting user guide — flujo de publicación gestionado del sitio. <https://docs.aws.amazon.com/amplify/latest/userguide/welcome.html>
- AWS (2025). Restricting access to an Amazon S3 origin (origin access control) — cerrar el origen para que solo se sirva por el CDN. <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-restricting-access-to-s3.html>
- AWS (2025). CloudFront cache policies and cache key — políticas de caché y composición de su clave. <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/controlling-the-cache-key.html>
- AWS (2025). CloudFront Functions — lógica ligera ejecutada en el borde. <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/cloudfront-functions.html>
- AWS (2025). Blocking public access to your Amazon S3 storage — bloqueo de acceso público del almacenamiento. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/access-control-block-public-access.html>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 17 en PDF · Recorrido de AWS en PDF · Manual integral

Evaluación — 205 Hosting progresivo con Amplify, S3 y CloudFront

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega aws-static-platform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

206 — OIDC de GitHub y GitLab hacia AWS sin secretos

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: iam · Estado: EXECUTABLECORE

Propósito

Eliminar las claves de acceso de larga duración de las canalizaciones, sustituyéndolas por federación con testigos de corta vida. La clase explica el mecanismo pieza a pieza —proveedor de identidad, condiciones de confianza, testigo y rol—, insiste en la parte donde se cometen los errores graves (la condición de sujeto mal escrita permite que cualquier repositorio del mundo asuma tu rol), y aborda la migración de decenas de canalizaciones sin romperlas.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar el intercambio de testigo por credenciales temporales.
2. Configurar el proveedor de identidad y la política de confianza correctamente.
3. Escribir condiciones de sujeto que no dejen huecos.
4. Migrar canalizaciones existentes sin interrumpirlas.
5. Verificar con pruebas negativas que nadie más puede asumir el rol.

Conceptos centrales

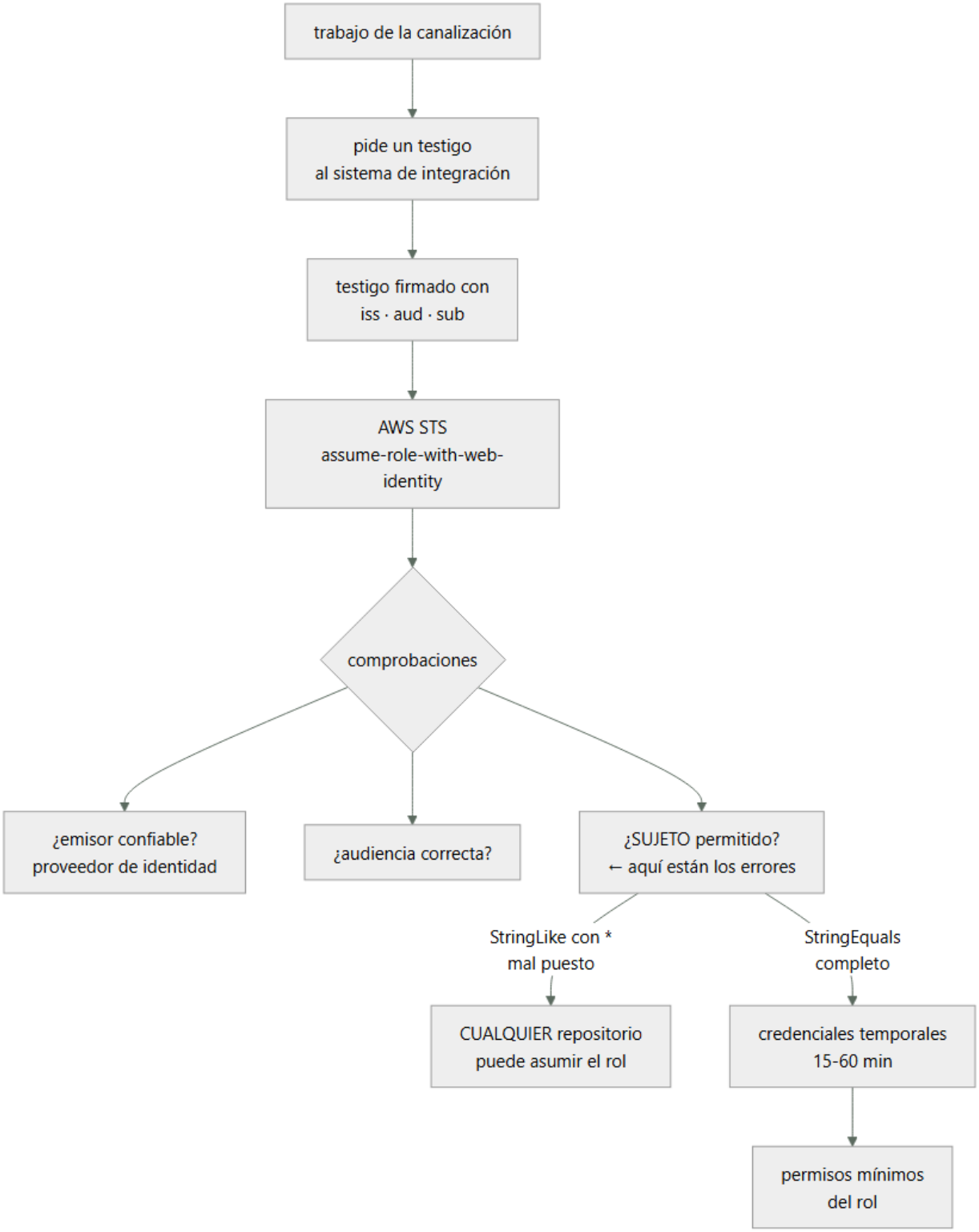
Concepto	Comprensión verificable
federación por identidad de carga	Intercambiar un testigo firmado por el sistema de integración por credenciales temporales del proveedor de nube.
proveedor de identidad	Recurso que declara en qué emisor se confía y con qué audiencia.
política de confianza	Regla del rol que dice quién puede asumirlo y bajo qué condiciones.
sujeto	Campo del testigo que identifica el repositorio, la rama, el entorno o la etiqueta que solicita el acceso.
audiencia	Destinatario declarado del testigo. Evita que un testigo emitido para otro servicio sirva aquí.
entorno protegido	Mecanismo del sistema de integración que exige aprobación antes de que un trabajo obtenga el testigo.

Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Cómo funciona el intercambio

El mecanismo tiene cuatro piezas y conviene entenderlas antes de configurarlas.

```
1  EL SISTEMA DE INTEGRACIÓN EMITE UN TESTIGO
   firmado por él, con vida de minutos, que dice
     iss   quién lo emite (el servicio de integración)
     aud   para quién es
     sub   QUIÉN lo pide: organización, repositorio, rama,
           entorno o etiqueta
```

```
     y otros campos: rama, flujo de trabajo, ejecutor…

2  AWS TIENE DECLARADO UN PROVEEDOR DE IDENTIDAD
   que dice «confío en los testigos firmados por este
   emisor, con esta audiencia»

3  EL ROL TIENE UNA POLÍTICA DE CONFIANZA
   que dice «pueden asumirme los testigos de ese proveedor
   cuyo sujeto cumpla ESTAS condiciones»

4  EL TRABAJO INTERCAMBIA testigo por credenciales
   temporales, de 15 a 60 minutos, con los permisos del rol
```

Y lo que se gana, que es lo que justifica la migración:

```
sin claves de acceso guardadas como secretos
  → no hay nada que robar del repositorio
  → no hay nada que rotar                          clase 159
  → si el repositorio se compromete, el atacante necesita
    además cumplir las condiciones del sujeto
credenciales de minutos, no de años
y cada uso queda registrado con el sujeto que lo pidió
  → auditoría atribuible                          clase 141
```

Y una comparación con lo que sustituye:

```
CLAVE DE ACCESO GUARDADA COMO SECRETO
  vive años
  sirve desde cualquier sitio del mundo
  se copia con un simple volcado de variables de entorno
  y su rotación es un procedimiento manual que nadie hace
                                                    ley 22

→ en la clase 179, cuatro servicios compartían una clave
  estática de tres años
```

Y el ámbito de aplicación, que es más amplio de lo que suele usarse:

```
el mismo mecanismo sirve para
  GitHub Actions y GitLab CI hacia AWS
  y también hacia Azure y Google Cloud
  cargas en Kubernetes hacia AWS                  clase 213
  y cualquier emisor que firme testigos con el formato
    estándar
```

2. Donde se cometen los errores graves

La configuración es corta y tiene un punto donde un error convierte el mecanismo en algo peor que una clave estática.

```
LA CONDICIÓN DE SUJETO

  ✗ CATASTRÓFICO
    "StringLike": { "...:sub": "*" }
    → CUALQUIER repositorio de CUALQUIER organización del
      mundo puede asumir tu rol
    → basta con que alguien sepa el nombre del rol

  ✗ MUY MALO
    "StringLike": { "...:sub": "repo:miorg/*" }
    → cualquier repositorio de tu organización, incluidos
      los que cree un becario
    → y si alguien puede crear un repositorio, puede
      desplegar en producción

  ✗ SUTIL Y FRECUENTE
    olvidar la condición de AUDIENCIA
    → un testigo emitido para otro servicio podría valer

  ✓ CORRECTO
    "StringEquals": {
      "...:aud": "<audiencia declarada>",
      "...:sub": "repo:miorg/mirepo:environment:produccion"
    }
    → un repositorio, un entorno, exacto
```

Y las formas de sujeto que conviene conocer:

```
repo:org/repo:ref:refs/heads/main       una rama concreta
repo:org/repo:environment:produccion    un entorno protegido
repo:org/repo:ref:refs/tags/v*          etiquetas (con
                                        StringLike, y con
                                        cuidado)
repo:org/repo:pull_request              propuestas de cambio
                                        ← NUNCA para producción
```

Y la última merece énfasis:

```
una propuesta de cambio la puede abrir cualquiera desde una
bifurcación
→ dar permisos de producción a ese sujeto equivale a dar
  permisos de producción a cualquiera
→ los trabajos de propuesta de cambio usan un rol de solo
  lectura, y nada más
```

El entorno protegido, que es la pieza que casi nadie usa y la que más aporta:

```
un entorno del sistema de integración puede exigir
  aprobación manual de personas concretas
  que la rama sea una determinada
  una espera mínima

y el testigo incluye el nombre del entorno en el sujeto
→ atar el rol de producción al entorno «produccion»
  significa que ningún flujo puede obtenerlo sin pasar por
  la aprobación                                  clase 106
```

Y una comprobación que hay que hacer siempre:

```
intenta asumir el rol desde
  otra rama                    → debe fallar
  otro repositorio             → debe fallar
  una bifurcación              → debe fallar
  un flujo sin el entorno      → debe fallar
→ y si alguna funciona, la condición está mal escrita
                                                  ley 22
```

3. Permisos del rol y separación

Quitar la clave estática no sirve de nada si el rol federado puede hacerlo todo.

```
UN ROL POR PROPÓSITO, NO UNO QUE VALGA PARA CUALQUIER COSA
  rol de lectura           propuestas de cambio: validar,
                           planificar, analizar
  rol de despliegue de
    preproducción          escribir en preproducción
  rol de despliegue de
    producción             escribir en producción, atado al
                           entorno protegido
  rol de publicación de
    artefactos             subir imágenes al registro

→ y cada uno con permisos mínimos y con su condición de
  sujeto propia                                  clase 134
```

Y los permisos que nunca debe tener el rol de la canalización:

```
modificar políticas de identidad o crear roles
  → si puede crear un rol con más permisos, tiene todos
desactivar el registro de auditoría
borrar copias de seguridad
modificar la propia política de confianza
  → podría ampliarse a sí mismo

→ estos permisos se ponen en una barrera de la organización,
  para que ni siquiera un administrador de la cuenta pueda
  concederlos                                    clase 169
```

Y dos controles que refuerzan:

```
DURACIÓN MÍNIMA
  la sesión dura lo que dure el trabajo, no una hora por
  defecto
```

```
CONDICIONES ADICIONALES EN LA POLÍTICA DE PERMISOS
  restringir por región
  restringir por etiqueta de recurso
  exigir que la petición venga de la sesión federada, no de
    una credencial cualquiera
```

Y una decisión de arquitectura sobre cuentas:

```
el rol de producción vive en la CUENTA de producción
la canalización no tiene ningún acceso permanente a esa
cuenta
→ y el paso de producción está en un flujo separado, con
  entorno protegido
→ así el compromiso del repositorio no da acceso a
  producción sin una aprobación humana        clases 108, 189
```

4. Migrar sin romper

Migrar decenas de canalizaciones a la vez es donde se rompen las cosas. El patrón es el de expandir y contraer.

```
1  CREAR el proveedor de identidad y los roles, con
   condiciones estrictas
   → sin tocar nada de lo existente

2  MIGRAR UNA canalización poco crítica
   y comprobar que funciona y que las pruebas negativas
   fallan como deben

3  MIGRAR EL RESTO por lotes
   las claves antiguas SIGUEN existiendo

4  MEDIR EL USO de cada clave antigua
   → una clave sin uso durante N días es candidata a
     desactivar
   → y las que siguen usándose revelan canalizaciones que
     nadie sabía que existían

5  DESACTIVAR, no borrar, y esperar
   → si algo se rompe, se reactiva en segundos

6  BORRAR, y con ellas los secretos del repositorio
```

Y el paso 4 es el que descubre cosas:

```
el informe de credenciales y los registros de acceso dicen
  qué claves existen, cuándo se usaron por última vez y
  desde dónde
→ las claves sin uso en 90 días son riesgo puro
→ y las que se usan desde direcciones inesperadas son otra
  cosa                                          clase 141
```

Lo que hay que vigilar una vez migrado:

```
usos del rol federado, con el sujeto que lo pidió
intentos de asunción DENEGADOS
  → un aumento significa que alguien está probando
claves de acceso de larga duración que existan todavía
  → función de aptitud: cero permitidas            clase 190
cambios en las políticas de confianza
  → auditar siempre; ampliar una condición es el camino
    para saltarse todo lo anterior
```

Y una limitación que hay que conocer:

```
no todo se puede federar
  herramientas de terceros que solo aceptan clave y secreto
  sistemas antiguos
→ para esos, credenciales de rotación automática y ámbito
  mínimo, con fecha de revisión escrita             ley 25
```

Y la lista de comprobación de la clase:

- ■ existe el proveedor de identidad con la audiencia declarada
- ■ ninguna condición de sujeto usa comodín amplio
- ■ la condición de audiencia está presente
- ■ producción está atada a un entorno protegido con

```
  aprobación
■ las propuestas de cambio usan un rol de solo lectura
■ hay un rol por propósito, con permisos mínimos
■ ningún rol de canalización puede crear roles ni modificar
  su confianza
■ la duración de la sesión es la del trabajo
■ las pruebas negativas desde otra rama, repositorio y
  bifurcación fallan
■ no queda ninguna clave de acceso de larga duración
■ hay función de aptitud que lo comprueba
■ se vigilan los intentos denegados y los cambios de
  confianza
```

Y el cierre que enlaza con la clase siguiente: con una canalización que puede desplegar sin secretos, queda desplegar algo. La aplicación sin servidores —funciones, pasarela de API y su definición como código— es la materia de la clase 207.

Ejemplo trabajado

CloudShop migra 41 canalizaciones a federación. Lo que sigue es la condición de sujeto mal escrita que encontraron en la primera revisión, la migración por lotes, y las once claves que aparecieron al medir el uso.

El punto de partida:

```
canalizaciones                                      41
claves de acceso de larga duración en secretos      23
  antigüedad media                            2,1 años
  con permisos de administrador                      6
  compartidas entre varios repositorios              4
rotaciones realizadas en 3 años                      0
```

El primer intento, y el fallo que encontró la revisión.

```
un equipo había migrado ya su canalización, en marzo
su política de confianza decía

  "Condition": {
    "StringLike": {
      "token.actions.githubusercontent.com:sub":
        "repo:cloudshop/*"
    }
  }

lo que eso permitía
  cualquier repositorio de la organización cloudshop podía
  asumir el rol
  → y ese rol tenía permisos de escritura en producción

y lo que lo hacía peor de lo que parecía
  crear un repositorio en la organización lo podía hacer
  cualquiera de los 214 miembros
  → cualquiera podía desplegar en producción creando un
    repositorio nuevo
  → sin aprobación, sin revisión, sin dejar rastro obvio

y faltaba además la condición de audiencia

la prueba negativa que lo demostró
  se creó un repositorio de prueba en la organización
  se escribió un flujo que asumía el rol
  → funcionó a la primera
  tiempo desde la idea hasta desplegar en producción: 6 min
```

Y el análisis:

```
esta configuración era PEOR que la clave estática que
sustituía
  la clave, al menos, estaba en un secreto de un repositorio
  concreto
  esto era accesible desde cualquier repositorio nuevo
→ y se había revisado y aprobado, porque «quitaba el
  secreto»                                    clase 191
```

La configuración corregida.

```
proveedor de identidad, uno por sistema de integración,
con la audiencia declarada

ROLES, uno por propósito

  cloudshop-ci-lectura
    sub   repo:cloudshop/tienda:pull_request
    aud   comprobada
    puede validar plantillas, planificar cambios, leer
          artefactos
    NO puede escribir nada

  cloudshop-ci-preproduccion
    sub   repo:cloudshop/tienda:ref:refs/heads/main
    puede desplegar en la cuenta de preproducción

  cloudshop-ci-produccion
    sub   repo:cloudshop/tienda:environment:produccion
    vive en la CUENTA de producción
    entorno protegido con aprobación de 2 personas de una
      lista, y espera mínima de 5 minutos
    permisos mínimos, restringidos por región y por etiqueta

  cloudshop-ci-artefactos
    sub   repo:cloudshop/tienda:ref:refs/heads/main
    puede subir al registro de imágenes, nada más

BARRERA DE ORGANIZACIÓN
  ningún rol de canalización puede
    crear o modificar roles
    modificar políticas de confianza
    desactivar el registro de auditoría
    borrar copias de seguridad
  → denegación explícita a nivel de organización, no de
    cuenta                                          clase 169
```

Las pruebas negativas, ejecutadas tras corregir:

```
✓  asumir el rol de producción desde otra rama      denegado
✓  asumir desde otro repositorio de la organización denegado
✓  asumir desde una bifurcación externa             denegado
✓  asumir desde un flujo sin el entorno             denegado
✓  asumir con un testigo de otra audiencia          denegado
✗  el rol de preproducción podía leer un bucket de
   producción
   → una política heredada del rol antiguo permitía
     s3:GetObject sobre un comodín
   → corregido; y añadida condición de etiqueta de entorno
✓  el rol de artefactos intentando desplegar        denegado
✓  el rol de canalización creando un rol            denegado
   por la barrera
```

La migración de las 41, por lotes.

```
semana 1     proveedor y roles creados; nada más tocado
semana 2     1 canalización interna, poco crítica
             pruebas negativas ejecutadas
semanas 3-6  32 canalizaciones, en lotes de 8
semanas 5-7  las 8 restantes, que eran las de producción
             → con entorno protegido, requirió coordinar
               aprobadores
semanas 7-11 MEDIR EL USO de las 23 claves antiguas
semana 12    desactivar (no borrar) las sin uso
semana 16    borrar y limpiar los secretos
```

Lo que apareció al medir el uso de las claves antiguas:

```
claves sin uso en 30 días                          12
  → desactivadas sin incidencia

claves aún en uso                                  11
  de canalizaciones ya migradas (residuos)          3
    → un paso del flujo seguía usando la clave
```

```
  de canalizaciones que NO estaban en la lista de 41    8
    · un trabajo programado en una máquina de un equipo
      de datos
    · un script en el portátil de una persona, ejecutado a
      mano cada lunes para subir tarifas       ← clase 192
    · una herramienta de un proveedor externo
    · dos integraciones de sistemas antiguos
    · un sistema de despliegue que se creía retirado en 2023
    · dos usos que nadie pudo explicar

→ el inventario decía 41 canalizaciones
→ la realidad tenía 49                                ley 24
```

Y el tratamiento de los 8:

```
5   migrados a federación o a roles asumidos desde la nube
2   no se pudieron federar (proveedor externo y sistema
    antiguo)
    → credenciales con rotación automática cada 30 días,
      permisos mínimos y fecha de revisión escrita   ley 25
1   se apagó: era el sistema de despliegue retirado
    → llevaba 2 años con credenciales de administrador
      válidas                                        ley 20
```

El resultado:

```
claves de acceso de larga duración            23           2
  con rotación automática                      0           2
  con permisos de administrador                6           0
roles con condición de sujeto amplia           1           0
entornos protegidos con aprobación             0           4
canalizaciones conocidas                      41          49
tiempo para que un miembro cualquiera
  despliegue en producción sin aprobación    6 min   imposible
intentos denegados registrados al mes         n/a          14
  → todos, flujos mal configurados tras un cambio de rama
```

Y la función de aptitud que lo mantiene:

```
«cero claves de acceso de larga duración sin excepción
 registrada»
  excepciones vivas                                    2
  con dueño y fecha de revisión                        2
  → comprobado en cada cambio de infraestructura
                                                clase 190
```

La lección que esta clase deja: la primera migración quitó el secreto y empeoró la seguridad, porque una condición de sujeto con comodín permitía que cualquiera de doscientos catorce miembros desplegara en producción creando un repositorio. Se había revisado y aprobado. Y al medir el uso de las claves antiguas apareció lo de siempre: ocho canalizaciones que no estaban en el inventario, incluida una que llevaba dos años con credenciales de administrador para un sistema que se creía retirado.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/206-oidc-de-github-y-gitlab-hacia-aws-sin-secretos/
lab.py
```

El laboratorio selecciona el motor de práctica iam y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa aws-oidc-federation en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una matriz de acceso mínimo con prueba de denegación. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye aws-oidc-federation para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

■■ Errores frecuentes

Síntoma	Causa probable	Corrección
Cualquiera de la organización puede desplegar en producción	La condición de sujeto usa un comodín sobre la organización o el repositorio	Usa comparación exacta con repositorio y entorno concretos, y añade siempre la condición de audiencia.
Un flujo de propuesta de cambio obtiene permisos de escritura	El sujeto de propuesta de cambio está permitido en un rol con escritura	Las propuestas de cambio solo asumen un rol de lectura; las bifurcaciones pueden abrirlas desde fuera.
El rol federado puede ampliarse a sí mismo	Tiene permisos para crear roles o modificar su política de confianza	Deniega esos permisos en una barrera de organización, no solo en la política del rol.
Quitar el secreto no mejoró nada	El rol federado conserva permisos amplios heredados del anterior	Un rol por propósito, con permisos mínimos y condiciones de región y etiqueta.
Al borrar las claves antiguas se rompen procesos desconocidos	Se borraron sin medir el uso previo	Mide el uso semanas, desactiva antes de borrar, y trata los usos inesperados como hallazgos de inventario.
Quedan credenciales estáticas que no se pueden federar	Herramientas de terceros o sistemas antiguos que solo aceptan clave y secreto	Rotación automática, ámbito mínimo y excepción registrada con dueño y fecha de revisión.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué cuatro piezas intervienen en el intercambio de testigo por credenciales?
2. ¿Por qué una condición de sujeto con comodín puede ser peor que una clave estática?
3. ¿Qué aporta atar el rol de producción a un entorno protegido?
4. ¿Qué permisos no debe tener nunca un rol de canalización y dónde se deniegan?
5. ¿Qué se descubre al medir el uso de las claves antiguas antes de borrarlas?

Referencias

- GitHub (2025). About security hardening with OpenID Connect — por qué un token efímero sustituye a la clave estática. <https://docs.github.com/en/actions/deployment/security-hardening-your-deployments/about-security-hardening-with-openid-connect>
- GitHub (2025). Configuring OpenID Connect in Amazon Web Services — configuración concreta del proveedor de identidad en AWS. <https://docs.github.com/en/actions/deployment/security-hardening-your-deployments/configuring-openid-connect-in-amazon-web-services>
- GitLab (2025). Connect to cloud services with OIDC — el procedimiento equivalente desde GitLab. <https://docs.gitlab.com/ee/ci/cloudservices/>
- AWS (2025). Create an OpenID Connect identity provider in IAM — alta del proveedor de identidad en IAM. <https://docs.aws.amazon.com/IAM/latest/UserGuide/idrolesproviderscreateoidc.html>
- AWS (2025). AssumeRoleWithWebIdentity and session policies — acotar la sesión asumida mediante políticas. <https://docs.aws.amazon.com/STS/latest/APIReference/APIAssumeRoleWithWebIdentity.html>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 17 en PDF · Recorrido de AWS en PDF · Manual integral

Anterior	Índice	Siguiente
← 205 · Hosting progresivo con Amplify, S3 y CloudFront	Parte 17 · Programa	207 · SAM, Lambda, API Gateway y despliegue serverless →

Evaluación — 206 OIDC de GitHub y GitLab hacia AWS sin secretos

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega aws-oidc-federation con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

207 — SAM, Lambda, API Gateway y despliegue serverless

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: serverless · Estado: EXECUTABLECORE

Propósito

Construir y desplegar una aplicación sin servidores en AWS con las decisiones que separan un prototipo de algo que aguanta producción. La clase cubre la definición como código con SAM, la configuración de funciones que de verdad importa —memoria, concurrencia, plazos y arranque en frío—, la elección de pasarela y su coste, y los tres asuntos que rompen estos sistemas: la concurrencia que agota la base, el reintento que duplica y el despliegue que no se puede revertir.

Resultados de aprendizaje

Al finalizar podrás:

1. Definir una aplicación sin servidores como código, con entornos separados.
2. Dimensionar memoria, concurrencia y plazos con criterio y no por defecto.
3. Elegir el tipo de pasarela por coste y por lo que necesitas de ella.
4. Controlar el arranque en frío y decidir cuándo pagar por evitarlo.
5. Desplegar de forma escalonada, con reversión automática.

Conceptos centrales

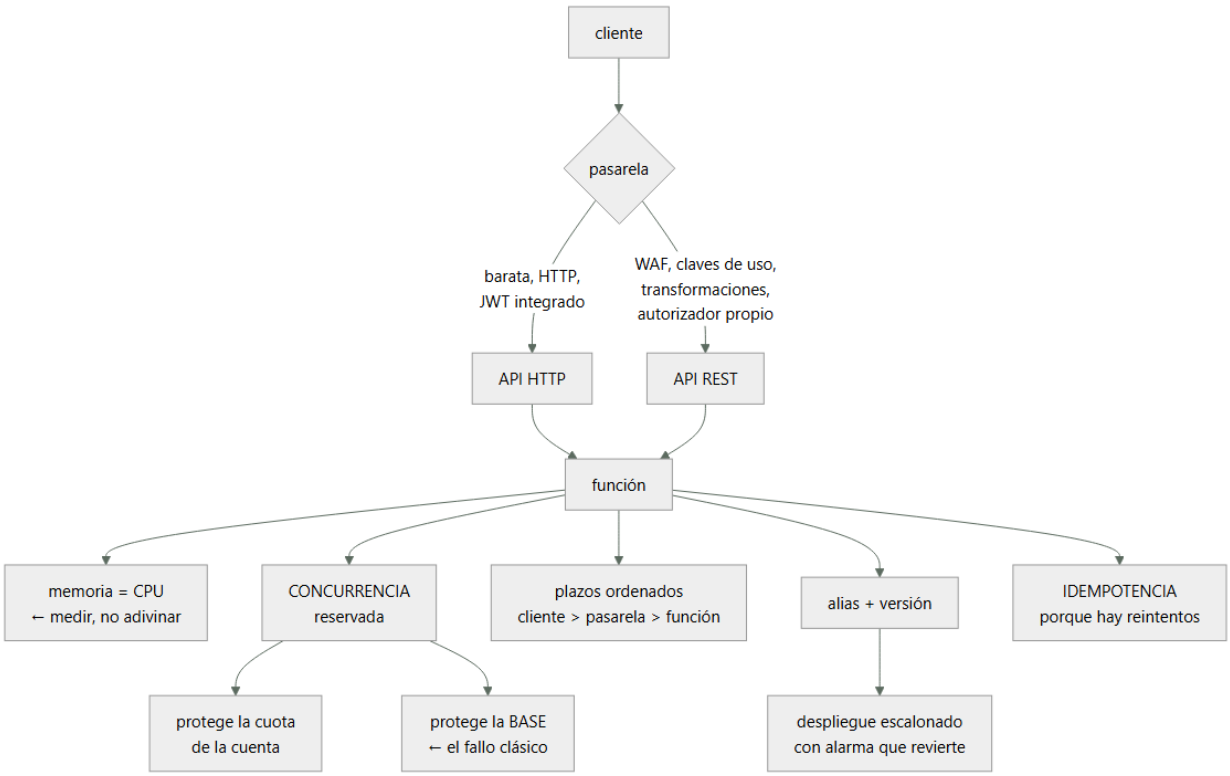
Concepto	Comprensión verificable
SAM	Extensión de CloudFormation que declara funciones, APIs y tablas con menos texto y despliega con un solo comando.
concurrencia reservada	Máximo de ejecuciones simultáneas de una función. Es a la vez un límite y una reserva de la cuota de la cuenta.
concurrencia aprovisionada	Instancias mantenidas calientes. Elimina el arranque en frío y se paga por hora.
arranque en frío	Retraso al crear un entorno de ejecución nuevo. Afecta a los percentiles altos, no a la media.
alias y versión	Puntero estable a una versión concreta de la función. Es lo que permite desplazar tráfico y revertir.
despliegue escalonado	Desplazamiento progresivo del tráfico al alias nuevo, con alarmas que lo revierten solo.

Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Definir como código, con entornos

Una plantilla de SAM declara funciones, rutas, permisos y recursos con mucho menos texto que la plantilla equivalente, y se despliega con un comando.

```
LO QUE UNA PLANTILLA DEBE TENER DESDE EL PRINCIPIO
  parámetros por entorno, sin valores incrustados
  nombres derivados del entorno, no fijos
  permisos por función, no un rol compartido
  etiquetas obligatorias: dueño, entorno, servicio  clase 142
  y ninguna referencia a recursos creados a mano
```

Y los errores de estructura que se pagan pronto:

```
✗ UNA PILA GIGANTE con todo
  → cada despliegue toca todo; un fallo bloquea la pila
  → y las pilas tienen límites de recursos

✓ PILAS POR CICLO DE VIDA
  lo que cambia a diario   funciones y API
  lo que cambia poco       tablas, buckets, colas
  → y las tablas de datos NUNCA en la misma pila que el
    código: un borrado de pila no debe poder llevarse los
    datos

✗ EL MISMO ROL PARA TODAS LAS FUNCIONES
  → una función comprometida alcanza todo         clase 189

✓ UN ROL POR FUNCIÓN, con permisos sobre los recursos
  concretos que usa
```

Y la política de borrado, que casi nadie configura y que evita desastres:

```
los recursos con datos llevan política de retención
  → al borrar la pila, la tabla y el bucket SOBREVIVEN
→ sin esto, un `delete-stack` equivocado en la cuenta
  equivocada borra los datos                      clase 166
```

Los entornos, que se hacen con cuentas y no con prefijos:

```
cuenta de desarrollo · preproducción · producción
```

```
→ la separación por cuenta es la única que impide de verdad
  que un despliegue de preproducción toque producción
                                                  clase 169
→ dentro de cada cuenta, una pila por servicio
```

Y el desarrollo local, que ahorra mucho tiempo y engaña en una cosa:

```
la emulación local sirve para la lógica
no reproduce
  latencias reales, límites de concurrencia, permisos,
  arranques en frío ni el comportamiento de la pasarela
→ las pruebas que importan se hacen en un entorno real,
  aunque sea efímero                              clase 104
```

2. Memoria, concurrencia y plazos

Tres ajustes que vienen con valores por defecto y que deciden coste, latencia y estabilidad.

La memoria, que es en realidad la CPU:

```
en Lambda, la CPU asignada es proporcional a la memoria
→ más memoria puede salir MÁS BARATO, porque la función
  termina antes

cómo se decide
  medir la misma carga con 512, 1.024, 1.792 y 3.008 MB
  y comparar duración × precio
  → hay un mínimo de coste, y casi nunca está en el valor
    por defecto

típicamente
  funciones de cálculo         se benefician mucho
  funciones de espera de red   no; la CPU no acelera esperar
```

La concurrencia, que es donde se rompen estos sistemas:

```
EL FALLO CLÁSICO
  una función sin límite de concurrencia escala a miles de
  ejecuciones simultáneas
  cada una abre una conexión a la base relacional
  → la base agota sus conexiones y CAE
  → y con ella, todo lo demás que la usa

Y es exactamente el problema de la clase 186
  el recurso saturado no es la función: es la base

LAS CORRECCIONES
  concurrencia reservada por función
    = número de conexiones que la base puede dar / conexiones
      por ejecución
  y un intermediario de conexiones (RDS Proxy) si la base
    es relacional
  → o una base pensada para esto                  clase 208
```

Y el otro efecto de la concurrencia reservada:

```
la cuota de concurrencia es DE LA CUENTA, compartida
→ una función que se dispara consume la cuota y las demás
  empiezan a ser rechazadas
→ reservar concurrencia a las funciones críticas es
  reservarles capacidad, no solo limitarlas
```

Los plazos, con la misma regla de la clase 196:

```
cliente > pasarela > función > llamadas de la función

y dos particularidades
  la API HTTP y la REST tienen un plazo máximo propio
    → si la función tarda más, la pasarela corta y la
      función SIGUE ejecutándose y facturando
  el plazo por defecto de una función es bajo, y el máximo
    es alto
    → poner el máximo «por si acaso» significa pagar
      15 minutos de una función colgada

regla   plazo = p99 esperado × 2 o 3, nunca el máximo
```

Y una consecuencia que sorprende:

```
una función con plazo de 15 min que se queda esperando una
dependencia caída
  consume concurrencia durante 15 minutos por invocación
  → agota la cuota y tumba lo demás
→ el plazo corto es un mecanismo de contención  clase 153
```

3. Pasarela, arranque en frío y coste

La elección de pasarela, que tiene consecuencias de coste importantes:

```
API HTTP
  más barata (del orden de un tercio)
  menor latencia añadida
  autorizador de testigo integrado                clase 209
  CORS por configuración
  NO tiene   claves de uso y planes, transformaciones de
             petición y respuesta, WAF directo en algunos
             casos, caché integrada

API REST
  todo lo anterior
  autorizadores propios
  WAF asociado
  caché de respuestas
  y despliegues por etapa con variables

FUNCIÓN CON URL DIRECTA
  sin pasarela; la más barata y la más limitada
  → útil para webhooks internos, no para una API pública

BALANCEADOR DE APLICACIÓN → función
  útil si ya hay balanceador y se quiere una sola entrada
```

Y el criterio:

```
empieza por API HTTP
pasa a REST solo si necesitas algo de su lista
→ y si la aplicación va detrás de CloudFront, muchas de
  esas necesidades (WAF, caché) se resuelven ahí
                                                clase 205
```

El arranque en frío, que se exagera y a la vez se ignora donde importa:

```
QUÉ ES
  crear un entorno de ejecución nuevo: descargar el
  paquete, arrancar el intérprete, ejecutar el código de
  inicialización

CUÁNTO
  lenguajes interpretados y paquetes pequeños  100-400 ms
  máquinas virtuales de lenguajes con arranque
    pesado                                     0,5-3 s
  con conexión a una red virtual, hoy, poco más
  y el código de inicialización propio, lo que tarde

A QUIÉN AFECTA
  a los percentiles altos, no a la media
  → con 100 peticiones/s, casi todo va caliente
  → con 1 petición cada 5 minutos, casi todo va frío
```

Y las correcciones, por orden de coste:

```
1  REDUCIR EL PAQUETE                           gratis
   quitar dependencias, empaquetar solo lo necesario

2  MOVER TRABAJO FUERA DE LA INICIALIZACIÓN     gratis
   → pero reutilizar conexiones ENTRE invocaciones sí va
     en la inicialización: se crean una vez

3  ELEGIR UN TIEMPO DE EJECUCIÓN LIGERO         gratis

4  CONCURRENCIA APROVISIONADA                   se paga
   instancias calientes permanentes
```

```
   → tiene sentido en el camino crítico con tráfico
     irregular
   → y se puede programar por horas

5  MANTENER CALIENTE CON INVOCACIONES PERIÓDICAS
   → truco antiguo, poco fiable con concurrencia; evitarlo
```

El coste, que se estima mal siempre en la misma dirección:

```
lo que se cuenta   invocaciones y duración
lo que se olvida
  peticiones de la pasarela            ← suele ser mayor
  transferencia de datos
  registros: ingesta y almacenamiento  ← el sorprendente
  llamadas a otros servicios por petición
  concurrencia aprovisionada, por hora

→ en sistemas de mucho tráfico y poca duración, la pasarela
  y los registros pueden superar al cómputo
```

4. Desplegar, revertir y ser idempotente

El despliegue escalonado, que SAM da hecho y casi nadie activa:

```
versión + alias
  cada despliegue publica una VERSIÓN inmutable
  el alias apunta a una versión
  → el tráfico va al alias

desplazamiento progresivo
  10 % durante 5 min, luego 100 %
  o lineal: 10 % cada minuto

alarmas asociadas
  si la tasa de error o la latencia superan el umbral
  durante el desplazamiento, el alias VUELVE solo a la
  versión anterior
  → reversión automática, sin nadie mirando      clase 102

comprobación previa y posterior
  funciones que validan antes de desplazar y después
```

Y lo que hay que tener en cuenta al revertir:

```
revertir el código es inmediato
revertir un cambio de ESQUEMA de datos, no
→ los cambios de datos siguen la regla de expandir y
  contraer, con el código tolerando ambas formas
                                                clase 188
```

La idempotencia, que en este entorno no es opcional:

```
POR QUÉ HAY REINTENTOS SIEMPRE
  invocación asíncrona: reintenta 2 veces por defecto
  desde una cola: reintenta hasta agotar y va a la cola de
    fallidos                                    clase 210
  desde un flujo de eventos: reprocesa el lote entero si
    falla un elemento
  el cliente reintenta ante un plazo vencido

→ toda función con efecto debe ser idempotente  clase 117
```

Y el mecanismo:

```
clave de idempotencia por operación
  del mensaje, de la cabecera del cliente, o derivada del
  contenido
registro de claves procesadas, con caducidad
y la comprobación ANTES del efecto, con escritura
  condicionada                                  clase 149
```

Lo que hay que vigilar:

```
invocaciones, errores y duración por función
EJECUCIONES RECHAZADAS por límite de concurrencia
  → señal de que la reserva es corta o algo se disparó
concurrencia usada frente a la reservada
```

```
mensajes en la cola de fallidos, y su antigüedad ley 13
plazos vencidos, separados de los errores
y el coste por función, no solo el total
```

Y la lista de comprobación de la clase:

- la plantilla no tiene valores incrustados por entorno
- los datos están en pilas separadas del código
- los recursos con datos tienen política de retención
- hay un rol por función, con permisos concretos
- la memoria se eligió midiendo, no por defecto
- hay concurrencia reservada en las funciones que tocan bases con conexiones limitadas
- los plazos son múltiplos del p99, no el máximo
- los plazos están ordenados cliente > pasarela > función
- la pasarela elegida corresponde a lo que se necesita
- el arranque en frío se midió antes de pagar por evitarlo
- el despliegue es escalonado con alarma y reversión automática
- toda función con efecto es idempotente
- hay alerta de ejecuciones rechazadas y de cola de fallidos
- el coste incluye pasarela, transferencia y registros

Y el cierre que enlaza con la clase siguiente: una función que escala a miles de ejecuciones necesita un almacén que escale igual, y eso cambia por completo cómo se diseña el modelo de datos. DynamoDB por patrones de acceso es la materia de la clase 208.

Ejemplo trabajado

CloudShop pasa su API de pedidos a funciones. Lo que sigue es el incidente del primer día de campaña, las cinco correcciones que salieron, y el análisis de coste que reveló dónde estaba el dinero de verdad.

El montaje inicial, con valores por defecto:

```
API REST → 14 funciones
memoria                        128 MB (por defecto)
plazo                          30 s
concurrencia                   sin límite
base                           PostgreSQL gestionada,
                               máximo 400 conexiones
despliegue                     directo, sin alias
funciona en pruebas            perfectamente
```

El incidente del primer día de campaña.

```
11:02   arranca la campaña; el tráfico se multiplica por 22
11:04   la base de datos deja de aceptar conexiones
11:04   CUANTO usa la base falla, incluidos el panel
        interno y los informes
11:31   se identifica la causa
12:20   servicio restablecido

qué pasó
  las funciones escalaron a 3.100 ejecuciones simultáneas
  cada una abría su propia conexión
  3.100 > 400
  → la base agotó conexiones y empezó a rechazar
  → las funciones fallaban, el cliente reintentaba, y los
    reintentos abrían más conexiones                clase 186

y un efecto secundario
  las 3.100 ejecuciones consumieron la cuota de concurrencia
  de la CUENTA
  → funciones de otros servicios empezaron a ser rechazadas
  → incluida la que procesa los correos de confirmación
```

Las cinco correcciones.

```
1  CONCURRENCIA RESERVADA, calculada
   conexiones disponibles para la API        320
   conexiones por ejecución                    1
   margen                                    20 %
   → concurrencia reservada de las funciones que tocan la
```

```
     base: 256 en total, repartida
       crear pedido        120
       consultar pedido     80
       modificar pedido     40
       resto                16
   y las funciones que NO tocan la base quedan sin reserva

   efecto   por encima de 256, las peticiones se rechazan
            con 429 rápido, en vez de tumbar la base
            → rechazar pronto es mejor que encolar
                                              clase 186

2  INTERMEDIARIO DE CONEXIONES
   se añadió un proxy de base de datos
   → 3.100 ejecuciones comparten 180 conexiones reales
   → la concurrencia reservada se pudo subir a 900
   coste    +140 €/mes

3  PLAZOS ORDENADOS Y CORTOS
   antes   cliente 60 s · pasarela 29 s · función 30 s
   → la pasarela cortaba a los 29 s y la función seguía
     ejecutándose 1 s más, facturando y ocupando
     concurrencia
   después cliente 12 s · pasarela 10 s · función 8 s
           llamadas de la función: 3 s

   efecto durante una degradación de la pasarela de pago
     antes    cada invocación ocupaba 30 s de concurrencia
     después  8 s
     → la cuota aguanta 3,7 veces más tiempo

4  MEMORIA, MEDIDA
   la función de crear pedido, con la misma carga

     memoria   duración   coste relativo
      128 MB    3.900 ms       1,00
      512 MB    1.010 ms       1,04
    1.024 MB      520 ms       1,07
    1.792 MB      340 ms       1,22

   → el mínimo estaba en 128 MB por poco, pero la latencia
     era inaceptable
   → se eligió 1.024 MB: 7 % más caro y 7,5 veces más rápido
   → y el p99 de la API bajó de 4,2 s a 0,7 s

   y la función de generar informes, de cálculo puro
      128 MB   48 s   1,00
    3.008 MB    2,4 s  0,52
   → más memoria, la MITAD de coste

5  DESPLIEGUE ESCALONADO
   alias y versiones
   desplazamiento 10 % durante 5 min, luego 100 %
   alarmas: tasa de error > 1 % o p99 > 1,5 s
   reversión automática

   en los 8 meses siguientes
     despliegues                          186
     revertidos automáticamente             7
     tiempo medio de reversión          2 min
     incidentes causados por despliegue     0
```

La idempotencia, que faltaba.

```
en el incidente, los reintentos del cliente crearon pedidos
duplicados
  pedidos duplicados detectados                    64
  cobrados dos veces                               11

corrección
  la petición de crear pedido lleva una clave de
  idempotencia generada por el cliente
  la función escribe con condición «si no existe esa clave»
```

```
  → un reintento devuelve el pedido original, no crea otro

y la prueba negativa
  enviar la misma petición 50 veces en paralelo
  → 1 pedido creado, 49 respuestas idénticas       ✓
```

El análisis de coste, que dio la sorpresa.

```
estimación inicial del equipo
  cómputo de funciones                          340 €/mes
  «lo demás es despreciable»

factura real del primer mes completo
  cómputo de funciones                          410 €
  peticiones de API Gateway (REST)              890 €   ←
  registros: ingesta                            620 €   ←
  registros: almacenamiento                     180 €
  transferencia                                 210 €
  proxy de base de datos                        140 €
  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  total                                       2.450 €

  → el cómputo era el 17 % de la factura
```

Y las tres correcciones de coste:

```
PASARELA
  se revisó qué se usaba de la API REST
    claves de uso        no
    transformaciones     no
    caché                no (la hace CloudFront)  clase 205
    WAF                  sí → pero se puede poner en
                         CloudFront
  → migración a API HTTP
  890 € → 310 €

REGISTROS
  las funciones registraban cada petición completa, con
  cuerpo
  → nivel de registro por entorno; en producción solo
    errores y muestreo del 1 % de las peticiones correctas
  → caducidad de 14 días en lugar de indefinida
  620 € → 95 €;  almacenamiento 180 € → 25 €

TRANSFERENCIA
  las respuestas incluían campos que el cliente no usaba
  210 € → 120 €

total                              2.450 € → 1.100 €
```

El resultado, seis meses después:

```
p99 de la API                            4,2 s       0,7 s
caídas de la base por agotamiento      1 (campaña)       0
pedidos duplicados                        64             0
despliegues revertidos a mano              3             0
  (automáticos)                            —             7
coste mensual                        2.450 €     1.100 €
  proporción de cómputo                 17 %        37 %
ejecuciones rechazadas en pico          n/a      412/día
  → a propósito: mejor rechazar que tumbar la base
```

La lección que esta clase deja: el sistema funcionaba perfectamente en pruebas y cayó en cuatro minutos el día de campaña, porque el recurso saturado no era la función sino las conexiones de la base —el mismo problema de la clase 186, con otra tecnología—. Y del análisis de coste salió lo que la hipótesis de la parte anticipaba: el cómputo era el 17 % de la factura, y las dos partidas mayores eran la pasarela y los registros, ninguna de las cuales se había estimado.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/207-sam-lambda-api-gateway-y-despliegue-serverless/
lab.py
```

El laboratorio selecciona el motor de práctica serverless y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa aws-serverless-api en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una función con límites, reintentos e idempotencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye aws-serverless-api para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

■■ Errores frecuentes

Síntoma	Causa probable	Corrección
La base de datos cae en cuanto hay un pico de tráfico	Las funciones escalan sin límite y cada ejecución abre una conexión	Calcula y aplica concurrencia reservada según las conexiones disponibles, y usa un intermediario de conexiones.
Una función descontrolada afecta a servicios que no tienen nada que ver	La cuota de concurrencia es de la cuenta y se consume entera	Reserva concurrencia a las funciones críticas; reservar es también proteger su capacidad.
Se factura tiempo de funciones que ya nadie está esperando	El plazo de la función es mayor que el de la pasarela, o se puso el máximo por si acaso	Ordena los plazos cliente > pasarela > función y fija el de la función en dos o tres veces el p99.
Se crean registros duplicados tras un pico o un fallo	La función no es idempotente y algo la reintenta	Clave de idempotencia por operación y escritura condicionada antes del efecto.
Un despliegue defectuoso llega a todo el tráfico	Se despliega sin alias ni desplazamiento progresivo	Publica versiones, desplaza tráfico por porcentaje y asocia alarmas que reviertan solas.
La factura triplica la estimación	Solo se estimó el cómputo; la pasarela y los registros suelen pesar más	Estima peticiones de pasarela, transferencia e ingesta de registros; ajusta nivel de registro y caducidad por entorno.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Por qué más memoria puede salir más barato en una función?
2. ¿Cómo se calcula la concurrencia reservada de una función que usa una base relacional?
3. ¿Qué ocurre si el plazo de la función supera el de la pasarela?
4. ¿Cuándo compensa pagar concurrencia aprovisionada?
5. ¿Qué partidas de coste se olvidan al estimar una aplicación sin servidores?

Referencias

- AWS (2025). Serverless Application Model developer guide — definición declarativa de la aplicación serverless. <https://docs.aws.amazon.com/serverless-application-model/latest/developerguide/what-is-sam.html>
- AWS (2025). Lambda function scaling and reserved concurrency — control del escalado y reserva de concurrencia. <https://docs.aws.amazon.com/lambda/latest/dg/lambda-concurrency.html>
- AWS (2025). Choosing between HTTP APIs and REST APIs — criterio para elegir entre ambos tipos de API. <https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-vs-rest.html>
- AWS (2025). Operating Lambda: performance optimization and cold starts — origen del arranque en frío y cómo se mitiga. <https://aws.amazon.com/blogs/compute/operating-lambda-performance-optimization-part-1/>
- AWS (2025). Amazon RDS Proxy — agrupación de conexiones para funciones. <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/rds-proxy.html>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 17 en PDF · Recorrido de AWS en PDF · Manual integral

Anterior	Índice	Siguiente
← 206 · OIDC de GitHub y GitLab hacia AWS sin secretos	Parte 17 · Programa	208 · DynamoDB por patrones de acceso y single-table design →

Evaluación — 207 SAM, Lambda, API Gateway y despliegue serverless

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega aws-serverless-api con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

208 — DynamoDB por patrones de acceso y single-table design

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: data · Estado: EXECUTABLECORE

Propósito

Diseñar en DynamoDB, donde el modelo se deduce de los patrones de acceso y no al revés, y donde equivocarse en la clave es la decisión más cara de cambiar de todo un sistema. La clase da el método —escribir los patrones antes de tocar nada—, explica el diseño de tabla única con sus ventajas y su coste real, cubre los índices, la capacidad y las particiones calientes, y aborda con honestidad cuándo DynamoDB no es la elección correcta.

Resultados de aprendizaje

Al finalizar podrás:

1. Escribir los patrones de acceso antes de diseñar el modelo.
2. Elegir clave de partición y de ordenación que repartan y sirvan las consultas.
3. Aplicar el diseño de tabla única donde aporta, y no donde no.
4. Dimensionar capacidad y detectar particiones calientes.
5. Decidir cuándo usar otra base de datos.

Conceptos centrales

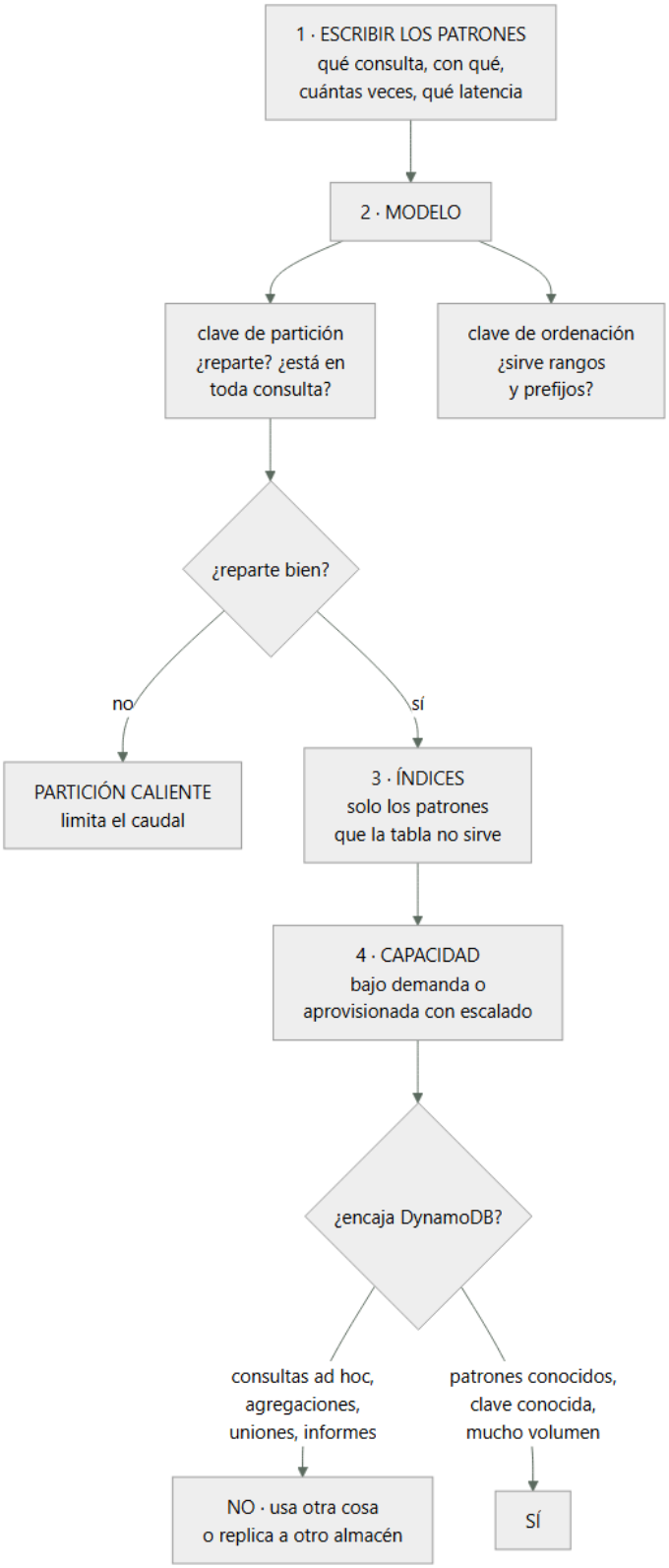
Concepto	Comprensión verificable
patrón de acceso	Consulta o escritura concreta que el sistema debe hacer, con su frecuencia y su latencia exigida.
clave de partición	Determina en qué partición vive el elemento. Decide el reparto de carga y no se puede cambiar.
clave de ordenación	Ordena los elementos dentro de una partición y permite consultas por rango y por prefijo.
tabla única	Guardar varias entidades en una tabla con claves genéricas, para servir consultas relacionadas en una sola operación.
índice secundario global	Vista con otra clave. Tiene su propia capacidad y es eventualmente consistente.
partición caliente	Clave que concentra el tráfico. Limita el caudal aunque la tabla tenga capacidad de sobra.

Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Primero los patrones, siempre

En una base relacional se modela el dominio y las consultas salen después. En DynamoDB eso no funciona: el modelo se deduce de las consultas.

```
EL DOCUMENTO QUE HAY QUE ESCRIBIR ANTES

  nº  patrón                        clave      frecuencia
```

```
   1  obtener pedido por id        id         alta
   2  listar pedidos de un cliente,
      del más reciente al más      cliente    alta
      antiguo, paginado
   3  listar pedidos de un cliente
      en un rango de fechas        cliente+f  media
   4  obtener las líneas de un
      pedido                       pedido     alta
   5  listar pedidos por estado
      para el panel de operaciones estado     baja
   6  obtener el cliente de un
      pedido                       cliente    alta

  y por cada uno
    ¿con qué dato se pide? ← esto es la clave
    ¿cuántas veces por segundo?
    ¿qué latencia se exige?
    ¿se necesita el dato más reciente o vale eventual?
```

Y la razón de que sea obligatorio:

```
DynamoDB solo sabe hacer dos cosas rápido
  obtener un elemento por su clave completa
  obtener un rango de elementos con la misma clave de
    partición, ordenados por la de ordenación

todo lo demás es un recorrido completo de la tabla
  → caro, lento, y que empeora al crecer
```

Y el patrón que revela que el modelo está mal:

```
si para servir una pantalla hacen falta 5 consultas y luego
unir en el código, el modelo no está bien
→ o el modelo cambia, o esa consulta no debería ir aquí
```

Y una advertencia sobre el proceso:

```
los patrones nuevos aparecen. El modelo debe poder
absorberlos con un índice, no con una migración
→ por eso conviene dejar atributos genéricos libres para
  índices futuros                                clase 181
```

2. Claves, y la partición caliente

La clave de partición es la decisión más cara de todo el diseño, porque no se puede cambiar: cambiarla es crear otra tabla y migrar.

```
UNA BUENA CLAVE DE PARTICIÓN
  1  está presente en casi todas las consultas
  2  tiene MUCHOS valores distintos
  3  el tráfico se reparte entre esos valores
  4  y ningún valor concentra demasiados elementos

UNA MALA
  estado del pedido      ← 5 valores; todo cae en «entregado»
  fecha del día          ← todo el tráfico de hoy en una
  tipo de entidad        ← peor todavía
  país                   ← si el 80 % es de un país
```

La partición caliente, que es el fallo característico:

```
CADA PARTICIÓN TIENE UN LÍMITE PROPIO
  del orden de 3.000 lecturas y 1.000 escrituras por segundo

→ una tabla con capacidad para 40.000 escrituras/s puede
  estar rechazando peticiones si todas van a la misma clave
→ el síntoma es un error de caudal excedido con la tabla
  «al 8 % de uso»
```

Y las soluciones, por orden de preferencia:

```
1  CAMBIAR LA CLAVE por una que reparta
   → lo correcto, si aún se está a tiempo

2  AÑADIR UN SUFIJO REPARTIDOR
   cliente#7 → cliente#7#3, con 10 sufijos
```

```
   → reparte la escritura
   → y obliga a hacer 10 consultas al leer y unir
   → solo si el patrón de lectura lo tolera

3  CACHÉ DELANTE para lecturas calientes
   → resuelve lectura, no escritura

4  AGREGAR ANTES DE ESCRIBIR
   si son 5.000 escrituras/s de contadores, agregar en
   ventanas y escribir una vez
```

La clave de ordenación, que es donde está la potencia:

```
con la misma clave de partición se puede pedir
  todo lo que empieza por un prefijo
  todo lo que está entre dos valores
  en orden ascendente o descendente
  con límite y paginación

y por eso las claves de ordenación se construyen para que
el orden lexicográfico signifique algo
  FECHA en formato ISO ordena cronológicamente
  jerarquías con separador: PEDIDO#2025-03#4471
```

Y la técnica que sirve la mayoría de las consultas:

```
clave de ordenación con prefijo por tipo
  CLIENTE#123 · PERFIL
  CLIENTE#123 · PEDIDO#2025-03-14#4471
  CLIENTE#123 · PEDIDO#2025-03-02#4390
  CLIENTE#123 · DIRECCION#casa

→ una consulta con prefijo «PEDIDO#» da los pedidos
  ordenados por fecha
→ una consulta sin prefijo da el cliente entero de una vez
```

3. Tabla única: qué aporta y qué cuesta

El diseño de tabla única consiste en guardar varias entidades juntas para poder traerlas en una sola operación.

```
QUÉ RESUELVE
  la pantalla que necesita cliente, sus pedidos y sus
  direcciones
  con tablas separadas: 3 consultas y unir en el código
  con tabla única: 1 consulta
  → y en un sistema con muchas llamadas, eso importa
                                                clase 152

QUÉ CUESTA, y se cuenta poco
  el modelo deja de ser legible: PK y SK genéricos
  cada patrón nuevo obliga a pensar de nuevo
  las herramientas de exploración no ayudan
  la incorporación de gente nueva es más lenta
  y un error de modelo afecta a todas las entidades
```

Y el criterio honesto:

```
USA TABLA ÚNICA CUANDO
  hay patrones que necesitan varias entidades juntas
  el volumen y la latencia lo justifican
  y el equipo entiende el modelo

NO LA USES CUANDO
  las entidades no se consultan juntas nunca
    → tablas separadas, más simples y sin coste
  el volumen es modesto
    → la ganancia no compensa la complejidad
  el equipo no ha trabajado así antes y no hay tiempo de
    aprender
    → un modelo de tabla única mal hecho es peor que tablas
      separadas
```

Y una regla práctica que reduce el riesgo:

```
agrupa en una tabla lo que se consulta junto y comparte
ciclo de vida
```

```
separado, lo demás
→ es el mismo criterio de cohesión de la clase 183
```

Los índices secundarios, con lo que hay que saber:

```
ÍNDICE GLOBAL
  otra clave de partición y de ordenación
  capacidad PROPIA, que se paga aparte
  eventualmente consistente, siempre
  → si su capacidad se agota, las ESCRITURAS de la tabla
    principal empiezan a fallar     ← el efecto que sorprende

ÍNDICE LOCAL
  misma partición, otra ordenación
  se define al crear la tabla y no se puede añadir después
  limita el tamaño de la partición a 10 GB
  → rara vez merece la pena

PROYECCIÓN
  qué atributos se copian al índice
  solo claves · algunos · todos
  → proyectar todo duplica el almacenamiento y el coste de
    escritura
  → proyectar solo lo necesario y buscar el resto en la
    tabla, si es poco frecuente
```

Y el error de índices más común:

```
crear un índice por cada consulta que se ocurre
→ cada escritura se replica a todos los índices
→ 5 índices = 6 escrituras por cada escritura lógica
→ y el coste de escritura se multiplica por seis
```

4. Capacidad, coste y cuándo no usarla

La capacidad, con la decisión que hay que tomar:

```
BAJO DEMANDA
  se paga por petición; escala sola
  + sin gestión, ideal para tráfico impredecible o bajo
  − más caro por unidad si el tráfico es alto y estable
  − tiene su propio escalado: un pico de 10× desde reposo
    puede encontrar límites al principio

APROVISIONADA CON ESCALADO AUTOMÁTICO
  se reserva capacidad y se ajusta por uso
  + más barata con tráfico alto y previsible
  − el escalado tarda minutos: no cubre un pico brusco
  − requiere vigilar el estrangulamiento

REGLA PRÁCTICA
  empieza bajo demanda, mide un mes, y cambia si el patrón
  es estable y el ahorro es real
```

El coste, con lo que suele olvidarse:

```
lectura y escritura                    lo que se cuenta
almacenamiento
ÍNDICES: su propia escritura y almacenamiento   ← olvidado
copias continuas y su almacenamiento
flujos de cambios y su consumo                  ← olvidado
tablas globales: escritura replicada en cada región
transferencia entre regiones

y dos decisiones que cambian mucho el coste
  LECTURA EVENTUAL en vez de fuerte: la mitad de precio
  → y en la mayoría de las consultas basta      clase 187
  ELEMENTOS PEQUEÑOS: la unidad de lectura cubre 4 KB
  → un elemento de 5 KB cuesta el doble que uno de 4
```

Cuándo NO usar DynamoDB, que es la parte honesta:

```
NO LA USES SI
  las consultas no se conocen de antemano
    informes, exploración, analítica ad hoc
  hacen falta agregaciones (sumas, medias, agrupaciones)
```

```
    → hay que mantenerlas a mano al escribir
  hacen falta uniones entre entidades no relacionadas
  hace falta búsqueda por texto
  el volumen es pequeño y el equipo conoce SQL
    → una base relacional gestionada será más rápida de
      construir y más barata de operar

Y EL PATRÓN QUE RESUELVE LA MAYORÍA DE ESTOS CASOS
  DynamoDB para el camino operativo, de alto volumen y
  patrones conocidos
  + flujo de cambios que replica a un almacén analítico
    para informes y exploración                  clase 150
  → cada uno hace lo que sabe hacer
```

Y las transacciones, que existen y tienen límites:

```
se pueden escribir varios elementos de forma atómica
  con límite de elementos por transacción
  y coste doble
→ útiles para invariantes concretos; no para sustituir a
  una base transaccional                         clase 187
```

Y la lista de comprobación de la clase:

- los patrones de acceso están escritos, con frecuencia y latencia
- la clave de partición está en casi todas las consultas
- la clave de partición reparte: muchos valores y tráfico distribuido
- la clave de ordenación permite prefijos y rangos útiles
- el uso de tabla única está justificado, no copiado
- los índices son los mínimos, con proyección ajustada
- se sabe que un índice saturado hace fallar las escrituras
- las lecturas usan consistencia eventual salvo donde no se pueda
- hay alerta de estrangulamiento y de clave caliente
- el coste incluye índices, flujos y copias
- lo que no encaja se replica a otro almacén, no se fuerza

Y el cierre que enlaza con la clase siguiente: con una API y un almacén que escalan, falta decidir quién puede llamar y qué se hace con lo que llega de fuera. Identidad de usuario, autorizadores y defensa en profundidad es la materia de la clase 209.

Ejemplo trabajado

CloudShop diseña el almacén de pedidos. Lo que sigue es el documento de patrones, el modelo que salió, el error de clave que costó una migración de seis semanas, y la decisión de qué NO poner ahí.

Los patrones, escritos antes de nada:

nº	patrón	se pide con	veces/s	latencia
1	obtener pedido por id	id	400	< 20 ms
2	pedidos de un cliente, recientes primero, paginado	cliente	180	< 50 ms
3	pedidos de un cliente en rango de fechas	cliente+fecha	20	< 50 ms
4	líneas de un pedido	pedido	400	< 20 ms
5	pedidos en estado «pendiente de envío», para el almacén	estado	2	< 1 s
6	datos del cliente de un pedido	cliente	400	< 20 ms
7	pedidos de un día para el cierre contable	fecha	1 vez/día	< 5 min
8	buscar pedidos por texto libre (atención al cliente)	texto	15	< 1 s
9	ventas por categoría y semana	—	informes	—

Y la primera decisión, tomada antes de modelar:

```
los patrones 8 y 9 NO van a DynamoDB
  8  búsqueda por texto → índice de búsqueda, alimentado
     por el flujo de cambios
  9  agregaciones y exploración → almacén analítico, por el
     mismo flujo                                   clase 150
```

```
→ forzarlos habría destrozado el modelo
```

El error de clave, y lo que costó.

```
primer diseño, hecho antes de escribir los patrones
  clave de partición    estado del pedido
  clave de ordenación   fecha#id
  razón                 «el panel del almacén consulta por
                        estado, y es la consulta que nos
                        pidieron primero»

lo que pasó en producción
  los estados son 6
  el 71 % de los pedidos acaba en «entregado»
  y todos los pedidos nuevos entran en «pendiente de pago»
  → dos particiones concentraban el 94 % de la escritura

  síntoma
    errores de caudal excedido a 900 escrituras/s
    con la tabla configurada para 20.000
    y el panel de uso mostrando un 5 %

  y el patrón 1, el más frecuente, no se podía servir:
  para obtener un pedido por id había que conocer su estado
  → se hacía un recorrido completo de la tabla
  → 340 ms de latencia y coste creciente

coste de la corrección
  tabla nueva, doble escritura durante la migración,
  comparación, y corte
  6 semanas de trabajo de 2 personas

qué lo habría evitado
  escribir los nueve patrones antes de elegir la clave
  → 3 horas de trabajo                          ley 14
```

El modelo definitivo, con tabla única.

```
PK                  SK                          entidad
CLIENTE#123         PERFIL                      cliente
CLIENTE#123         DIRECCION#casa              dirección
CLIENTE#123         PEDIDO#2025-03-14#4471      resumen de
                                                pedido
CLIENTE#123         PEDIDO#2025-03-02#4390      resumen
PEDIDO#4471         CABECERA                    pedido
PEDIDO#4471         LINEA#001                   línea
PEDIDO#4471         LINEA#002                   línea

cómo se sirve cada patrón
  1  obtener PEDIDO#4471 / CABECERA          → 1 operación
  2  consultar CLIENTE#123 con prefijo
     «PEDIDO#», descendente, límite 20       → 1 operación
  3  consultar CLIENTE#123 entre
     «PEDIDO#2025-01» y «PEDIDO#2025-03»     → 1 operación
  4  consultar PEDIDO#4471 con prefijo
     «LINEA#»                                → 1 operación
  6  el resumen de pedido bajo CLIENTE ya trae lo necesario
     → 0 operaciones extra

y la pantalla de detalle de cliente
  consultar CLIENTE#123 sin prefijo
  → perfil, direcciones y últimos pedidos en UNA operación
  → antes eran 3 consultas y unión en el código
```

Y la justificación de la tabla única, escrita:

```
se usa porque los patrones 2, 3 y 6 necesitan cliente y
pedidos juntos, con 400 peticiones/s
no se usa para las líneas de producto del catálogo, que
nunca se consultan con lo anterior y viven en su propia
tabla                                          clase 183
```

Los índices, los mínimos.

```
GSI-1   para el patrón 5 (panel del almacén)
```

```
  PK    ESTADO#pendiente-envio
  SK    fecha#id
  proyección   solo id, cliente, fecha, importe
  → 2 consultas/s; no justifica proyectar todo

  y aquí SÍ hay concentración: todos los pendientes en una
  clave
  → aceptable porque son 2 consultas/s y pocos elementos
  → decisión registrada, con la señal que la reabriría:
    «si el panel pasa de 50 consultas/s o si los pendientes
     superan 100.000 elementos»                clase 190

GSI-2   para el patrón 7 (cierre diario)
  PK    FECHA#2025-03-14
  SK    id
  proyección   solo claves
  → se ejecuta una vez al día, fuera de hora punta

NO se creó ningún índice más
  se propusieron 4 más para consultas del panel interno
  → se resolvieron en el almacén analítico          clase 150
  → habrían multiplicado por 3 el coste de escritura
```

La capacidad y el coste.

```
primer mes, bajo demanda
  lecturas                    980 M     490 €
  escrituras                   41 M     205 €
  almacenamiento              210 GB     53 €
  índices (escritura)          82 M     410 €   ←
  flujo de cambios             41 M      18 €
  copias continuas                        68 €
  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  total                                1.244 €

los índices costaban más que las escrituras de la tabla
  → porque GSI-1 recibía cada escritura de pedido y
    proyectaba 4 atributos
  → se ajustó para que solo se indexen los pedidos en
    estados activos, borrando la clave del índice al
    entregar
  → escrituras de índice: 82 M → 19 M;  410 € → 95 €

segundo ajuste
  el 92 % de las lecturas no necesitaban consistencia fuerte
  → cambiadas a eventual:  490 € → 268 €

tercer ajuste, tras 2 meses de datos
  el tráfico resultó estable y alto
  → capacidad aprovisionada con escalado automático
  → 268 € → 171 € en lecturas

total                          1.244 € → 620 €
```

La comprobación de reparto, hecha antes de salir:

```
se simuló un día de campaña con datos reales
  claves de partición distintas                 41.200
  máxima proporción de escritura en una clave      0,8 %
  máxima proporción de lectura en una clave        2,1 %
    → un cliente empresarial con muchos pedidos
    → aceptable; muy por debajo del límite de partición

y la alerta que quedó montada
  «errores de caudal excedido > 0 durante 1 minuto»
  → distingue el estrangulamiento del error de aplicación
```

El resultado:

```
latencia p99 del patrón 1                 340 ms       11 ms
operaciones para la pantalla de cliente        3           1
errores de caudal con la tabla al 5 %         sí          no
coste mensual                              1.244 €     620 €
índices                                        6           2
patrones servidos fuera de DynamoDB            0           2
```

La lección que esta clase deja: la clave de partición se eligió por la primera consulta que pidió negocio y no por los nueve patrones, y esa decisión de una tarde costó seis semanas de migración. El segundo hallazgo fue de coste: los índices costaban más que la tabla, porque recibían cada escritura y proyectaban atributos que casi nadie leía. Y las dos consultas que peor encajaban —búsqueda por texto y agregaciones— se resolvieron sacándolas de DynamoDB, no forzando el modelo.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/208-dynamodb-por-patrones-de-acceso-y-single-table-design/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa aws-dynamodb-model en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye aws-dynamodb-model para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

Errores frecuentes

Síntoma	Causa probable	Corrección
Errores de caudal excedido con la tabla al 5 % de uso	Partición caliente: el tráfico se concentra en pocas claves	Elige una clave con muchos valores y tráfico repartido; si ya es tarde, reparte con sufijo, cachea o agrega antes de escribir.
Servir una pantalla exige cinco consultas y unir en el código	El modelo se diseñó desde el dominio y no desde los patrones	Escribe los patrones con su frecuencia y latencia antes de modelar, y agrupa en la clave de ordenación lo que se consulta junto.
Obtener un elemento por su identificador requiere recorrer la tabla	La clave de partición no está presente en el patrón más frecuente	La clave de partición debe aparecer en casi todas las consultas; si no, el modelo está al revés.
Las escrituras de la tabla empiezan a fallar sin motivo aparente	Un índice secundario global agotó su capacidad	Vigila la capacidad de cada índice por separado y reduce índices y proyecciones al mínimo necesario.
El coste de escritura es varias veces el esperado	Cada escritura se replica a todos los índices, con proyección completa	Crea solo los índices que sirvan patrones reales, proyecta lo mínimo y saca del índice lo que deja de ser consultable.
Los informes y las búsquedas son lentos y caros	Se fuerzan consultas ad hoc y agregaciones sobre DynamoDB	Replica por el flujo de cambios a un índice de búsqueda y a un almacén analítico, y deja aquí solo el camino operativo.

Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

Preguntas de comprobación

1. ¿Por qué en DynamoDB el modelo se deduce de los patrones y no al revés?
2. ¿Qué cuatro propiedades tiene una buena clave de partición?
3. ¿Por qué puede haber estrangulamiento con la tabla al 5 % de uso?
4. ¿Qué le ocurre a las escrituras de la tabla si se satura un índice global?
5. ¿En qué casos conviene sacar un patrón fuera de DynamoDB?

Referencias

- AWS (2025). DynamoDB developer guide: best practices for designing partition keys — reparto de claves de partición para evitar puntos calientes. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/bp-partition-key-design.html>
- AWS (2025). Single-table design considerations — cuándo compensa una sola tabla y cuándo no. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/bp-general-nosql-design.html>
- DeBrie, A. (2020). The DynamoDB Book — modelado guiado por los patrones de acceso. <https://www.dynamodbbook.com/>
- AWS (2025). Secondary indexes and their capacity implications — coste en capacidad de los índices secundarios. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/SecondaryIndexes.html>
- AWS (2025). DynamoDB Streams and change data capture — captura de cambios para propagar estado a otros servicios. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.html>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Anterior	Índice	Siguiente
← 207 · SAM, Lambda, API Gateway y despliegue serverless	Parte 17 · Programa	209 · Cognito, JWT authorizers, WAF y defensa en profundidad →

Evaluación — 208 DynamoDB por patrones de acceso y single-table design

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega aws-dynamodb-model con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

209 — Cognito, JWT authorizers, WAF y defensa en profundidad

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: security · Estado: EXECUTABLECORE

Propósito

Resolver la identidad de los usuarios finales y la defensa del perímetro público sin construir nada propio que haya que mantener. La clase cubre Cognito y los testigos que emite, cómo se validan bien —que es donde están los fallos de seguridad reales—, cuándo conviene un autorizador propio y cuándo no, y el filtrado de aplicación con su parte incómoda: las reglas gestionadas bloquean tráfico legítimo, y desplegarlas en modo bloqueo sin medir antes rompe el negocio.

Resultados de aprendizaje

Al finalizar podrás:

1. Distinguir los tres testigos de Cognito y usar cada uno donde corresponde.
2. Validar un testigo correctamente, comprobando todo lo que hay que comprobar.
3. Elegir entre autorizador integrado y propio con criterio.
4. Desplegar filtrado de aplicación sin bloquear tráfico legítimo.
5. Componer las capas de defensa sabiendo qué cubre cada una.

Conceptos centrales

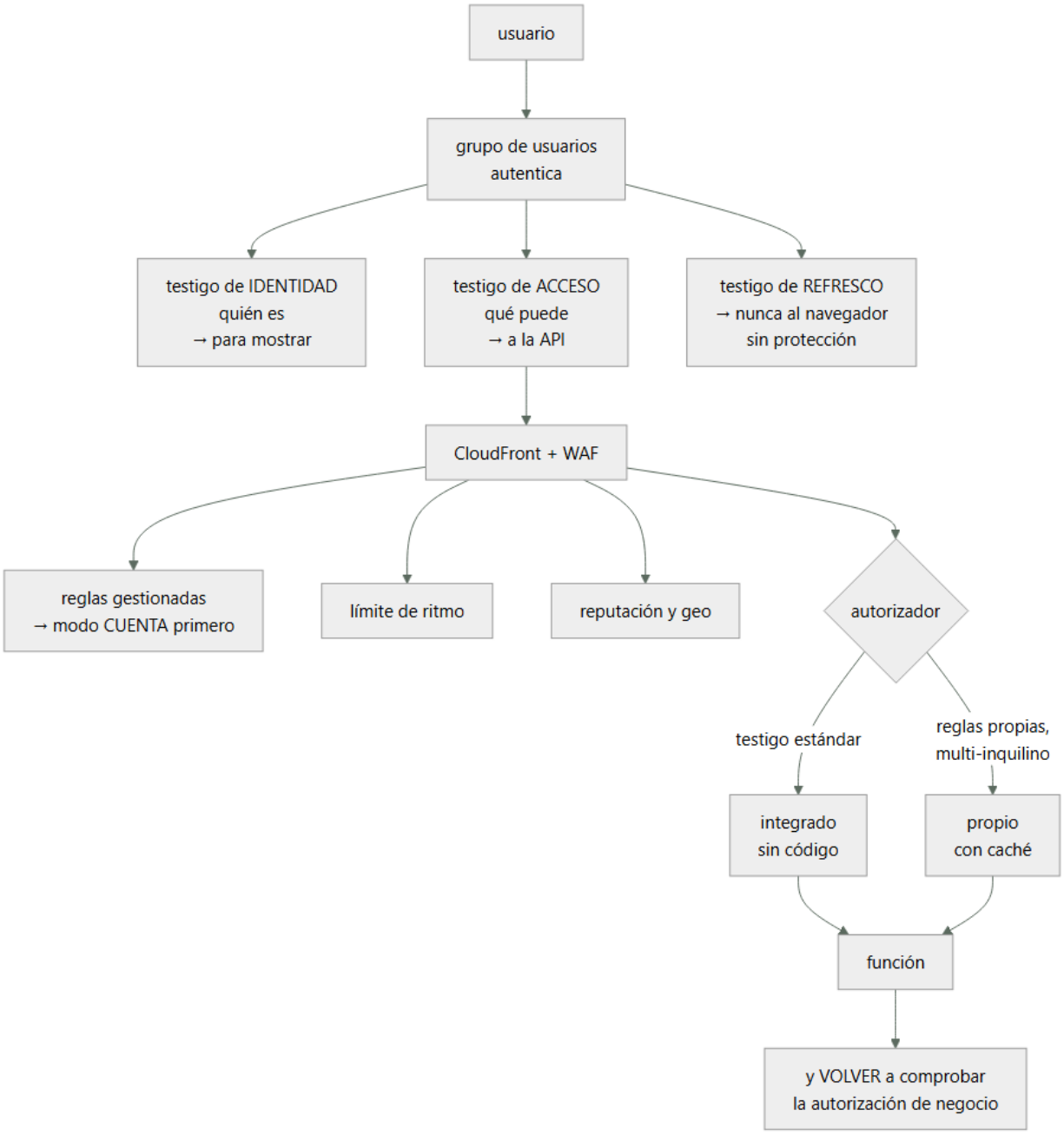
Concepto	Comprensión verificable
grupo de usuarios	Directorio de usuarios finales que autentica y emite testigos. Es el proveedor de identidad.
grupo de identidades	Componente que canjea un testigo por credenciales temporales de AWS. Cosa distinta del anterior.
testigo de identidad	Contiene quién es el usuario. Sirve para mostrar datos, no para autorizar llamadas.
testigo de acceso	Contiene qué puede hacer, con sus ámbitos. Es el que se envía a la API.
autorizador	Componente de la pasarela que valida el testigo antes de invocar la función.
modo cuenta	Despliegue de una regla de filtrado que registra lo que bloquearía sin bloquearlo.

Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Los tres testigos, y cuál va a dónde

Cognito emite tres testigos y confundirlos es la causa de la mitad de los problemas.

```
TESTIGO DE IDENTIDAD
  dice QUIÉN es el usuario: nombre, correo, atributos
  destinado a la aplicación cliente, para mostrar datos
  ✗ NO se usa para autorizar llamadas a la API
  → su audiencia es el cliente, no la API

TESTIGO DE ACCESO
  dice QUÉ puede hacer: ámbitos, grupos, cliente
  es el que se envía en la cabecera de autorización
  ✓ este es el que valida la API

TESTIGO DE REFRESCO
  sirve para obtener testigos nuevos sin volver a
  autenticarse
  vida larga (días o meses)
```

```
  ✗ NUNCA en almacenamiento accesible por scripts del
    navegador
  → cookie con marca de solo servidor, o almacenamiento
    seguro del dispositivo
```

Y el error de diseño más frecuente:

```
enviar el testigo de identidad a la API porque «trae el
correo del usuario»
→ la API acaba autorizando con un testigo que no era para
  ella
→ y si la aplicación cliente cambia, la API se rompe

si la API necesita el correo, se añade como ámbito o se
busca por el identificador del sujeto
```

El grupo de identidades, que es otra cosa y se confunde con el anterior:

```
GRUPO DE USUARIOS      autentica personas y emite testigos
GRUPO DE IDENTIDADES   canjea un testigo por credenciales
                       temporales de AWS

→ el segundo solo hace falta si el cliente debe llamar
  DIRECTAMENTE a servicios de AWS (subir a un bucket, por
  ejemplo)
→ y entonces el rol que recibe debe estar restringido al
  prefijo del propio usuario, no al bucket entero
                                                clase 134
```

Y una advertencia sobre el acceso directo desde el cliente:

```
dar credenciales de AWS al navegador amplía mucho el alcance
→ si es evitable, mejor una URL prefirmada emitida por la
  API, con vida corta y ámbito exacto
```

2. Validar bien un testigo

Aquí están los fallos de seguridad reales, y son siempre los mismos.

```
LO QUE HAY QUE COMPROBAR, SIN SALTARSE NADA

1  LA FIRMA, contra las claves públicas del emisor
   ✗ el fallo grave: decodificar sin verificar
     → cualquiera puede fabricar un testigo

2  EL ALGORITMO ESPERADO
   ✗ aceptar el algoritmo que diga el testigo
     → «ninguno» o cambio a algoritmo simétrico con la
       clave pública como secreto

3  EL EMISOR
   ✗ no comprobarlo → vale un testigo de otro directorio

4  LA AUDIENCIA (o el cliente)
   ✗ no comprobarla → vale un testigo emitido para otra
     aplicación del mismo directorio

5  EL USO DEL TESTIGO
   comprobar que es de acceso y no de identidad

6  LA CADUCIDAD, con margen de reloj pequeño

7  LOS ÁMBITOS o grupos que exige esta operación
```

Y el detalle operativo que rompe en producción:

```
LAS CLAVES PÚBLICAS SE CACHEAN
  descargarlas en cada petición añade latencia y depende
  del emisor
  cachearlas para siempre rompe cuando rotan
  → cachear con caducidad y refrescar si aparece un
    identificador de clave desconocido
```

El autorizador de la pasarela, con la elección:

```
INTEGRADO (validación de testigo estándar)
  la pasarela valida firma, emisor, audiencia y ámbitos
```

```
  sin código propio
  + nada que mantener, sin arranque en frío, gratis
  − no puede decidir con lógica de negocio

PROPIO (función autorizadora)
  código que devuelve permitido o denegado, y contexto
  + reglas propias: multi-inquilino, listas, permisos finos
  − una función más en el camino crítico
  − CACHÉ obligatoria, o se ejecuta en cada petición

  y el detalle de la caché
    la clave de caché debe incluir lo que hace variar la
    decisión
    → si solo cachea por el testigo y la decisión depende
      del recurso, autoriza de más   ← fallo de seguridad
```

Y la regla que este programa mantiene:

```
el autorizador comprueba QUE EL TESTIGO ES VÁLIDO
la función vuelve a comprobar QUE ESTE USUARIO PUEDE HACER
ESTO CON ESTE RECURSO

→ un pedido no se lee porque el testigo sea válido, sino
  porque el pedido es de ese cliente
→ y esa comprobación no la puede hacer la pasarela
```

Y el fallo que resulta de olvidarlo:

```
referencia directa insegura
  GET /pedidos/4471 con un testigo válido de otro cliente
  → si la función no comprueba la propiedad, devuelve el
    pedido ajeno
→ es de los hallazgos más frecuentes en revisiones
                                                clase 189
```

3. Filtrado de aplicación sin romper el negocio

El filtrado de aplicación se despliega mal casi siempre, y la forma de hacerlo bien es lenta a propósito.

```
LO QUE APORTA
  reglas gestionadas contra ataques conocidos
  límite de ritmo por dirección o por identificador
  reputación de origen y bloqueo por geografía
  reglas propias por ruta y por cabecera

LO QUE HAY QUE SABER ANTES
  las reglas gestionadas producen FALSOS POSITIVOS
  → bloquean peticiones legítimas con contenido que parece
    un ataque
  → un campo de texto libre con comillas, una descripción
    con SQL de ejemplo, un cuerpo grande
```

Y por eso el despliegue tiene un orden obligatorio:

```
1  DESPLEGAR EN MODO CUENTA
   registra lo que bloquearía, sin bloquear

2  MEDIR 2-4 SEMANAS
   ¿cuántas peticiones bloquearía? ¿cuáles son legítimas?

3  AJUSTAR
   excluir reglas concretas en rutas concretas
   excluir campos del cuerpo que dan falsos positivos
   → NO desactivar el grupo entero

4  ACTIVAR EL BLOQUEO por grupos de reglas, no todo a la vez

5  VIGILAR los bloqueos y revisarlos
   → cada bloqueo legítimo es un ajuste pendiente

→ activar en bloqueo el primer día es como cerrar la salida
  sin registrar antes                          clase 200
```

Y el límite de ritmo, que es la regla que más aporta y la más fácil:

```
por dirección de origen: protege de lo básico
```

```
por identificador de usuario o de sesión: protege mejor
por ruta: el registro y el acceso llevan límites más
  estrictos que el catálogo

y hay que decidir qué se devuelve al bloquear
  429 con cabecera de reintento es honesto y ayuda al
  cliente legítimo
```

Dónde poner el filtrado, que tiene consecuencias:

```
en CloudFront    filtra en el borde, antes de la pasarela
                 → más barato: lo bloqueado no llega
                 → y protege también el contenido estático
en la pasarela   filtra ahí; si hay CloudFront delante, se
                 duplica
en el balanceador para arquitecturas con contenedores
                                              clase 212

→ con CloudFront delante, ponerlo ahí y restringir la
  pasarela para que SOLO acepte tráfico de esa distribución
  → si no, se puede saltar el filtro llamando a la pasarela
    directamente                              clase 205
```

4. Capas, y qué cubre cada una

La defensa en profundidad se cita mucho y se comprueba poco. Lo útil es saber qué cubre cada capa y qué no.

```
CAPA                 CUBRE                    NO CUBRE
filtrado en el borde ataques conocidos,       lógica de
                     volumen, geografía       negocio

autorizador          testigo inválido,        si ESTE usuario
                     caducado, de otro        puede ver ESTE
                     emisor                   recurso

función              propiedad del recurso,   testigo robado
                     reglas de negocio

permisos de la       alcance del código       errores de
función              si es comprometido       lógica

datos                cifrado, perímetro       acceso legítimo
                                              mal usado
```

Y las comprobaciones que corresponden a esta clase:

```
■ llamar a la API con un testigo de identidad en vez de
  acceso                          → debe fallar
■ llamar con un testigo de otra aplicación del mismo
  directorio                      → debe fallar
■ llamar con un testigo caducado  → debe fallar
■ llamar con la firma alterada    → debe fallar
■ llamar con algoritmo «ninguno»  → debe fallar
■ pedir el pedido de otro cliente con testigo válido
                                  → debe fallar
■ llamar a la pasarela saltándose la distribución
                                  → debe fallar
■ superar el límite de ritmo      → 429, no 500
■ enviar una carga de ataque conocida → bloqueada
■ enviar texto legítimo con comillas → NO bloqueada
```

Y la última es tan importante como las demás: una defensa que bloquea a los clientes es un incidente.

Lo que hay que vigilar:

```
autorizaciones denegadas, por motivo
  → un pico de «firma inválida» es alguien probando
  → un pico de «caducado» suele ser un fallo de refresco
    en el cliente
bloqueos del filtrado, por regla
límite de ritmo alcanzado, por ruta
intentos de autenticación fallidos por usuario
y el uso de testigos de refresco
```

Y dos decisiones operativas que conviene tomar pronto:

```
DURACIÓN DE LOS TESTIGOS
  acceso   corta (15-60 min): limita el daño de uno robado
  refresco larga, pero REVOCABLE
  → y hay que tener probado cómo se revoca a un usuario
    concreto                                        ley 22

QUÉ PASA CUANDO EL DIRECTORIO NO RESPONDE
  los testigos ya emitidos siguen valiendo hasta caducar
  → la validación no llama al directorio en cada petición
  → pero no se pueden emitir nuevos: los usuarios ya dentro
    siguen; los que entran, no
  → conviene saberlo antes de calcular el techo de
    disponibilidad                                clase 185
```

Y la lista de comprobación de la clase:

- la API valida el testigo de ACCESO, no el de identidad
- se comprueban firma, algoritmo, emisor, audiencia, uso y caducidad
- las claves públicas se cachean con caducidad y se refrescan ante clave desconocida
- el testigo de refresco no es accesible desde scripts
- si hay autorizador propio, su caché incluye lo que hace variar la decisión
- la función comprueba la propiedad del recurso
- el filtrado se desplegó en modo cuenta antes de bloquear
- hay límite de ritmo por ruta y por usuario
- la pasarela solo acepta tráfico de la distribución
- la revocación de un usuario está probada
- las diez pruebas negativas se han ejecutado

Y el cierre que enlaza con la clase siguiente: con la entrada protegida, el trabajo que no puede hacerse en la petición se envía a procesar más tarde, y ahí aparecen los duplicados, los mensajes fallidos y el reproceso. Bus de eventos, colas, cola de fallidos e idempotencia es la materia de la clase 210.

Ejemplo trabajado

CloudShop protege su API pública. Lo que sigue son los tres fallos que encontró la revisión de seguridad, el despliegue del filtrado que empezó rompiendo el registro de usuarios, y el estado final con sus pruebas negativas.

Fallo 1 · La API validaba el testigo equivocado.

```
el cliente enviaba el testigo de IDENTIDAD
el autorizador estaba configurado para aceptarlo
razón   «trae el correo, que la API necesita»

lo que eso permitía
  el testigo de identidad no lleva ámbitos
  → la API no podía distinguir un usuario normal de uno
    con permisos de administración
  → la distinción se hacía leyendo un atributo del testigo
    que el propio usuario podía modificar desde su perfil

  la prueba negativa
    un usuario cambió su atributo «rol» a «admin» desde la
    pantalla de perfil
    → obtuvo acceso al panel de administración
    tiempo desde la idea hasta conseguirlo: 4 minutos

corrección
  la API valida el testigo de ACCESO
  los permisos vienen de GRUPOS del directorio, no de
  atributos editables por el usuario
  el atributo «rol» se eliminó de los editables
  y el correo se obtiene por el identificador del sujeto
```

Fallo 2 · La función no comprobaba la propiedad.

```
GET /pedidos/{id}
  el autorizador validaba el testigo    ✓
  la función devolvía el pedido          ✗ sin comprobar de
                                           quién era

la prueba
```

```
  un usuario con sesión válida pidió /pedidos/4471
  → recibió el pedido de otro cliente, con dirección,
    teléfono e importe

y los identificadores eran secuenciales
  → se podían recorrer todos                    clase 189

corrección
  la función comprueba que el pedido pertenece al sujeto
  del testigo
  → y como el modelo tiene el pedido bajo CLIENTE#, la
    comprobación es la propia clave de la consulta
                                                clase 208
  identificadores cambiados a opacos
  y la comprobación añadida como prueba negativa permanente
```

Fallo 3 · Se podía saltar el filtrado.

```
el filtrado estaba en CloudFront
la pasarela aceptaba peticiones de cualquier origen
→ llamando a la URL de la pasarela directamente se
  saltaba el filtrado, el límite de ritmo y los registros

se comprobó en los registros de acceso
  peticiones directas a la pasarela              8.400/día
  de ellas, de rastreadores y escáneres          7.900
  de ellas, de un cliente móvil antiguo mal
    configurado                                    500

corrección
  la pasarela exige una cabecera secreta que solo añade la
  distribución, y rechaza el resto
  el cliente móvil antiguo se corrigió en la versión
  siguiente, con un plazo de 8 semanas para la retirada
                                                clase 188
```

El despliegue del filtrado, que empezó mal.

```
primer intento, febrero
  se activaron 4 grupos de reglas gestionadas en modo
  BLOQUEO, el mismo día

  a las 3 horas
    el registro de usuarios nuevos había caído un 34 %
    nadie relacionó las dos cosas

  a los 2 días, atención al cliente reportó que había
  usuarios que no podían registrarse

  causa
    una regla gestionada bloqueaba cuerpos con ciertos
    patrones
    los apellidos con apóstrofo —O'Brien, D'Angelo— y las
    direcciones con comillas coincidían
    → 1 de cada 3 registros con esos caracteres se
      bloqueaba, devolviendo un 403 sin explicación

  pérdida estimada                      2 días × 34 %
                                        de registros

segundo intento, marzo, con el orden correcto
  1  los 4 grupos en MODO CUENTA
  2  medición de 3 semanas
       peticiones que se bloquearían            41.200
       claramente maliciosas                    38.900
       LEGÍTIMAS                                 2.300
         · apellidos y direcciones con apóstrofo
         · descripciones de producto de proveedores con
           fragmentos de HTML
         · un cliente que subía ficheros grandes por la API
  3  ajustes
       exclusión de la regla concreta en el campo de
         apellido y dirección, en la ruta de registro
       exclusión en el campo de descripción de la ruta de
```

```
        catálogo interno
      límite de tamaño ajustado en la ruta de subida
      → NO se desactivó ningún grupo entero
4  bloqueo activado grupo a grupo, con una semana entre
   cada uno
5  revisión semanal de bloqueos

resultado tras 6 meses
  peticiones bloqueadas                    36.400/mes
  bloqueos legítimos reportados                   4
  → los 4, ajustados en menos de un día
```

El límite de ritmo, por ruta:

```
ruta                       límite            motivo
/auth/login                5 / 5 min / IP    fuerza bruta
/auth/registro             3 / hora / IP     cuentas falsas
/api/pedidos (POST)        20 / min /usuario abuso
/api/catalogo              600 / min / IP    tráfico normal
resto                      2.000 / 5 min /IP suelo general

respuesta al bloquear   429 con cabecera de reintento
```

Las diez pruebas negativas, ejecutadas:

```
✓  testigo de identidad en vez de acceso        rechazado
✓  testigo de otra aplicación del directorio    rechazado
✓  testigo caducado                             rechazado
✓  firma alterada                               rechazado
✓  algoritmo «ninguno»                          rechazado
✓  pedido de otro cliente con testigo válido    rechazado
✓  llamada directa a la pasarela                rechazada
✓  superar el límite de ritmo                   429 correcto
✓  carga de ataque conocida                     bloqueada
✓  apellido con apóstrofo                       NO bloqueado
```

Y una prueba adicional que se añadió tras el incidente:

```
✓  registrar un usuario llamado O'Brien, con dirección
   «Rúa d'Ouro, 3», y una descripción con <b>negrita</b>
   → los tres pasan
   → se ejecuta en cada despliegue del filtrado
```

El resultado:

```
escalada de privilegios por atributo         posible   imposible
lectura de pedidos ajenos                    posible   imposible
peticiones que saltan el filtrado          8.400/día         0
registros bloqueados por falso positivo      34 %       0,01 %
intentos de fuerza bruta que llegan a
  la función                                 todos            0
coste del filtrado                             —       210 €/mes
coste de cómputo evitado por bloquear
  en el borde                                  —      -390 €/mes
```

La lección que esta clase deja: los tres fallos de seguridad no estaban en la configuración del filtrado ni del directorio: estaban en usar el testigo equivocado, en no comprobar de quién era el recurso y en dejar un camino que se saltaba todo. Y el incidente más caro del capítulo lo causó la propia defensa: activar las reglas gestionadas en modo bloqueo el primer día tumbó un tercio de los registros durante dos días, y a nadie se le ocurrió relacionarlo hasta que llamó un cliente apellidado O'Brien.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/209-cognito-jwt-authorizers-waf-y-defensa-en-profundidad/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.

2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa aws-identity-edge en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye aws-identity-edge para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

■■ Errores frecuentes

Síntoma	Causa probable	Corrección
Un usuario se concede permisos de administración editando su perfil	La autorización se basa en un atributo del testigo de identidad que el usuario puede modificar	Valida el testigo de acceso y deriva los permisos de grupos del directorio, no de atributos editables.
Un usuario válido lee recursos de otro	El autorizador valida el testigo pero la función no comprueba la propiedad del recurso	Comprueba en la función que el recurso pertenece al sujeto del testigo, y usa identificadores opacos.
Se puede evitar el filtrado llamando a la pasarela directamente	La pasarela acepta tráfico de cualquier origen	Exige una cabecera secreta que solo añade la distribución y rechaza el resto.
Cae el registro de usuarios tras activar el filtrado	Reglas gestionadas en modo bloqueo desde el primer día, con falsos positivos	Despliega en modo cuenta, mide semanas, excluye reglas concretas en campos concretos y activa el bloqueo por grupos.
Un autorizador propio autoriza de más	La clave de caché no incluye lo que hace variar la decisión	Incluye en la clave de caché el recurso y cuanto influya en el resultado, o desactiva la caché.
El cliente pierde la sesión cada poco o la mantiene demasiado	Duraciones de testigo mal elegidas y refresco mal guardado	Testigo de acceso corto, refresco largo pero revocable y guardado fuera del alcance de scripts, con revocación probada.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué diferencia hay entre el testigo de identidad y el de acceso, y cuál va a la API?
2. ¿Qué siete comprobaciones exige validar bien un testigo?
3. ¿Qué comprueba el autorizador y qué debe volver a comprobar la función?
4. ¿Por qué no se despliega el filtrado en modo bloqueo el primer día?
5. ¿Qué ocurre con los testigos ya emitidos si el directorio deja de responder?

Referencias

- AWS (2025). Amazon Cognito user pools: using tokens — qué contiene cada token y cuánto dura. <https://docs.aws.amazon.com/cognito/latest/developerguide/amazon-cognito-user-pools-using-tokens-with-identity-providers.html>
- AWS (2025). Verifying a JSON Web Token — validación de la firma y de las reclamaciones. <https://docs.aws.amazon.com/cognito/latest/developerguide/amazon-cognito-user-pools-using-tokens-verifying-a-jwt.html>
- AWS (2025). AWS WAF: testing rules in count mode — probar reglas en modo recuento sin bloquear tráfico legítimo. <https://docs.aws.amazon.com/waf/latest/developerguide/web-acl-testing.html>
- OWASP (2025). API Security Top 10: broken object level authorization — el fallo de autorización que el diseño debe cerrar. <https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/>
- RFC 8725 — JSON Web Token best current practices. <https://www.rfc-editor.org/rfc/rfc8725>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 17 en PDF · Recorrido de AWS en PDF · Manual integral

Anterior	Índice	Siguiente
← 208 · DynamoDB por patrones de acceso y single-table design	Parte 17 · Programa	210 · EventBridge, SQS, DLQ, replay e idempotencia →

Evaluación — 209 Cognito, JWT authorizers, WAF y defensa en profundidad

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega aws-identity-edge con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.

- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

210 — EventBridge, SQS, DLQ, replay e idempotencia

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: messaging · Estado: EXECUTABLECORE

Propósito

Procesar trabajo fuera de la petición sin perder mensajes, sin duplicar efectos y sin quedarse con una cola de fallidos que nadie mira. La clase distingue el bus de eventos de la cola por lo que cada uno resuelve, fija los parámetros que de verdad importan —visibilidad, reintentos, lotes—, y desarrolla las dos cosas que este programa lleva señalando desde la parte 09: la idempotencia no es opcional y la cola de fallidos sin alerta ni procedimiento de reproceso es una pérdida de datos en diferido.

Resultados de aprendizaje

Al finalizar podrás:

1. Elegir entre bus de eventos, cola y flujo según lo que haga falta.
2. Configurar visibilidad, reintentos y lotes de forma coherente.
3. Implementar idempotencia con clave y escritura condicionada.
4. Operar la cola de fallidos con alerta, diagnóstico y reproceso.
5. Evitar los tres fallos que arruinan estos sistemas.

Conceptos centrales

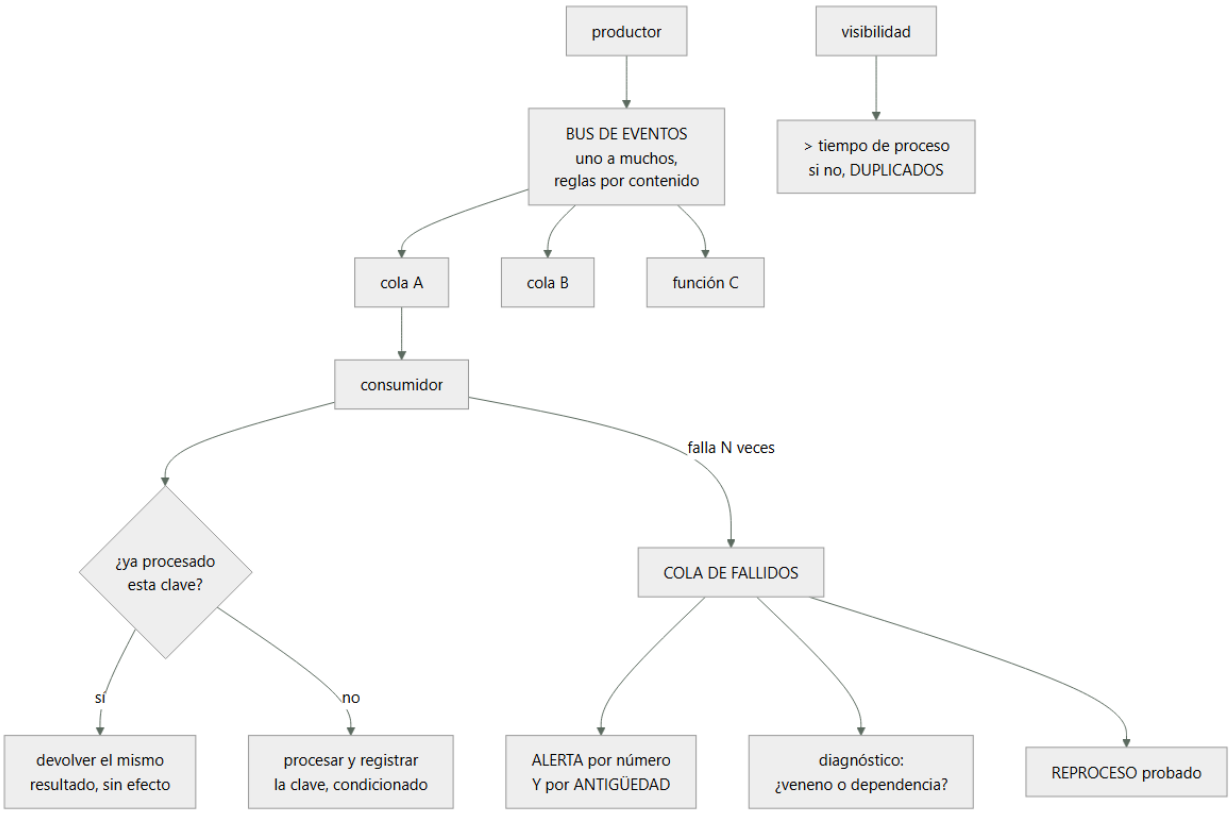
Concepto	Comprensión verificable
bus de eventos	Encaminador que entrega un evento a varios destinos según reglas de contenido. Uno a muchos.
cola	Almacén de mensajes con un consumidor lógico, entrega al menos una vez y control de visibilidad.
tiempo de visibilidad	Plazo durante el que un mensaje tomado no se entrega a otro consumidor. Debe superar el tiempo de proceso.
cola de fallidos	Destino de los mensajes que agotaron sus reintentos. Sin alerta ni reproceso, es una papelera.
idempotencia	Que procesar el mismo mensaje dos veces produzca el mismo resultado que procesarlo una.
fallo parcial de lote	Informar de qué elementos del lote fallaron para que solo esos se reintenten.

Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Bus, cola y flujo: qué resuelve cada uno

Los tres transportan mensajes y no son intercambiables.

```
BUS DE EVENTOS (EventBridge)
  encamina un evento a VARIOS destinos según reglas sobre
  su contenido
  + desacopla al productor de sus consumidores: añadir un
    consumidor no toca al productor              clase 148
  + reglas declarativas, archivo y reproducción
  + integración con eventos del propio proveedor
  – no garantiza orden
  – sin control fino de reintentos por consumidor

COLA (SQS)
  almacena mensajes para UN consumidor lógico
  + control de visibilidad, reintentos y cola de fallidos
  + amortigua picos: el consumidor va a su ritmo
  + variante ordenada con agrupación por clave
  – uno a uno; para varios consumidores hacen falta varias
    colas

FLUJO (Kinesis)
  registro ordenado por partición, con varios lectores
  independientes y reproducción por posición
  + orden dentro de la partición y reproceso desde un punto
  – capacidad por fragmento; el reparto se decide con la
    clave                                        clase 116
```

Y el patrón que combina bien y es el habitual:

```
productor → BUS → regla → COLA → función

por qué la cola en medio y no la función directamente
  amortigua picos: la función consume a su ritmo
  da reintentos controlados y cola de fallidos
  y protege a la base de la concurrencia         clase 207

→ conectar el bus directamente a la función pierde las tres
```

Y la decisión sobre el contenido del mensaje:

```
HECHO CON DATOS   «pedido confirmado» + los datos
  + el consumidor no necesita llamar de vuelta
  − el mensaje envejece; hay que versionarlo   clase 188

SOLO REFERENCIA   «pedido 4471 confirmado»
  + mensaje pequeño y siempre válido
  − cada consumidor llama al productor: N llamadas más
  − y si el productor está caído, no se puede procesar

→ en general, HECHO CON LOS DATOS que el consumidor
  necesita, versionado y con esquema validado en el
  productor                                    clase 188
```

2. Los parámetros que importan

Los valores por defecto de una cola producen duplicados y pérdidas. Estos son los que hay que decidir.

```
TIEMPO DE VISIBILIDAD
  cuánto tiempo un mensaje tomado queda invisible

  REGLA   visibilidad > tiempo máximo de proceso
          típicamente 6 veces el plazo de la función

  si es MENOR que el proceso
    el mensaje reaparece mientras aún se está procesando
    → otro consumidor lo coge
    → DOS ejecuciones del mismo mensaje
    → y el síntoma es «duplicados aleatorios bajo carga»

RECUENTO MÁXIMO DE RECEPCIONES
  cuántas veces se reintenta antes de ir a la cola de
  fallidos
  demasiado bajo   un fallo transitorio pierde el mensaje
  demasiado alto   un mensaje venenoso bloquea durante horas
  típico   3 a 5

TAMAÑO DE LOTE
  cuántos mensajes recibe la función por invocación
  lote grande   más eficiente, menos invocaciones
  lote grande   y un fallo reintenta EL LOTE ENTERO
  → salvo que se informe de fallo parcial

FALLO PARCIAL DE LOTE
  la función devuelve qué identificadores fallaron
  → solo esos se reintentan
  → SIN esto, un mensaje malo en un lote de 10 hace que
    los 9 buenos se reprocesen una y otra vez
  → y si no son idempotentes, se duplican nueve efectos

RETENCIÓN
  cuánto vive un mensaje en la cola: hasta 14 días
  → y en la cola de fallidos conviene el máximo, para tener
    tiempo de reprocesar

ESPERA LARGA
  el consumidor espera si no hay mensajes
  → menos peticiones vacías, menos coste y menos latencia
  → debería estar siempre activada
```

Y una relación que hay que respetar:

```
visibilidad ≥ plazo de la función × tamaño de lote
→ porque la función procesa el lote entero dentro de un
  solo tiempo de visibilidad
```

Y el orden, cuando hace falta:

```
la cola ordenada garantiza orden DENTRO de un grupo
  → el grupo es la clave: cliente, pedido, cuenta
  → y limita el caudal por grupo

y la pregunta previa
  ¿de verdad hace falta orden, o basta con que las
```

```
  operaciones sean conmutativas?              clase 203
  → «−1 unidad» no necesita orden; «stock = 4», sí
```

3. Idempotencia, en concreto

Con entrega al menos una vez, reintentos y lotes, el mismo mensaje se procesará dos veces. No es una posibilidad remota: ocurre.

```
DE DÓNDE VIENEN LOS DUPLICADOS
  visibilidad menor que el proceso
  reintento tras un plazo vencido en el que el trabajo sí
    se hizo
  lote reintegrado entero por un solo elemento fallido
  el productor que reintenta al no recibir confirmación
  y el reproceso desde la cola de fallidos
```

El mecanismo, en tres piezas:

```
1  CLAVE DE IDEMPOTENCIA
   identificador del mensaje, o del suceso de negocio
   → nunca generada por el consumidor
   → y debe ser la misma en el reintento: si el productor
     genera una nueva cada vez, no sirve de nada

2  REGISTRO DE CLAVES PROCESADAS
   tabla con la clave, el resultado y una caducidad
   → la caducidad debe superar la retención de la cola

3  ESCRITURA CONDICIONADA, antes del efecto
   «inserta esta clave si no existe»
   si falla la condición → ya se procesó: devolver el
   resultado guardado
   si tiene éxito → procesar
```

Y el detalle que decide si funciona:

```
¿QUÉ PASA SI EL PROCESO FALLA DESPUÉS DE REGISTRAR LA
CLAVE?
  el mensaje se reintenta, ve la clave y no hace nada
  → el efecto NUNCA se produce

→ por eso el registro se hace en tres estados
    EN CURSO   con caducidad corta
    HECHO      con el resultado
    y si un reintento encuentra EN CURSO caducado, lo
    reclama
```

Y la alternativa que evita todo esto cuando se puede:

```
OPERACIONES NATURALMENTE IDEMPOTENTES
  «poner el estado a ENVIADO» lo es
  «incrementar el contador» no lo es
  «insertar con clave única del negocio» lo es

→ modelar la operación como asignación en vez de como
  incremento resuelve el problema sin infraestructura
                                              clase 149
```

Y la prueba que hay que ejecutar:

```
enviar el mismo mensaje 50 veces en paralelo
→ un solo efecto, 50 respuestas idénticas         ley 22
```

4. La cola de fallidos, operada de verdad

La cola de fallidos se configura y luego se olvida. Este programa la ha visto llena y sin mirar en la clase 120 y en la 132.

```
LO QUE PASA SI NADIE LA MIRA
  los mensajes caducan a los 14 días
  → pérdida de datos silenciosa                    ley 13
  → y cuando se descubre, ya no se pueden recuperar
```

Lo que hace falta, en cuatro piezas:

```
1  ALERTA POR NÚMERO Y POR ANTIGÜEDAD
   «hay mensajes» es la alerta obvia
   «hay un mensaje de más de 1 hora» es la que sirve
```

```
   → la antigüedad detecta el goteo que el número esconde

2  DIAGNÓSTICO: ¿por qué está ahí?
   VENENO       el mensaje es incorrecto y fallará siempre
     → esquema inválido, campo que falta, dato imposible
     → reprocesarlo no sirve; hay que corregir o descartar
   DEPENDENCIA  el mensaje es correcto y algo estaba caído
     → reprocesar funciona
   CÓDIGO       un fallo de la función
     → corregir y reprocesar

   → y para distinguirlos hay que guardar el motivo del
     fallo con el mensaje

3  REPROCESO PROBADO
   un procedimiento —no un comando improvisado— que
     mueve mensajes de la cola de fallidos a la principal
     con límite de ritmo, para no tumbar lo que se
       recuperó
     y con posibilidad de reprocesar uno solo
   → y probado antes de necesitarlo                ley 22

4  DESCARTE EXPLÍCITO
   los mensajes veneno se descartan a propósito, con
   registro de quién y por qué
   → no se dejan caducar
```

Y una advertencia sobre el reproceso masivo:

```
reprocesar 40.000 mensajes de golpe
  → dispara la concurrencia
  → tumba la base                                clase 207
  → y genera una segunda tanda de fallidos

→ reproceso con límite de ritmo, y por lotes
```

Los tres fallos que arruinan estos sistemas, resumidos:

```
1  visibilidad menor que el proceso  → duplicados
2  sin idempotencia                  → duplicados con efecto
3  cola de fallidos sin alerta ni    → pérdida silenciosa
   reproceso
```

Lo que hay que vigilar:

```
antigüedad del mensaje más viejo en la cola  ← la clave
profundidad de la cola y su tendencia
mensajes en la cola de fallidos, y su antigüedad
invocaciones fallidas del consumidor
recepciones repetidas del mismo mensaje
  → señal directa de visibilidad mal puesta
y el retraso entre que ocurre el hecho y se procesa
  → es la medida que le importa al negocio
```

Y la lista de comprobación de la clase:

- ■ el bus se usa para uno a muchos y la cola para amortiguar
- ■ hay cola entre el bus y la función, no conexión directa
- ■ los eventos son hechos con datos, versionados y validados
- ■ la visibilidad supera el proceso máximo por lote
- ■ el recuento de reintentos está decidido, no por defecto
- ■ el fallo parcial de lote está activado
- ■ la espera larga está activada
- ■ toda operación con efecto es idempotente
- ■ la clave de idempotencia la genera el productor
- ■ el registro de claves distingue en curso de hecho
- ■ hay alerta por antigüedad en la cola de fallidos
- ■ se guarda el motivo del fallo con el mensaje
- ■ el procedimiento de reproceso existe y se ha ejecutado
- ■ el reproceso tiene límite de ritmo
- ■ los mensajes veneno se descartan con registro

Y el cierre que enlaza con la clase siguiente: con la API, el almacén, la seguridad y el proceso asíncrono en pie, falta saber qué está pasando cuando algo va mal. Observabilidad en AWS, definida como código, es la materia de la clase 211.

Ejemplo trabajado

CloudShop procesa la confirmación de pedidos de forma asíncrona. Lo que sigue son los duplicados que aparecieron bajo carga, la cola de fallidos con 41.000 mensajes que nadie miraba, y el reproceso que tumbó la base la primera vez.

El montaje inicial:

```
función de crear pedido → bus de eventos
  regla «pedido.confirmado» → función de correo
  regla «pedido.confirmado» → función de inventario
  regla «pedido.confirmado» → función de facturación

todo conectado directamente del bus a las funciones
sin colas en medio
sin idempotencia
cola de fallidos configurada en las funciones asíncronas
```

Problema 1 · Correos duplicados bajo carga.

```
síntoma   en campaña, algunos clientes recibían 2 y hasta 3
          correos de confirmación
          en tráfico normal, nunca

diagnóstico
  la invocación asíncrona desde el bus reintenta 2 veces
  ante error
  bajo carga, la función de correo superaba su plazo por
  la latencia del proveedor de correo
  → el envío SÍ se había hecho; el plazo venció al esperar
    la respuesta
  → el bus reintentaba, y se enviaba otra vez

  correos duplicados en la campaña                  1.840
  quejas recibidas                                     61

corrección
  1  cola entre el bus y cada función
     → amortigua, y da control de reintentos propio
  2  idempotencia con el identificador del evento
  3  el plazo de la función subido a 3 × el p99 del
     proveedor, y el envío marcado como hecho ANTES de
     esperar la confirmación final
```

Problema 2 · Duplicados aleatorios en inventario.

```
síntoma   tras añadir las colas, aparecieron descuentos de
          stock duplicados, sin patrón claro

diagnóstico
  tiempo de visibilidad de la cola          30 s (por defecto)
  plazo de la función                       25 s
  tamaño de lote                            10 mensajes
  tiempo real de proceso del lote           hasta 90 s

  → el lote tardaba más que la visibilidad
  → los mensajes reaparecían mientras se procesaban
  → otro consumidor los tomaba

  la relación que se había incumplido
    visibilidad ≥ plazo × tamaño de lote
    30 s  <  25 s × 10

corrección
  visibilidad                              300 s
  tamaño de lote                            10
  plazo de la función                       25 s
  y fallo parcial de lote activado

  y la señal que lo habría detectado antes
    «recepciones repetidas del mismo mensaje»
    → estaba disponible y nadie la miraba          ley 15
```

Y el efecto del fallo parcial de lote:

```
antes   1 mensaje malo en un lote de 10
        → el lote entero se reintentaba
        → 9 mensajes buenos reprocesados hasta 5 veces
        → 45 efectos duplicados por cada mensaje malo
después → solo el malo se reintenta
```

Problema 3 · La cola de fallidos con 41.000 mensajes.

```
se descubrió al revisar el coste de almacenamiento de colas

  mensajes en la cola de fallidos de facturación     41.200
  el más antiguo                                  13 días
  → 14 días es la retención: a la mañana siguiente
    empezaban a caducar

  alerta configurada   «número de mensajes > 0»
  → sí existía
  → salía a un canal creado en 2023, sin suscriptores
                                          ley 15, clase 194

  qué eran los 41.200
    38.900  fallos de un periodo de 4 horas en que el
            servicio de facturación estuvo caído
            → mensajes CORRECTOS, reprocesables
     2.180  mensajes con un campo nuevo que la función
            antigua no entendía
            → veneno por versión de esquema      clase 188
       120  mensajes con importe negativo, de un error de
            un integrador
            → veneno real
```

El reproceso, que salió mal la primera vez.

```
primer intento
  se movieron los 41.200 mensajes a la cola principal de
  golpe

  a los 40 segundos
    la función de facturación escaló a 2.800 ejecuciones
    la base de datos agotó conexiones           clase 207
    → cayó facturación Y el resto de servicios que usan
      esa base
    → y 9.400 mensajes volvieron a la cola de fallidos

  duración del incidente                          38 min

segundo intento, con procedimiento
  concurrencia reservada de la función           40
  reproceso por lotes de 500, con espera entre lotes
  ritmo efectivo                              ~120 mensajes/s
  duración                                        6 min
  fallos                                             0

y los venenos
  2.180 de esquema  → la función se actualizó primero para
                      tolerar el campo nuevo, y luego se
                      reprocesaron
    120 de importe  → descartados con registro de quién y
                      por qué; y se avisó al integrador
```

Lo que quedó montado.

```
ARQUITECTURA
  productor → bus → regla → COLA → función
  una cola por consumidor, con su cola de fallidos

PARÁMETROS, por consumidor
  consumidor    visibilidad  lote  reintentos  plazo
  correo           180 s      5        3        30 s
  inventario       300 s     10        5        25 s
  facturación      600 s     10        5        50 s
  todos con espera larga y fallo parcial activados

IDEMPOTENCIA
  tabla de claves procesadas, con estados EN CURSO y HECHO
```

```
  clave = identificador del evento, generado por el
    productor
  caducidad de la clave: 21 días (> 14 de retención)
  escritura condicionada antes del efecto

  prueba negativa
    mismo mensaje 50 veces en paralelo → 1 efecto      ✓

COLA DE FALLIDOS
  retención al máximo: 14 días
  motivo del fallo guardado como atributo del mensaje
  ALERTAS
    «hay mensajes con más de 1 hora»    → canal de guardia
    «el más antiguo supera 7 días»      → escalado
    → la de antigüedad es la que sirve; la de número se
      dispara con un solo mensaje transitorio
  panel con desglose por motivo

  PROCEDIMIENTO DE REPROCESO
    escrito, con límite de ritmo y por lotes
    con opción de reprocesar un mensaje concreto
    ejecutado por alguien que no lo escribió       ley 22
    → en el primer ensayo, 3 de 9 pasos necesitaron
      aclaración
    tiempo de reproceso de 10.000 mensajes         2 min
```

El resultado, seis meses después:

```
correos duplicados por campaña             1.840           0
descuentos de stock duplicados               340           0
mensajes en cola de fallidos           41.200 (max)   máx. 92
antigüedad máxima en cola de fallidos     13 días      41 min
mensajes caducados y perdidos          casi 41.200         0
reprocesos ejecutados                          1           7
  que causaron incidente                       1           0
mensajes descartados con registro              0         214
```

La lección que esta clase deja: los duplicados no eran aleatorios: eran visibilidad menor que el proceso del lote, una relación aritmética que se puede comprobar antes de desplegar. La cola de fallidos tenía cuarenta y un mil mensajes correctos a un día de caducar, con una alerta que existía y salía a un canal sin nadie. Y el reproceso, hecho sin procedimiento, tumbó la base y generó nueve mil cuatrocientos fallidos nuevos: la recuperación se convirtió en el segundo incidente.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/210-eventbridge-sqs-dlq-replay-e-idempotencia/lab.
py
```

El laboratorio selecciona el motor de práctica messaging y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa aws-event-platform en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un flujo que declara orden, entrega y manejo de errores. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye aws-event-platform para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

■■ Errores frecuentes

Síntoma	Causa probable	Corrección
Aparecen duplicados solo bajo carga	El tiempo de visibilidad es menor que el proceso del lote y el mensaje reaparece mientras se procesa	Fija visibilidad mayor que plazo por tamaño de lote y vigila las recepciones repetidas del mismo mensaje.
Un mensaje incorrecto provoca decenas de efectos duplicados	El lote entero se reintegra por un solo elemento fallido	Activa el informe de fallo parcial de lote para que solo se reintenten los elementos que fallaron.
Se pierden mensajes sin que nadie se entere	La cola de fallidos caduca a los 14 días y su alerta va a un canal sin nadie	Alerta por antigüedad además de por número, dirígela a un canal con guardia y comprueba que llega.
El reproceso masivo provoca un segundo incidente	Se devolvieron todos los mensajes de golpe y la concurrencia tumbó la base	Reprocesa por lotes con límite de ritmo y concurrencia reservada, siguiendo un procedimiento probado.
Reprocesar no arregla nada para parte de los mensajes	Son mensajes veneno: esquema inválido o datos imposibles	Guarda el motivo del fallo, distingue veneno de dependencia, corrige el consumidor antes de reprocesar y descarta lo irreparable con registro.
El efecto nunca llega a producirse pese a los reintentos	Se registró la clave de idempotencia y el proceso falló después	Registra en dos estados, en curso con caducidad y hecho con resultado, y permite reclamar los en curso caducados.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué resuelve el bus que no resuelve la cola, y por qué conviene poner una cola en medio?
2. ¿Qué relación deben cumplir visibilidad, plazo y tamaño de lote?
3. ¿Qué aporta informar del fallo parcial de un lote?
4. ¿Por qué la alerta por antigüedad es mejor que la de número en la cola de fallidos?
5. ¿Cómo se distingue un mensaje veneno de uno que falló por una dependencia caída?

Referencias

- AWS (2025). Amazon SQS: visibility timeout and dead-letter queues — tiempo de visibilidad y destino de los mensajes fallidos. <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-visibility-timeout.html>
- AWS (2025). Lambda event source mapping: partial batch responses — reintento parcial de un lote sin reprocesarlo entero. <https://docs.aws.amazon.com/lambda/latest/dg/services-sqs-errorhandling.html>
- AWS (2025). Amazon EventBridge: rules, archives and replay — archivo y reproducción de eventos ya emitidos. <https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-what-is.html>
- AWS (2025). Powertools for AWS Lambda: idempotency — idempotencia mediante clave y almacén de estado. <https://docs.powertools.aws.dev/lambda/python/latest/utilities/idempotency/>
- Hohpe, G. y Woolf, B. (2003). Enterprise Integration Patterns — canal de mensajes inválidos y reintentos. <https://www.enterpriseintegrationpatterns.com/>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 17 en PDF · Recorrido de AWS en PDF · Manual integral

Anterior	Índice	Siguiente
← 209 · Cognito, JWT authorizers, WAF y defensa en profundidad	Parte 17 · Programa	211 · CloudWatch, X-Ray y observabilidad como código →

Evaluación — 210 EventBridge, SQS, DLQ, replay e idempotencia

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega aws-event-platform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

211 — CloudWatch, X-Ray y observabilidad como código

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: observability · Estado: EXECUTABLECORE

Propósito

Montar observabilidad en AWS que sirva para diagnosticar y no para decorar, y declararla como código junto al servicio que la produce. La clase cubre registros estructurados con su coste —que en la clase 207 resultó ser la segunda partida de la factura—, métricas propias sin arruinarse en dimensiones, trazado distribuido con muestreo, y la parte que decide si esto funciona: la alerta se define en el mismo cambio que el servicio, o no existe.

Resultados de aprendizaje

Al finalizar podrás:

1. Emitir registros estructurados con contexto y controlar su coste.
2. Publicar métricas propias sin multiplicar el gasto por dimensiones.
3. Trazar peticiones extremo a extremo con muestreo razonable.
4. Declarar paneles, objetivos y alertas como código.
5. Distinguir lo que hay que vigilar de lo que solo hay que poder consultar.

Conceptos centrales

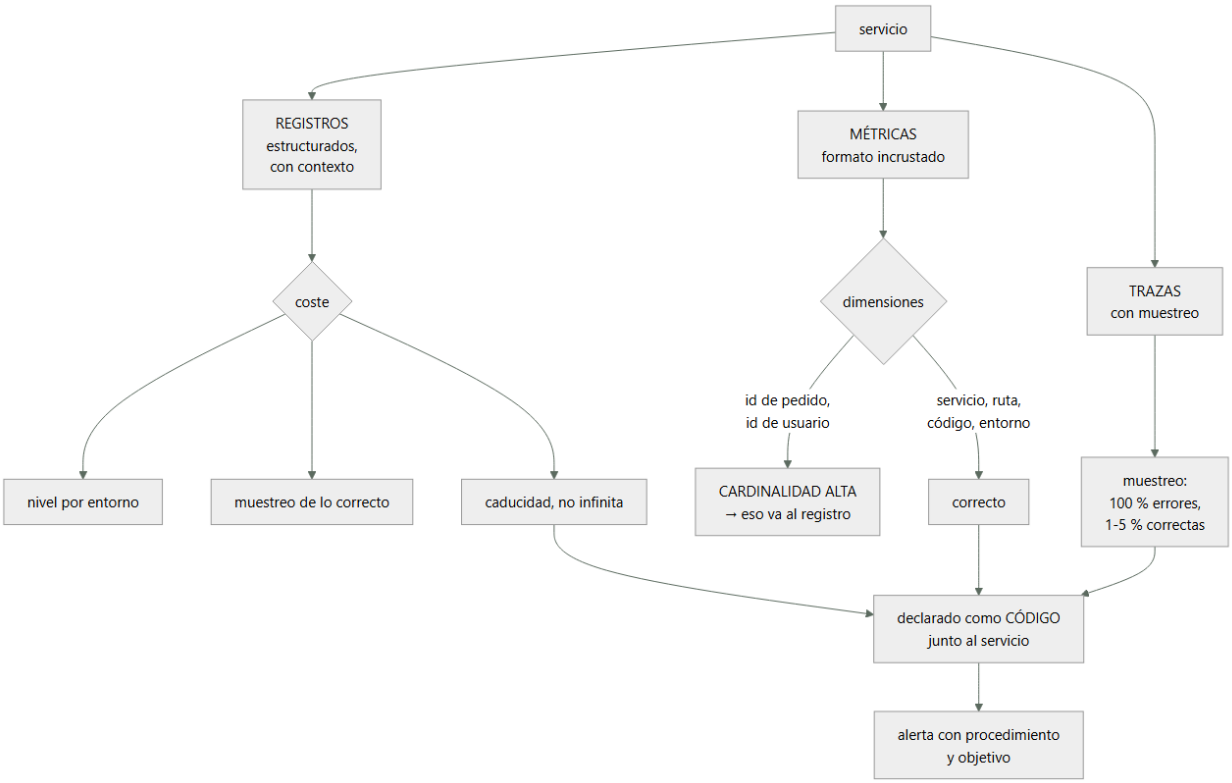
Concepto	Comprensión verificable
registro estructurado	Línea en formato de datos con campos consultables, no texto libre.
métrica de formato incrustado	Métrica publicada dentro de un registro, sin llamada aparte ni coste de API.
dimensión	Etiqueta de una métrica. Cada combinación distinta es una métrica facturable.
cardinalidad	Número de valores distintos de una dimensión. Alta cardinalidad multiplica el coste.
traza	Registro del recorrido de una petición por los servicios, con tiempos por tramo.
observabilidad como código	Paneles, objetivos y alertas declarados junto al servicio y desplegados con él.

Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Registros: estructura y coste

En AWS los registros se cobran por ingesta y por almacenamiento, y la ingesta suele ser la partida sorpresa.

```
LO PRIMERO: ESTRUCTURA
  ✗ "Error procesando pedido 4471 del cliente 123"
  ✓ {"nivel":"error","mensaje":"fallo al confirmar",
     "pedido":"4471","cliente":"123",
     "traza":"1-65f...","servicio":"pedidos",
     "version":"1.4.2","duracion_ms":412}

→ el primero se busca con expresiones regulares y falla
→ el segundo se consulta con filtros por campo
```

Y el contexto que debe llevar toda línea:

```
identificador de traza          ← el que une todo   clase 121
servicio y versión
entorno
identificador de la petición
identificador del usuario o del inquilino, si aplica
y la duración, cuando la operación termina
```

El coste, con los tres controles que lo dominan:

```
1  NIVEL POR ENTORNO
   desarrollo   depuración
   producción   información y errores; depuración activable
                por variable sin redesplegar
   → registrar cada petición con su cuerpo en producción es
     el error más caro y el más común

2  MUESTREO DE LO CORRECTO
   el 100 % de los errores
   el 1-5 % de las peticiones correctas
   → y el 100 % de las peticiones de un usuario concreto
     cuando se está diagnosticando, activable

3  CADUCIDAD
   ✗ por defecto, muchos grupos se crean sin caducidad
```

```
  ✓ 14-30 días en producción, y lo que haga falta
    conservar se exporta a almacenamiento barato
  → función de aptitud: ningún grupo sin caducidad
                                                clase 190
```

Y el dato de la clase 207, que justifica todo esto:

```
ingesta de registros                          620 €/mes
almacenamiento                                180 €/mes
cómputo de las funciones                      410 €/mes
→ los registros costaban el doble que el cómputo
tras aplicar nivel, muestreo y caducidad      120 €/mes
```

Y una advertencia sobre lo que nunca debe ir a un registro:

```
contraseñas, testigos, claves
datos personales completos
números de tarjeta
cuerpos de petición sin filtrar
→ los registros se consultan por mucha gente y se conservan
→ y una captura de registros es un dato sensible
                                                clase 189
```

2. Métricas propias y la trampa de las dimensiones

Las métricas de infraestructura vienen dadas. Las que importan —las de negocio y las de servicio— hay que publicarlas.

```
CÓMO PUBLICARLAS
  ✗ una llamada a la API de métricas por cada valor
    → latencia añadida en el camino crítico y coste por
      llamada
  ✓ FORMATO INCRUSTADO: la métrica va dentro del registro
    con una sección de metadatos
    → sin llamada, sin latencia, y el registro sirve además
      como detalle
```

La trampa de las dimensiones, que produce facturas absurdas:

```
cada COMBINACIÓN distinta de dimensiones es una métrica
facturable

  dimensiones      valores      métricas resultantes
  servicio            8
  ruta               24
  código de estado    6
  → 8 × 24 × 6 = 1.152 métricas          razonable

  añade id_de_pedido con 400.000 valores
  → 460 millones de métricas             desastre

REGLA
  dimensión = algo de lo que quieras ver la EVOLUCIÓN
  identificador = algo con lo que quieras BUSCAR
  → la evolución va a métricas; la búsqueda, a registros
```

Y las métricas que de verdad hacen falta, por servicio:

```
LAS CUATRO SEÑALES                              clase 121
  latencia por percentil
  tráfico
  errores, separando los del cliente de los del sistema
  saturación: el recurso que se agota primero

Y LAS DE NEGOCIO, que son las que se echan de menos
  pedidos confirmados por minuto
  pagos rechazados
  retraso entre el hecho y su procesamiento    clase 210
  → una caída de pedidos confirmados detecta cosas que
    ninguna métrica técnica ve                   ley 15
```

Y una advertencia sobre los percentiles:

```
las métricas agregadas por minuto no permiten calcular el
percentil real del periodo mayor
→ el «p99 del día» calculado a partir de p99 por minuto no
```

```
  es el p99 del día
→ para percentiles exactos hacen falta métricas con
  distribución o el detalle en registros
```

Las trazas, con la decisión de muestreo:

```
el trazado distribuido muestra el recorrido de una petición
y cuánto tarda cada tramo                          clase 121

muestreo
  100 % de las peticiones con error
  100 % de las lentas (por encima de un umbral)
  1-5 % del resto
  → trazar todo cuesta mucho y aporta poco

y lo que hay que propagar
  la cabecera de traza a TODAS las llamadas, incluidas las
  asíncronas: por la cola, dentro del mensaje
  → sin esto, la traza se corta en el primer salto
    asíncrono                                      clase 210
```

3. Declararlo como código

La observabilidad montada a mano por la consola envejece y desaparece. Declararla junto al servicio es lo que la mantiene viva.

```
QUÉ SE DECLARA
  el grupo de registros, con su caducidad
  las métricas y sus filtros
  el panel del servicio
  los objetivos de nivel de servicio
  las alertas, con su destino y su procedimiento enlazado
  → todo en la misma plantilla que el servicio

QUÉ SE GANA
  un servicio nuevo nace observable                clase 171
  las alertas se revisan en la misma propuesta de cambio
  y al retirar el servicio, se retira su observabilidad
                                                     ley 23
```

Y la regla que lo hace efectivo:

```
FUNCIÓN DE APTITUD                                 clase 190
  ningún servicio se despliega sin
    grupo de registros con caducidad
    las cuatro señales publicadas
    al menos un objetivo declarado
    al menos una alerta con procedimiento enlazado
→ y el carril fácil lo trae hecho, así que casi nunca falla
```

Alertas: qué merece despertar a alguien.

```
ALERTA (despierta)
  el objetivo de nivel de servicio se está consumiendo
  demasiado rápido                                 clase 125
  el flujo crítico no funciona
  la cola de fallidos tiene mensajes antiguos      clase 210
  el retraso de proceso supera el acordado

AVISO (revisar en horario)
  tendencias, saturación creciente, coste

NI UNA NI OTRA
  «la CPU está al 80 %»        ← no es un síntoma de usuario
  «hubo un error»              ← los errores ocurren
  «un despliegue terminó»
```

Y las alertas que este programa ha demostrado que faltan siempre:

```
ALERTA POR AUSENCIA
  «esta función no se ha ejecutado en N minutos»
  → un trabajo programado que deja de dispararse no genera
    ningún error                                     ley 13

ALERTA POR ANTIGÜEDAD
  «el mensaje más viejo de la cola de fallidos supera 1 h»
```

```
  «este certificado no se renueva desde hace 45 días»
                                                clase 196
  «este dispositivo lleva 2 h sin reportar»      clase 203
```

Y la comprobación de que la alerta funciona:

```
provocar la condición y ver si llega, a quien tiene que
llegar                                              ley 22
→ en este programa, las alertas que salían a canales sin
  suscriptores han aparecido en cuatro clases distintas
```

Y una decisión sobre destino:

```
las alertas van a UN canal con guardia, no a canales
nuevos por servicio
→ crear un canal por servicio garantiza que alguno quede
  sin nadie                                         ley 15
```

4. Consultar, y lo que no hay que vigilar

Hay una diferencia importante entre lo que hay que vigilar y lo que hay que poder consultar.

```
VIGILAR   pocas cosas, con alerta, revisadas
CONSULTAR mucho, sin alerta, disponible cuando haga falta

→ confundirlos produce cientos de alertas y ningún
  diagnóstico                                    clase 125
```

Las consultas que hay que tener preparadas, no improvisar:

```
«dame todas las líneas de esta traza»
«dame los errores de este servicio en la última hora,
 agrupados por tipo»
«dame las peticiones de este usuario»
«dame las peticiones más lentas y qué tramo domina»
«compara esta hora con la misma hora de ayer»

→ guardadas y enlazadas desde el procedimiento de la alerta
→ improvisar consultas durante un incidente cuesta minutos
  que no se tienen                              clase 127
```

Y la correlación, que es lo que hace útil todo lo demás:

```
de una alerta → al panel del servicio
del panel → a las trazas del periodo
de una traza → a los registros de esa petición
de los registros → a la línea de cambios: ¿qué se desplegó?
                                                clase 122

→ si esa cadena tiene un eslabón roto, el diagnóstico se
  hace a ojo
```

Los objetivos de nivel de servicio, declarados:

```
por flujo de usuario, no por componente         clase 123
  «el 99,5 % de las confirmaciones de pedido en menos de
   800 ms, medidas en el borde»
con presupuesto de error y alerta por ritmo de consumo
  → y esa es la alerta que despierta, no el error suelto
```

Y la lista de comprobación de la clase:

- ■ todos los registros son estructurados
- ■ toda línea lleva identificador de traza, servicio, versión y entorno
- ■ el nivel de registro depende del entorno y se puede cambiar sin redesplegar
- ■ hay muestreo de las peticiones correctas
- ■ ningún grupo de registros carece de caducidad
- ■ no se registran secretos ni datos personales completos
- ■ las métricas se publican en formato incrustado
- ■ ninguna dimensión tiene cardinalidad alta
- ■ hay métricas de negocio, no solo técnicas
- ■ la cabecera de traza se propaga también por las colas
- ■ el muestreo de trazas cubre el 100 % de errores y lentas
- ■ paneles, objetivos y alertas están en la plantilla del servicio

- hay alertas por ausencia y por antigüedad
- cada alerta tiene procedimiento enlazado
- las alertas se han probado provocando la condición
- las consultas de diagnóstico están guardadas

Y el cierre que enlaza con la clase siguiente: hasta aquí todo ha sido sin servidores. Cuando la carga no encaja en ese modelo —procesos largos, dependencias pesadas, control del entorno— aparecen los contenedores. Registro de imágenes, orquestación gestionada y escalado es la materia de la clase 212.

Ejemplo trabajado

CloudShop monta la observabilidad de su plataforma. Lo que sigue es la factura de registros que disparó el proyecto, la métrica que costaba 3.100 € al mes por una dimensión, y la alerta que llevaba ocho meses sin llegar a nadie.

El punto de partida:

```
coste mensual de observabilidad              4.980 €
  ingesta de registros                       2.140 €
  almacenamiento de registros                  680 €
  métricas propias                           1.740 €
  trazas                                       420 €

y lo que se obtenía a cambio
  paneles                                       31
  abiertos en el último mes                      4
  alertas configuradas                          89
  disparadas en el último mes                  612
  que resultaron accionables                    41 (6,7 %)
```

Los registros: dónde estaba el gasto.

```
ingesta por servicio
  api-pedidos                              1.190 €
  api-catalogo                               420 €
  procesador-eventos                         310 €
  resto                                      220 €

qué registraba api-pedidos
  cada petición, con el cuerpo completo de entrada y salida
  cada consulta a la base, con sus parámetros
  cada llamada saliente, con cabeceras
  y en nivel de depuración, porque nunca se cambió al
    pasar a producción

  volumen                                  1,4 TB/mes
  líneas por petición                            14
```

Y dos hallazgos al revisar el contenido:

```
1  los cuerpos de petición incluían el testigo de
   autorización completo
   → 41 personas tenían acceso de lectura a los registros
   → un testigo válido en texto plano, conservado sin
     caducidad                                  clase 189

2  el grupo de registros no tenía caducidad
   → 19 meses de historia acumulada
   → 8,4 TB almacenados, de los cuales lo consultado en el
     último año eran los últimos 11 días
```

Las correcciones:

```
nivel por entorno                    depuración → información
  con variable para activar depuración por 30 min sin
  redesplegar
muestreo de peticiones correctas      100 % → 3 %
  errores                             100 %, siempre
cuerpos                               eliminados; solo campos
                                      seleccionados
testigos y datos personales           filtrados en el emisor
caducidad                             sin límite → 21 días
exportación de lo necesario           a almacenamiento barato

líneas por petición                        14 → 2
```

```
volumen                              1,4 TB → 96 GB/mes
ingesta                              2.140 € → 165 €
almacenamiento                         680 € → 40 €
```

La métrica que costaba 3.100 €… que resultaron ser 1.740 €.

```
al desglosar el coste de métricas propias

  métrica                        dimensiones      series
  pedidos_procesados      servicio, ruta, estado     144
  latencia_operacion      servicio, operacion        320
  errores_por_tipo        servicio, tipo              96
  tiempo_confirmacion     servicio, ID_DE_PEDIDO  412.000  ←

  la última la había añadido un equipo para «poder ver el
  tiempo de cada pedido»
  → 412.000 series, cada una con muy pocos puntos
  → 1.410 € de los 1.740 €

corrección
  el tiempo por pedido va al REGISTRO, con el identificador
  como campo consultable
  la MÉTRICA queda con dimensiones servicio y tipo de
  pedido: 24 series
  y para ver un pedido concreto, se consulta el registro

coste de métricas               1.740 € → 290 €
```

Y la regla que quedó escrita:

```
¿quieres ver la EVOLUCIÓN de esto agrupado?   → métrica
¿quieres BUSCAR un caso concreto?             → registro
→ y una función de aptitud que rechaza dimensiones con más
  de 200 valores previstos                     clase 190
```

La alerta que llevaba ocho meses sin llegar a nadie.

```
al auditar las 89 alertas
  con destino a un canal con guardia                34
  con destino a canales creados por equipos          41
    de ellos, canales SIN suscriptores               11
  con destino a un buzón de correo compartido        14
    → el buzón tenía 12.400 correos sin leer

y entre las 11 de canales vacíos
  «la cola de fallidos de facturación tiene mensajes»
  → la del incidente de la clase 210, que llevaba 8 meses
    disparándose sin que nadie la viera            ley 15

corrección
  destino único: canal de guardia por turno
  las 41 alertas de equipos se revisaron
    → 23 se convirtieron en avisos de horario laboral
    → 12 se retiraron por no ser accionables
    →  6 pasaron a alerta real
  prueba de extremo a extremo de cada alerta:
    provocar la condición y comprobar que llega
    → 7 de 40 no llegaron a la primera             ley 22
```

La observabilidad como código.

```
la plantilla de cada servicio incluye ahora
  grupo de registros con caducidad
  filtros de métrica
  panel del servicio
  objetivo de nivel de servicio
  alertas con procedimiento enlazado

y la plantilla de servicio nuevo lo trae hecho
  → en 6 meses se crearon 9 servicios
  → 9 de 9 nacieron con panel, objetivo y alerta
  → antes, la media era de 5 semanas hasta tener alerta,
    y 3 de 11 servicios nunca la tuvieron

función de aptitud
```

```
  «ningún servicio sin grupo con caducidad, cuatro señales,
   objetivo y alerta con procedimiento»
  fallos en 6 meses                                    2
    → los 2, servicios antiguos, corregidos
```

La cadena de diagnóstico, comprobada con un incidente real:

```
03:14  alerta: el objetivo de confirmación de pedido está
       consumiendo presupuesto de error 14× más rápido de
       lo normal
03:14  el mensaje enlaza el panel del servicio
03:15  el panel muestra p99 disparado desde las 03:02
03:15  la línea de cambios: despliegue de precios a las
       03:01                                  clase 122
03:16  las trazas del periodo: el tramo lento es la llamada
       a precios
03:17  los registros de esas trazas: error de conexión al
       almacén de precios
03:18  reversión automática ya en curso; confirmada
03:22  restablecido

tiempo hasta identificar la causa                4 min
  antes de este proyecto, la media era de       52 min
```

El resultado:

```
coste de observabilidad                  4.980 €      620 €
ingesta de registros                      1,4 TB      96 GB
series de métricas                       412.500         584
alertas configuradas                          89          63
  disparadas al mes                          612          71
  accionables                           41 (6,7 %)  63 (89 %)
alertas que no llegaban a nadie                11           0
servicios sin panel ni objetivo                 3           0
tiempo medio hasta identificar causa         52 min       4 min
testigos en texto plano en registros            sí          no
```

La lección que esta clase deja: de casi cinco mil euros mensuales de observabilidad, una sola dimensión con cuatrocientos mil valores costaba mil cuatrocientos, y lo que ese equipo quería —ver el tiempo de un pedido concreto— se resuelve con un campo en el registro. Los registros costaban más que el cómputo y contenían testigos válidos en texto plano. Y de ochenta y nueve alertas, once salían a canales sin nadie, incluida la de la cola de fallidos que ocho meses después provocó el incidente de la clase 210.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/211-cloudwatch-x-ray-y-observabilidad-como-codigo/
lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa aws-observability en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye aws-observability para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

■■ Errores frecuentes

Síntoma	Causa probable	Corrección
La factura de registros supera a la de cómputo	Nivel de depuración en producción, cuerpos completos y sin muestreo ni caducidad	Nivel por entorno activable sin redesplegar, muestreo de las peticiones correctas, campos seleccionados y caducidad obligatoria.
El coste de métricas se dispara sin explicación	Una dimensión de alta cardinalidad crea una serie por valor	Las dimensiones son para ver evolución agrupada; lo que sirve para buscar un caso concreto va al registro.
La traza se corta al llegar a un proceso asíncrono	La cabecera de traza no se propaga dentro del mensaje de la cola	Incluye el contexto de traza en el mensaje y recupéralo en el consumidor.
Una alerta lleva meses disparándose sin que nadie actúe	Sale a un canal creado por un equipo y sin suscriptores	Destino único con guardia y prueba de extremo a extremo provocando la condición.
Un servicio lleva semanas en producción sin panel ni alerta	La observabilidad se monta a mano después del despliegue	Declárala en la plantilla del servicio y exígela con una función de aptitud; el carril por defecto debe traerla hecha.
Los registros contienen testigos o datos personales	Se registran cuerpos completos sin filtrar	Filtra en el emisor, registra solo campos seleccionados y trata el archivo de registros como dato sensible.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué campos debe llevar toda línea de registro y por qué?
2. ¿Qué distingue algo que debe ser dimensión de algo que debe ser campo de registro?
3. ¿Qué muestreo de trazas resulta razonable y por qué no se traza todo?
4. ¿Qué dos tipos de alerta faltan casi siempre y qué detectan?
5. ¿Qué eslabones tiene la cadena de diagnóstico desde la alerta hasta la causa?

Referencias

- AWS (2025). CloudWatch embedded metric format — emitir métricas desde el propio registro de la función. <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/CloudWatchEmbeddedMetricFormat.html>

- AWS (2025). CloudWatch Logs Insights query syntax — sintaxis de consulta de registros para el diagnóstico. <https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/CWLQuerySyntax.html>
- AWS (2025). AWS X-Ray sampling rules — muestreo de trazas y su efecto en el coste. <https://docs.aws.amazon.com/xray/latest/devguide/xray-console-sampling.html>
- OpenTelemetry (2025). Context propagation — propagación de contexto entre servicios. <https://opentelemetry.io/docs/concepts/context-propagation/>
- Google (2018). The Site Reliability Workbook: alerting on SLOs — alertar por consumo del presupuesto de error. <https://sre.google/workbook/alerting-on-slos/>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 17 en PDF · Recorrido de AWS en PDF · Manual integral

Anterior	Índice	Siguiente
← 210 · EventBridge, SQS, DLQ, replay e idempotencia	Parte 17 · Programa	212 · ECR, ECS Fargate, ALB y autoscaling →

Evaluación — 211 CloudWatch, X-Ray y observabilidad como código

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega aws-observability con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

212 — ECR, ECS Fargate, ALB y autoscaling

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: container · Estado: EXECUTABLECORE

Propósito

Ejecutar contenedores en AWS sin gestionar servidores, con las decisiones que separan un despliegue que funciona de uno que aguanta. La clase cubre el registro de imágenes y su higiene, la elección entre capacidad gestionada y máquinas propias, la configuración de servicio y balanceador que evita cortar peticiones, y el escalado: por qué la señal correcta casi nunca es la CPU y por qué el escalado siempre llega tarde al pico.

Resultados de aprendizaje

Al finalizar podrás:

1. Publicar imágenes con exploración, inmutabilidad y caducidad.
2. Elegir entre capacidad gestionada y máquinas propias con cifras.
3. Configurar servicio, comprobaciones y drenaje sin cortar peticiones.
4. Escalar con la señal correcta y con margen para el retraso.
5. Dimensionar CPU y memoria midiendo, no copiando.

Conceptos centrales

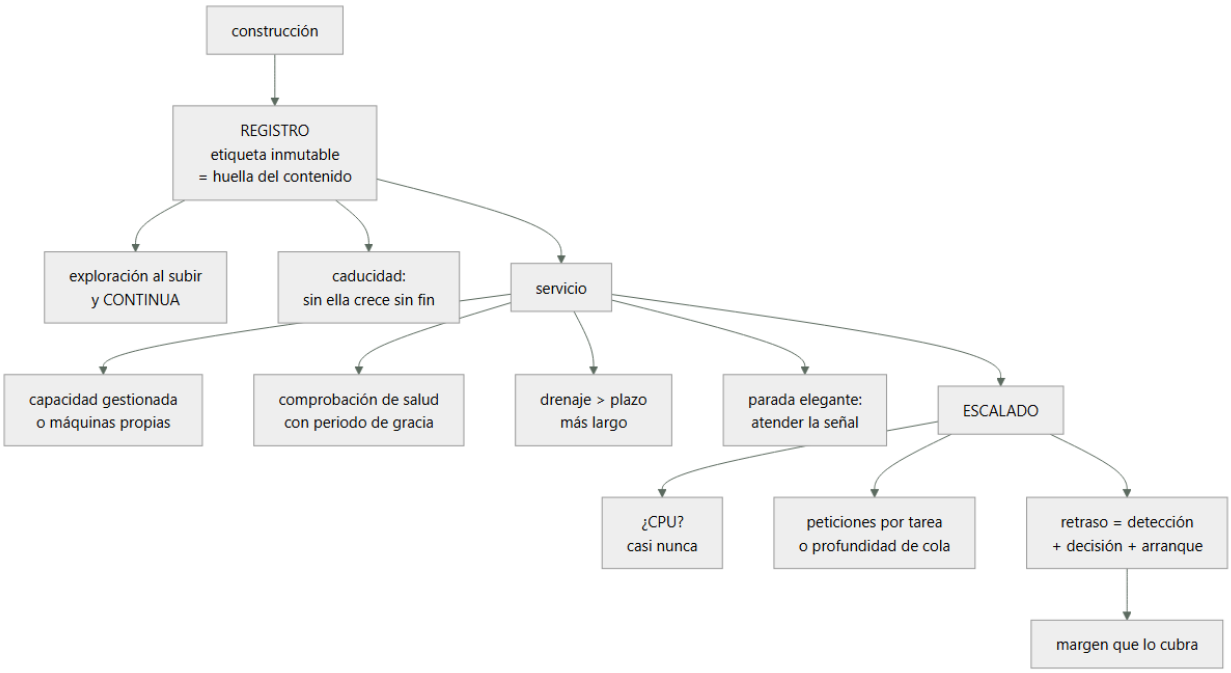
Concepto	Comprensión verificable
registro de imágenes	Almacén de imágenes de contenedor, con exploración de vulnerabilidades y reglas de caducidad.
etiqueta inmutable	Etiqueta que no se puede sobrescribir. Evita que 'la misma versión' cambie de contenido.
definición de tarea	Declaración de qué imagen, con qué recursos, permisos y variables se ejecuta.
capacidad gestionada	Ejecución sin administrar instancias. Se paga por recursos pedidos y por tiempo.
drenaje de destino	Tiempo que el balanceador deja terminar las peticiones en curso antes de retirar una tarea.
retraso de escalado	Suma de detección, decisión y arranque. Es lo que hay que cubrir con margen.

Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. El registro y la higiene de imágenes

El registro parece un detalle y produce dos problemas caros: imágenes que cambian bajo el mismo nombre y crecimiento sin límite.

```
ETIQUETAS INMUTABLES
  ✗ etiqueta «latest» o «v1.4» sobrescribible
    → dos despliegues con la misma etiqueta ejecutan cosas
      distintas
    → y revertir no lleva a lo que había
  ✓ etiqueta = huella del contenido, o versión inmutable
    → la etiqueta identifica un contenido exacto
    → y el despliegue referencia la huella, no la etiqueta
                                                clase 106

EXPLORACIÓN
  al subir: bloquea lo que tenga vulnerabilidades graves
  CONTINUA: una imagen desplegada hace tres meses puede
    tener hoy una vulnerabilidad conocida
  → sin exploración continua, solo se examina lo nuevo
                                                  ley 13

CADUCIDAD
  reglas: conservar las N últimas de cada rama, borrar las
  demás; borrar las sin etiqueta a los pocos días
  → sin reglas, el registro crece indefinidamente y factura
  → y las imágenes viejas con vulnerabilidades siguen ahí
    disponibles                                   ley 25
```

Y dos decisiones sobre la imagen misma:

```
TAMAÑO
  imagen base mínima, construcción en varias etapas
  → el tiempo de arranque depende de la descarga
  → 800 MB frente a 90 MB son segundos en cada arranque,
    y eso se paga en cada escalado

SIN SECRETOS DENTRO
  las variables con secretos se inyectan en ejecución desde
  el gestor de secretos, no se construyen dentro
  → una imagen se puede descargar y examinar
```

Y el permiso que hay que separar y casi nunca se separa:

```
ROL DE EJECUCIÓN     el que usa el agente para descargar la
                     imagen y escribir registros
ROL DE LA TAREA      el que usa TU código para llamar a
                     servicios
→ mezclarlos da al código permisos de infraestructura
                                                clase 134
```

2. Capacidad: gestionada o propia

La elección entre ejecutar sin administrar instancias o gestionarlas se decide con cifras, no con preferencia.

```
CAPACIDAD GESTIONADA (Fargate)
  + sin instancias que parchear, escalar ni vigilar
  + aislamiento por tarea
  + se paga por lo pedido, por segundo
  − precio por unidad de recurso mayor
  − tamaños en combinaciones fijas: pedir 1 vCPU y 2 GB
    puede obligar a pagar más
  − arranque algo más lento
  − sin acceso al anfitrión: agentes especiales, GPU o
    almacenamiento local quedan limitados

MÁQUINAS PROPIAS
  + más barato por unidad, sobre todo con compromisos o
    capacidad sobrante                          clase 143
  + control del anfitrión
  − hay que parchear, escalar y vigilar el conjunto
  − el empaquetado deja hueco desperdiciado
  − y ese trabajo consume capacidad del equipo     ley 23
```

Y el cálculo honesto:

```
el ahorro por unidad de las máquinas propias solo se
realiza si el empaquetado es alto
  utilización real típica de un conjunto mal empaquetado
  35-50 %
  → a esa utilización, la capacidad gestionada suele salir
    igual o más barata

regla práctica
  empieza con capacidad gestionada
  mide utilización y coste durante meses
  pasa a máquinas propias solo si el volumen es alto,
    estable y hay quien lo opere
```

Y el modo intermedio que suele ganar:

```
capacidad gestionada para lo variable y lo poco usado
máquinas propias con compromiso para la base estable
→ y el reparto se decide con el histórico, no a ojo
                                                clase 143
```

Y una nota sobre el precio por interrupción:

```
la capacidad interrumpible es mucho más barata y puede
retirarse con poco aviso
→ vale para trabajo por lotes y para parte de la capacidad
  de servicios tolerantes
→ nunca para el 100 % de un servicio con usuarios
```

3. Servicio y balanceador sin cortar peticiones

La configuración por defecto de un servicio detrás de un balanceador corta peticiones en cada despliegue. Estos son los parámetros que lo evitan.

```
COMPROBACIÓN DE SALUD, con dos puntos       clase 196
  /vivo    ¿el proceso está en pie?      → reinicio
  /listo   ¿puede atender ahora?         → recibir tráfico
  y /listo NO depende de dependencias blandas

PERIODO DE GRACIA AL ARRANCAR
  tiempo antes de empezar a evaluar la salud
  ✗ demasiado corto → la tarea se mata mientras arranca,
    y entra en un bucle de reinicios
  ✓ mayor que el arranque real medido, con margen
```

```
DRENAJE DE DESTINO
  ✗ por defecto suele ser corto
  ✓ mayor que el plazo más largo de las peticiones
  → si no, cada despliegue corta lo que estaba en curso

PARADA ELEGANTE
  la aplicación debe ATENDER la señal de terminación
    dejar de aceptar nuevas
    terminar las que tiene
    cerrar conexiones
  ✗ si no la atiende, se la mata al agotar el plazo
  → y ese plazo debe superar la petición más larga

ORDEN CORRECTO EN UN DESPLIEGUE
  1  arranca la tarea nueva
  2  pasa /listo
  3  el balanceador la registra
  4  el balanceador quita la vieja del reparto
  5  DRENAJE
  6  señal de terminación a la vieja
  7  parada elegante
```

Y los parámetros de despliegue del servicio:

```
PORCENTAJE MÍNIMO SANO   100 %  → nunca menos capacidad de
                                  la nominal
PORCENTAJE MÁXIMO        200 %  → permite arrancar las
                                  nuevas antes de retirar
                                  las viejas
→ con mínimo 100 y máximo 200 el despliegue no reduce
  capacidad en ningún momento
→ con los valores por defecto (50/200 en algunos casos) se
  puede quedar a la mitad durante el despliegue

CIRCUITO DE DESPLIEGUE
  si las tareas nuevas no llegan a estar sanas, el servicio
  vuelve solo a la versión anterior
  → hay que activarlo; no viene activado
```

Y una decisión sobre el tipo de balanceador:

```
de aplicación (capa 7)   rutas, cabeceras, reintentos
de red (capa 4)          latencia mínima, IP fija, otros
                         protocolos
→ y la elección es la misma discusión de la clase 196
```

4. Escalar con la señal correcta

El escalado automático por CPU es el valor por defecto y casi nunca es lo correcto.

```
POR QUÉ LA CPU FALLA COMO SEÑAL
  la mayoría de los servicios de aplicación esperan a la
  red y a la base, no calculan
  → una tarea saturada de peticiones en espera puede tener
    la CPU al 30 %
  → es exactamente el caso de la clase 186: el recurso
    saturado era el grupo de conexiones

LAS SEÑALES QUE SÍ FUNCIONAN
  peticiones por tarea (del balanceador)
    → directa, y fácil de fijar con una prueba de carga
  latencia p99 del destino
    → escalar antes de incumplir el objetivo
  profundidad de cola por consumidor
    → para trabajadores asíncronos              clase 210
  concurrencia en vuelo
    → la mejor medida de saturación real

  y la CPU, solo si el servicio de verdad calcula
```

El retraso de escalado, que es lo que hace que llegue tarde:

```
detección de la métrica         30-120 s
decisión y disparo              30-60 s
arranque de la tarea            20-90 s (descarga + arranque)
```

```
paso de la comprobación         10-60 s
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
TOTAL típico                    1,5 a 5 minutos

→ un pico que sube en 60 segundos NO lo cubre el escalado
→ hay que llegar al pico con capacidad ya presente
```

Y de ahí salen tres decisiones:

```
1  MARGEN QUE CUBRA EL RETRASO
   capacidad base = pico previsible × (1 + tasa de subida ×
   retraso)
   → o dicho simple: no ir al límite

2  ESCALADO PROGRAMADO para lo previsible
   la campaña empieza a las 11:00 → subir a las 10:30
   → es más fiable que reaccionar

3  SUBIR RÁPIDO, BAJAR DESPACIO
   escalar hacia arriba con umbral bajo y sin espera
   escalar hacia abajo con espera de varios minutos
   → evita oscilar, y bajar de más cuesta más que subir de
     más
```

El dimensionado de CPU y memoria, que se copia y debería medirse:

```
lo que ocurre al copiar el valor de otro servicio
  memoria de más   se paga y no se usa
  memoria de menos   el proceso muere sin aviso claro
  CPU de menos     latencia alta con «CPU al 100 %», que en
                   contenedores puede ser estrangulamiento
                   por cuota, no falta real de máquina

cómo se decide
  ejecutar la carga real y observar uso de CPU, memoria
  máxima y estrangulamiento
  fijar memoria por encima del pico observado con margen
  y la CPU según la latencia objetivo, no según el uso medio
```

Y la lista de comprobación de la clase:

- ■ las etiquetas del registro son inmutables
- ■ el despliegue referencia la huella, no la etiqueta
- ■ hay exploración al subir y continua
- ■ el registro tiene reglas de caducidad
- ■ la imagen es mínima y no contiene secretos
- ■ el rol de ejecución y el de la tarea están separados
- ■ la elección de capacidad se justificó con utilización y coste
- ■ hay puntos separados de vivo y de listo
- ■ el periodo de gracia supera el arranque medido
- ■ el drenaje supera el plazo más largo
- ■ la aplicación atiende la señal de terminación
- ■ mínimo sano 100 % y máximo 200 %
- ■ el circuito de despliegue está activado
- ■ la señal de escalado no es la CPU salvo que corresponda
- ■ el margen cubre el retraso de escalado, medido
- ■ hay escalado programado para lo previsible
- ■ CPU y memoria se fijaron midiendo

Y el cierre que enlaza con la clase siguiente: cuando los contenedores dejan de ser unos pocos servicios y pasan a ser una plataforma con muchos equipos, la orquestación gestionada se queda corta y aparece Kubernetes con sus propios problemas. Es la materia de la clase 213.

Ejemplo trabajado

CloudShop mueve tres servicios a contenedores. Lo que sigue es el despliegue que cortaba peticiones, el escalado que llegaba tarde a cada campaña, y la comparación de coste que cambió la decisión de capacidad.

Problema 1 · Cada despliegue cortaba peticiones.

```
síntoma   en cada despliegue, entre 200 y 400 peticiones
          fallaban con error de conexión
          duraba unos 40 segundos
```

```
          se había normalizado: «se despliega de madrugada»

  diagnóstico, parámetro a parámetro
    drenaje de destino                  30 s (por defecto)
    plazo más largo de petición         45 s (exportaciones)
    → las peticiones de más de 30 s se cortaban

    parada elegante
    → la aplicación NO atendía la señal de terminación
    → se la mataba al agotar el plazo, con peticiones vivas

    porcentaje mínimo sano              50 %
    → durante el despliegue el servicio operaba con la mitad
      de tareas
    → y en hora punta eso bastaba para saturar el resto

    periodo de gracia                   30 s
    arranque real medido                52 s
    → las tareas nuevas se mataban por «no sanas» antes de
      terminar de arrancar
    → el servicio entraba en bucle y el despliegue tardaba
      12 minutos

  corrección
    drenaje                             60 s
    plazo de parada                     70 s
    parada elegante implementada: deja de aceptar, termina lo
      que tiene, cierra
    mínimo sano                         100 %
    máximo                              200 %
    periodo de gracia                   120 s
    circuito de despliegue              activado

  resultado
    peticiones cortadas por despliegue     300 → 0
    duración del despliegue             12 min → 3 min
    despliegues en horario laboral         no → sí
```

Problema 2 · El escalado llegaba tarde.

```
  configuración inicial
    señal          CPU media > 70 %
    mínimo         4 tareas
    máximo         40

  lo que pasaba en cada campaña
    11:00  arranca la promoción; el tráfico ×18 en 90 s
    11:00  latencia p99 sube de 180 ms a 4.100 ms
    11:00  CPU de las tareas: 38 %       ← no dispara nada
    11:03  la latencia dispara alertas
    11:04  alguien escala a mano
    11:09  capacidad suficiente

    9 minutos de degradación en cada campaña

  diagnóstico
    el servicio esperaba a la base y al servicio de precios
    → la CPU nunca subía
    → el recurso saturado era la concurrencia y las
      conexiones                                  clase 186

  corrección
    1  SEÑAL
       peticiones por tarea, del balanceador
       umbral fijado con prueba de carga: el codo estaba en
       240 peticiones/min por tarea → objetivo 160

    2  RETRASO MEDIDO
       detección                        60 s
       decisión                         40 s
       arranque + descarga de imagen    52 s
       comprobación de salud            25 s
       ■■■
```

```
      total                          3 min 17 s

      y la imagen se redujo de 740 MB a 110 MB
      → arranque 52 s → 21 s
      → total 2 min 46 s

  3  MARGEN
     el tráfico sube ×18 en 90 s
     → el escalado no puede cubrirlo
     → capacidad base subida para absorber los primeros
       3 minutos

  4  ESCALADO PROGRAMADO
     las campañas tienen hora conocida
     → subir a 26 tareas a las 10:30
     → y volver a la política automática a las 13:00

  5  SUBIR RÁPIDO, BAJAR DESPACIO
     subida sin espera; bajada con 10 min de espera

resultado en la campaña siguiente
  p99 máximo                      4.100 ms → 310 ms
  minutos de degradación               9 → 0
  intervenciones manuales               1 → 0
  coste del escalado programado    +14 €/campaña
```

La decisión de capacidad, revisada con datos.

```
se empezó con capacidad gestionada; a los 5 meses se
planteó pasar a máquinas propias «porque es más barato»

los datos
  coste actual con capacidad gestionada     3.180 €/mes
  recursos pedidos                          124 vCPU · 248 GB
  utilización media real                    31 %

  estimación con máquinas propias
    instancias necesarias con empaquetado del 65 %
      → 9 instancias grandes            1.940 €/mes
    con compromiso de 1 año             1.360 €/mes
    parcheado, escalado del conjunto,
      vigilancia                        ~2,5 días-persona/mes

  y el cálculo honesto
    ahorro bruto                          1.820 €/mes
    coste del trabajo (2,5 días × ~450 €) 1.125 €/mes
    ahorro neto                             695 €/mes

decisión
  NO migrar todavía
  motivo   695 €/mes no compensa añadir un conjunto de
           instancias que parchear a un equipo de 6
                                                    ley 23
  qué la reabriría
           si el gasto supera 6.000 €/mes
           o si aparece una carga con GPU o con requisitos
           de anfitrión                          clase 190

y lo que sí se hizo, que dio más
  ajustar los tamaños pedidos, que estaban copiados
    servicio A   2 vCPU / 4 GB → 0,5 vCPU / 2 GB
    servicio B   2 vCPU / 4 GB → 1 vCPU / 2 GB
    servicio C   4 vCPU / 8 GB → 2 vCPU / 4 GB
  utilización media                    31 % → 62 %
  coste                          3.180 € → 1.710 €
```

Y el detalle de cómo se fijaron esos tamaños:

```
se ejecutó la carga real durante una semana midiendo
  uso de CPU p95, memoria máxima y estrangulamiento por
  cuota

  servicio A   CPU p95 0,21 vCPU · memoria máx 1,3 GB
               estrangulamiento 0 %
```

```
  servicio C   CPU p95 1,7 vCPU · memoria máx 3,1 GB
               estrangulamiento 4 % con 2 vCPU
               → se dejó en 2 vCPU: el 4 % ocurría en el
                 arranque, no en régimen
```

La higiene del registro, que nadie había mirado:

```
imágenes almacenadas                              8.410
  con etiqueta                                    1.240
  SIN etiqueta (capas huérfanas)                  7.170
tamaño                                             2,1 TB
coste                                            210 €/mes
etiquetas mutables                                  sí
exploración                                      al subir
exploración continua                                no

tras aplicar reglas
  conservar las 10 últimas por rama, borrar el resto
  borrar sin etiqueta a los 7 días
  etiquetas inmutables, con huella del contenido
  exploración continua activada

  imágenes                                 8.410 → 310
  tamaño                                   2,1 TB → 74 GB
  coste                                    210 € → 8 €

y la exploración continua encontró
  3 imágenes en producción con vulnerabilidades graves
  publicadas DESPUÉS de su construcción
  → la más antigua llevaba 4 meses desplegada    ley 13
```

El resultado:

```
peticiones cortadas por despliegue          300          0
duración del despliegue                  12 min       3 min
minutos de degradación por campaña            9          0
utilización de recursos                    31 %        62 %
coste de cómputo                        3.180 €     1.710 €
coste del registro                        210 €         8 €
imágenes con vulnerabilidad grave en
  producción                                  3          0
```

La lección que esta clase deja: el despliegue cortaba peticiones por cuatro parámetros por defecto a la vez —drenaje corto, sin parada elegante, mínimo sano al 50 % y gracia menor que el arranque—, y ninguno era un error de diseño. El escalado llegaba tarde porque la señal era la CPU en un servicio que espera a la red, y la mejora mayor no fue cambiar la señal sino reducir la imagen de 740 a 110 MB. Y el ahorro que se buscaba en el tipo de capacidad estaba en realidad en los tamaños pedidos, que estaban copiados de otro servicio.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/212-ecr-ecs-fargate-alb-y-autoscaling/lab.py
```

El laboratorio selecciona el motor de práctica container y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa aws-ecs-platform en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una imagen mínima, escaneada y ejecutada sin privilegios. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye aws-ecs-platform para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

■■ Errores frecuentes

Síntoma	Causa probable	Corrección
Cada despliegue corta peticiones en curso	Drenaje menor que el plazo más largo y la aplicación no atiende la señal de terminación	Aumenta el drenaje por encima de la petición más larga e implementa la parada elegante.
Las tareas nuevas entran en bucle de reinicio	El periodo de gracia es menor que el tiempo real de arranque	Mide el arranque y fija el periodo de gracia con margen sobre él.
El servicio pierde capacidad durante los despliegues	El porcentaje mínimo sano permite bajar del cien por cien	Fija mínimo sano 100 % y máximo 200 %, y activa el circuito de despliegue.
El escalado no reacciona aunque la latencia se dispare	La señal es la CPU y el servicio espera a la red, no calcula	Escala por peticiones por tarea, latencia o profundidad de cola, con el umbral fijado en una prueba de carga.
El escalado llega tarde a cada pico	El retraso de detección, decisión y arranque no está cubierto por margen	Mide el retraso, reduce el tamaño de la imagen, deja margen de capacidad y programa la subida para los picos previsibles.
Una imagen desplegada hace meses tiene vulnerabilidades conocidas	Solo hay exploración al subir, y las etiquetas son mutables	Activa exploración continua, usa etiquetas inmutables con huella y aplica reglas de caducidad al registro.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Por qué las etiquetas mutables impiden revertir con seguridad?
2. ¿Qué condiciones deben cumplirse para que las máquinas propias salgan realmente más baratas?
3. ¿Cuál es el orden correcto de un despliegue sin cortar peticiones?
4. ¿Por qué la CPU suele ser mala señal de escalado y cuáles funcionan mejor?
5. ¿Qué compone el retraso de escalado y cómo se compensa?

Referencias

- AWS (2025). Amazon ECR: image tag mutability and lifecycle policies — inmutabilidad de etiquetas y caducidad de imágenes. <https://docs.aws.amazon.com/AmazonECR/latest/userguide/image-tag-mutability.html>

- AWS (2025). Amazon ECS deployment types and circuit breaker — despliegue con corte automático ante fallo. <https://docs.aws.amazon.com/AmazonECS/latest/developerguide/deployment-type-ecs.html>
- AWS (2025). Application Load Balancer: deregistration delay — retirada ordenada de destinos del balanceador. <https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-target-groups.html>
- AWS (2025). Service auto scaling for Amazon ECS — políticas de escalado automático del servicio. <https://docs.aws.amazon.com/AmazonECS/latest/developerguide/service-auto-scaling.html>
- AWS (2025). AWS Fargate pricing and task sizing — dimensionar la tarea junto con su coste. <https://aws.amazon.com/fargate/pricing/>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 17 en PDF · Recorrido de AWS en PDF · Manual integral

Anterior	Índice	Siguiente
← 211 · CloudWatch, X-Ray y observabilidad como código	Parte 17 · Programa	213 · EKS, IRSA, GitOps y operación de clúster →

Evaluación — 212 ECR, ECS Fargate, ALB y autoscaling

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega aws-ecs-platform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

213 — EKS, IRSA, GitOps y operación de clúster

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: kubernetes · Estado: EXECUTABLECORE

Propósito

Operar Kubernetes gestionado en AWS sabiendo qué se gana y qué se paga, porque es la decisión de plataforma que más capacidad de equipo consume. La clase cubre la identidad de las cargas sin credenciales, la reconciliación desde el repositorio, los tres asuntos que causan casi todos los incidentes —peticiones y límites mal puestos, actualizaciones de versión y complementos— y la pregunta que hay que contestar antes: ¿hace falta Kubernetes, o basta con lo de la clase 212?

Resultados de aprendizaje

Al finalizar podrás:

1. Decidir si Kubernetes compensa frente a orquestación más simple.
2. Dar identidad a las cargas sin credenciales estáticas.
3. Operar el clúster desde el repositorio, con reconciliación.
4. Configurar peticiones y límites sin provocar expulsiones ni desperdicio.
5. Planificar las actualizaciones de versión y de complementos.

Conceptos centrales

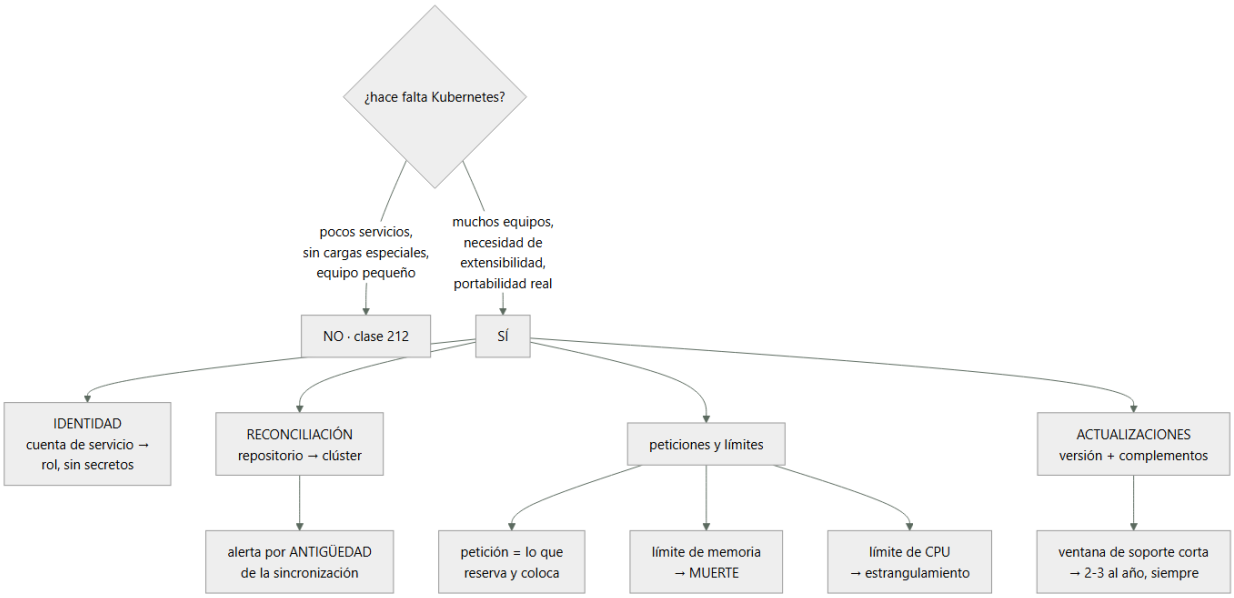
Concepto	Comprensión verificable
identidad de carga (IRSA)	Mecanismo por el que una cuenta de servicio del clúster obtiene credenciales temporales de AWS, sin secretos.
reconciliación	Bucle que compara el estado declarado en el repositorio con el real y corrige la diferencia.
petición de recursos	Lo que la carga declara necesitar. Determina dónde se coloca y la capacidad total requerida.
límite de recursos	Techo que la carga no puede superar. Con memoria significa muerte; con CPU, estrangulamiento.
complemento	Componente añadido al clúster (red, DNS, métricas, controladores). Cada uno es una dependencia de versión.
presupuesto de interrupción	Declaración de cuántas réplicas pueden estar caídas a la vez durante mantenimiento.

Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. ¿Hace falta Kubernetes?

Es la pregunta que hay que contestar primero, y la respuesta honesta suele decepcionar.

```
LO QUE KUBERNETES DA DE VERDAD
  un modelo declarativo extensible: se pueden definir
    recursos propios y controladores
  un ecosistema enorme de componentes ya hechos
  la misma interfaz en cualquier nube y en local
  planificación fina: afinidades, tolerancias, prioridades
  y una comunidad con respuestas para casi todo

LO QUE CUESTA
  actualizaciones frecuentes y obligatorias
  complementos que hay que mantener y que se rompen entre sí
  una capa más de red, de identidad y de diagnóstico
  y personas que sepan operarlo

  → entre 1 y 3 personas dedicadas para una plataforma
    seria                                          ley 23
```

Y el criterio de decisión:

```
NO HACE FALTA SI
  son unos pocos servicios web y trabajadores
  el equipo es pequeño
  no hay cargas con requisitos especiales
  → la orquestación gestionada de la clase 212 hace lo
    mismo con una fracción del trabajo

SÍ COMPENSA SI
  hay muchos equipos que necesitan autonomía sobre una base
    común                                       clase 171
  hacen falta operadores y recursos propios
  hay cargas heterogéneas: GPU, procesos largos, trabajos
    por lotes, bases de datos con operador
  la portabilidad entre nubes es un requisito REAL y no
    declarativo                                 clase 158
```

Y una advertencia sobre la portabilidad, que es el argumento más usado:

```
el manifiesto es portable; el resto casi nunca
  balanceadores, almacenamiento, identidad, registro,
  certificados y controladores son específicos
→ la portabilidad real exige diseñarla y probarla, no viene
  de usar Kubernetes                            clase 158
```

Y la decisión sobre el modo de ejecución de los nodos:

```
GRUPOS DE NODOS GESTIONADOS
  instancias que se parchean con ayuda del proveedor
  + control, coste por unidad menor, cualquier carga
  − hay que planificar reemplazos y drenajes

CAPACIDAD SIN NODOS
  un pod por unidad de capacidad, sin instancias
  + nada que parchear
  − más caro por unidad, con límites de funciones

  → mezcla habitual: sin nodos para lo variable y grupos
    gestionados para la base                    clase 212
```

2. Identidad sin credenciales

Dar a una carga acceso a servicios de AWS con una clave montada en un secreto es el patrón que hay que eliminar, y el mecanismo es el mismo de la clase 206.

```
CÓMO FUNCIONA
  el clúster expone un emisor de testigos
  se declara como proveedor de identidad en AWS
  la cuenta de servicio del pod se anota con un rol
  el pod recibe un testigo montado, con vida corta
  y lo intercambia por credenciales temporales

LO QUE SE GANA
  sin claves en secretos ni en variables
  credenciales de minutos
  y permisos por CARGA, no por nodo
```

Y el patrón que sustituye, que sigue siendo frecuente:

```
✗ EL ROL DEL NODO
  dar permisos al rol de la instancia
  → TODOS los pods del nodo los tienen
  → incluidos los de otro equipo que aterricen ahí
  → alcance enorme desde cualquier pod comprometido
                                              clase 189

  y además hay que bloquear el acceso de los pods al
  servicio de metadatos del nodo, o pueden pedir esas
  credenciales aunque tengan la suya
```

Y los errores de configuración de la identidad de carga:

```
la condición del rol debe atar
  el emisor del clúster concreto
  el espacio de nombres
  y el NOMBRE de la cuenta de servicio

✗ condición solo por emisor
  → cualquier cuenta de servicio de cualquier espacio de
    nombres puede asumir el rol
  → es el mismo fallo de la clase 206, con otra forma
```

Y una comprobación obligatoria:

```
desde un pod de otro espacio de nombres, intentar asumir
el rol
→ debe fallar                                   ley 22
```

Los secretos, que en Kubernetes son un objeto sin cifrar por defecto:

```
un objeto de tipo secreto está codificado, no cifrado
→ hay que activar el cifrado en reposo del almacén del
  plano de control
→ y para secretos de verdad, obtenerlos del gestor externo
  en tiempo de ejecución, no guardarlos en el clúster
→ y nunca en un manifiesto del repositorio      clase 106
```

3. Operar desde el repositorio

Aplicar manifiestos a mano desde un portátil produce clústeres que nadie sabe reconstruir. La reconciliación resuelve eso.

```
EL MODELO
  el estado deseado vive en un repositorio
  un controlador dentro del clúster lo compara con el real
  y corrige la diferencia continuamente

LO QUE APORTA sobre desplegar desde la canalización
  el clúster no necesita credenciales de la canalización:
    tira, no le empujan
  la deriva se corrige sola
  el repositorio es la verdad, y se puede reconstruir
  y el historial de cambios es el del repositorio
                                                clase 103
```

Y la alerta imprescindible:

```
«la sincronización lleva N minutos sin completarse»
  o «el estado real difiere del declarado desde hace N»

→ un bucle de reconciliación parado no genera ningún error:
  simplemente deja de aplicar cambios              ley 13
→ y en la clase 179 esa prueba negativa falló porque la
  alerta iba a un canal sin nadie
```

La separación por equipos, que es la razón de usar Kubernetes en muchos casos:

```
espacio de nombres por equipo o por servicio
cuotas de recursos por espacio
  → un equipo no puede consumir la capacidad de los demás
rangos de límites por defecto
políticas de admisión
  → qué imágenes se admiten (firmadas, del registro propio)
  → qué no se permite: privilegios, red del anfitrión,
    montajes sensibles
  → y las etiquetas obligatorias                clase 142
políticas de red
  → por defecto, denegar; y abrir lo declarado  clase 201
```

Y la advertencia sobre la admisión:

```
un control de admisión que rechaza sin explicar por qué se
rodea: la gente pide excepciones o crea recursos por otro
camino                                            ley 16
→ el mensaje debe decir qué falta y cómo arreglarlo
→ y el carril fácil —la plantilla de servicio— debe cumplir
  solo                                          clase 190
```

4. Peticiones, límites y actualizaciones

Peticiones y límites son la causa de la mayoría de los incidentes de plataforma, y se ponen mal en las dos direcciones.

```
PETICIÓN
  lo que el planificador RESERVA
  determina en qué nodo cabe y cuánta capacidad total hace
  falta

LÍMITE
  el techo
  memoria por encima del límite → el proceso MUERE
  CPU por encima del límite     → estrangulamiento, latencia

LOS CUATRO ERRORES
  1  sin petición
     el pod se coloca en cualquier sitio y compite
     → y es el primero en ser expulsado cuando falta
       memoria
  2  petición mucho mayor que el uso
     capacidad reservada y desperdiciada
     → el conjunto crece sin necesidad, y la factura con él
  3  límite de memoria ajustado al uso medio
     un pico normal mata el proceso
```

```
     → y el síntoma es «se reinicia solo y no sé por qué»
  4  límite de CPU muy bajo
     estrangulamiento constante: latencia alta con el nodo
     ocioso                                    clase 186
```

Y las reglas que funcionan:

```
petición de memoria ≈ uso p95 observado
límite de memoria   ≈ petición × 1,5 a 2
petición de CPU     ≈ uso p95 observado
límite de CPU       a menudo, NINGUNO
  → dejar que use el hueco libre cuando lo hay
  → salvo que haga falta aislar por contrato

y medir siempre antes: la carga real durante días
```

Y tres mecanismos que evitan sorpresas:

```
PRESUPUESTO DE INTERRUPCIÓN
  «al menos 2 réplicas siempre disponibles»
  → el drenaje de un nodo lo respeta
  → sin esto, un mantenimiento puede dejar el servicio en
    cero réplicas durante segundos

CLASES DE PRIORIDAD
  lo crítico expulsa a lo que no lo es cuando falta espacio

REPARTO ENTRE ZONAS
  restricciones que obligan a repartir las réplicas
  → sin ellas, las 3 réplicas pueden acabar en el mismo
    nodo o en la misma zona                  clase 185
```

Las actualizaciones, que son el trabajo continuo de esta plataforma:

```
LA VENTANA DE SOPORTE ES CORTA
  cada versión se soporta poco más de un año
  → hay que actualizar 2 o 3 veces al año, siempre
  → no es opcional ni aplazable                  ley 25

QUÉ HAY QUE COMPROBAR EN CADA ACTUALIZACIÓN
  interfaces retiradas: manifiestos que dejan de ser
    válidos
  compatibilidad de cada complemento con la versión nueva
  compatibilidad del cliente y de las herramientas
  y de los operadores instalados

EL ORDEN
  1  revisar avisos de retirada y corregir manifiestos
  2  actualizar en un clúster de prueba idéntico
  3  actualizar el plano de control
  4  actualizar complementos
  5  reemplazar nodos por lotes, respetando el presupuesto
     de interrupción
```

Y lo que se olvida:

```
los COMPLEMENTOS tienen su propio calendario
  red, DNS interno, controlador de balanceador, métricas,
  autoescalado, controlador de certificados
→ cada uno es una dependencia con su matriz de
  compatibilidad
→ y un clúster con quince complementos tiene quince cosas
  que pueden bloquear una actualización          ley 23
```

Y la lista de comprobación de la clase:

- ■ la decisión de usar Kubernetes está justificada
- ■ las cargas obtienen credenciales por identidad de carga
- ■ el rol del nodo no tiene permisos de aplicación
- ■ los pods no pueden alcanzar el servicio de metadatos
- ■ las condiciones del rol atan espacio de nombres y cuenta de servicio
- ■ el clúster se reconcilia desde el repositorio
- ■ hay alerta por antigüedad de la sincronización
- ■ hay cuotas por espacio de nombres
- ■ hay política de admisión con mensajes que explican

- [ ] hay política de red con denegación por defecto
- [ ] todas las cargas tienen peticiones, medidas
- [ ] los límites de memoria dan margen sobre el pico
- [ ] hay presupuestos de interrupción
- [ ] las réplicas se reparten entre zonas
- [ ] el calendario de actualización está planificado
- [ ] la matriz de compatibilidad de complementos está escrita

Y el cierre que enlaza con la clase siguiente: todo lo montado en esta parte genera factura, y a estas alturas ya han aparecido tres partidas que nadie había estimado. Presupuestos, análisis de coste, etiquetado y automatización es la materia de la clase 214.

Ejemplo trabajado

CloudShop monta una plataforma de contenedores para siete equipos. Lo que sigue es la decisión de adoptarlo, los tres incidentes del primer año —todos de peticiones y límites o de actualizaciones— y lo que costó de verdad.

La decisión, con el método de la clase 152.

```
situación   14 servicios en orquestación gestionada,
            funcionando bien

lo que se pedía
  7 equipos quieren autonomía para desplegar sin
    coordinarse
  2 cargas con GPU para el equipo de datos     clase 175
  trabajos por lotes con planificación por prioridad
  un operador de base de datos que el equipo de datos ya usa
  y «portabilidad», que al preguntar resultó no ser un
    requisito real                              clase 158

lo que ya estaba resuelto
  despliegue, escalado y balanceo             clase 212

veredicto
  3 de las 5 necesidades exigen Kubernetes o lo facilitan
  mucho
  y hay 2 personas que pueden dedicarse a la plataforma
  → SE ADOPTA, y se migran solo las cargas que lo
    necesitan

y la decisión que se registró
  los 14 servicios web SIGUEN en orquestación gestionada
  motivo   funcionan, y moverlos añade trabajo sin
           beneficio
  qué la reabriría   si la plataforma demuestra reducir el
           tiempo de despliegue de un servicio nuevo por
           debajo del actual                    clase 190
```

Incidente 1 · «Los pods se reinician solos», mes 2.

```
síntoma   el servicio de informes se reiniciaba varias
          veces al día, sin errores en sus registros
          la última línea era siempre distinta

diagnóstico
  el pod moría por exceso de memoria
  límite de memoria                    512 Mi
  uso medio                            410 Mi
  uso p99                              780 Mi (al generar
                                       informes grandes)
  → el límite estaba puesto sobre el uso MEDIO

y por qué no había errores
  el proceso se mata sin darle oportunidad de registrar
  nada
  → el único rastro era el motivo de terminación del
    contenedor, que nadie miraba                ley 15

corrección
  petición de memoria      = p95 observado = 620 Mi
  límite                   = petición × 1,8 = 1.100 Mi
  y alerta nueva: «contenedores terminados por memoria > 0»
```

```
reinicios al día                          6 → 0
```

Incidente 2 · «Latencia alta con los nodos al 30 %», mes 4.

```
síntoma   el servicio de catálogo tenía p99 de 2,4 s
          los nodos estaban al 30 % de CPU
          escalar el número de réplicas no mejoraba nada

diagnóstico
  límite de CPU del contenedor          200 milicores
  el contenedor era estrangulado el 74 % del tiempo
  → la métrica de estrangulamiento existía y no estaba en
    ningún panel                                clase 211

  la confusión
    el panel mostraba «CPU del nodo: 30 %»
    y la conclusión era «sobra capacidad»
    → pero el contenedor tenía su propio techo

corrección
  petición de CPU          = p95 = 320 milicores
  límite de CPU            eliminado
  → el contenedor usa el hueco libre del nodo cuando lo hay

p99                                     2,4 s → 210 ms
réplicas necesarias                        12 → 6
coste                                   -840 €/mes
```

Y la observación:

```
quitar el límite de CPU redujo a la mitad las réplicas
necesarias
→ el límite estaba causando el problema que se intentaba
  resolver escalando
```

Incidente 3 · La actualización de versión, mes 8.

```
situación   la versión del clúster entraba en fin de soporte
            en 6 semanas
            se planificó la actualización para un martes

lo que salió mal
  el clúster de prueba se había creado hacía 4 meses y no
  era idéntico: le faltaban 3 complementos que producción
  sí tenía
  → la prueba pasó sin problemas

  en producción
    el controlador del balanceador no era compatible con la
    versión nueva
    → los servicios nuevos dejaron de obtener balanceador
    → los existentes siguieron funcionando
    tiempo hasta detectarlo                    3 días
    → porque no había despliegues de servicios nuevos hasta
      entonces                                   ley 13

    y 41 manifiestos usaban una interfaz retirada
    → la reconciliación empezó a fallar en silencio para
      esos recursos
    → alerta de antigüedad de sincronización: NO existía

correcciones
  el clúster de prueba se genera del mismo repositorio que
  producción y se recrea antes de cada actualización
  matriz de compatibilidad de los 12 complementos, escrita
    y comprobada antes de actualizar
  revisión automática de interfaces retiradas en la
    canalización                               clase 190
  alerta de antigüedad de sincronización, con destino a
    guardia
  calendario: 3 actualizaciones al año, en fecha fijada

la siguiente actualización, mes 13
  duración                                     4 h
  incidencias                                  0
```

```
  manifiestos corregidos antes                    17
  complementos que hubo que subir primero          4
```

La identidad, montada bien desde el principio:

```
identidad de carga por cuenta de servicio
condición del rol atada a emisor + espacio de nombres +
  nombre de la cuenta
acceso de los pods al servicio de metadatos del nodo:
  bloqueado
rol del nodo: solo lo imprescindible para unirse al clúster

prueba negativa
  desde un pod del espacio de nombres «equipo-datos»,
  intentar asumir el rol de «equipo-pagos»
  → denegado                                       ✓
  desde un pod, leer las credenciales del nodo
  → bloqueado                                      ✓
```

Lo que costó de verdad, medido al año:

```
coste de cómputo del clúster                4.120 €/mes
coste del plano de control                     68 €/mes

y el coste que no aparece en la factura
  actualizaciones (3 al año, ~3 días cada una)  9 días
  mantenimiento de complementos                18 días
  soporte a los 7 equipos                      44 días
  incidentes de plataforma                     11 días
  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  total                                        82 días/año
                                      ≈ 0,4 personas

y lo que se esperaba   «2 personas dedicadas»
→ resultó menos de lo previsto, porque los 14 servicios web
  se quedaron fuera y la reconciliación evitó mucho soporte
```

Y el balance que el equipo escribió:

```
lo que la plataforma dio
  los 7 equipos despliegan sin coordinarse
  las cargas con GPU y los trabajos por lotes funcionan
  el operador de base de datos del equipo de datos también

lo que NO dio
  portabilidad: 4 de los 12 complementos son específicos
  del proveedor, y los balanceadores y el almacenamiento
  también                                       clase 158
  → la portabilidad se había citado como motivo y no se
    materializó, lo cual estaba previsto y registrado
```

El resultado:

```
equipos que despliegan sin coordinarse         0           7
reinicios por memoria                       6/día          0
p99 del catálogo                           2,4 s       210 ms
réplicas del catálogo                         12           6
incidentes por actualización                   1           0
servicios web migrados innecesariamente        0           0
capacidad de equipo consumida                n/d      0,4 pers.
```

La lección que esta clase deja: los dos primeros incidentes fueron peticiones y límites mal puestos, y en el segundo el límite de CPU era la causa del problema que se estaba intentando resolver añadiendo réplicas. El tercero fue una actualización cuya prueba pasó porque el clúster de prueba no era idéntico al de producción, y sus consecuencias tardaron tres días en verse porque nada las hacía visibles. Y la decisión más rentable de todo el proyecto fue no migrar los catorce servicios que ya funcionaban.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/213-eks-irsa-gitops-y-operacion-de-cluster/lab.py
```

El laboratorio selecciona el motor de práctica kubernetes y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa aws-eks-gitops en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es manifiestos declarativos con estado observado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye aws-eks-gitops para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

■■ Errores frecuentes

Síntoma	Causa probable	Corrección
Los pods se reinician sin dejar rastro en sus registros	El límite de memoria está ajustado al uso medio y un pico mata el proceso	Fija la petición en el p95 observado y el límite entre 1,5 y 2 veces la petición; alerta sobre terminaciones por memoria.
Latencia alta con los nodos ociosos y escalar no ayuda	Límite de CPU bajo que provoca estrangulamiento del contenedor	Fija la petición de CPU en el p95 y considera no poner límite; vigila la métrica de estrangulamiento.
Cualquier pod puede usar los permisos de otro equipo	Los permisos están en el rol del nodo o la condición del rol solo ata al emisor	Usa identidad de carga con condición por espacio de nombres y cuenta de servicio, y bloquea el acceso al servicio de metadatos.
Los cambios del repositorio dejan de aplicarse sin que nadie lo note	El bucle de reconciliación falla en silencio	Alerta por antigüedad de la sincronización y por diferencia persistente entre lo declarado y lo real.
Una actualización rompe producción pese a haber pasado en pruebas	El clúster de prueba no era idéntico y faltaban complementos	Genera el clúster de prueba del mismo repositorio, mantén una matriz de compatibilidad de complementos y revisa las interfaces retiradas antes.
Un mantenimiento deja un servicio sin réplicas disponibles	No hay presupuesto de interrupción ni reparto entre zonas	Declara presupuestos de interrupción y restricciones de reparto por zona.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿En qué casos no compensa Kubernetes frente a orquestación gestionada más simple?
2. ¿Por qué dar permisos al rol del nodo es un problema de alcance?
3. ¿Qué diferencia hay entre petición y límite, y qué ocurre al superar cada uno?
4. ¿Qué alerta detecta que la reconciliación ha dejado de funcionar?
5. ¿Qué hay que comprobar antes de actualizar la versión del clúster?

Referencias

• AWS (2025). IAM roles for service accounts (IRSA) — identidad por cuenta de servicio sin claves estáticas. <https://docs.aws.amazon.com/eks/latest/userguide/iam-roles-for-service-accounts.html>

• AWS (2025). Amazon EKS Kubernetes version support and upgrade — ventana de soporte y procedimiento de actualización. <https://docs.aws.amazon.com/eks/latest/userguide/kubernetes-versions.html>

• Kubernetes (2025). Managing resources for containers: requests and limits — peticiones y límites como contrato de recursos. <https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/>

• Argo CD (2025). GitOps and application health — estado de salud de la aplicación en el flujo GitOps. <https://argo-cd.readthedocs.io/en/stable/>

• AWS (2025). EKS Best Practices Guide — prácticas recomendadas de operación del clúster gestionado. <https://docs.aws.amazon.com/eks/latest/best-practices/introduction.html>

• Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 17 en PDF · Recorrido de AWS en PDF · Manual integral

Anterior	Índice	Siguiente
← 212 · ECR, ECS Fargate, ALB y autoscaling	Parte 17 · Programa	214 · Budgets, Cost Explorer, etiquetado y FinOps automatizado →

Evaluación — 213 EKS, IRSA, GitOps y operación de clúster

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega aws-eks-gitops con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

214 — Budgets, Cost Explorer, etiquetado y FinOps automatizado

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: finops · Estado: EXECUTABLECORE

Propósito

Controlar el coste de lo que se ha montado en esta parte, donde ya han aparecido tres partidas que nadie había estimado. La clase fija el etiquetado como requisito de creación y no como limpieza posterior, explica cómo se lee una factura para encontrar el dinero de verdad, y automatiza lo que funciona: presupuestos con acción, detección de anomalías y retirada de lo que nadie usa. Con la advertencia de siempre: la policía del coste se rodea; el carril fácil, no.

Resultados de aprendizaje

Al finalizar podrás:

1. Imponer etiquetado en la creación, con barrera y no con auditoría.
2. Leer la factura para encontrar las partidas que dominan.
3. Configurar presupuestos y detección de anomalías que actúen.
4. Atribuir el coste por servicio y por unidad de negocio.
5. Retirar lo que no se usa, de forma automática y segura.

Conceptos centrales

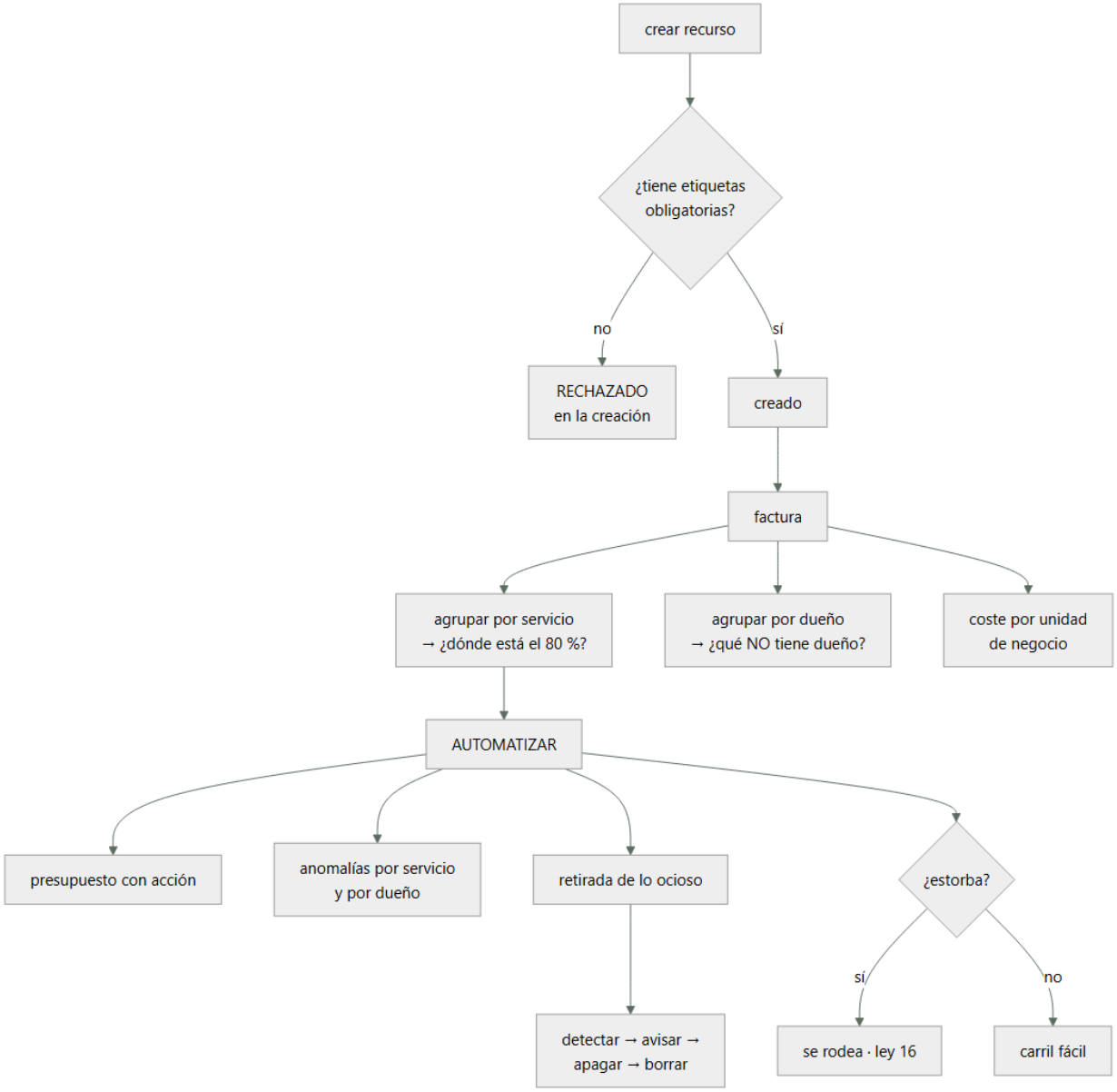
Concepto	Comprensión verificable
etiqueta de asignación	Etiqueta activada para que aparezca como dimensión en el informe de costes. Sin activar, no sirve.
coste por unidad de negocio	Coste dividido entre pedidos, usuarios o transacciones. Es la única cifra comparable en el tiempo.
presupuesto con acción	Presupuesto que además de avisar ejecuta algo: notificar, restringir o apagar.
detección de anomalías	Modelo que compara el gasto con el patrón previo y avisa de desviaciones.
coste no atribuido	Gasto que no cae en ningún servicio ni dueño. Es la parte que nadie reduce.
recurso ocioso	Recurso creado, facturado y sin uso. La categoría más fácil de eliminar y la que más se repite.

Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Etiquetar en la creación, no después

El proyecto de etiquetado que empieza etiquetando lo que ya existe fracasa siempre: mientras se etiqueta lo viejo, se crea lo nuevo sin etiquetas.

```
EL ORDEN CORRECTO
  1  decidir el conjunto MÍNIMO de etiquetas obligatorias
  2  impedir la creación sin ellas
  3  y solo entonces, etiquetar lo existente

→ al revés, nunca se termina                        ley 25
```

Y el conjunto mínimo, que debe ser corto:

```
dueño          el equipo, no una persona          ley 20
servicio       a qué sistema pertenece
entorno        producción, preproducción, desarrollo
centro de coste  a quién se le imputa

→ cuatro. Más de seis y nadie las pone bien
→ y con valores CERRADOS, no texto libre:
  «prod», «Prod», «producción» y «PRD» son cuatro grupos
  distintos en el informe
```

Cómo se impide la creación, por orden de fuerza:

```
1  BARRERA DE ORGANIZACIÓN
   deniega crear sin las etiquetas
   → ni un administrador de la cuenta puede saltárselo
                                                clase 169

2  POLÍTICA DE ETIQUETAS
   fija los valores permitidos por clave

3  FUNCIÓN DE APTITUD EN LA CANALIZACIÓN
   la plantilla que no las declara no despliega
                                                clase 190

4  LA PLANTILLA DE SERVICIO NUEVO LAS TRAE HECHAS
   → esto es lo que hace que casi nunca falle nada
                                                clase 171
```

Y la advertencia de la parte 14:

```
si poner las etiquetas cuesta trabajo, la gente creará los
recursos por otro camino, o pondrá valores basura para pasar
el control                                          ley 16
→ el carril fácil las pone solo; la barrera es la red de
  seguridad, no el mecanismo principal
```

Y lo que no se puede etiquetar, que hay que tratar aparte:

```
transferencia de datos entre zonas y regiones
coste compartido de servicios comunes
tráfico de salida sin recurso asociado
→ estos se reparten por regla, escrita y acordada
→ y hay que decir cuánto se reparte y con qué criterio
```

2. Leer la factura

La factura se mira mal: se abre el total, se dice «ha subido» y se cierra. Estas son las cinco vistas que encuentran el dinero.

```
1  POR SERVICIO, ordenado de mayor a menor
   → el 80 % del gasto suele estar en 3 o 4 líneas
   → y esas son las únicas que merece la pena optimizar

2  POR DUEÑO
   → y sobre todo: cuánto NO tiene dueño
   → el coste no atribuido no lo reduce nadie      ley 20

3  POR ENTORNO
   → si preproducción cuesta más del 25 % de producción,
     hay algo encendido que no debería

4  POR TIPO DE USO dentro del servicio grande
   no basta «almacenamiento: 4.100 €»
   hace falta: peticiones, almacenamiento por clase,
   transferencia, recuperaciones
   → el desglose es donde está la sorpresa

5  COSTE POR UNIDAD DE NEGOCIO
   coste / pedidos del mes
   → es la ÚNICA cifra comparable entre meses
   → sin ella, «el coste ha subido un 20 %» no dice si
     mejoró o empeoró                            clase 142
```

Y las partidas que este programa ha visto sorprender, ya en esta parte:

```
peticiones de la pasarela         > cómputo    clase 207
ingesta de registros              > cómputo    clase 211
escrituras de índices             > tabla      clase 208
transferencia entre zonas         casi siempre olvidada
series de métricas de alta
  cardinalidad                     1.410 €/mes  clase 211
invalidaciones de caché                         clase 205
capas huérfanas del registro de
  imágenes                         210 €/mes    clase 212
```

Y una técnica que ahorra tiempo:

```
no persigas el porcentaje: persigue el importe
  «este servicio subió un 400 %»  → eran 3 € y ahora son 15
  «este subió un 6 %»             → eran 12.000 €
→ ordena siempre por importe de la variación, no por
  porcentaje
```

Los descuentos por compromiso, con la parte que se hace mal:

```
se compromete SOLO la base estable, medida con meses de
histórico
  → comprometer el pico deja capacidad pagada sin usar
  → y comprometer a 3 años algo que cambiará en 1 es peor
    que no comprometer nada

y hay que vigilar la COBERTURA y la UTILIZACIÓN
  cobertura     qué proporción del uso está cubierta
  utilización   qué proporción de lo comprometido se usa
  → utilización por debajo del 95 % es dinero tirado
```

3. Automatizar lo que funciona

Los informes que alguien debe mirar dejan de mirarse. Lo que funciona es lo que actúa.

Presupuestos con acción:

```
POR CUENTA Y POR SERVICIO, no solo global
  → un presupuesto global se supera y nadie sabe por qué

CON UMBRALES ESCALONADOS
  50 %, 80 %, 100 % del previsto para la fecha
  y sobre PREVISIÓN, no solo sobre gasto acumulado
  → avisar al 100 % el día 28 no sirve de nada

CON ACCIONES REALES en entornos no productivos
  al 100 % en desarrollo → restringir la creación de
    recursos nuevos
  al 120 % → apagar lo que se pueda apagar
  → en producción, avisar; nunca apagar automáticamente
```

Detección de anomalías, que encuentra lo que el presupuesto no:

```
el presupuesto detecta que se gasta de más EN TOTAL
la anomalía detecta que un servicio concreto se ha salido
de su patrón, aunque el total esté bien

configurarla por servicio Y por dueño
umbral en importe absoluto, no en porcentaje
  → si no, avisa de cada servicio pequeño que se dobla
y con destino a alguien que actúe                clase 211
```

La retirada de lo ocioso, que es donde está el dinero fácil:

```
LO QUE SIEMPRE APARECE
  volúmenes desconectados de cualquier máquina
  copias antiguas de volúmenes, sin caducidad
  direcciones fijas reservadas y sin asociar
  balanceadores sin destinos
  instancias paradas cuyo disco se sigue pagando
  bases de datos de pruebas de hace un año
  entornos efímeros que no caducaron              ley 25
  puntos privados creados y no usados            clase 200
  registros sin caducidad                        clase 211
  imágenes sin etiqueta                          clase 212

EL PROCESO SEGURO, en cuatro pasos
  1  DETECTAR y listar, con dueño si lo hay
  2  AVISAR al dueño, con plazo
  3  APAGAR o desasociar, sin borrar, y esperar
  4  BORRAR si nadie se ha quejado

→ el paso 3 es el que evita el desastre: si algo se rompe,
  se vuelve a encender en segundos
```

Y dos automatismos que dan mucho por poco:

```
APAGADO POR HORARIO en entornos no productivos
  desarrollo y preproducción apagados por la noche y el
  fin de semana
  → 168 h/semana → 50 h/semana = 70 % menos
  → y el arranque debe ser automático, o la gente
    desactivará el apagado                          ley 16

CADUCIDAD EN LOS ENTORNOS EFÍMEROS
  nacen con fecha de destrucción
  → sin ella, quedan para siempre                    ley 25
```

4. Que no se convierta en policía

La parte 14 ya mostró qué pasa cuando el control de coste se plantea como vigilancia: se rodea.

```
LO QUE NO FUNCIONA
  el informe mensual de «los que más gastan»
  pedir justificación de cada recurso
  bloquear sin explicar
  → produce recursos creados por caminos raros y valores
    de etiqueta inventados                          ley 16

LO QUE SÍ
  que el equipo vea SU coste, en su panel, junto a sus
    demás señales                                  clase 211
  que la plantilla por defecto ya sea la barata
  que el coste por unidad de negocio sea el indicador, no
    el total
  → así el equipo que crece un 40 % no aparece como
    culpable
  y que la retirada de lo ocioso la haga la plataforma, no
    el equipo
```

Y la medida que dice si el sistema funciona:

```
PROPORCIÓN DE COSTE ATRIBUIDO
  ¿qué porcentaje del gasto tiene dueño identificable?
  → por debajo del 90 %, el resto de los esfuerzos son
    ruido
  → en la clase 179 pasó del 38 % al 96 %

y la segunda: COSTE POR UNIDAD DE NEGOCIO, su tendencia
```

Lo que hay que vigilar:

```
gasto diario frente a previsión, por cuenta
coste no atribuido, y su tendencia
coste por unidad de negocio
cobertura y utilización de compromisos
recursos ociosos detectados y retirados
anomalías abiertas
```

Y la lista de comprobación de la clase:

- ■ hay cuatro o cinco etiquetas obligatorias, con valores cerrados
- ■ están activadas como etiquetas de asignación
- ■ no se puede crear un recurso sin ellas
- ■ la plantilla de servicio nuevo las trae hechas
- ■ el coste compartido se reparte por una regla escrita
- ■ se mira el desglose dentro de los servicios grandes
- ■ se ordena por importe de variación, no por porcentaje
- ■ hay coste por unidad de negocio y se sigue su tendencia
- ■ hay presupuestos por cuenta y por servicio, sobre previsión
- ■ en entornos no productivos, los presupuestos actúan
- ■ hay detección de anomalías por servicio y por dueño
- ■ hay apagado por horario en no producción, con arranque automático
- ■ los entornos efímeros nacen con fecha de destrucción
- ■ la retirada de lo ocioso pasa por apagar antes de borrar
- ■ solo se compromete la base estable, y se vigila la utilización
- ■ el coste atribuido supera el 90 %

Y el cierre que enlaza con la clase siguiente: con el sistema construido, protegido, observado y con su coste bajo control, queda lo que este programa exige siempre antes de dar algo por terminado: comprobar que sobrevive a que se caiga una región, y comprobarlo ejecutándolo. Es la materia de la clase 215.

Ejemplo trabajado

CloudShop revisa el coste de la plataforma que ha montado en esta parte. Lo que sigue es la factura desglosada, las cuatro partidas que nadie había estimado, y la automatización que redujo el gasto un 41 % sin tocar la arquitectura.

La factura del primer mes completo:

```
total                                      28.400 €

por servicio, ordenado
  cómputo de contenedores                   5.830   20,5 %
  base de datos gestionada                  4.210   14,8 %
  transferencia de datos                    3.940   13,9 %  ←
  registros y métricas                      3.120   11,0 %  ←
  almacenamiento de objetos                 2.680    9,4 %
  pasarela de API                           2.140    7,5 %  ←
  DynamoDB                                  1.920    6,8 %
  balanceadores                             1.310    4,6 %
  funciones                                   980    3,5 %
  registro de imágenes                        610    2,1 %  ←
  resto                                     1.660    5,9 %

por dueño
  con dueño identificable                  17.100   60,2 %
  SIN DUEÑO                                11.300   39,8 %  ←

por entorno
  producción                               16.900
  preproducción                             7.200   43 % de
                                                    producción
  desarrollo                                3.100
  sin etiquetar                             1.200
```

Y las tres lecturas inmediatas:

```
el cómputo, que era lo que se vigilaba, es el 20 %
cuatro de las diez partidas mayores no se habían estimado
y el 40 % del gasto no tiene dueño                 ley 20
```

Las cuatro partidas no estimadas, desglosadas.

```
TRANSFERENCIA DE DATOS · 3.940 €
  salida a internet                          1.100
  ENTRE ZONAS                                2.190   ←
  entre regiones                               410
  otros                                        240

  el tráfico entre zonas venía de
    · el clúster hablando con la base sin preferencia de
      zona
    · réplicas de servicio repartidas y balanceo que
      ignoraba la zona
  corrección   preferencia de zona en el balanceo interno
               y lectura desde la réplica de la misma zona
  2.190 € → 640 €

REGISTROS Y MÉTRICAS · 3.120 €
  ya se había reducido de 4.980 en la clase 211, y quedaba
  el clúster
    ingesta del clúster                      1.840
    → cada contenedor escribía a nivel de depuración
  corrección   nivel por espacio de nombres, muestreo y
               caducidad de 14 días
  3.120 € → 780 €

PASARELA DE API · 2.140 €
  la migración de REST a HTTP de la clase 207 se había
  hecho solo en un servicio
  → 3 servicios seguían en REST sin usar nada de lo suyo
```

```
  2.140 € → 810 €

REGISTRO DE IMÁGENES · 610 €
  ya diagnosticado en la clase 212: capas huérfanas
  610 € → 24 €
```

El coste sin dueño: qué era.

```
11.300 € sin dueño identificable

  al inventariar
    transferencia de datos (no etiquetable)       3.940
    servicios comunes: nombres, identidad,
      registro central                            1.870
    recursos con etiquetas mal escritas           2.410
      «prod» / «Prod» / «producción» / «PRD»
      → cuatro grupos para un entorno
    RECURSOS OCIOSOS                              2.180  ←
    recursos creados antes de la política           900

  los ociosos, detallados
    41 volúmenes desconectados                      610 €
    118 copias de volúmenes sin caducidad           480 €
    9 direcciones fijas sin asociar                  32 €
    4 balanceadores sin destinos                    118 €
    2 bases de datos de pruebas de 2024             740 €
    31 entornos efímeros no destruidos              200 €
```

Y el detalle que resume el hallazgo:

```
las dos bases de datos de pruebas llevaban 14 meses
encendidas
la más cara la creó alguien que ya no está en la empresa
y nadie la reclamó al apagarla                ley 20, ley 25
```

Lo que se montó.

```
ETIQUETADO
  cinco etiquetas obligatorias: dueño, servicio, entorno,
  centro de coste, caducidad (opcional pero recomendada)
  valores CERRADOS por política de etiquetas
    entorno ∈ {prod, pre, dev, sandbox}
  barrera de organización: sin etiquetas, no se crea
  plantilla de servicio nuevo: las trae hechas

  y la corrección de lo existente
    2.410 € de etiquetas mal escritas → normalizadas en
    una semana con un script

PRESUPUESTOS
  por cuenta y por servicio, sobre previsión
  umbrales 50 / 80 / 100 %
  en desarrollo y preproducción
    al 100 %  → se restringe la creación de recursos nuevos
    al 120 %  → se apagan las cargas apagables
  en producción, solo aviso

ANOMALÍAS
  por servicio y por dueño, umbral de 150 € absolutos
  destino: el canal de guardia                 clase 211
  → en 6 meses, 11 anomalías
    · 7 reales (una prueba de carga olvidada, un trabajo
      en bucle, un índice nuevo, 4 crecimientos legítimos)
    · 4 falsas, por campañas previstas

APAGADO POR HORARIO
  desarrollo y preproducción: apagados de 20:00 a 7:00 y
  los fines de semana
  arranque automático a las 7:00, y bajo demanda con un
  botón que tarda 4 minutos
  → el botón fue lo que evitó que la gente desactivara el
    apagado                                      ley 16
  7.200 + 3.100 = 10.300 € → 4.180 €

ENTORNOS EFÍMEROS
```

```
  nacen con caducidad de 7 días, ampliable a 30 con motivo
  → los 31 existentes se destruyeron tras avisar

RETIRADA DE LO OCIOSO, automatizada
  detección semanal
  aviso al dueño con 7 días de plazo
  desasociar o apagar (sin borrar) y esperar 14 días
  borrar si nadie reclama
  → en 6 meses: 214 recursos retirados, 3 reclamaciones,
    3 restauraciones en menos de 5 minutos
```

El resultado, a los tres meses:

```
total mensual                            28.400 €   16.700 €
  transferencia                           3.940 €      920 €
  registros y métricas                    3.120 €      780 €
  pasarela                                2.140 €      810 €
  registro de imágenes                      610 €       24 €
  no productivos                         10.300 €    4.180 €
  recursos ociosos                        2.180 €        0 €
coste atribuido                            60,2 %     94,1 %
coste por pedido                          0,082 €    0,046 €
etiquetas con valores inconsistentes        2.410 €      0 €
```

Y la comprobación de que no se había convertido en policía:

```
recursos creados por caminos alternativos
  antes de la barrera (estimado)                desconocido
  después, detectados por inventario                      0

quejas del equipo sobre el control de coste
  al implantar                                            6
  a los 3 meses                                           0
  → las 6 se resolvieron con el botón de arranque bajo
    demanda y con ampliar la caducidad de los entornos
    efímeros de 3 a 7 días

y la señal que se vigila
  «número de excepciones vivas a la política de etiquetas»
  4, todas con dueño y fecha                    clase 190
```

La lección que esta clase deja: el cómputo, que era lo único que se vigilaba, resultó ser el 20 % de la factura, y cuatro de las diez partidas mayores no se habían estimado. El 40 % del gasto no tenía dueño y dos mil ciento ochenta euros al mes eran recursos que no usaba nadie, incluidas dos bases de datos de pruebas encendidas catorce meses. Y del ahorro total, la medida que más aportó no fue ninguna optimización técnica: fue apagar por la noche lo que no es producción.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/214-budgets-cost-explorer-etiquetado-y-finops-automatizado/lab.py
```

El laboratorio selecciona el motor de práctica finops y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa aws-cost-control en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un cálculo trazable con unidad, supuesto y sensibilidad. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye aws-cost-control para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

■■ Errores frecuentes

Síntoma	Causa probable	Corrección
El proyecto de etiquetado nunca termina	Se empezó etiquetando lo existente mientras se seguía creando sin etiquetas	Impide primero la creación sin etiquetas y solo después corrige lo antiguo.
El informe de costes muestra el mismo entorno repartido en varios grupos	Las etiquetas admiten texto libre	Fija valores cerrados por política de etiquetas y normaliza lo existente.
Se optimiza el cómputo y la factura no baja	El cómputo no era la partida dominante	Ordena la factura por importe, desglosa dentro de los servicios grandes y ataca las tres o cuatro líneas que suman el ochenta por ciento.
Nadie reduce una parte importante del gasto	Ese gasto no tiene dueño identificable	Mide la proporción de coste atribuido y reparte lo no etiquetable con una regla escrita; por debajo del 90 % lo demás es ruido.
El presupuesto avisa cuando ya no se puede hacer nada	Está definido sobre gasto acumulado y no sobre previsión	Configura umbrales sobre la previsión de cierre y añade acciones reales en entornos no productivos.
El control de coste genera recursos creados por caminos raros	Se planteó como vigilancia y no como camino fácil	Que la plantilla por defecto ya sea la barata, que cada equipo vea su coste junto a sus señales y que la retirada la haga la plataforma.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Por qué hay que impedir la creación sin etiquetas antes de etiquetar lo existente?
2. ¿Qué cinco vistas de la factura encuentran el dinero de verdad?
3. ¿Por qué se ordena por importe de la variación y no por porcentaje?
4. ¿Qué detecta la detección de anomalías que no detecta el presupuesto?
5. ¿Qué paso del proceso de retirada evita convertir una limpieza en un incidente?

Referencias

- AWS (2025). Cost allocation tags and tag policies — etiquetas de asignación de coste y políticas que las obligan. <https://docs.aws.amazon.com/awsaccountbilling/latest/aboutv2/cost-alloc-tags.html>
- AWS (2025). AWS Budgets and budget actions — presupuestos con acción automática al superarse. <https://docs.aws.amazon.com/cost-management/latest/userguide/budgets-managing-costs.html>
- AWS (2025). AWS Cost Anomaly Detection — detección de desviaciones de gasto sin umbral fijo. <https://docs.aws.amazon.com/cost-management/latest/userguide/manage-ad.html>
- FinOps Foundation (2025). FinOps Framework: capabilities and maturity — las capacidades del marco y su nivel de madurez. <https://www.finops.org/framework/>
- AWS (2025). Savings Plans and Reserved Instances: coverage and utilization — cobertura y utilización de los compromisos de compra. <https://docs.aws.amazon.com/savingsplans/latest/userguide/what-is-savings-plans.html>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 17 en PDF · Recorrido de AWS en PDF · Manual integral

Anterior	Índice	Siguiente
← 213 · EKS, IRSA, GitOps y operación de clúster	Parte 17 · Programa	215 · Multi-región, Route 53, failover y game day →

Evaluación — 214 Budgets, Cost Explorer, etiquetado y FinOps automatizado

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega aws-cost-control con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

215 — Multi-región, Route 53, failover y game day

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: reliability · Estado: EXECUTABLECORE

Propósito

Comprobar que el sistema sobrevive a la pérdida de una región, y comprobarlo ejecutándolo. La clase cubre los modos de despliegue multirregión con su coste real, la conmutación por nombres y sus límites —que casi siempre son el cuello—, la replicación de datos con su decisión de consistencia, y el ejercicio dirigido: cómo se prepara, cómo se ejecuta y por qué la mayoría de lo que encuentra no es de infraestructura.

Resultados de aprendizaje

Al finalizar podrás:

1. Elegir el modo multirregión que corresponde al objetivo, con su coste.
2. Conmutar por nombres conociendo el retraso real, no el teórico.
3. Replicar datos decidiendo qué se pierde y qué se bloquea.
4. Preparar y ejecutar un ejercicio dirigido con seguridad.
5. Corregir lo que el ejercicio encuentre, que rara vez es lo previsto.

Conceptos centrales

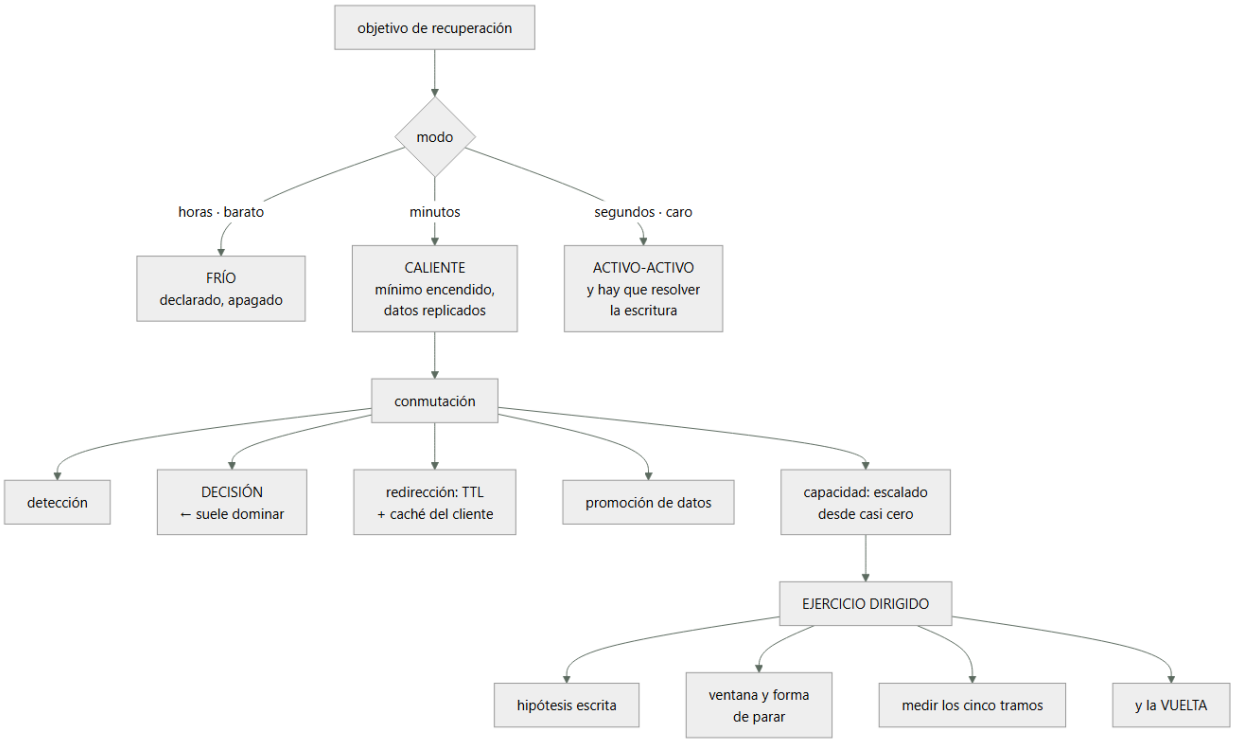
Concepto	Comprensión verificable
activo-pasivo en frío	La segunda región existe declarada y apagada. Barata y lenta de activar.
activo-pasivo en caliente	La segunda región tiene capacidad mínima encendida y datos replicados.
activo-activo	Ambas regiones sirven tráfico. Caro, y obliga a resolver la escritura en dos sitios.
comprobación de salud del nombre	Sonda que decide si un destino se anuncia. Su intervalo y umbrales fijan el tiempo de conmutación.
ejercicio dirigido	Simulación planificada de un fallo real, con hipótesis escrita, ventana acordada y forma de parar.
hipótesis del ejercicio	Lo que se espera que ocurra, escrito antes. Lo valioso es dónde falla.

Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Modos multirregión y su coste

La elección se hace por el objetivo de recuperación, y el objetivo se decide con el coste de la indisponibilidad, no con lo que suene bien.

```
FRÍO
  la infraestructura está declarada como código y no
  desplegada; los datos, en copias replicadas
  plazo   horas
  coste   almacenamiento de copias y poco más
  riesgo  el despliegue completo nunca se ha ejecutado
          → y es exactamente lo que falla         ley 22

CALIENTE (piloto)
  capacidad mínima encendida, datos replicados en continuo
  plazo   minutos
  coste   30-40 % de una región completa
  riesgo  la capacidad mínima tarda en escalar al 100 %

ACTIVO-ACTIVO
  ambas regiones sirven
  plazo   segundos
  coste   más del doble de una región
  riesgo  hay que resolver la escritura en dos sitios, que
          es el problema difícil                clase 187
```

Y la decisión, con la aritmética de siempre:

```
coste de la indisponibilidad = minutos esperados × pérdida
por minuto

y se compara con el coste del modo siguiente
→ si pasar de caliente a activo-activo cuesta 6.400 €/mes y
  evita 390 € de pérdida esperada, no se compra
                                      clases 164, 185
```

Y la advertencia sobre activo-activo, que se propone más de lo que se justifica:

```
ACTIVO-ACTIVO OBLIGA A DECIDIR DÓNDE SE ESCRIBE
  un escritor global      → latencia alta desde la otra
                            región
```

```
  escritura por región    → hay que particionar los datos
                            por región y que no se crucen
  escritura en ambas      → conflictos, y hay que
                            resolverlos              ley 21

→ la mayoría de los sistemas que dicen ser activo-activo
  son activo-pasivo con lectura en las dos
```

Y lo que hay que replicar además de los datos:

```
imágenes de contenedor y artefactos            clase 212
secretos y claves de cifrado
configuración y banderas de función
certificados
reglas de filtrado
y la propia canalización, capaz de desplegar allí
→ una región secundaria sin registro de imágenes propio no
  puede arrancar nada                           clase 185
```

2. Conmutar por nombres, con los límites reales

La conmutación por nombres es el mecanismo habitual y su tiempo real casi nunca coincide con el previsto.

```
LOS COMPONENTES DEL RETRASO

1  DETECCIÓN
   comprobación de salud: intervalo × fallos consecutivos
   típico 30 s × 3 = 90 s
   → configurable, y es lo más fácil de reducir

2  PROPAGACIÓN DEL CAMBIO
   el servicio de nombres reevalúa y deja de anunciar

3  TTL DEL REGISTRO
   los resolutores mantienen la respuesta hasta que expira
   → si el TTL es 300 s, hasta 5 minutos       clase 195

4  CACHÉ DEL CLIENTE
   navegadores, bibliotecas y entornos de ejecución cachean
   por su cuenta
   → algunos entornos cachean para siempre dentro del
     proceso
   → es el tramo que NADIE mide y que en la clase 195
     retrasó 40 minutos una conmutación

5  CAPACIDAD Y DATOS EN DESTINO
   escalar desde capacidad mínima: minutos   clase 212
   promover la réplica de datos a primaria
```

Y la conclusión práctica:

```
el TTL debe ser menor que el plazo prometido   clase 195
y aun así, el plazo real lo fija la capa que MÁS cachea
→ por eso el plazo se MIDE, no se calcula
```

Lo que hay que comprobar en la comprobación de salud, que suele estar mal:

```
✗ comprueba que el balanceador responde
  → responde aunque la base esté caída
✓ comprueba un punto que ejerza el camino crítico
  → incluida la base, con una consulta ligera

✗ una sola sonda desde un sitio
✓ varias, desde regiones distintas, con acuerdo por mayoría
  → evita conmutar por un problema de red local
```

Y dos decisiones que evitan conmutaciones falsas:

```
HISTÉRESIS
  salir con pocos fallos, volver con muchos aciertos
  → evita ir y venir                            clase 196

CONMUTACIÓN MANUAL DISPONIBLE
  un interruptor que fuerza el cambio, y otro que lo
  impide
  → durante un incidente confuso, decidir a mano suele ser
```

```
    mejor que dejar que un umbral decida
```

Y el tramo que domina en la práctica:

```
en la clase 179, la conmutación tardaba 2 h 10 frente a la
hora declarada
  y los dos tramos que dominaban eran DECIDIR y REDIRIGIR,
  no arrancar nada
→ la decisión humana es parte del plazo, y hay que
  cronometrarla                                  clase 166
```

3. Datos: qué se pierde y qué se bloquea

La parte difícil de multirregión son los datos, y la decisión es siempre la misma: qué se pierde al conmutar y qué se bloquea para no perder nada.

```
REPLICACIÓN ASÍNCRONA
  la escritura confirma en la región primaria y viaja
  después
  + latencia de escritura baja
  − al conmutar se pierde lo que no llegó a viajar
  → el objetivo de pérdida es el retraso de replicación
  → y ese retraso hay que VIGILARLO con alerta  clase 161

REPLICACIÓN SÍNCRONA ENTRE REGIONES
  la escritura confirma cuando llega a las dos
  + no se pierde nada
  − 60-150 ms añadidos a CADA escritura
  → rara vez compensa; es lo que se descartó en la
    clase 187

TABLAS GLOBALES (multi-escritor)
  se escribe en cualquier región y se replica
  + latencia baja en todas
  − resolución de conflictos por «el último gana», con las
    trampas de los relojes                      clase 187
  → solo sirve si los conflictos son imposibles por diseño:
    cada dato lo escribe una sola región         ley 21
```

Y la decisión que hay que tomar y escribir:

```
por cada conjunto de datos
  ¿cuánto se puede perder?     → fija el modo de replicación
  ¿se puede escribir en la
   secundaria?                 → si no, hay que bloquear
  ¿qué pasa con lo que estaba
   a medias?                   → reconciliación   clase 203
```

La vuelta, que es donde se pierden datos de verdad:

```
CONMUTAR ES LA MITAD FÁCIL
  volver exige que la región original se ponga al día con
  lo escrito en la secundaria
  y durante ese tiempo NO puede haber dos escritores
                                                  ley 21

EL ERROR CLÁSICO
  la región original vuelve, la comprobación de salud pasa,
  el nombre la vuelve a anunciar
  → y durante unos minutos se escribe en las dos
  → datos divergentes, imposibles de reconciliar

LA CORRECCIÓN
  la vuelta es SIEMPRE manual y con pasos
  1  bloquear escritura en la secundaria
  2  esperar a que la original se sincronice
  3  promover la original
  4  redirigir
  5  desbloquear
  → y el paso 1 exige que exista un modo de solo lectura
```

Y una comprobación que hay que hacer antes:

```
¿el sistema tiene modo de solo lectura?
  → si no, no se puede conmutar de vuelta con seguridad
  → y conviene tenerlo también para el mantenimiento
```

4. El ejercicio dirigido

Un plan de continuidad no comprobado no funciona. Este programa lo ha demostrado en la clase 166 y en la 179.

La preparación, que es la mitad del trabajo:

```
HIPÓTESIS ESCRITA, antes
  «al perder la región primaria, el tráfico conmutará en
   menos de 15 minutos, con pérdida menor de 1 minuto de
   datos, y las funciones X e Y quedarán degradadas»
  → lo valioso del ejercicio es dónde falla la hipótesis

ALCANCE Y VENTANA
  qué se rompe exactamente, y qué no se toca
  ventana acordada con negocio, no de madrugada la primera
    vez
  → de madrugada no hay nadie para observar y no hay
    tráfico realista

FORMA DE PARAR
  un botón que restaura el estado, decidido antes
  y un criterio claro: «si la pérdida supera X, se para»

OBSERVACIÓN
  quién mira qué, y qué cifras se anotan
  y una persona SOLO tomando notas

COMUNICACIÓN
  atención al cliente y negocio, avisados
  → un ejercicio que genera un incidente en el sistema de
    tiquetes ha fallado en algo más
```

La escala, que evita el desastre en el primer intento:

```
1  en papel: recorrer el procedimiento leyéndolo
   → encuentra los pasos que no existen
2  en un entorno de prueba
3  en producción, con una parte del tráfico
4  en producción, completo

→ empezar por el 4 es cómo se convierte un ejercicio en un
  incidente
```

Los cinco tramos que hay que cronometrar:

```
1  desde el fallo hasta la detección
2  desde la detección hasta la DECISIÓN
3  desde la decisión hasta la redirección efectiva
4  desde la redirección hasta la capacidad completa
5  desde ahí hasta la operación normal

y además
  la pérdida de datos real, medida
  y el tiempo de la VUELTA
```

Qué encuentra un ejercicio, con la evidencia del programa:

```
en la clase 179, de 4 pruebas de continuidad, 1 falló y la
conmutación tardó el doble de lo declarado
en la clase 198, la primera conmutación funcionó y la
replicación no se reanudó sola
en la clase 204, 6 de 15 pruebas fallaron y NINGUNA por
error de diseño

→ lo que encuentran los ejercicios casi nunca es
  infraestructura: son procedimientos, permisos, datos que
  faltan y decisiones que nadie sabía tomar    leyes 22, 24
```

Y la disciplina posterior:

```
cada hallazgo se convierte en una acción con dueño y fecha
el procedimiento se corrige EN EL MOMENTO, no después
y se repite el ejercicio hasta que pase entero
→ un ejercicio que encuentra problemas y no se repite ha
  servido para la mitad
```

Y la lista de comprobación de la clase:

- el modo multirregión corresponde al objetivo y su coste está comparado con la pérdida esperada
- imágenes, secretos, certificados y canalización están replicados
- la comprobación de salud ejerce el camino crítico
- hay sondas desde varias regiones y acuerdo por mayoría
- hay histéresis y conmutación manual disponible
- el TTL es menor que el plazo prometido
- se ha medido cuánto cachea cada cliente
- el retraso de replicación tiene alerta
- existe modo de solo lectura
- el procedimiento de vuelta es manual y con pasos
- el ejercicio tiene hipótesis escrita y forma de parar
- se cronometran los cinco tramos y la pérdida real
- los hallazgos tienen dueño y fecha
- el ejercicio se repite hasta pasar entero

Y el cierre que enlaza con la clase siguiente: con el sistema construido, protegido, observado, con su coste controlado y su continuidad comprobada, queda ponerlo todo junto y llevarlo a producción de verdad. Es la materia de la clase 216, que además cierra la parte 17.

Ejemplo trabajado

CloudShop hace su primer ejercicio dirigido de pérdida de región. Lo que sigue es la hipótesis escrita, lo que ocurrió de verdad, y los ocho hallazgos —de los cuales seis no eran de infraestructura.

El montaje, antes del ejercicio:

```
modo              activo-pasivo en caliente
primaria          eu-west-1
secundaria        eu-central-1, al 15 % de capacidad
datos             base relacional con réplica asíncrona
                  DynamoDB con tabla global
                  objetos replicados
nombres           registro con comprobación de salud,
                  TTL de 60 s
objetivo declarado   conmutar en < 15 min, pérdida < 1 min
coste del modo       4.100 €/mes (frente a 11.800 € de
                     activo-activo)
```

La hipótesis, escrita tres días antes:

```
1  la comprobación de salud detectará el fallo en < 2 min
2  la conmutación por nombres se completará en < 5 min
3  la capacidad escalará al 100 % en < 8 min
4  la pérdida de datos será < 30 s
5  la búsqueda y las recomendaciones quedarán degradadas;
   el flujo de compra funcionará completo
6  el tiempo total será < 15 min
7  la vuelta se podrá hacer en < 30 min
```

La preparación:

```
escala     se hizo en papel (2 semanas antes) y en
           preproducción (1 semana antes)
           → el recorrido en papel encontró que el paso 4 del
             procedimiento no existía: nadie sabía quién
             promueve la base

ventana    martes, 14:00-17:00, con tráfico real del 30 % de
           un día normal
           → no de madrugada, a propósito

parada     restaurar el estado devolviendo el registro de
           nombres a la primaria; criterio de parada: pérdida
           de datos > 5 min o más de 20 min sin servicio

observación  6 personas: 1 ejecuta, 1 toma notas y
           cronometra, 4 observan sus áreas

aviso      atención al cliente, negocio y el socio principal
```

Lo que ocurrió, tramo a tramo:

```
14:00:00  se corta el tráfico a la primaria simulando el
          fallo

14:01:34  detección: la comprobación de salud marca la
          primaria como no sana
          previsto < 2 min                    ✓ 1 min 34 s

14:01:34  el registro deja de anunciar la primaria
14:09:20  el tráfico está mayoritariamente en la secundaria
          previsto < 5 min                    ✗ 7 min 46 s

          por qué
            el TTL era 60 s, correcto
            PERO la aplicación móvil, en su versión 4.1,
            cachea la resolución dentro del proceso hasta
            reiniciar
            → el 22 % del tráfico móvil siguió llegando a
              la primaria durante todo el ejercicio
            → nadie lo había medido               clase 195

14:04:10  se decide promover la base de datos
14:04:10  ...y nadie tiene permiso para hacerlo
14:11:45  se localiza a la persona con permiso
14:13:02  la base secundaria es promovida
          → 8 min 52 s perdidos en un permiso   ← hallazgo

14:13:02  pérdida de datos medida: 41 segundos
          previsto < 30 s                     ✗ por poco

14:14:00  el escalado empieza
14:26:30  capacidad al 100 %
          previsto < 8 min                    ✗ 12 min 30 s

          por qué
            la región secundaria no tenía las imágenes de
            contenedor más recientes: el registro replicaba
            con retraso y faltaban 2 de 9
            → 2 servicios tardaron en arrancar
                                       clases 212, 185

14:26:30  operación normal… casi
          la búsqueda NO funcionaba en absoluto
          previsto: degradada                 ✗ caída

          por qué
            el índice de búsqueda no se replicaba a la
            secundaria; nadie lo había incluido en el plan
            → y el flujo de compra SÍ dependía de la
              búsqueda para el listado inicial
            → dependencia declarada blanda que era dura
                                       clases 185, 201

TIEMPO TOTAL              26 min 30 s
previsto                  < 15 min                  ✗
```

La vuelta, que fue peor:

```
15:30  se inicia la vuelta
15:31  se restaura el tráfico a la primaria
15:31  la primaria estaba en pie y su comprobación de salud
       pasaba
       → el registro la volvió a anunciar automáticamente
       → y durante 6 minutos se escribió en las DOS
         regiones

15:37  se detecta al ver escrituras en ambas
15:37  parada del ejercicio; tráfico forzado a la
       secundaria
15:52  reconciliación manual: 214 pedidos escritos en la
       primaria durante esos 6 minutos, que la secundaria
       no tenía
       → recuperados con esfuerzo, ninguno perdido
```

```
       → pero en un incidente real, con más tráfico, esto
         habría sido grave                          ley 21

causa  no existía modo de solo lectura
       y el procedimiento de vuelta era automático, no
       manual con pasos
```

Los ocho hallazgos:

#	hallazgo	tipo	acción
1	permiso de promoción de base solo lo tenía una persona, no localizable	PERMISOS	3 personas con acceso de emergencia
2	procedimiento sin el paso de quién promueve	PROCESO	escrito y probado por otro
3	la app móvil 4.1 cachea la resolución para siempre	CLIENTE	corregido en 4.3; y medido
4	el registro de imágenes replicaba con retraso	DATOS	replicación síncrona de etiquetas de producción
5	el índice de búsqueda no se replicaba	DATOS	incluido en el plan
6	la búsqueda era dependencia dura del listado	DISEÑO	listado con respaldo cacheado
7	no existía modo de solo lectura	DISEÑO	implementado
8	la vuelta era automática y produjo dos escritores	PROCESO	vuelta manual, con 5 pasos

```
de infraestructura                                   2
de procedimiento, permisos, cliente y diseño         6
```

El segundo ejercicio, tres meses después:

```
detección                            1 min 12 s     ✓
conmutación efectiva                 3 min 40 s     ✓
  → tras corregir la app móvil, el tráfico residual
    bajó al 0,4 %
decisión y promoción                    58 s        ✓
  → con 3 personas con permiso y procedimiento escrito
pérdida de datos                          22 s       ✓
capacidad al 100 %                   6 min 10 s     ✓
búsqueda                               degradada    ✓
  → el listado usa respaldo cacheado
TIEMPO TOTAL                        10 min 40 s     ✓

vuelta, manual con 5 pasos
  bloquear escritura en secundaria          40 s
  esperar sincronización                 4 min 20 s
  promover la original                      55 s
  redirigir                              2 min 10 s
  desbloquear                               30 s
  total                                  8 min 35 s
  dos escritores simultáneos                    0
```

El coste, decidido con estos datos:

```
se volvió a plantear activo-activo
  coste adicional                       7.700 €/mes
  mejora del plazo              10 min 40 s → ~40 s
  pérdida esperada evitada, calculada con el histórico
  de incidentes regionales                410 €/mes

decisión   NO
registrado con la señal que lo reabriría
  «si el negocio de empresa con penalización por
   indisponibilidad supera el 15 % de los ingresos»
                                              clase 190
```

La lección que esta clase deja: la hipótesis falló en cinco de sus siete puntos, y el tramo que más tiempo consumió no fue técnico: fueron ocho minutos y cincuenta y dos segundos buscando a quien tuviera permiso para promover la base. De los ocho hallazgos, seis no eran de infraestructura. Y lo más peligroso ocurrió en la vuelta, que nadie había ensayado: seis minutos con dos escritores a la vez, que en un incidente real habrían dejado datos divergentes.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/215-multi-region-route-53-failover-y-game-day/lab.py
```

El laboratorio selecciona el motor de práctica reliability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa aws-dr-drill en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un escenario de fallo con objetivo y recuperación medida. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye aws-dr-drill para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

Errores frecuentes

Síntoma	Causa probable	Corrección
La conmutación tarda mucho más de lo previsto pese a un TTL corto	Algún cliente cachea la resolución dentro del proceso y no respeta el TTL	Mide cuánto tarda cada cliente en seguir un cambio de nombre y corrige los que cachean indefinidamente.
La región secundaria no puede arrancar los servicios	Faltan imágenes, secretos, certificados o la propia canalización	Replica todo lo necesario para desplegar, no solo los datos, y compruébalo arrancando desde cero allí.
Al volver a la región original se escribe en las dos a la vez	La vuelta es automática y la comprobación de salud vuelve a anunciarla en cuanto está en pie	Haz la vuelta manual con pasos: bloquear escritura, sincronizar, promover, redirigir, desbloquear; y ten modo de solo lectura.
Durante la conmutación nadie puede ejecutar un paso crítico	El permiso lo tiene una sola persona y el procedimiento no lo nombra	Varias personas con acceso, procedimiento escrito y probado por alguien que no lo escribió.
Un servicio declarado degradable resulta imprescindible	La dependencia estaba declarada blanda y nunca se probó apagándola	Inyecta el fallo y comprueba; corrige con respaldo cacheado o valor por defecto antes del ejercicio siguiente.
El ejercicio se convierte en un incidente real	Se empezó por el escenario completo en producción, sin recorrido previo ni forma de parar	Escala el ejercicio: papel, prueba, parte del tráfico y completo; con criterio de parada y botón de restauración acordados.

Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

Preguntas de comprobación

1. ¿Qué distingue el modo caliente del activo-activo y qué problema añade el segundo?
2. ¿Qué cinco componentes forman el retraso de una conmutación por nombres?
3. ¿Por qué el plazo real se mide y no se calcula?
4. ¿Qué cinco pasos debe tener la vuelta y por qué es manual?
5. ¿Qué tipo de hallazgos produce un ejercicio dirigido con más frecuencia?

Referencias

- AWS (2025). Disaster recovery options in the cloud — las cuatro estrategias de recuperación y su coste. <https://docs.aws.amazon.com/whitepapers/latest/disaster-recovery-workloads-on-aws/disaster-recovery-options-in-the-cloud.html>
- AWS (2025). Route 53 health checks and DNS failover — conmutación por fallo gobernada desde el DNS. <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/dns-failover.html>
- AWS (2025). Amazon DynamoDB global tables — réplica multirregión activa-activa de la base de datos. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/GlobalTables.html>
- Google (2018). The Site Reliability Workbook: disaster role playing and testing — ensayo de desastre con roles asignados. <https://sre.google/workbook/table-of-contents/>
- Basiri, A. y otros (2016). Chaos engineering — experimentos con hipótesis y radio de alcance. <https://ieeexplore.ieee.org/document/7503833>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 215 Multi-región, Route 53, failover y game day

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega aws-dr-drill con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

216 — Proyecto: CloudShop productivo en AWS

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 8
Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Poner en producción el sistema completo de CloudShop en AWS con lo de las once clases anteriores, y comprobarlo con las pruebas negativas de toda la parte. La clase da el orden de construcción, el entregable y los criterios. Y cierra la parte 17: corrige las cinco predicciones de la clase 204 —tres acertadas, una subestimada y una fallada—, actualiza el recuento de leyes, añade la ley 26 y escribe la hipótesis de la parte 18.

Resultados de aprendizaje

Al finalizar podrás:

1. Construir el sistema completo en el orden que evita rehacer.
2. Comprobar con las pruebas negativas de toda la parte.
3. Medir coste, latencia y disponibilidad con cifras comparables.
4. Corregir las cinco predicciones de la clase 204 con evidencia.
5. Escribir la hipótesis de la parte 18 en forma refutable.

Conceptos centrales

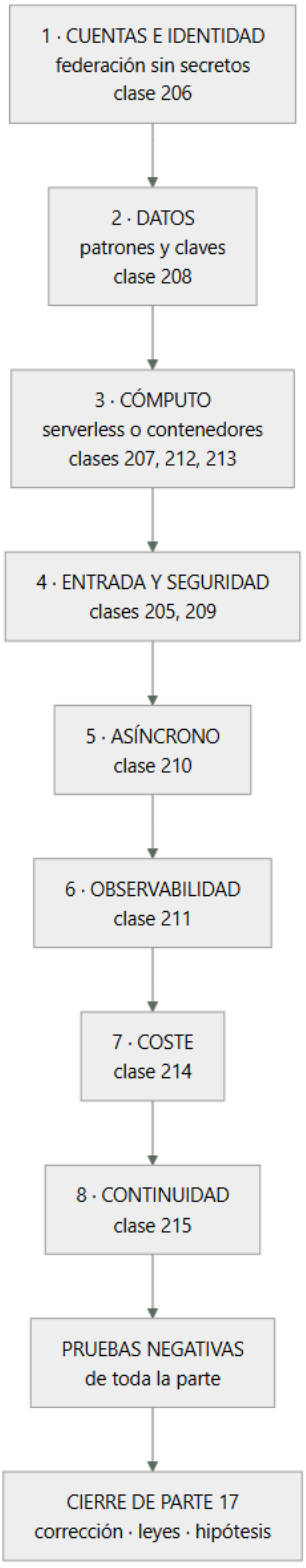
Concepto	Comprensión verificable
sistema productivo	El que tiene usuarios reales, guardia, presupuesto y continuidad comprobada. No el que funciona.
orden de construcción	Cuentas e identidad primero, diagnóstico y coste al final. Lo que condiciona a lo demás, antes.
valor por defecto	Configuración inicial de un servicio. Está elegida para que la demostración funcione.
ley 26	El valor por defecto está elegido para que la demostración funcione, no para que el sistema aguante.
prueba negativa de parte	Comprobación acumulada de las once clases, ejecutada sobre el sistema entero.
hipótesis de parte	Afirmación refutable escrita antes de estudiar, que la parte siguiente corrige con evidencia.

Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. El encargo y su orden

El encargo. Llevar a producción la plataforma de pedidos de CloudShop en AWS: con usuarios reales, guardia, presupuesto y continuidad comprobada.

El orden, por coste de cambio:

```
1  CUENTAS E IDENTIDAD                              clase 206
   una cuenta por entorno; federación sin secretos
   barreras de organización: nadie puede crear roles ni
   desactivar auditoría desde una canalización
   → si esto llega tarde, hay que migrar todo lo anterior

2  DATOS                                            clase 208
   patrones de acceso escritos ANTES
   claves de partición que reparten
   lo que no encaja, replicado a otro almacén
   → la decisión más cara de cambiar de la parte

3  CÓMPUTO                                clases 207, 212, 213
   funciones para lo que encaja; contenedores para lo demás
   concurrencia reservada calculada por las conexiones
   memoria y tamaños medidos, no copiados

4  ENTRADA Y SEGURIDAD                     clases 205, 209
   distribución, bucket privado, capa 7 detrás
   testigo de acceso validado con las siete comprobaciones
   filtrado en modo cuenta antes de bloquear

5  ASÍNCRONO                                        clase 210
   bus, cola por consumidor, idempotencia
   cola de fallidos con alerta por antigüedad y reproceso
   probado

6  OBSERVABILIDAD                                   clase 211
   declarada junto al servicio; alertas que llegan a alguien

7  COSTE                                            clase 214
   etiquetas obligatorias en la creación; presupuestos con
   acción; retirada de lo ocioso

8  CONTINUIDAD                                      clase 215
   segunda región, y el ejercicio dirigido que la comprueba
```

Y la regla que resume la parte:

```
en cada uno de los ocho pasos, la primera tarea es
REVISAR LOS VALORES POR DEFECTO
→ porque están elegidos para que la demostración funcione
                                                    ley 26
```

Y el error de método que hunde este proyecto:

```
empezar por el paso 3, que es lo divertido
→ y descubrir en el paso 2 que la clave de partición está
  mal y hay que migrar                          clase 208
```

2. El entregable, las pruebas y la evaluación

El documento, con lo que hay que poder enseñar:

```
1  el problema, con la cifra que lo demuestra
2  patrones de acceso, escritos y fechados antes del modelo
3  arquitectura: cuentas, servicios, datos, entrada, salida
4  la lista de VALORES POR DEFECTO cambiados y por qué
5  concurrencia, plazos y capacidad, con su cálculo
6  seguridad: testigos, autorización, filtrado, perímetro
7  observabilidad: qué se vigila y qué solo se consulta
8  coste: desglose, atribución y coste por unidad de negocio
9  continuidad: modo, plazos medidos y ejercicio ejecutado
10 pruebas negativas, con los fallos publicados
11 lo que NO se hace, y por qué
```

Las pruebas negativas de la parte, que son el criterio de terminado:

- leer un objeto por la URL directa del bucket
- asumir el rol de producción desde otra rama o repositorio
- llamar a la API con testigo de identidad, caducado o alterado
- pedir el recurso de otro usuario con testigo válido
- llamar a la pasarela saltándose la distribución
- registrar un nombre con apóstrofo (que NO se bloquee)

- superar el límite de ritmo y recibir 429
- enviar la misma petición 50 veces en paralelo
- enviar el mismo mensaje 50 veces a la cola
- dejar un mensaje en la cola de fallidos y esperar la alerta
- reprocesar 10.000 mensajes sin tumbar la base
- agotar la concurrencia y comprobar que se rechaza en vez de caer
- desplegar una versión rota y ver la reversión automática
- desplegar durante tráfico y contar peticiones cortadas
- provocar la condición de cada alerta y ver si llega
- crear un recurso sin etiquetas
- borrar la pila y comprobar que los datos sobreviven
- perder la región primaria y cronometrar los cinco tramos
- volver a la primaria sin dos escritores

Y los criterios de evaluación, publicados antes:

```
1  los patrones de acceso están fechados antes del modelo  3
2  la lista de valores por defecto cambiados es explícita  3
3  no queda ninguna credencial de larga duración           3
4  la concurrencia está calculada, no puesta a ojo         2
5  la autorización comprueba la propiedad del recurso      3
6  toda operación con efecto es idempotente                3
7  cada alerta se ha probado provocando la condición       2
8  el coste atribuido supera el 90 %                       2
9  hay coste por unidad de negocio                         2
10 la continuidad se midió ejecutándola                    3
11 las pruebas negativas se ejecutaron y hay fallos
   publicados                                              3
12 está escrito lo que no se hace                          1
```

Y el 11 pesa como los que más, con la evidencia acumulada:

```
proporción de pruebas negativas que han fallado la primera
vez en este programa
  clase 144    3 de 11      27 %
  clase 168    5 de 11      45 %
  clase 179    9 de 31      29 %
  clase 189    3 de 14      21 %
  clase 204    6 de 15      40 %
→ un proyecto con cero fallos no las ejecutó       ley 22
```

3. Cierre de la parte 17: corrección de las cinco predicciones

Las cinco predicciones de la clase 204, corregidas con la evidencia de las clases 205 a 215.

1. «bajar a un proveedor concreto revelará que buena parte de lo que hemos tratado como decisiones de arquitectura son valores por defecto; y estarán mal elegidos para producción en más de la mitad de los casos»

   CORRECTA, y con margen. De los valores por defecto que hubo que revisar en las once clases, la cuenta salió así: bucket con alojamiento estático y política pública, error 404 convertido en 200, política de caché heredada, invalidación con comodín, memoria de función en el mínimo, plazo en 30 s, sin concurrencia reservada, sin alias, visibilidad de cola en 30 s, sin informe de fallo parcial, grupos de registros sin caducidad, nivel de depuración en producción, etiquetas de imagen mutables, registro sin caducidad, drenaje corto, porcentaje mínimo sano al 50 %, periodo de gracia menor que el arranque, circuito de despliegue desactivado y escalado por CPU. Diecinueve, y los diecinueve había que cambiarlos.

2. «el diseño de la base de datos por patrones de acceso será la decisión más cara de cambiar de la parte, y la que más veces se toma sin haber escrito los patrones»

   CORRECTA en las dos mitades. Fue la más cara: seis semanas de dos personas para cambiar una clave de partición que se había elegido en una tarde, por la primera consulta que pidió negocio. Y se tomó sin escribir los patrones, que habrían costado tres horas. Pero incompleta en algo: el

error más CARO fue ese, y el más FRECUENTE fue otro —los valores por defecto, que aparecieron en casi todas las clases—.

3. «la eliminación de secretos mediante federación resultará sencilla técnicamente y lenta organizativamente: el obstáculo será quién tiene permiso para cambiarla»

   FALLADA, y en las dos mitades. La parte técnica NO era sencilla: tenía una trampa que dejó el sistema PEOR que con la clave estática, porque una condición de sujeto con comodín permitía a cualquiera de doscientos catorce miembros desplegar en producción creando un repositorio. Y la lentitud no vino de los permisos: vino de que el inventario decía cuarenta y una canalizaciones y había cuarenta y nueve. Predijimos el obstáculo equivocado, y lo más grave es que dimos por resuelta la parte que tenía el fallo.

4. «más de la mitad de los problemas del proyecto productivo no serán de AWS sino de lo mismo de siempre: leyes 25, 15 y 22»

   CORRECTA. Una política pública añadida «para probar» y no retirada; ocho canalizaciones fuera del inventario, una con credenciales de administrador para un sistema retirado en 2023; once alertas a canales sin suscriptores, entre ellas la de la cola de fallidos que ocho meses después dejó cuarenta y un mil mensajes a un día de caducar; dos bases de datos de pruebas encendidas catorce meses; treinta y un entornos efímeros que nunca caducaron; y en el ejercicio de continuidad, seis de ocho hallazgos que no eran de infraestructura.

5. «el coste real será entre dos y cuatro veces la estimación inicial, y la diferencia estará casi toda en partidas que no son cómputo»

   CORRECTA EN EL MECANISMO Y CORTA EN LA CIFRA. La segunda mitad se cumplió por completo: el cómputo resultó ser el 17 % de la factura de la API y el 20 % de la de la plataforma, y las partidas sorpresa fueron pasarela, registros, índices, transferencia entre zonas y capas huérfanas del registro de imágenes. Pero el factor no fue de dos a cuatro: fue de SIETE, porque la estimación inicial solo contaba invocaciones y duración.

Marcador: tres correctas, una correcta y subestimada, una fallada. Y la fallada enseña más que las otras cuatro: dimos por resuelta la parte técnica y pusimos el problema en las personas, y resultó que la parte técnica tenía una trampa que empeoraba la seguridad mientras parecía mejorarla.

4. Recuento de leyes, ley 26 e hipótesis de la parte 18

El recuento de leyes, cerrada la parte 17.

```
ley 13  lo que no se mira deja de funcionar en silencio       45
ley 15  la señal existe y nadie la mira                       34
ley 22  un procedimiento nunca ejecutado no funciona          29
ley 14  el coste se decide al crear, no al pagar              28
ley 16  un control que estorba se rodea                       26
ley 20  lo que no tiene dueño se filtra y se desperdicia      26
ley 21  el acoplamiento vive en quién escribe                 21
ley 23  la capacidad la limita lo que ya se mantiene          13
ley 25  lo provisional sobrevive a su motivo                  12
ley 24  lo que no está en el diagrama no se analiza           11
ley 19  la compensación hace invisible el fallo               10
ley 17  se optimiza la medida, no el objetivo                 10
ley 18  lo asíncrono traslada la garantía, no la elimina       8
```

Y la parte 17 obliga a escribir una ley nueva, que es la más específica de todas y la que más dinero y latencia ha movido en esta parte:

```
LEY 26
```

```
  el valor por defecto está elegido para que la
  demostración funcione, no para que el sistema aguante

apariciones en esta parte                                    5
  clase 205   alojamiento estático del bucket, 404 a 200,
              política de caché heredada, invalidación con
              comodín
  clase 207   memoria mínima, plazo de 30 s, sin
              concurrencia reservada, sin alias
  clase 210   visibilidad de 30 s, sin informe de fallo
              parcial de lote
  clase 211   grupos de registros sin caducidad, nivel de
              depuración en producción
  clase 212   drenaje corto, mínimo sano al 50 %, gracia
              menor que el arranque, escalado por CPU

y lo que la distingue
  no habla de descuido ni de falta de dueño
  habla de que el valor inicial está OPTIMIZADO PARA OTRA
  COSA: para que funcione en cinco minutos en un tutorial
  → el remedio no es vigilar: es revisar la lista de
    valores por defecto como primera tarea de cada servicio
```

La hipótesis de la parte 18 (clases 217 a 228, Azure en producción), escrita antes de estudiarla para que la clase 228 la corrija:

1. los valores por defecto de Azure estarán mal elegidos para producción en una proporción parecida a los de AWS, pero fallarán en otro sitio: los de AWS tienden a ser permisivos en red y almacenamiento; los de Azure lo serán en ámbito de identidad y de asignación de permisos
   ley 26

2. la jerarquía de grupos de administración y suscripciones resultará ser el equivalente del plan de direcciones: se decide en una tarde, condiciona una década y renumerarla costará meses
   ley 14

3. la identidad será el eje de toda la parte: lo que en AWS se resuelve con roles y políticas se resolverá aquí con el directorio, y el error más frecuente será un ÁMBITO DE ASIGNACIÓN demasiado amplio —el equivalente exacto de la condición de sujeto con comodín de la clase 206

4. la mayoría de los conceptos técnicos se corresponderán uno a uno entre las dos nubes; lo que NO se corresponderá es el modelo operativo, y trasladar «la misma arquitectura» costará más en operación que en código
   clase 158

5. los problemas del proyecto productivo volverán a ser, en mayoría, de las leyes 25, 15 y 22. Y aquí añadimos algo incómodo: si esta predicción vuelve a acertar por cuarta parte consecutiva, dejará de tener mérito y pasará a ser una descripción, no una hipótesis. Lo que habrá que preguntarse entonces no es si acertamos, sino por qué, sabiéndolo, sigue ocurriendo

Y el cierre de la parte 17: de once clases, lo que más dinero y latencia movió no fue ninguna decisión de arquitectura, sino diecinueve valores por defecto que había que cambiar y que funcionaban perfectamente sin cambiarlos. La parte 18 hace el mismo recorrido en Azure, empezando por lo que allí condiciona todo lo demás: la jerarquía de suscripciones y grupos de administración. Es la clase 217.

Ejemplo trabajado

El sistema de CloudShop en producción en AWS, montado con el método de la parte. Lo que sigue es el resumen del entregable, la lista de valores por defecto cambiados, y el resultado de las diecinueve pruebas negativas —de las que fallaron siete.

La arquitectura, en una página:

```
CUENTAS       organización con 4 cuentas: producción,
              preproducción, desarrollo, seguridad
```

```
              barreras: sin crear roles, sin desactivar
              auditoría, sin borrar copias

IDENTIDAD     federación desde el repositorio, 4 roles por
              propósito, producción atada a entorno
              protegido con 2 aprobaciones
              credenciales de larga duración: 2, ambas de
              terceros, con rotación y fecha de revisión

DATOS         DynamoDB con tabla única para pedidos y
              clientes; 9 patrones escritos y fechados
              búsqueda y analítica alimentadas por el flujo
              de cambios
              PostgreSQL solo para facturación heredada

CÓMPUTO       funciones para la API de pedidos
              contenedores para búsqueda y para el
              procesador de eventos
              Kubernetes solo para las cargas del equipo de
              datos

ENTRADA       CloudFront → capa 7 → API HTTP → funciones
              bucket privado con control de acceso al origen
              filtrado con 4 grupos, ajustados tras 3
              semanas en modo cuenta

ASÍNCRONO     bus → 4 colas → 4 consumidores
              todos idempotentes; todas con cola de fallidos

OBSERVABILIDAD declarada en la plantilla de cada servicio
              584 series de métricas, 63 alertas, 1 canal

COSTE         5 etiquetas obligatorias, barrera de creación
              presupuestos con acción en no producción
              retirada automática de lo ocioso

CONTINUIDAD   activo-pasivo en caliente en eu-central-1
              ejercicio ejecutado 2 veces
```

La lista de valores por defecto cambiados, que fue el entregable más útil:

servicio	por defecto	cambiado a
S3	alojamiento estático	desactivado
S3	acceso público	bloqueado
S3	sin versionado	activado
CloudFront	caché heredada	política propia por ruta
CloudFront	sin cabeceras seg.	política de cabeceras
CloudFront	404 → 200 con index	función de borde por extensión
Lambda	128 MB	1.024 MB (medido)
Lambda	plazo 30 s	8 s
Lambda	sin concurrencia reservada	reservada, por conexiones
Lambda	sin alias	alias + escalonado
API	REST	HTTP
SQS	visibilidad 30 s	300 s (por lote)
SQS	sin fallo parcial	activado
SQS	retención 4 días	14 días
CloudWatch	sin caducidad	21 días
CloudWatch	nivel depuración	información
ECR	etiquetas mutables	inmutables
ECR	sin caducidad	10 últimas por rama
ECS	drenaje 30 s	60 s
ECS	mínimo sano 50 %	100 %
ECS	gracia 30 s	120 s
ECS	sin circuito	activado
ECS	escalado por CPU	peticiones por tarea
DynamoDB	lectura fuerte	eventual salvo 2 patrones

```
total cambiados                                          24
que funcionaban sin cambiarlos                           24
```

Y la observación que el equipo escribió:

```
los veinticuatro funcionaban
ninguno daba error
ninguno aparecía en ninguna alerta
y los veinticuatro habrían causado un problema en
producción, entre coste, latencia, pérdida de datos o
seguridad                                                ley 26
```

Las diecinueve pruebas negativas: siete fallaron.

```
✓  URL directa del bucket                          denegada
✓  rol de producción desde otra rama               denegado
✓  testigo de identidad, caducado o alterado       rechazado
✗  recurso de otro usuario con testigo válido
   → 2 de 11 rutas no comprobaban la propiedad; las 2 eran
     rutas nuevas añadidas después de la revisión
                                                   clase 209
✓  llamada a la pasarela saltándose la distribución denegada
✓  registro con apóstrofo                          aceptado
✓  límite de ritmo                                 429
✓  misma petición 50 veces en paralelo             1 efecto
✗  mismo mensaje 50 veces a la cola
   → el consumidor de notificaciones no era idempotente:
     se había añadido en el último mes sin la comprobación
✓  mensaje en cola de fallidos → alerta             41 s
✗  reprocesar 10.000 mensajes
   → tumbó la base la primera vez: el procedimiento tenía
     límite de ritmo y quien lo ejecutó no lo usó porque
     no estaba en el primer paso                   clase 210
✓  agotar la concurrencia                          429, sin
                                                   caída
✓  versión rota                                    revertida
                                                   en 2 min
✗  desplegar durante tráfico
   → 14 peticiones cortadas en el servicio de búsqueda:
     su aplicación no atendía la señal de terminación
                                                   clase 212
✗  provocar la condición de cada alerta
   → 5 de 63 no llegaron; 3 por umbral mal puesto y 2 por
     destino equivocado
✓  recurso sin etiquetas                           rechazado
✗  borrar la pila y comprobar los datos
   → la tabla de idempotencia NO tenía política de
     retención y se borró
   → el sistema siguió funcionando, pero perdió la memoria
     de lo procesado: 1.100 mensajes en vuelo se habrían
     duplicado al reintentarse
✗  perder la región primaria
   → 12 min 40 s frente a los 10 min 40 s del último
     ejercicio: había 2 servicios nuevos sin réplica de
     imagen en la secundaria
✓  volver sin dos escritores                       correcto
```

Y el análisis de las siete:

```
dos por servicios AÑADIDOS después de la última revisión
  (rutas sin comprobación de propiedad, consumidor sin
   idempotencia)
dos por procedimientos con pasos en mal orden o umbrales
  mal puestos
dos por omisiones al crecer (retención de una tabla nueva,
  réplica de imágenes de servicios nuevos)
una por una aplicación que no atendía la señal

→ SEIS de las siete se deben a lo mismo: el sistema creció
  y las comprobaciones no crecieron con él
→ y por eso las pruebas negativas se ejecutan
  PERIÓDICAMENTE, no una vez                       ley 22
```

Las cifras del sistema en producción, tras tres meses:

```
p99 del flujo de compra                  < 500 ms    412 ms
disponibilidad observada                   99,8 %    99,86 %
techo por dependencias                     99,84 %   —
coste mensual                            12.000 €   16.700 €
coste por pedido                           0,050 €    0,046 €
tiempo de conmutación de región            15 min    12 min 40 s
pérdida de datos en conmutación              1 min     22 s
coste atribuido                               90 %    94,1 %
alertas por turno                            < 2       0,7
proporción de alertas accionables           > 80 %      89 %
despliegues al mes                             —        94
  revertidos automáticamente                   —         5
  que causaron incidente                       —         0
```

Y las tres cosas que se decidió no hacer, registradas:

```
no adoptar activo-activo: 7.700 €/mes frente a 410 € de
  pérdida evitada; se revisa si el negocio de empresa
  supera el 15 % de los ingresos
no migrar los servicios web a Kubernetes: funcionan
no unificar todo en contenedores: las funciones cubren su
  caso con menos operación
```

La lección que este proyecto deja: el entregable que más valor tuvo no fue el diagrama ni el cálculo de capacidad: fue la tabla de veinticuatro valores por defecto cambiados, todos los cuales funcionaban sin cambiarlos. Y de las diecinueve pruebas negativas, seis de las siete que fallaron se debieron a que el sistema creció y las comprobaciones no crecieron con él: rutas nuevas sin comprobar propiedad, un consumidor nuevo sin idempotencia, una tabla nueva sin retención y dos servicios nuevos sin réplica de imagen.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/216-proyecto-cloudshop-productivo-en-aws/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa cloudshop-aws en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye cloudshop-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.

- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

■■ Errores frecuentes

Síntoma	Causa probable	Corrección
Hay que migrar datos a mitad del proyecto	Se empezó por el cómputo y la clave de partición se eligió por la primera consulta que pidió negocio	Escribe los patrones de acceso con su frecuencia y latencia antes de modelar, y fecha ese documento.
Todo funciona en pruebas y falla el primer día de carga real	Los valores por defecto están elegidos para que la demostración funcione	Revisa la lista de valores por defecto como primera tarea de cada servicio y documenta cuáles cambiaste y por qué.
Las comprobaciones pasan pero el sistema tiene huecos nuevos	Se añadieron servicios y rutas después de la última revisión	Ejecuta las pruebas negativas periódicamente y añade una por cada capacidad nueva, no solo al terminar.
Borrar una pila se lleva datos por delante	Los recursos con datos no tienen política de retención y viven en la misma pila que el código	Separa las pilas por ciclo de vida y pon retención a todo lo que guarde estado, incluidas las tablas auxiliares.
El procedimiento de recuperación se ejecuta mal bajo presión	Los pasos críticos no están al principio ni son obligatorios	Pon el límite de ritmo y los pasos de seguridad como primer paso, y haz que lo ejecute alguien que no lo escribió.
La estimación de coste se queda muy corta	Solo se contó el cómputo	Estima pasarela, registros, índices, transferencia entre zonas y almacenamiento de imágenes; suelen sumar más que el cómputo.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Por qué la primera tarea de cada servicio es revisar sus valores por defecto?
2. ¿Cuál de las cinco predicciones de la clase 204 falló y qué enseña su fallo?
3. ¿Qué dice la ley 26 y en qué se distingue de la 25?
4. ¿Qué proporción de pruebas negativas ha fallado la primera vez en este programa?
5. ¿Por qué seis de las siete pruebas fallidas del proyecto tenían la misma causa de fondo?

Referencias

- AWS (2025). Well-Architected Framework — los pilares como lista de verificación del proyecto. <https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html>
- AWS (2025). Serverless Application Lens — la lente específica para arquitecturas serverless. <https://docs.aws.amazon.com/wellarchitected/latest/serverless-applications-lens/welcome.html>
- AWS (2025). Security Reference Architecture — arquitectura de referencia de los servicios de seguridad. <https://docs.aws.amazon.com/prescriptive-guidance/latest/security-reference-architecture/welcome.html>
- Beyer, B. y otros (2018). The Site Reliability Workbook — prácticas de operación aplicadas al servicio productivo. <https://sre.google/workbook/table-of-contents/>
- Basiri, A. y otros (2016). Chaos engineering — inyección controlada de fallos para validar la resiliencia. <https://ieeexplore.ieee.org/document/7503833>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 216 Proyecto: CloudShop productivo en AWS

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega cloudshop-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 22 — Especializaciones, certificaciones y práctica profesional

273 — Mapeo AWS, Azure, Google Cloud, Kubernetes y FinOps

Parte: 22 — Especializaciones, certificaciones y práctica profesional Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: assessment · Estado: EXECUTABLECORE

Propósito

Traducir entre proveedores sin engañarse: dar el mapa de equivalencias entre las tres nubes, los contenedores y la disciplina de coste, y —más importante— dónde la equivalencia se rompe. La clase enseña el método de traducción por restricción en vez de por nombre, y las diferencias reales que este programa ha medido y que ninguna tabla de equivalencias recoge.

Resultados de aprendizaje

Al finalizar podrás:

1. Traducir entre proveedores por restricción, no por nombre de servicio.
2. Reconocer dónde la equivalencia aparente esconde una diferencia real.
3. Usar el mapa de las cinco capas para orientarse en una nube nueva.
4. Evaluar una traducción con las preguntas que la validan.
5. Aplicar el mismo método a los contenedores y a la disciplina de coste.

Conceptos centrales

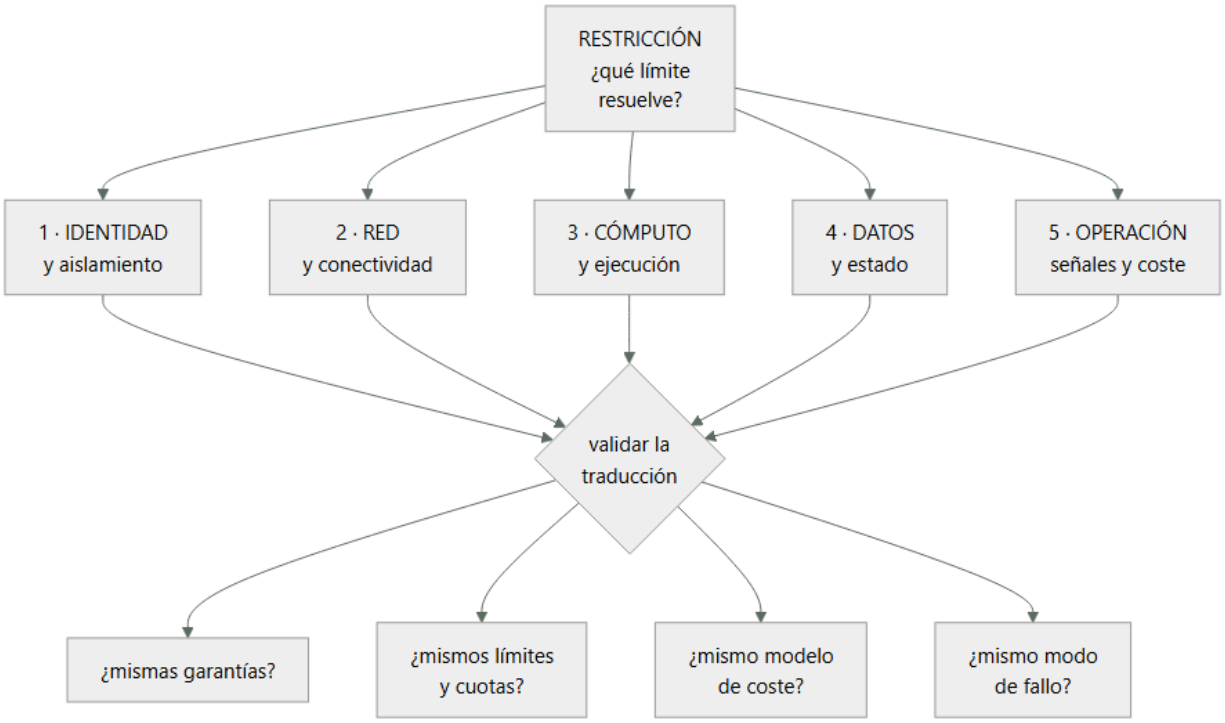
Concepto	Comprensión verificable
traducción por restricción	Buscar cómo la otra nube resuelve el mismo límite físico, en vez de buscar el servicio con nombre parecido.
equivalencia falsa	Dos servicios con el mismo propósito y garantías, límites o modelo de coste distintos.
unidad de aislamiento	El contenedor administrativo donde vive la separación: cuenta, suscripción o proyecto.
plano de control	La interfaz que crea y modifica recursos. Sus cuotas y su latencia difieren mucho entre nubes.
portabilidad aparente	Creer que usar la misma tecnología en tres nubes elimina las diferencias. No lo hace.
coste de traducción	Lo que cuesta operar en dos nubes: no es el doble de aprender, es el doble de operar.

Modelo mental

Una especialización combina fundamentos, evidencia de proyectos y juicio bajo restricciones; una insignia sin práctica no sustituye esa combinación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Traducir por restricción, no por nombre

Las tablas de equivalencias por nombre producen errores caros. El método que funciona empieza por la restricción.

```
EL MÉTODO
  1  ¿qué RESTRICCIÓN resuelve esto?
     → «necesito que un servicio pruebe su identidad ante
       otro sin credencial permanente»
  2  ¿cómo la resuelve la nube destino?
  3  ¿con qué garantías, límites y coste?
  4  ¿y con qué modo de fallo?

→ y solo entonces se escribe el nombre del servicio
→ empezar por el nombre es lo que produce las sorpresas
```

Y las cinco capas en las que conviene organizarse:

```
1  IDENTIDAD Y AISLAMIENTO
   quién es quién, qué puede, y dónde vive la separación
   → cuenta · suscripción · proyecto
2  RED Y CONECTIVIDAD
   direccionamiento, rutas, resolución de nombres, salida
   a internet, interconexión
3  CÓMPUTO Y EJECUCIÓN
   máquinas, contenedores, funciones, escalado
4  DATOS Y ESTADO
   objetos, bloques, relacional, no relacional, analítico,
   mensajería
5  OPERACIÓN
   señales, registros, políticas, coste, cuotas

→ en toda nube existen las cinco
→ y aprender una nube nueva es rellenar esta rejilla, no
  leer un catálogo
```

Y el mapa por restricción, que es lo transferible:

```
RESTRICCIÓN                        DÓNDE MIRAR EN CUALQUIER NUBE
«que un servicio pruebe quién es   identidad de carga de
sin credencial fija»               trabajo, federada
«separar entornos para que un       unidad de aislamiento
fallo no cruce»                     administrativa
```

```
«que el tráfico solo llegue a quien debe»        segmentación de red y reglas por identidad
«que un cambio no afecte a todos a la vez»       despliegue progresivo del proveedor o propio
«que el dato sobreviva a un administrador comprometido»   copia en otra unidad de aislamiento, inmutable
«saber quién gasta qué»                          etiquetas y jerarquía administrativa
«no pasar de N peticiones por segundo al plano de control»   cuotas del plano de control

→ y este mapa no cambia cuando cambian los productos
```

2. Dónde la equivalencia se rompe

Lo que ninguna tabla de equivalencias dice, y este programa ha medido.

```
1  LA UNIDAD DE AISLAMIENTO NO ES EQUIVALENTE
   cuenta, suscripción y proyecto se parecen y difieren en
     cuánto cuesta crear una nueva
     qué se hereda desde arriba
     y qué queda compartido aunque parezca separado
   → y de aquí salen los errores de aislamiento más caros
                                          clases 219, 231

2  LOS LÍMITES POR DEFECTO SON MUY DISTINTOS
   dos nubes con el «mismo» servicio de funciones tienen
   concurrencias por defecto de órdenes distintos
   → y en la región secundaria, los valores por defecto
     mandan                                clase 262, ley 26

3  EL MODELO DE COSTE CAMBIA LA ARQUITECTURA
   donde el tráfico entre zonas se cobra, la arquitectura
   correcta es otra
   donde el análisis se cobra por dato leído, el formato y
   la partición valen más que el motor    clase 243, ley 28
   → y una traducción literal puede multiplicar la factura

4  LA COHERENCIA DE LOS ALMACENES DE OBJETOS
   parecen iguales y sus garantías de lectura tras
   escritura, de listado y de sobrescritura no lo son
   → y esto rompe canales de datos silenciosamente
                                                ley 29

5  LA RESOLUCIÓN DE NOMBRES Y LA SALIDA A INTERNET
   quién resuelve, desde dónde, y qué pasa con las
   respuestas privadas difiere mucho          clase 195
   → es la fuente número uno de «funciona en una nube y no
     en la otra»

6  EL PLANO DE CONTROL
   su latencia y sus cuotas de peticiones difieren
   → y una herramienta de infraestructura como código que
     funciona en una nube se estrangula en otra
                                          clases 128, 262

7  Y LA SEMÁNTICA DE LOS PERMISOS
   denegar explícito, herencia, ámbito y evaluación no
   funcionan igual
   → una política traducida literalmente puede conceder
     más de lo que concedía                   clase 231
```

Y la regla que resume todo lo anterior:

```
LO QUE SE TRADUCE BIEN
  el propósito y la forma de la arquitectura

LO QUE NO SE TRADUCE
  garantías, límites, modelo de coste y modos de fallo

→ y esos cuatro son los que deciden si el sistema
  funciona
→ por eso la traducción se VALIDA midiendo, no leyendo
```

3. Contenedores y la portabilidad aparente

El caso especial que más expectativas defrauda.

```
LO QUE SÍ APORTA UNA CAPA COMÚN DE CONTENEDORES
  el empaquetado y el modelo de despliegue se parecen
  las descripciones de carga de trabajo son casi iguales
  y el conocimiento del equipo se traslada

LO QUE NO ELIMINA
  la identidad y su federación con la nube
  la red: direccionamiento, salida, balanceo, resolución
  el almacenamiento persistente y sus garantías
  el registro de imágenes y su disponibilidad
  las cuotas del plano de control
  el coste, muy distinto por nube
  y la operación: actualizaciones, versiones y su ritmo

→ la portabilidad es del ARTEFACTO, no del SISTEMA
→ y la parte que no se traslada es la que da los
  incidentes                                clase 185
```

Y el coste real de operar en dos nubes:

```
NO ES EL DOBLE DE APRENDER: ES EL DOBLE DE OPERAR
  dos inventarios                            clase 253
  dos modelos de permisos                    clase 231
  dos formas de red                          clase 194
  dos conjuntos de cuotas                    clase 262
  dos catálogos de alertas y dos guardias    clase 257
  dos formas de facturar                     clase 270
  y ensayos en las dos                       clase 261

→ y por eso multinube por defecto es caro
→ y multinube por una RAZÓN concreta —normativa,
  adquisición, un servicio que solo existe en una— se
  sostiene

y la pregunta que ordena la decisión
  «¿qué riesgo concreto estamos comprando con este coste?»
  → si la respuesta es «no depender de un proveedor», hay
    que cuantificar cuánto vale eso y compararlo
```

Y el mapa de la disciplina de coste, que también se traduce:

```
lo que existe en las tres, con otros nombres
  jerarquía administrativa para atribuir
  etiquetas y su obligatoriedad
  exportación detallada de facturación
  compromisos con descuento
  presupuestos y alertas
  y capacidad interrumpible

lo que cambia de verdad
  qué se cobra por transferencia entre zonas y regiones
  cómo se factura el almacenamiento frío y su
    recuperación
  cómo se cobra la analítica: por dato leído o por
    capacidad reservada
  y el retraso con que llega el detalle de coste

→ y esos cuatro cambian decisiones de diseño
                                             clase 270
```

4. Cómo se valida una traducción

Una traducción sobre el papel no vale nada. Estas son las comprobaciones que la convierten en fiable.

```
LAS CUATRO PREGUNTAS DE VALIDACIÓN
  1  ¿MISMAS GARANTÍAS?
     coherencia, durabilidad, orden, exactamente una vez
     → y aquí casi siempre hay una diferencia
  2  ¿MISMOS LÍMITES Y CUOTAS?
     por defecto y ampliables, por región
  3  ¿MISMO MODELO DE COSTE?
```

```
     qué se cobra por operación, por dato, por hora
  4  ¿MISMO MODO DE FALLO?
     ¿falla rápido o se degrada? ¿qué se ve cuando falla?

→ y las cuatro se responden con documentación y con una
  prueba
```

Y la prueba mínima antes de comprometerse:

```
PROTOTIPO DE UNA SEMANA
  el camino crítico de extremo a extremo
  con identidad real, red real y datos de tamaño
    realista
  midiendo latencia, coste por operación y comportamiento
    al fallar
  y provocando un fallo a propósito         clase 261

→ una semana de esto ahorra meses
→ y descubre las diferencias que ninguna tabla recoge
```

Y cómo se demuestra esta competencia:

```
LO QUE NO VALE
  «conozco las tres nubes»
  → y la parte 22 predijo que esto rendiría peor que
    conocer una a fondo                     clase 264

LO QUE VALE
  «migramos el canal de datos y descubrimos que las
   garantías de listado del almacén de objetos eran
   distintas; lo detectamos con una prueba de una semana
   antes de comprometernos»
  «traducimos la política y concedía más de lo que
   concedía la original; lo cogió la comprobación
   automática»
  «el mismo diseño costaba 3,2 veces más por el cobro de
   tráfico entre zonas, y rediseñamos la colocación»

→ efecto, mecanismo y cifra               clase 275
```

Y la lista de comprobación de la clase:

- ■ traduzco por restricción, no por nombre de servicio
- ■ uso la rejilla de cinco capas para orientarme
- ■ compruebo garantías, límites, coste y modo de fallo
- ■ sé que la unidad de aislamiento no es equivalente
- ■ reviso los valores por defecto, sobre todo en la región secundaria
- ■ compruebo la semántica de permisos, no solo su forma
- ■ verifico resolución de nombres y salida a internet
- ■ vigilo las cuotas del plano de control
- ■ no confundo portabilidad del artefacto con la del sistema
- ■ cuantifico el coste de operar en dos nubes
- ■ hago un prototipo de una semana antes de comprometerme
- ■ y provoco un fallo en ese prototipo

Y el cierre que enlaza con la clase siguiente: con el mapa y el método de traducción, queda la parte que examina esto por escrito y con tiempo limitado. Las preguntas de escenario y la estrategia de examen son la materia de la clase 274.

Ejemplo trabajado

CloudShop traduce tres piezas entre nubes. Lo que sigue son las cuatro diferencias que el prototipo de una semana encontró y ninguna tabla recogía, la política traducida que concedía de más, y la cuenta del coste real de operar en dos nubes.

Traducción 1 · El canal de datos.

```
qué se traducía
  ingesta a almacén de objetos, catálogo, y consultas
  analíticas                                clase 242

la tabla de equivalencias decía
  almacén de objetos      →  almacén de objetos
```

```
  catálogo               →  catálogo
  motor de consulta      →  motor de consulta

→ traducción aparentemente trivial
```

Y lo que el prototipo de una semana encontró:

```
DIFERENCIA 1 · garantías de listado
  el proceso dependía de listar un prefijo justo después
  de escribir
  → en el origen, el listado era inmediato
  → en el destino, podía tardar
  → y el trabajo posterior procesaba ficheros de menos
    SIN ERROR                                  ley 29

  cómo se detectó
    prueba con 40.000 ficheros escritos y listados de
    inmediato
    → 61 casos de listado incompleto
  cómo se arregló
    manifiesto explícito de ficheros, en vez de listar

DIFERENCIA 2 · modelo de coste de la consulta
  origen   por dato leído
  destino  por capacidad reservada
  → el mismo conjunto de 41 informes
    origen, tras optimizar               12 USD/informe
    destino con capacidad mínima         por debajo del
                                         umbral, pero con
                                         suelo mensual
  → por debajo de cierto volumen el destino era más caro
  → y por encima, mucho más barato
  → la decisión dependía del volumen, no del motor

DIFERENCIA 3 · cuota del plano de control
  la herramienta de infraestructura como código aplicaba
  341 recursos
  → en el destino se estrangulaba a mitad
  → aplicación fallida de forma intermitente  clase 128
  → se resolvió partiendo el estado y limitando el
    paralelismo

DIFERENCIA 4 · zonas horarias en las particiones
  la convención por defecto del catálogo diferia
  → y las particiones diarias se desplazaban unas horas
  → los informes diarios cuadraban salvo en el límite del
    día                                    clase 243
```

Y la valoración:

```
coste del prototipo                    1 semana, 1 persona
diferencias encontradas                                  4
  que ninguna tabla de equivalencias recogía             4
  que habrían llegado a producción sin error visible     2

→ las dos silenciosas eran las peligrosas
```

Traducción 2 · La política que concedía de más.

```
política original
  permitir lectura de un prefijo concreto del almacén, a
  un rol de servicio, con denegación explícita para todo
  lo demás                                   clase 231

la traducción literal
  se mantuvo la concesión y se omitió la denegación
  explícita
  → porque en la nube destino la herencia y la evaluación
    funcionaban distinto y «parecía innecesaria»

lo que produjo
  el rol heredaba desde el nivel superior un permiso de
  lectura sobre el contenedor entero administrativo
  → y sin la denegación explícita, la herencia ganaba
  → el rol podía leer 214 conjuntos de datos en vez de 1
```

Y cómo se detectó:

```
no en la revisión humana: la revisión lo aprobó

lo cogió la comprobación automática de permisos efectivos
  → que compara «qué puede hacer realmente esta identidad»
    antes y después                          clase 217
  → 214 recursos accesibles frente a 1 esperado

→ la lección: se compara el EFECTO, no el texto de la
  política
→ y esa comprobación se añadió a la cadena para toda
  traducción
```

La cuenta del coste real de operar en dos nubes.

```
CloudShop tenía una razón concreta para la segunda nube
  un cliente del sector público exigía residencia en una
  región que el proveedor principal no ofrecía
                                           clase 280

y se midió lo que costaba mantenerla

  inventario y etiquetado, segunda nube      0,4 personas
  modelo de permisos y su auditoría          0,3
  red y conectividad                         0,3
  cuotas y capacidad                         0,2
  alertas, guardia y procedimientos          0,6
  facturación y atribución                   0,2
  ensayos                                    0,2
  actualizaciones y versiones                0,3
                                             ■■■■■■
                                             2,5 personas

  más el coste de infraestructura            41.000 USD/mes
  y el sobrecoste por menor volumen
    (compromisos peores)                     +9 %
```

Y la decisión que se tomó con esa cifra:

```
ingresos del cliente que exigía la segunda nube
                                        1,9 M USD/año
coste de mantenerla                ~2,5 personas + 492.000
                                        USD/año

→ se mantuvo, y con la cifra escrita en el registro de
  decisión                                clase 272
→ con condición de revisión: «si estos ingresos bajan de
  900.000, se revisa»

y lo que NO se hizo
  no se replicó el resto de la plataforma en la segunda
  nube «por si acaso»
  → solo lo que ese cliente necesitaba
  → 6 servicios de 41
```

Y la comparación con lo que se había propuesto al principio:

```
propuesta inicial   «seamos multinube en todo, para no
                    depender de un proveedor»
  coste estimado    ~7 personas + 1,4 M USD/año
  riesgo evitado    no cuantificado

y la pregunta que la desmontó
  «¿cuál es el escenario concreto del que nos protege, y
  cuánto nos costaría si ocurriera?»
  → el escenario realista era una subida de precios o una
    pérdida de servicio prolongada
  → y la protección real requería mantener el sistema
    EJECUTÁNDOSE en las dos, no solo poder migrar
  → lo que multiplicaba la cifra

→ se decidió: multinube donde hay una razón concreta;
  portabilidad razonable en lo demás
→ y «portabilidad razonable» se definió: contratos, datos
```

```
  exportables y decisiones registradas, no abstracciones
  propias                                  clase 267
```

Y la rejilla de cinco capas, rellenada para orientarse.

```
el equipo mantuvo una tabla de una página por nube

  capa            qué mirar             dónde difiere más
  identidad       unidad de aislamiento herencia y
                  y federación          denegación
  red             direccionamiento,     resolución de
                  salida, balanceo      nombres privados
  cómputo         escalado, arranque,   límites por
                  interrumpible         defecto
  datos           garantías del almacén listado y coste
                  y del relacional      por operación
  operación       señales, políticas,   retraso del
                  cuotas, coste         detalle de coste

→ una página por nube, actualizada cuando algo sorprende
→ y usada para entrar en una nube nueva en días
```

La lección que esta clase deja: la traducción del canal de datos parecía trivial y el prototipo de una semana encontró cuatro diferencias, dos de ellas silenciosas —un listado incompleto y un desplazamiento de particiones— que habrían llegado a producción sin producir ningún error. Y la política traducida pasó la revisión humana mientras concedía acceso a 214 conjuntos de datos en vez de uno: lo cogió comparar permisos efectivos, no leer el texto.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-22-specializations-certifications-career/273-mapeo-aws-azure-google-cloud-kubernetes-y-finops/lab.py
```

El laboratorio selecciona el motor de práctica assessment y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa certification-map en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una evaluación por escenarios con rúbrica y evidencia trazable. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye certification-map para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [ ] lab.py termina con código 0 y genera JSON válido.
- [ ] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [ ] Existe una prueba positiva y una prueba negativa con evidencia.
- [ ] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [ ] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [ ] Otra persona puede repetir el recorrido sin conocimiento tácito.

Errores frecuentes

Síntoma	Causa probable	Corrección
Un servicio equivalente se comporta distinto en producción	Se tradujo por nombre y no se compararon garantías, límites, coste y modo de fallo	Traduce por restricción y valida las cuatro preguntas con documentación y con un prototipo del camino crítico.
Una política traducida concede más de lo que concedía	La herencia y la evaluación de permisos difieren y la denegación explícita se omitió	Compara permisos efectivos antes y después, no el texto de la política; automatiza esa comprobación en la cadena.
El canal de datos procesa registros de menos sin dar error	Las garantías de listado y de lectura tras escritura del almacén no son iguales	Usa manifiestos explícitos en vez de listar, y prueba con volumen realista escribiendo y listando de inmediato.
La infraestructura como código se aplica a medias e intermitentemente	Se alcanzan las cuotas de peticiones del plano de control, que difieren por nube	Parte el estado, limita el paralelismo y vigila la cuota del plano de control como una más del inventario.
El mismo diseño cuesta varias veces más en la otra nube	El modelo de coste cambia lo que es una buena arquitectura	Revisa qué se cobra por transferencia, por operación y por dato leído; rediseña colocación y formato antes de migrar.
Se adopta multinube por no depender de un proveedor y el coste se dispara	No se cuantificó el riesgo evitado ni el coste de operar dos nubes	Exige un escenario concreto y su coste; multinube donde hay una razón, portabilidad razonable en lo demás.

Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

Preguntas de comprobación

1. ¿Por qué se traduce por restricción y no por nombre de servicio?
2. ¿Cuáles son las cinco capas de la rejilla y para qué sirven?
3. ¿Qué cuatro cosas no se traducen nunca y por qué deciden el resultado?
4. ¿Qué aporta y qué no aporta una capa común de contenedores?
5. ¿Qué comprueba un prototipo de una semana que ninguna tabla recoge?

Referencias

- AWS (2024). Compare AWS and Azure services — equivalencias de servicio entre AWS y Azure. <https://learn.microsoft.com/azure/architecture/aws-professional/services>
- Google Cloud (2024). Google Cloud for AWS professionals — equivalencias de servicio para quien viene de AWS. <https://cloud.google.com/docs/get-started/aws-comparison>
- CNCF (2024). Kubernetes conformance and portability — la certificación de conformidad como garantía de portabilidad. <https://www.cncf.io/certification/software-conformance/>
- FinOps Foundation (2024). FOCUS: FinOps cost and usage specification — coste comparable entre nubes. <https://focus.finops.org/>
- Brewer, E. (2012). CAP twelve years later: how the rules have changed — por qué las garantías no se traducen. <https://www.infoq.com/articles/cap-twelve-years-later-how-the-rules-have-changed/>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 22 en PDF · Manual integral

Evaluación — 273 Mapeo AWS, Azure, Google Cloud, Kubernetes y FinOps

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega certification-map con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [ ] Resultado reproducible por una segunda persona.
- [ ] Evidencia vinculada a cada afirmación técnica.
- [ ] Prueba negativa o de fallo incluida.
- [ ] Costos y permisos expresados con unidades y alcance.
- [ ] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.